

Tópicos em Engenharia de Software

Texto Preliminar - Versão ECI-ES Turma 2025 02

Geraldo Xexéo

9 de agosto de 2025 01:46

Copyright © 2025 Geraldo Xexéo.

Todos os direitos reservados.

Esta é um pré-publicação com o texto parcial em trabalho.

Apenas os alunos da cadeira de Engenharia de Software de Engenharia de Computação e Informação, Turma de 2025/02, estão autorizados a ter uma cópia digital e uma cópia impressa deste livro.

Contato com o autor pelo e-mail xexeo@cos.ufrj.br.

Sumário

Lista de Figuras	viii
Lista de Tabelas	xii
1. Prefácio: A Engenharia de Software e o Mundo Real	2
1.1. Como Ler Esse Livro	2
1.2. Níveis de Dificuldade	3
1.3. A Engenharia de Software Está Sempre Certa?	3
1.4. Como cobrar? A pergunta mais comum nos cursos de ES.	4
I. Entendendo o Software	6
2. Introdução a Engenharia de Software	7
2.1. O que é Software	8
2.2. Por que software é importante	9
2.2.1. A Ubiquidade do Software	11
2.3. Quais os riscos que software traz?	11
2.3.1. Acontecimentos Recentes (2024)	13
2.3.2. Riscos Éticos	13
2.4. Por que é difícil fazer software?	13
2.4.1. O Relatório CHAOS	15
2.5. O que é Engenharia de Software	17
2.5.1. Uma Visão da Engenharia de Software	18
2.6. Por que a Engenharia de Software é necessária?	18
2.6.1. Diferenças para outras engenharias	18
2.7. Qual o Segredo do Sucesso de um Projeto de Software	19
2.8. O Corpo de Conhecimento da ES	20
2.9. As normas da Engenharia de Software	21
2.10. SEMAT	21
2.10.1. O Kernel Essencial	22
2.11. Quem é o Engenheiro de Software	23
2.12. Exercícios	24

3. Partes Interessadas	26
3.1. O Que São Partes Interessadas	28
3.1.1. Usuários	29
3.1.2. Perspectivas dos Usuários	30
3.2. Partes Interessadas e Valor	31
3.3. Gerência das Partes Interessadas	32
3.4. A abordagem SEMAT para Partes Interessadas	33
3.5. Caracterização da Parte Interessada	33
3.5.1. Interesses e Objetivos	33
3.6. Classificação das Partes Interessadas	35
3.6.1. O que é poder?	36
3.7. Análise das Partes Interessadas	38
3.8. Representação das Partes Interessadas	39
3.9. Engajamento das Partes Interessadas	39
3.10. Matriz RACI	40
3.11. Conclusão	41
3.12. Exercícios	41
4. Valor	43
4.1. Conceituação Genérica de Valor	44
4.2. Valor na Economia	44
4.2.1. Utilidade	45
4.2.2. Microeconomia Moderna e o Custo de Oportunidade	46
4.3. Valor em Marketing	46
4.3.1. Os Elementos do Valor - B2C	48
4.3.2. Os Elementos do Valor - B2B	49
4.3.3. Outras Técnicas	51
4.4. Priorização Baseada em Valor para o Cliente	51
4.4.1. Técnicas simples de ordenação	52
4.4.2. Método MoSCoW	52
4.4.3. Análise de Kano	53
4.4.4. Prioridade e Consenso	54
4.4.5. Usando mais de uma dimensão	55
4.4.6. Prioridade e Dependências	57
4.5. Valor Financeiro de Projetos	57
4.5.1. Custo Total de Propriedade	59
4.5.2. Retorno do Investimento	60
4.5.3. Valor Presente Líquido	61
4.5.4. Taxa Interna de Retorno	61
4.6. Valor em Software	62
4.6.1. Triângulo do Valor	63
4.6.2. Valor em Métodos Ágeis	63
4.7. Willingness to Pay e o Preço de Software	64
4.8. Conclusão	65
4.9. Exercícios	65
5. 5W2H	67
5.1. As Perguntas Certas	68

5.2.	Usos do 5W2H	68
5.2.1.	O <i>Framework de Zachman</i>	69
5.3.	Onde Está o Valor?	70
5.4.	Várias Perguntas para cada Palavra	71
5.4.1.	Perguntas sobre o que	71
5.4.2.	Perguntas sobre quem	71
5.4.3.	Perguntas sobre quando	72
5.4.4.	Perguntas sobre onde	72
5.4.5.	Perguntas sobre como	72
5.4.6.	Perguntas sobre por que	73
5.4.7.	Perguntas sobre quanto	73
5.5.	Exercícios	73
6.	Qualidade	74
6.1.	Tipos de Qualidade	77
6.2.	Dimensões da Qualidade	77
6.3.	Qualidade é Sempre por um Ponto de Vista	82
6.4.	Qualidade de Produto e Qualidade de Processo	82
6.5.	Falhas e Defeitos	83
6.6.	Quanto Custa a Qualidade	83
6.7.	Gestão da Qualidade	84
6.8.	Exercícios	85
7.	Processo de Software	86
7.1.	Processos de Software segundo a ISO/IEC/IEEE 12207:2017	87
7.2.	O Modelo Genérico de Pressman	88
7.3.	Cascata versus Protótipos	88
7.3.1.	Processo em Cascata	89
7.3.2.	Prototipagem	90
7.4.	Iteração e Incrementos	91
7.5.	Processos Prescritivos	93
7.5.1.	RUP	93
7.6.	Processos Ágeis	94
7.6.1.	O Manifesto Ágil	94
7.6.2.	Evolução da Agilidade	95
7.6.3.	Críticas aos Processos Ágeis	95
7.6.4.	XP	95
7.6.5.	Scrum	96
7.6.6.	Continuous Delivery	97
7.7.	Importância da Estruturação de Processos Organizacionais	98
7.8.	O Capability Maturity Model Integration (CMMI)	98
7.9.	O Modelo MPS.BR para Software	99
II.	Especificando o Software	101
8.	Software Project Canvas	102
8.1.	O que são Canvas	103
8.2.	Projetos, Agilidade e Canvas	104
8.2.1.	O <i>Project Model Canvas</i>	105

8.3.	O <i>Software Project Canvas</i>	107
8.3.1.	Problemas e Oportunidades	108
8.3.2.	Objetivos e Metas	109
8.3.3.	Requisitos Funcionais, Não-Funcionais e Restrições	109
8.3.4.	Requisitos Funcionais	110
8.3.5.	Requisitos Não-Funcionais e Restrições	110
8.3.6.	Partes Interessadas	111
8.3.7.	Equipe	112
8.3.8.	Subsídios e Premissas	112
8.3.9.	Entregas	112
8.3.10.	Riscos	113
8.3.11.	Esforço	114
8.4.	A Metodologia do <i>Software Project Canvas</i>	114
8.4.1.	Conceber Projeto	114
8.4.2.	Concepção Iterativa e Incremental	116
8.4.3.	Ordens Alternativas de Preenchimento	116
8.4.4.	Verificar Proposta	117
8.4.5.	Validação	119
8.4.6.	Compartilhar Projeto	119
8.5.	Conclusão	119
9.	Modelagem de Processos de Negócio	121
9.1.	Processos de Negócio	122
9.2.	Exemplos de Processos	123
9.3.	Representação de Processos de Negócio	124
9.4.	Por que Entender os Processos de Negócio	124
9.5.	Exercícios	124
10.	BPMN	125
10.1.	Usos de BPMN	126
10.2.	Participantes	126
10.3.	Símbolos básicos	127
10.4.	Atividades	127
10.5.	Eventos	128
10.5.1.	Eventos Iniciais	129
10.5.2.	Eventos Finais	130
10.5.3.	Eventos Intermediários	131
10.6.	<i>Gateways</i>	132
10.7.	Fluxos	134
10.8.	Alguns Exemplos	134
10.9.	Eventos Anexados a Atividades	135
10.10.	Exercícios	135
11.	Oportunidades e Problemas	136
11.1.	Oportunidades	137
11.2.	Problemas	137
11.3.	Tipos de Problemas	138
11.4.	Identificando Problemas	138
11.4.1.	A Análise de Pareto	139
11.4.2.	Calculando a Quantidade	140

11.4.3.	Usando o Impacto dos Problemas	141
11.4.4.	Buscando Causas	143
11.4.5.	Organização do Diagrama Espinha de Peixe	145
11.5.	Caracterização do Problema	147
11.6.	Oportunidade e 5W2H	147
11.6.1.	Buscando Oportunidades com as Partes Interessadas	147
11.7.	Propondo Oportunidades ao Cliente	148
11.8.	Oportunidade e Valor	149
11.9.	Exercícios	149
12.	Requisitos	150
12.1.	Definição Formal	151
12.1.1.	Exemplos de Requisitos	152
12.2.	Requisitos x Benefícios	154
12.3.	Tipos de Requisitos	154
12.3.1.	Requisitos Funcionais	154
12.3.2.	Requisitos não funcionais	155
12.3.3.	Restrições	155
12.3.4.	Outros tipos de requisitos	156
12.4.	O Papel dos Requisitos no Projeto	156
12.5.	Engenharia de Requisitos	157
12.6.	Elicitação de Requisitos	157
12.6.1.	Processo de Elicitação de Requisitos	159
12.7.	Níveis de Abstração dos Requisitos	161
12.8.	O Risco de Requisitos Falsos	161
12.9.	Características de um Bom Requisito	162
12.9.1.	Efeitos de Requisitos com Problemas	163
12.10.	Atributos dos Requisitos	163
12.10.1.	Volere e o <i>snowcard</i>	164
12.11.	Priorizando Requisitos	165
12.12.	Requisitos Mudam com o Tempo	166
12.13.	Requisitos e Necessidades	166
12.14.	Requisitos de Acordo com Normas IEEE	167
12.14.1.	Requisitos Funcionais na Norma IEEE	168
12.14.2.	O Documento de Especificação de Requisitos	169
12.15.	A Visão do Novo Sistema	170
12.16.	Descrevendo Requisitos	170
12.16.1.	Exemplos de Requisitos	171
12.17.	Exercício	172
13.	Casos de Uso	173
13.1.	Conceituação de Caso de Uso	175
13.2.	Conceitos Importantes nos Casos de Uso	176
13.2.1.	Ator	177
13.2.2.	Objetivos	178
13.2.3.	Cenários	178
13.2.4.	As Alternativas	182
13.2.5.	Casos de Uso em UML	182
13.3.	A Narrativa do Caso de Uso	182

13.4.	O Nível de Abstração	183
13.5.	O Escopo do Caso de Uso	184
13.6.	Casos de Uso Especiais	185
13.7.	Partes do Caso de Uso	185
13.8.	Exemplo de Um Caso de Uso	186
13.9.	Levantando Casos de Usos	187
13.10.	Diagramas de Caso de Uso	190
13.10.1.	Especificando o <i>Subject</i>	190
13.10.2.	Generalização entre Atores	191
13.10.3.	Relações Entre Casos de Uso	191
13.10.4.	Usando Cardinalidades em Casos de Uso	194
13.11.	O Que o Diagrama de Caso de Uso Não Informa	195
13.12.	Exercícios	196
14.	Histórias do Usuário	197
14.1.	Conceituação de Histórias do Usuário	199
14.2.	Um <i>Template</i> Padrão para Histórias de Usuário	200
14.2.1.	Outros formatos possíveis	200
14.3.	INVEST	201
14.3.1.	Independente	201
14.3.2.	Negociável	203
14.3.3.	Com Valor	203
14.3.4.	Estimável	203
14.3.5.	Pequena	204
14.3.6.	Testável	204
14.4.	Estados de Uma História do Usuário	204
14.5.	Exercícios	205
III.	Gerenciando de Software	206
15.	O Que é um Projeto	207
15.1.	Recursos e Entregas	208
15.2.	Partes Interessadas	208
15.3.	Sucesso, Fracasso e Riscos	208
15.3.1.	A Incerteza dos Projetos e os Riscos	209
15.3.2.	Benefícios da Gerência de Projeto	209
15.4.	Pontos Críticos ou Pontos Chave de Sucesso	210
15.5.	Conclusão	210
16.	Gestão de Projetos de Software	211
16.1.	Técnicas Tradicionais de Gestão de Projeto	212
16.1.1.	Estrutura Analítica de Projeto	212
16.1.2.	Controlgrama de Gantt	213
16.1.3.	Redes PERT/CPM	214
16.1.4.	Algoritmo para cálculo das datas em uma rede PERT/CPM	217
16.2.	Uso das Técnicas Tradicionais	219
16.3.	Técnicas Ágeis de Gestão de Projeto	219
16.3.1.	Kanban	219

IV. Implementando o Software	220
17. Testes	222
17.1. Terminologia	222
17.2. Introdução aos Conceitos de Teste	223
V. Medindo o Software	225
18. Medidas e Estimativas do Tamanho do Software	226
18.1. Esforço	227
18.2. Algumas Medidas Conceitualmente Simples	228
18.2.1. A Medida Mais Simples: Linhas de Código	229
18.2.2. Pontos de História	230
18.2.3. Tamanhos de Camisa	231
18.3. Visão Geral dos Métodos de Estimativa	231
18.4. Métodos de Estimativa Baseado em Opinião	232
18.4.1. O Método de Estimativa do PERT	232
18.4.2. O Método de Delphi	233
18.4.3. O Planning Poker	233
19. Análise de Pontos de Função	235
19.1. Visão Geral de Pontos de Função	236
19.2. Procedimento de Contagem	236
19.3. Funções Transacionais	237
19.4. Funções de Dados	237
19.5. Identificando Funções de Negócio	238
19.5.1. Identificando Saídas	238
19.5.2. Identificando Consultas	239
19.6. Identificando Entradas	239
19.6.1. Entendendo as Lógicas de Processamento	239
19.7. Identificando Arquivos Internos	240
19.7.1. Identificando Arquivos Externos	240
19.7.2. Identificando Itens de Dados (DETs)	241
19.8. As Perguntas	243
19.9. Cálculo dos Pfs Finais	243
19.10. Conclusão	244
20. Uma Visão Reduzida do Modelo COCOMO II	245
20.1. Dois Modelos em Um	246
20.2. Esforço em Função do Tamanho	246
20.3. Existe uma Relação Entre Esforço e Tempo	247
20.3.1. Cálculo dos Fatores de Escala	248

Lista de Figuras

2.1.	Um software executando na web e parte do seu código.	8
2.2.	Sala de treinamento replicando exatamente a sala controle de uma usina nuclear, feita originalmente com tecnologia analógica e modernizada com computadores e software	10
2.3.	Esquema da virtualização do hardware.	11
2.4.	Tamanho de código de alguns projetos conhecidos.	15
2.5.	Curva de falha do hardware	16
2.6.	Curva de falha do software	16
2.7.	Pirâmide que explica o escopo da Engenharia de Software.	18
2.8.	Mind map para as áreas de conhecimento da Engenharia de Software: Fonte:(IEEE Computer Society, 2014a)	20
2.9.	O Kernel Essencial do SEMAT. Fonte:(Jacobson, Ng et al., 2013)	22
2.10.	Os Espaços de Atividades no Kernel do SEMAT. Fonte:(Jacobson, Ng et al., 2013)	23
2.11.	As Competências no Kernel do SEMAT. Fonte:(Jacobson, Ng et al., 2013)	23
3.1.	O Kernel Essencial do SEMAT. Fonte: (Jacobson, Ng et al., 2013)	27
3.2.	O impacto das partes interessadas.	28
3.3.	3 tipos comuns de usuários dos sistemas. Fonte:próprio autor	31
3.4.	Quadro para o tratamento das partes interessadas. Fonte: (PMI, 2017)	35
3.5.	Classificação das partes interessadas. Fonte: (Mitchell, Agle e Wood, 1997)	39
4.1.	Elementos do Valor B2C. Baseado em:Almquist, Senior e Bloch, 2016	48
4.2.	Hierarquia das Necessidades de Maslow. Fonte: Wikipedia Commons por Felipe Sanchez (CC-BY-SA 3.0) e J. Finkelstein (GFDL)	49
4.3.	Elementos do Valor B2B. Baseado em: Almquist, Cleghorn e Sherer, 2018	50
4.4.		54
4.5.	Gráfico com quadrantes para comparar prioridades usando benefício e custo.	56
4.6.	Gráfico com quadrantes, determinados arbitrariamente pelo usuário, para os itens de projeto listados na Tabela 4.2. As setas mostram uma possível sequência de prioridade. Os itens C e D foram considerados desnecessários ao projeto.	57
4.7.	Cálculo do valor futuro de um investimento de R\$10.000,00 por mês com uma taxa de juros de 1%.	59

Lista de Figuras

4.8.	Gráfico do TCO de impressoras em função de da quantidade de páginas impressas. Percebe-se que após aproximadamente 3500 páginas é melhor comprar uma impressora a laser mais cara, caso esse fosse o único fator de análise.	60
4.9.	O triângulo do valor para a maioria dos projetos.	63
5.1.	O Framework de Zachman fornece uma visão da arquitetura empresarial a partir de 6 dimensões definidas pelo 5WH. Reprinted with permission by John A. Zachman, Zachman International.	70
6.1.	Mapa Mental da organização da área do conhecimento Qualidade de Software no SWEBOK 3.0. Fonte:(IEEE Computer Society, 2014a)	76
6.2.	A Qualidade é a pedra basilar da Engenharia de Software. Fonte:(Pressman e Maxim, 2014)	76
6.3.	Modelo de Qualidade de McCall. Fonte:(McCall, Richards e Walters, 1977)	78
6.4.	O Modelo Fuzzy de Qualidade de Software. Fonte:(Belchior, 1997)	80
6.5.	Modelo de Qualidade de Produto de Software da norma ISO 25010. Fonte:(ISO, 2011)	80
6.6.	Modelo de Qualidade em uso de Software da norma ISO 25010. Fonte:(ISO, 2011)	81
6.7.	Diferentes pontos de vista implicam em diferentes avaliações de qualidade. Fonte: próprio autor	82
7.1.	Desenho representativo de iterações em uma imagem, aumentando o detalhe a cada passo.	92
7.2.	Desenho representativo de incrementos em uma imagem.	92
8.1.	O Software Project Canvas. Imagem do autor	103
8.2.	O Business Model Canvas. Fonte: Osterwalder, Pigneur e Clark (2010)	104
8.3.	Um quadro Scrum/Kanban muito simples. Fonte: por Jess Iasovski, CC BY-SA 3.0	105
8.4.	O processo de planejamento usando o Project Model Canvas. Fonte: Finocchio Jr. (2013)	106
8.5.	Taxonomia de Requisitos Não Funcionais. Fonte: Sommerville (2015)	111
8.6.	Descrição do processo de criação do Software Project Canvas. Fonte: imagem do autor	115
8.7.	Descrição do sub-processo Conceber Projeto, do processo de criação do Software Project Canvas. Fonte: imagem do autor	115
8.8.	Desenhos alternativos, que levam a processos alternativos para a fase de Conceber Projeto.	118
10.1.	Um exemplo de processo descrito em BPMN. Fonte: imagem do autor	126
10.2.	Exemplo de uso de um evento paralelo em BPMN	133
10.3.	Exemplo do atendimento da ordem de serviço em BPMN	135
11.1.	Gráfico com a quantidade de problemas.	141
11.2.	Pareto com custo	143
11.3.	Um diagrama de Espinha de Peixe	144
11.4.	Um diagrama de Espinha de Peixe resumido, construído com uma ferramenta de <i>mind map</i>	145

Lista de Figuras

11.5.	Um diagrama de Espinha de Peixe expandido, construído com uma ferramenta de <i>mind map</i>	145
11.6.	Um Diagrama de Causa Raiz vertical, onde a causa raiz está marcada. . . .	146
12.1.	Um cartão Volere.	153
12.2.	Exemplo de um diagrama de casos de uso	153
12.3.	Exemplo de um mapa mental.	154
12.4.	Pequena taxonomia de requisitos não funcionais. Fonte: (Sommerville, 2015)	156
12.5.	O papel dos requisitos no desenvolvimento do sistema. Fonte:(J. Robertson e S. Robertson, 2006)	157
12.6.	A elicitação de requisitos é um processo iterativo onde, por aproximações sucessivas, o desenvolvedor consegue a aprovação de uma especificação de requisitos.	160
12.7.	O processo de elicitação de requisitos, adaptado de Christel e Kand (1992).	160
12.8.	Cada requisito se posicionará em algum ponto desse gráfico, de acordo com as notas de avaliação.	165
13.1.	Exemplo de cenário principal de caso de uso.	174
13.2.	Exemplo de um caso de uso.	175
13.3.	Representação de um ator em um diagrama de caso de uso.	177
13.4.	Representação de dois atores identificados.	177
13.5.	Exemplo de cenário principal do caso de uso sacar dinheiro	179
13.6.	Representação abstrata de um cenário principal e dos fluxos alternativos . .	180
13.7.	Representação abstrata de cada alternativa investigada nesse caso.	181
13.8.	Exemplo de cenário principal do caso de uso atribuir notas	181
13.9.	Exemplo de cenário principal do caso de uso emitir boletim	188
13.10.	Os elementos básicos de um diagrama de caso de uso são: atores, casos de uso e associações. Fonte: do autor	190
13.11.	Indicando o sistema ao qual os casos de uso se referem. Fonte:do autor. . . .	191
13.12.	Nesse diagrama de casos de uso, o gerente é um especialização de funcionário, logo pode fazer tudo que o funcionário faz, mais alguns casos de uso que são exclusivos dele. Fonte: do autor	192
13.13.	Um <i>Profissional de Saúde</i> é a generalização de Enfermeira, Médico ou Auxiliar de Enfermagem, mas nenhum ator é um profissional de saúde se não for de um dos outros tipos. Fonte: do Autor.	192
13.14.	Representações das associações entre casos de uso.	193
13.15.	Exemplo da associação de inclusão entre casos de uso	193
13.16.	Um exemplo da associação de extensão entre casos de uso.	194
13.17.	Exemplo da associação de herança entre casos de uso	195
13.18.	Exemplo do uso de cardinalidades em casos de uso. Fonte: do autor	195
14.1.	Exemplo de uma história do usuário em um cartão. Fonte: do autor.	198
14.2.	Partes encontradas em um cartão de história do usuário, segundo um modelo original da Connextra (Patton e Economy, 2014a).	200
16.1.	EAP em formato texto, com o mesmo significado da Figura 16.2. Fonte: do autor	213
16.2.	Diagrama com modelo EAP equivalente ao da Figura 16.1. Fonte: do autor .	214
16.3.	Controlgrama de Gantt feito com o software Project Libre. Fonte: do autor .	215
16.4.	Modelo PERT original para a Tabela 16.1	215

Lista de Figuras

16.5.	Um diagrama de rede construído com o software MSPProject, software altamente configurável. Vemos atividades completas, em andamento. Também é possível notar as atividades no caminho crítico. Fonte: do autor	216
16.6.	Diagrama de rede para a Tabela 16.1, feito no software Project Libre. Fonte: o autor	216
18.1.	Ilustração da sequência do Planning Poker com duas rodadas	234
19.1.	O processo geral de contagem de Pontos de Função. Fonte: do autor	236
19.2.	Planilha de registro para ajudar a contar os Pontos de Função	244
20.1.	Efeitos das constantes A e B na relação entre tamanho e esforço	247

Lista de Tabelas

2.1. Resolução de projetos de software por tamanho e por método de desenvolvimento, para mais de 10.000 projetos. Fonte:(The Standish Group, 2015)	17
2.2. Fatores de Sucesso de um Projeto de Software	19
3.1. Exemplo de formulário para registro do interesse e objetivo da parte interessada.	34
3.2. Exemplo de matriz de cruzamento para registro do interesse e objetivo da parte interessada.	35
3.3. Matriz de Engajamento: C significa comportamento corrente, D o comportamento desejado.	36
3.4. Bases do Poder Social - Resumo	38
3.5. Exemplo de Matriz RACI	41
4.1. Resolvendo a classe de Kano por meio do cruzamento de duas perguntas (Moorman, 2012)	54
4.2. Benefício e esforço para itens de um project backlog imaginário.	56
4.3. Valores usados para calcular o TCO de uma impressora.	60
4.4. Tabela de cálculo do Valor Presente Líquido, considerando saldos diferentes em cada período. Foi usada uma taxa de 5% a cada período. A fórmula para cada Valor Presente é $Caixa_i / (1 + Taxa)^{Período}$	61
5.1. Perguntas e respostas a partir da notícia.	69
6.1. Custos típicos de software por tamanho e nível de qualidade. Fonte:(C. Jones e Bonsignour, 2012)	84
8.1. Ações recomendadas por (Finocchio Jr., 2013) de acordo grau de importância do risco	114
10.1. Símbolos para os eventos que podem aparecer no início de um processo.	128
10.2. Símbolos para os eventos que podem aparecer no início de um processo.	130
10.3. Símbolos para os eventos que podem aparecer no fim de um fluxo de processo.	131
10.4. Eventos que podem aparecer no meio de um processo.	132
10.5. Símbolos para os gateway.	133
11.1. Uma contagem de tipos de erros para uso da técnica de Pareto	140

Lista de Tabelas

11.2. Custo médio relativo dos problemas	142
11.3. Custo total relativo	142
12.1. Adequação das formas de elicitação propostas por K. E. Wiegers e Beatty (2013) e alguns tipos de projeto.	158
12.2. Processo de Elicitação de Requisitos segundo Christel e Kand (1992).	159
13.1. Verbos comumente usados para nomear casos de uso	178
13.2. Uma forma alternativa de representar os cenários possíveis a partir de um cenário principal e as alternativas associadas	182
13.3. Ícones que representam o nível de abstração dos casos de uso. Fonte:(Cockburn, 2000)	184
13.4. Ícones que representam o nível de escopo dos casos de uso. Fonte:(Cockburn, 2000)	184
13.6. Associações que podem ocorrer entre atores e casos de uso.	190
16.1. Tabela de tarefas de um projeto	217
16.2. Tabela de tarefas de um projeto, com as datas de início e término mais cedo . .	217
16.3. Tabela de tarefas de um projeto, com as datas de início e término mais cedo e mais tarde.	218
19.1. Tabela de apoio a determinação do tipo de função transacional. P=possível, N=Não permitida, O=obrigatório, O*= obrigatório que pelo menos uma das condições aconteça.	240
19.2. Cálculo da complexidade das saídas externas.	241
19.3. Cálculo da complexidade das entradas externas.	241
19.4. Cálculo da complexidade das consultas externas.	242
19.5. Cálculo da complexidade dos arquivos lógicos internos.	242
19.6. Cálculo da complexidade das interfaces lógicas externas.	242
20.1. Conversão de Pontos de Função não ajustados para linhas de código. Fonte: (B. Boehm, Abts e Brad Clark, 1997)	246
20.2. Valores das constantes no modelo COCOMO II padrão	248

Explicação

Esse livro é escrito simultaneamente com outro, que trata de Análise de Sistemas. Eles possuem capítulos comuns.

Prefácio: A Engenharia de Software e o Mundo Real

As a rule, software systems do not work well until they have been used, and have failed repeatedly, in real applications

(David Parnas)

Conteúdo

1.1.	Como Ler Esse Livro	2
1.2.	Níveis de Dificuldade	3
1.3.	A Engenharia de Software Está Sempre Certa?	3
1.4.	Como cobrar? A pergunta mais comum nos cursos de ES.	4

1.1. Como Ler Esse Livro

A Engenharia de Software existe há um pouco mais de meio século, e se desenvolveu por muitos caminhos distintos, em função do domínio de aplicação, das partes interessadas e dos praticantes.

O que este livro apresenta é uma visão, ou uma versão, de várias possíveis abordagens a uma introdução à Engenharia de Software. Para o livro, foram selecionadas técnicas historicamente representativas e técnicas com perspectivas de serem usadas nos próximos cinco e dez anos, segundo a visão do autor. Ele é uma versão da miríade de possibilidades de escrever um livro de Engenharia de Software, no conteúdo e na organização.

O leitor não deve ler esse livro como uma verdade absoluta, apesar de todos os esforços terem sido feitos para que ele apresente apenas teorias e práticas sólidas, mas sim como uma abordagem

específica, e sempre com uma leitura crítica em função da sua realidade, ou da realidade da sua organização, e da possibilidade de aplicar o que aqui está descrito.

Ao escolher esse livro, é preciso estar consciente que toda Engenharia de Software aqui descrita é primeiro uma teoria, e depois uma simplificação. Basicamente, é um modelo que, quando aplicado na vida real, deve responder a imprevistos, que exigem adaptação, uso de técnicas adicionais e, não raramente, a criação de soluções inovadoras.

1.2. Níveis de Dificuldade

Existe uma piada em que o professor, na sala de aula ensina um problema muito simples, algo como $x = 4 * 5 + 3$, na prova pede algo mais complicado, algo como $x = \sqrt{\left(\frac{94+32}{13^5}\right)}$, e no mundo real encontramos algo ainda mais complicado, que seja $\sum_1^n \frac{n^p}{p^n}$.

É verdade que, nos livros, começamos com os exemplos simples, o que os americanos chamam de *toy-problems*, porque precisamos que a aplicação da teoria seja entendida, e o que é muito complexo, não pode ser entendido.

Existem várias formas de discutir a dificuldade de problemas. O **Framework Cynefin** explica as dificuldades que podem surgir na tomada de decisões, em níveisremova to:

1. claro (*clear*), composto pelos *known knowns*, isto é os “conhecidos conhecidos”, onde a situação é estável e a relação causa e efeito conhecida;
2. complicado, composto pelos *known unknowns*;
3. complexo, os *unknown unknowns*;
4. caótico, e
5. confuso, não há clareza em que domínio se encontra.

1

Outra classificação fala de problemas “domados” (*tame*), para os quais existem soluções conhecidas, problemas críticos, que causam crises e precisam de respostas imediatas, e problemas malditos (*wicked*), que não tem solução e muitas vezes não podem ser resolvidos, mas apenas ter o seu estado alterado. ^{2 3 4 5}

1.3. A Engenharia de Software Está Sempre Certa?

O que chamamos de Engenharia de Software é formado por um corpo de conhecimento que foi criado de muitas formas. Algumas coisas foram inventadas e postas em prática, outras foram percebidas ou encontradas em projetos de sucesso, outras partiram de raciocínios e considerações sobre a prática, etc. Elas vieram de pesquisadores e profissionais do mercado, que algumas vezes possuem outros interesses além de contribuir para esse corpo de conhecimento, como vender produtos, escrever artigos, ser promovidos, etc. Algumas vezes essas pessoas têm opiniões discordantes.

¹https://en.wikipedia.org/wiki/Cynefin_framework

²<https://www.leadershipcentre.org.uk/artofchangemaking/theory/critical-tame-and-wicked-problems/>

³http://www.drbrd.com/problems_and_solutions/three_types_of_problems.html

⁴https://en.wikipedia.org/wiki/Wicked_problem

⁵<https://pubsonline.informs.org/doi/epdf/10.1287/mnsc.14.4.B141>

1. Prefácio: A Engenharia de Software e o Mundo Real

Por exemplo, é provável que a polêmica entre “documentadores” e “programadores”, ou seja, aqueles que defendem projetos de software fortemente documentados, ou passando por processos extremamente rígidos, e aqueles que defendem processos ágeis fortemente focados na entrega de programas executando o que o usuário deseja, nunca termine, até porque parte dessa discussão está ligada aos contextos onde o software é criado. Um jogo pequeno feito por uma equipe *indie* tem requisitos de qualidade diferentes que um software que controla um míssil atômico ou um aparelho médico.

Hoje, a verdade é que a Engenharia de Software, como um todo, não é uma Ciência Exata, do tipo em que se pode provar com certeza que algo é verdade, mesmo que com uma margem de erro. Apesar de ser uma Engenharia, porque cria artefatos, e parte dela ser exata, grande parte dela é ligada ao trabalho das pessoas, e seus erros.

Vamos analisar um caso específico, de uma ideia que faz bastante sucesso na Engenharia de Software, e acabou se tornando quase que universalmente aceitos como verdades absolutas, os Padrões de Projeto de software propostos pela gang dos quatro.

Esses padrões de projeto (*design patterns*), como *Singleton*, *Facade* ou *Observer*. são formas de organizar programas de computador que precisam de certas funcionalidades. Por exemplo, o padrão *Observer* mostra uma solução para que vários objetos, observadores, possam estar cientes do valor atual de um outro objeto, observado, basicamente por meio do que chamamos de um *call back*.

Esses padrões de projeto são realmente as melhores soluções? Como podemos saber isso?

A maioria dos leitores **aceita** o discurso dos autores, que nos garantem que certas práticas são melhores que outras, dando algumas explicações, porque ela está “vestida” com a “autoridade” de um livro, e referenciada como boa opinião por vários outros profissionais.

Poucos leitores, portanto, avaliam cada padrão de projeto isoladamente, pesam vantagens e desvantagens, e decidem se querem usá-los ou não. De certa forma, aceitam o que está escrito em um livro técnico como um religioso aceita o que está escrito em um livro sagrado. Esse fenômeno é bem conhecido pelos estudiosos da Ciência e Tecnologia.

Novamente, sugiro que os leitores deste livro o encarem de forma crítica, principalmente com o passar do tempo. Alguma informação que está aqui tem seu valor intrínseco, outras um valor histórico, e ainda outras tem seu valor momentâneo e passageiro, devido ao avanço das tecnologias.

1.4. Como cobrar? A pergunta mais comum nos cursos de ES.

Uma expectativa muito comum do iniciante na Engenharia de Software é que ela vai poder ser usada facilmente para responder a pergunta: quanto devo cobrar pelo software que produzo? Esse valor, porém, não é discutido na área. A questão importante para a Engenharia de Software é o custo, não o preço. O preço depende de outros fatores, como a oferta e a demanda do mercado. Preços de produtos e serviços são estudados por outras áreas do conhecimento, como a Economia e o Marketing. Mesmo assim, e fortemente baseado no conhecimento dessas áreas, é possível apresentar algumas informações sobre preço.

Muitas vezes o software é cobrado em função do seu custo, ao qual é adicionado uma margem de lucro. Então, quem escolher essa forma de trabalho vai encontrar, nesse livro, algumas

1. Prefácio: A Engenharia de Software e o Mundo Real

informações úteis. Também é comum que seja cobrado em “horas”, isto é, tempo trabalhado, e dentro desse valor cobrado uma parte é o lucro da organização.

O custo de software poder ser previsto ou calculado de várias maneiras, como por meio de horas trabalhadas, linhas de código, relatórios e telas, as estatísticas que permitem alguma comparação estão relacionadas a uma medida conhecida como Pontos de Função, que será tratada no Capítulo 19. No caso da previsão, sempre haverá uma incerteza, tanto por motivos da essência do software, quanto pela capacidade das técnicas. Além disso, todo projeto tem riscos. A Engenharia de Software tem entre suas metas aumentar a precisão das previsões e diminuir o risco de projeto.

Nesta forma de cobrança, é preciso prever o custo de alguma forma, e isto é feito em função do tamanho do esforço necessário para produzir o software, que pode ser equacionado a partir da funcionalidade entregue ao cliente. A medida mais aceita em todo mundo, principalmente pelos governos e órgãos regulatórios e de padronização é o Ponto de Função (IFPUG, 2010). No momento é razoável saber, como exemplo, que um relatório simples pode ser avaliado em 4 pontos de função.

No mercado mundial, segundo Czarnacka-Chrobot (2012), com dados do ISBSG, um ponto de função custa entre US\$ 300 e US\$ 1000, com uma média de US\$750, mas foram encontrados projetos com valores desde US\$ 17 até US\$ 2727 na base consultada, incluindo os *outliers*. São dados que incluem custos não só de desenvolvimento, mas também de apoio.

Com esses mesmos dados, o preço por hora varia entre US\$60 até US\$105, com uma média de US\$ 80, havendo extremos entre US\$7 e US\$ 570.

Um ponto de função pode custar entre US\$ 1600 em média no Japão, até US\$ 125 em média na Índia. Não é de se espantar que tanto software seja terceirizado para a Índia. Em relação ao tamanho dos projetos, é comum que quanto maior o software, maior o preço do ponto de função.

No Brasil, devido a regulamentação do governo, software é muitas vezes precificado de duas formas: pelo uso de Pontos de Função (PF) e por Unidades de Serviço Técnico (USTs). USTs são acordadas por projeto, havendo uma validação com a prática do próprio governo em outros projetos. Os preços oscilam entre R\$ 250,00 até R\$ 1.500,00 (FATTO, 2020). É importante frisar que existe uma regra de contagem de Pontos de Função destinada aos contratos do governo brasileiro que vai além do modelo original ou do padrão IFPUG (Brasil, 2018).

Isso significa que, para o governo brasileiro, uma tela com um relatório simples, como a quantidade de pedidos de aposentadoria por estado, por exemplo, pode custar algo entre R\$ 1.000,00 e R\$ 6.000,00.

A variação de preços é explicada por motivos como vários fatores de tecnologia, incluindo linguagem de programação, requisitos de operação continuada e de segurança, e ainda variações do mercado, como a oferta e demanda de profissionais, e outras.

Parte I.

Entendendo o Software

Introdução a Engenharia de Software

Most software today is very much like an Egyptian pyramid with millions of bricks piled on top of each other, with no structural integrity, but just done by brute force and thousands of slaves.

(Alan Kay)

Conteúdo

2.1.	O que é Software	8
2.2.	Por que software é importante	9
2.3.	Quais os riscos que software traz?	11
2.4.	Por que é difícil fazer software?	13
2.5.	O que é Engenharia de Software	17
2.6.	Por que a Engenharia de Software é necessária?	18
2.7.	Qual o Segredo do Sucesso de um Projeto de Software	19
2.8.	O Corpo de Conhecimento da ES	20
2.9.	As normas da Engenharia de Software	21
2.10.	SEMAT	21
2.11.	Quem é o Engenheiro de Software	23
2.12.	Exercícios	24

Este capítulo faz uma pequena introdução a área de Engenharia de Software.

2.1. O que é Software

Software é uma palavra da língua inglesa usada em contraposição a palavra **hardware**, que descreve máquinas e aparatos físicos de forma geral. Assim, um computador seria dividido em uma parte física, composta de circuitos eletrônicos, e uma parte não-física, composta do programa que o faz funcionar.

Quem inventou a palavra software?

Há dúvidas sobre quem passou a usar a palavra no contexto da computação, sendo que Alan Turing é um dos citados, e Paul Niquete reivindica ter usado o termo ao falar desde 1953. Normalmente o termo é associado a sua primeira publicação, por Tukey (1958). Um registro anterior, de 1956, em um artigo de Richard R. Carhart (Niquete, 2006) usa a palavra software com a aceção de pessoas envolvidas com o computador. Em francês existe a palavra *logiciel*, que os portugueses traduziram para logiciário. Este termo foi tentado no Brasil mas não chegou a ter força

Atualmente o termo software é associado não apenas ao programa de computador, mas também as suas estruturas de dados e sua documentação. Pressman e Maxim (2014) definem **software** como:

1. **instruções** (programas de computador) que quando executadas fornecem as funções, características e desempenho desejados;
2. **estruturas de dados** que habilitam o programa a manipular informação de forma adequada, e
3. **informação** que descreve a operação e uso do programa.

Já a IEEE define software como “programas de computador, procedimentos, e, possivelmente, documentação e dados pertinentes a operação de um sistema computacional” (IEEE, 2017b).

A Figura 2.1 mostra uma página do Sistema Acadêmico do CEDERJ e parte do código para sua criação. Apesar de HTML não ser uma linguagem de programação, é parte do que chamamos de software.

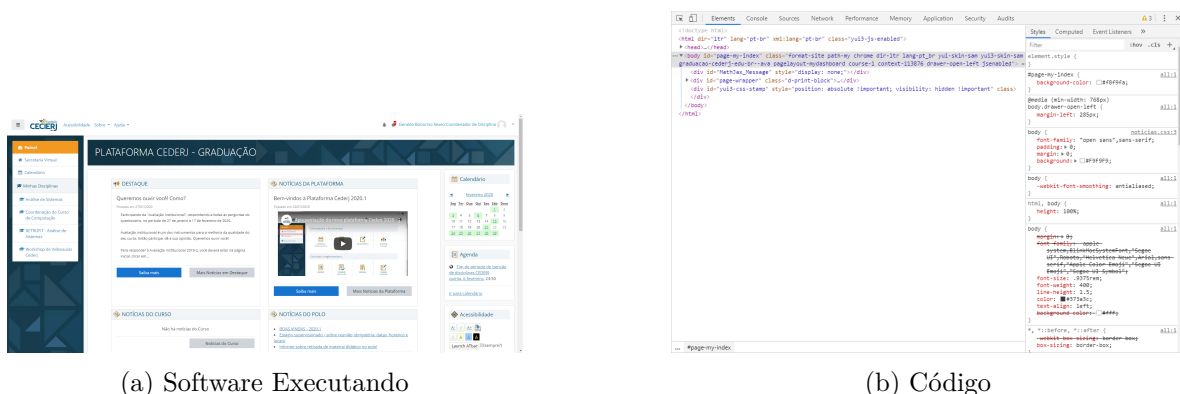


Figura 2.1.: Um software executando na web e parte do seu código.

Um computador atual não funciona sem software¹. Na verdade, o computador que usamos para nossas tarefas diárias possui camadas e camadas de software para funcionar. É o software que instrui a máquina construída com circuito eletrônicos sobre os passos que ela deve seguir.

¹No passado computadores podiam ser programadas apenas com fiações, dispensando o software.

Mesmo em uma CPU moderna, que normalmente vemos como *hardware*, já existe software embutido e até sistemas operacionais instalados (Vaughan-Nichols, 2017).

Software é considerado um bem imaterial. É bastante difícil, por exemplo, calcular o seu **valor**. Quanto foi gasto para produzi-lo, quanto ele traz de lucro, quanto é gasto para mantê-lo, quanto seria gasto para substituí-lo seriam formar alternativas de fazer isso. Produtos como o sistema de reserva de passagens SABRE, originalmente criado para a American Airlines (Head, 2002), se tornaram tão importantes que se transformaram em negócios isolados (Levere, 2001), adquirindo valor próprio como negócio.

Outra característica do software é que seu custo de desenvolvimento é basicamente composto pelo esforço das pessoas envolvidas em produzi-lo e mantê-lo. Software não exige ou consome matéria prima, e o uso de máquinas para produzi-lo é basicamente proporcional ao número de computadores necessários para as pessoas realizando tarefas, mais uns poucos computadores necessários para manter atividades automáticas². Isto não significa, porém, que seja barato desenvolver software, ao contrário, os custos de produção e manutenção de software são bastante altos e podem explodir se o projeto não for bem gerenciado. Gerenciar esses custos é uma das preocupações principais da Engenharia de Software.

Software pode ser encontrado de várias formas: texto em arquivos digitais ou impressos, desenhos, arquivos binários ou em outros formatos, bits na memória do computador, documentações em diferentes mídias, como texto, e vídeos. Este documento, por exemplo, é possui uma versão digital no formato PDF que contém, dentro dele, programas em PostScript que instruem como deve ser impresso ou visualizado. Este é um exemplo da ubiquidade do software, isto é, ele está presente ao nosso redor e não percebemos mais essa presença.

2.2. Por que software é importante

Basicamente, software, na forma de dados e programas de computador, é o que hoje faz funcionar, monitora e controla todo o mundo. Se no início os computadores eram máquinas extremamente pesadas e caras que viviam em porões resfriados, trabalhando em modo *batch*, hoje praticamente todo equipamento que usamos possui alguma automação, seja das grandes usinas geradoras de energia a cafeteira elétrica que usa essa energia.

Como tudo começou

O primeiro programa de computador, criado para um computador mecânico que nunca ficou pronto, foi feito por Ada Augusta King, nascida Byron, condessa de Lovelace. Ela criou e publicou um programa para a Máquina Analítica de Babbage, mostrando suas potencialidades.

Toda infraestrutura depende da geração de energia, que é planejada e controlada por meio de software, em vários níveis da produção. No Brasil, por exemplo, a ONS, o Operador Nacional do Sistema Elétrico, tem sistemas sofisticados que sabem o status de cada peça usada na transmissão de energia. Toda informação é trocada de forma digital, dos telefones às grandes redes de televisão, passando pela internet propriamente dita. Automóveis chegam a ter dezenas de processadores. Aparelhos domésticos podem se conectar à internet.

²Mais recentemente, principalmente com o introdução dos *Large Language Models*, LLMs, o custo de treinamento, incluindo tempo de máquina, custo do material usado para o aprendizado e gestão da qualidade dos resultados, passou também a ser relevante para a geração de modelos

2. Introdução a Engenharia de Software

Por exemplo, a foto da Figura 2.2 mostra a sala de controle de uma usina de energia nuclear de tecnologia original americana, construída antes das massivas transformações das estruturas de comando e controle dessa indústria para software. É possível ver nessa foto dezenas de controles de hardware, porém também é possível ver que parte dos controles já foi modernizada para sistemas de software.



Figura 2.2.: Sala de controle de uma usina nuclear. Fonte: foto de Rodrigo Costa dos Santos

Assim, dentro dessa realidade, toda a economia é extremamente dependente do funcionamento de programas de computador de diferentes graus de sofisticação. Um freio ABS, por exemplo, possui software em várias camadas, uma servindo como defesa dos erros que as outras podem cometer.

Outra caracterização da importância pode ser dado pelo tamanho da indústria de software mundial, avaliada em 2024 em 698,8 bilhões de dólares³, enquanto toda a indústria de TI foi estimado em 11,7 trilhões de dólares em 2024⁴. Além disso, em 2 de agosto de 2025, quatro das seis maiores empresas em valor de mercado eram empresas de tecnologia da informação cujo principal diferencial é seu software: Microsoft, Apple, Alphabet e Meta⁵, sendo que a Amazon ainda está nesse grupo.

Essas são enormes empresas de software de uso geral, existe um mercado ainda para software dedicado, como os que controlam um aparelho médico ou fazem uma usina de energia funcionar.

O custo de software geralmente excede o de hardware, até mesmo em sistemas complexos. Além disso, várias funções que antes eram feitas por hardware, hoje são feitas por software, o que apresenta vantagens, mas também aumenta a complexidade do que pode ser feito e consequentemente aumenta a complexidade do software, e também gera novos riscos para os sistemas.

Outra tendência é a virtualização do hardware. Isso significa que funções típicas de hardware são repassadas para camadas de software, sendo que isso acaba afetando também a forma de

³<https://valor.globo.com/patrocinado/dino/noticia/2024/02/09/mercado-de-sofwarees-estudo-preve-us-68-gh.html>

⁴<https://www.businessresearchinsights.com/pt/market-reports/information-technology-it-market-108885>

⁵<https://companiesmarketcap.com/>

se pensar o hardware. A Figura 2.3 mostra uma arquitetura que pode ser encontrada em um servidor hoje em dia⁶.

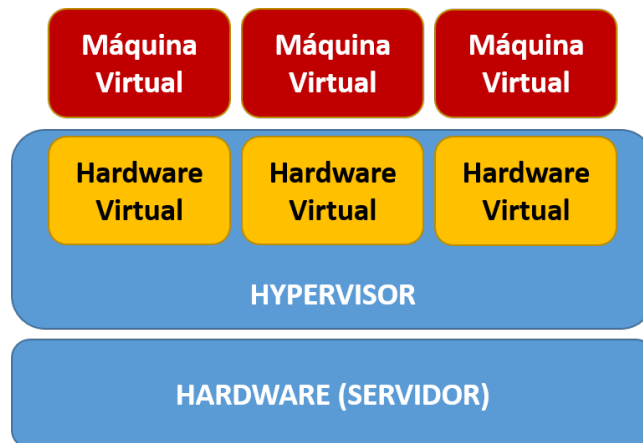


Figura 2.3.: Esquema da virtualização do hardware.

2.2.1. A Ubiquidade do Software

Imagine uma pessoa qualquer em uma cidade moderna qualquer. Vamos chamá-la de João. João acorda porque toca um alarme em um relógio digital. Esse relógio está ligado na rede elétrica, que é controlada por software. O relógio contém software dentro dele. João levanta e liga a TV digital, que contém software, para ver as notícias. Todos os equipamentos da estação de notícias são digitais e contém software. Ele olha seu celular, com sistemas operacionais e aplicativos altamente sofisticados, para verificar mensagens que vieram em uma rede de dados que contém camadas e camadas de software. Em 10 minutos de vida, João utilizou provavelmente **centenas de milhões de linhas de código** sem ao menos perceber sua dependência da indústria de Tecnologia da Informação.

A verdade é que o software, e toda a Tecnologia da Informação, se tornou algo **ubíquo**. Essa palavra indica um tipo de ideia ou objeto que é pervasiva, está em todo lugar, ao ponto que não percebemos mais sua presença.

2.3. Quais os riscos que software traz?

Sendo tão importante, é claro que software também traz riscos às organizações e às pessoas. As duas maiores facetas desse risco são as perdas de vida humana, animais ou bens, por erro do software em execução, e as grandes perdas financeiras causadas por software em execução ou durante a sua criação, ou seja, em um projeto de desenvolvimento de software.

Acidentes de aviação onde houve influência do software tiveram grande repercussão na mídia, como o caso dos aviões Boeing 737 Max (Travis, s.d.). Um dos casos mais discutidos tecnicamente, ligado a perda de vidas humanas, é o do equipamento de terapia por radiação Therac-25 (Levenson e Turner, 1993), onde uma falha em um sistema de proteção feito em

⁶É interessante notar que também existe um movimento contrário, de trazer funções desenvolvidas em software para o hardware, permitindo por sua vez que o software possa ser mais sofisticado, em um ciclo de *feedback* positivo, o que aconteceu por exemplo na sofisticação das placas gráficas (Savage, 2020)

2. Introdução a Engenharia de Software

software causou que doses altíssimas fossem aplicadas a várias pessoas, causando morte ou lesões em pelo menos 6 pessoas.

Alguns casos são interessantes de citar, sendo tragédias técnicas e financeiras, mas que não causaram perdas de vidas. O primeiro lançamento do foguete europeu Ariane V, em 1996, resultou em uma autodestruição automática devido a uma exceção não tratada no software (Lions, 1996), não só atrasando o projeto em anos, mas também criando um problema para a imagem do projeto e impactando o prêmio do seguro de lançamento de satélites para toda a indústria.

No Brasil, a Volkswagen, em 2006, anunciou o *recall* de 123 mil veículos para trocar um software que controlava o funcionamento dos componentes eletrônicos dos automóveis (Folha Online, 2006). No caso, alguns dos automóveis tinham o chip dentro do motor, o que obrigava a abri-lo, operação complicada e que ninguém gosta que seja feita em seu carro.

Já em prejuízos financeiros, por projetos atrasados ou abandonados, os casos se repetem ano após ano. Pequenos problemas de software, como aplicativos de celular parando de funcionar, ou erros de cálculo em fórmulas que usamos em planilhas eletrônicas, podem ter um grande valor agregado na economia, se somadas todas as horas de trabalho perdidas ou ainda os prejuízos reais causados. A Tricentis (2019) estimou que as perdas devidas a falhas de software somaram US\$1,72 trilhões em 2017, afetando 3,68 bilhões de pessoas e com uma perda acumulada de 268 anos. Já Krasner (2021) identificou um custo total de US\$ 2.08 causado por baixa qualidade de software, com mais US\$ 1.31 trilhões a serem gastos no futuro para correções.

Projetos inacabados com altos valores desperdiçados são também bastante comuns. Um dos maiores fracassos relatados aconteceu nas décadas de 80 e 90, quando a Administração Federal de Aviação americana (FAA) tentou criar um novo sistema de controle do tráfego aéreo. Reconhecido como ambicioso demais, um dos seus requisitos era de 99.99999% de confiabilidade⁷, o sistema foi planejado para custar, na época, US\$ 500 por linha de código, o que já era cinco vezes mais do que o padrão da indústria para processos bem gerenciados. Em 1995 o custo já tinha chegado de US\$ 700 a US\$ 900 por linha de código. Inicialmente orçado em US\$ 2.5 bilhões, em 1994 as estimativas subiram para US\$ 7 bilhões e o projeto foi abandonado. Como resultado positivo, apenas algumas de suas partes foram adotadas como partes de projetos menores (Gibbs, 1994).

Entre outros grandes fracassos podemos adicionar o *FBI* tendo que jogar fora US\$ 170 milhões em um sistema de apoio a identificação de terroristas, a *Ford Motor Company* abandonando um projeto de US\$ 200 milhões de um novo sistema integrado de compras e o *McDonald's* declarando US\$ 170 milhões em prejuízo por um software para automatizar toda sua operação (Carr, 2005).

Não apenas erros de programação ou problemas em projetos, mas falhas de design na interface e falta de dados em software funcionando também já influenciaram acidentes graves. Os acidentes com aviões B757, do voo AA965 em Cali, de 20/12/1995, com 159 fatalidades, e do voo ALW301 em *Puerto Plata*, de 6/2/1996, com 189 fatalidades, foram, em parte, associados a problema na interface homem-computador (Ladkin, 1996).

Problemas de software podem também fazer com que equipamentos não cumpram sua missão de proteção. O acidente do voo 801 da *Korean Air*, em 3/12/1984, em Guam, que causou 228 mortes, por exemplo, poderia ter sido prevenido se não houvesse um erro no sistema do Radar de Altitude Segura Mínima (Bellovin, 1997).

Existem muitos outros riscos que são relatados diariamente. Equipamentos podem ser controlados por pessoas estranhas a sua operação, câmeras podem ser monitoradas por pessoas sem

⁷o equivalente a dizer que o sistema só poderia ficar sem funcionar perfeitamente 3,15s em um ano

esse direito, dados pessoais podem ser obtidos de forma ilegal ou legal mas sem o conhecimento dessas pessoas, com grandes ameaças a privacidade, defeitos ou *bugs* podem fazer equipamentos pararem de funcionar a qualquer momento, etc. Uma boa fonte de notícias sobre o assunto é o *The Risks Digest: Forum on Risks to the Public in Computers and Related Systems*, moderado por Peter G. Neumann⁸.

2.3.1. Acontecimentos Recentes (2024)

Como exemplo de que os riscos acontecem e são constantes, Prajapati (2024) nos fornece uma lista de problemas em 2024, incluindo atrasos em projetos e problemas já na operação. Entre eles estão:

- Em julho um update rotineiro de um sistema de cibersegurança da *CrowdStrike* afetou 8.5 milhões de computadores, levando até 72 horas para ser corrigido e com perdas estimadas de 3 bilhões de dólares.
- A divisão de software da Volkswagen causou um prejuízo de 5 bilhões de dólares e queda na satisfação de funcionários em 25%, com atrasos de até 2 anos nos projetos.
- Um projeto de implementação da Oracle na cidade de Birmingham levou a um prejuízo de 38 milhão de dólares.

2.3.2. Riscos Éticos

Mais recentemente, também passamos a nos preocupar com os riscos ligados a decisões eticamente difíceis ou contestáveis. Com o uso de prática de Inteligência Artificial no controle de decisões que afetam seres humanos, desde a autorização de entrada em um prédio, até cirurgias cardíacas por robôs autônomos, passando por carros que não precisam de motoristas, software é responsável por tomar decisões que são difíceis, ou comumente demonstram viés, mesmo quando feitas por seres humanos.

Um automóvel autônomo, por exemplo, pode ter que decidir entre atropelar um pedestre ou bater em um poste, colocando em risco a vida de seus passageiros. Como tomar essa decisão? Mesmo os filósofos discutem isso no famoso “dilema do bonde”

2.4. Por que é difícil fazer software?

A Engenharia de software nasceu porque fazer software é difícil. O nome da área foi estabelecido em uma conferência da OTAN em 1968 cujo objeto era descobrir como resolver um problema de engenharia: o projeto, a produção e a manutenção com sucesso de sistemas de software úteis (Naur e Randell, 1969).

Em 1972, Dijkstra descreveu bem o problema de desenvolver software, na sua palestra de aceitação do prêmio Turing. Se colocando no lugar de um programador do futuro que voltou no tempo, ou seja, um programador que vivia aproximadamente agora, ele conta: “quando não havia máquinas, programar não era um problema de forma alguma; quando nós tínhamos poucos computadores fracos, programar se tornou um problema, e agora nós temos computadores gigantescos, programar se tornou um problema gigantesco” (Dijkstra, 1972). Ou seja, como temos computadores superpoderosos podemos tentar resolver problemas difíceis.

⁸<http://catless.ncl.ac.uk/Risks/>

2. Introdução a Engenharia de Software

Além de difíceis, parte dos problemas que tratamos são conhecidos como **wicked problems**, cuja tradução poderia ser problemas perversos. Esses problemas se caracterizam por (Conklin, 2005):

- Não serem entendidos até que uma solução seja desenvolvida
- Não haver uma regra de parada para a solução
- Suas soluções não serem certas ou erradas, mas sim melhores, aceitáveis ou não boas o suficiente
- Serem essencialmente únicos e novos
- Cada solução possível ser uma operação única e com consequências
- Não possuírem soluções alternativa

Somam-se a isso outros desafios.

Um deles é a necessidade de atender as demandas das pessoas em um contexto social, compondo diferentes partes interessadas. Isso implica em problemas de comunicação, política organizacional, e muitas vezes interesses diversos, não alinhados com um projeto único, ou mesmo com visões conflitantes sobre objetivos, prioridades e critérios de sucesso. Esse cenário exige do engenheiro de software não apenas competência técnica, mas também habilidades interpessoais, sensibilidade ética e capacidade de negociação, para equilibrar expectativas e construir consenso em torno de soluções viáveis e sustentáveis. Essa complexidade social (Conklin, 2005) geralmente ocorre junto com um problema perverso (*wicked problem*) e é função do número e da diversidade de partes interessadas envolvidas em um projeto.

A atenção específica às partes interessadas, em qualquer organização ou projeto, é uma tendência razoavelmente moderna. Apenas em 1984 foi publicado o livro seminal dessa área, “*Strategic Management: A Stakeholder Approach*”, por Freeman (1984), e seus efeitos ainda não são plenamente sentidos em todas as organizações. Mais recentemente, com novas visões sobre a missão de uma organização moderna e as exigências de ESG, houve um novo impulso para essa forma de gestão. Enquanto o paradigma dominante por décadas foi que uma organização deveria apenas maximizar o valor para o acionista, o *Business Roundtable*, uma associação de CEO’s de empresas representando aproximadamente um quinto do valor do mercado de ações americano, considera que “ao tomar decisões o conselho deve considerar o interesse de todos os constituintes da empresa, incluindo as partes interessadas tais como empregados, clientes, fornecedores e a comunidade onde a empresa faz negócios, quando fazer isso contribui de forma direta e significativa na construção da criação de valor a longo prazo”.

Partes Interessadas

Esse tópico será tratado detalhadamente no Capítulo 3. Uma definição simples é “toda pessoa, organização ou entidade que pode ser, ou acredita que pode ser, beneficiada ou prejudicada por um projeto”.

Outro problema é o técnico. Projetos de software envolvem pessoas trabalhando em um grande número de partes. Um software de funcionalidade relativamente simples pode ter milhares de linhas de código. Projetos podem durar anos. Tudo isso implica na necessidade de técnicas de gestão de projeto, garantia de qualidade, e outros assuntos que compõem a Engenharia de Software.

Em especial, não só a complexidade de um projeto de software aumenta de forma mais que linear com a quantidade de funcionalidade entregue, quanto os erros de estimativa aumentam mais ainda (Gibbs, 1994). Isso é bem diferente de grande parte da indústria, onde produzir mais pode inclusive baixar os preços.

2. Introdução a Engenharia de Software

Para entender o tamanho de um software atual, é possível lembrar que foram necessárias 145.000 linhas de código para o software de orientação da Apollo XI, que pousou na Lua em 1969, já o sistema Android versão 2.2, lançado em 2010, continha 12 milhões de linhas de código (P. Johnson, 2012). A Figura 2.4 mostra o tamanho comparativo de alguns software conhecidos, que chega a 86 milhões de linhas de código para o Mac OS X 10.4. Isso mostra que mesmo um programa pequeno que funcione em sistema operacional como Android, Windows ou MacOS depende de milhões de linhas de código funcionarem corretamente. Para comparação, uma das máquinas mais complexas existente no mundo, o Boeing 747, tem aproximadamente 6 milhões de partes distintas, e um automóvel apenas 30 mil.

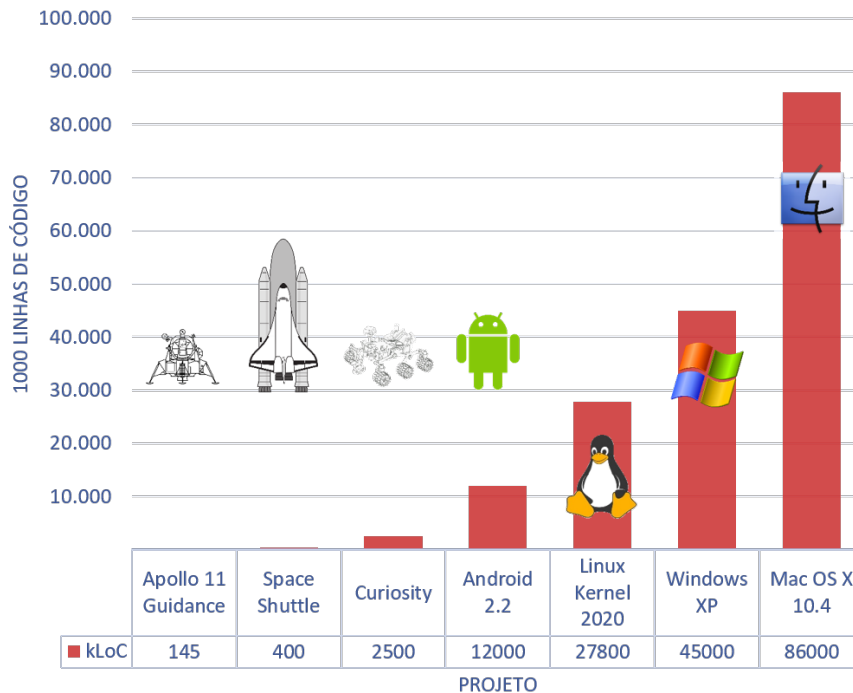


Figura 2.4.: Tamanho de código de alguns projetos conhecidos.

Outra questão importante do software é que dificilmente ele se mantém constante ao longo do tempo quando em operação. Dois são os motivos dessa mudança constante: a descoberta de *bugs* que precisam ser consertados e a alteração dos requisitos originais com a evolução do ambiente em que funcionam. Por exemplo, em um software de gerência de recursos humanos, qualquer alteração nas leis trabalhistas pode exigir uma modificação no software para que fique de acordo com as novas leis. O problema é que qualquer modificação no software tem uma chance de inserir novos *bugs*.

Isso leva a discussão da curva de falha do software. A maioria dos autores de Engenharia de Software concorda que essa curva, indicada na Figura 2.6, é bem diferente da curva de falha do hardware, Figura 2.5, a qual já é bem modelada estatisticamente. Por causa dessa diferença, é difícil entender como um produto de software vai reagir no futuro, já que cada mudança feita pode causar novas falhas.

2.4.1. O Relatório CHAOS

Pesquisas de campo também mostram que muitos projetos de software falham ou acabam com problemas. Desde de 1994 o *Standship Group* realiza uma pesquisa chamada “CHAOS

2. Introdução a Engenharia de Software

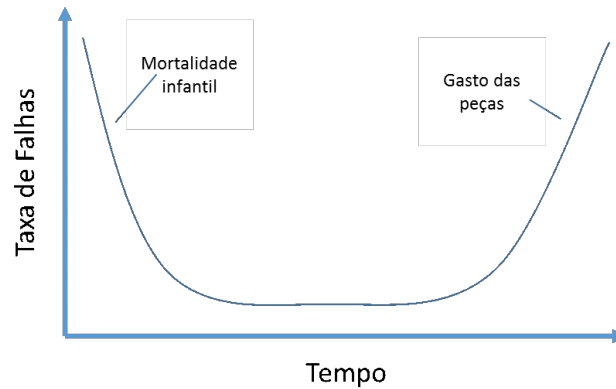


Figura 2.5.: Curva de falha do hardware. Fonte: (Pressman e Maxim, 2014)

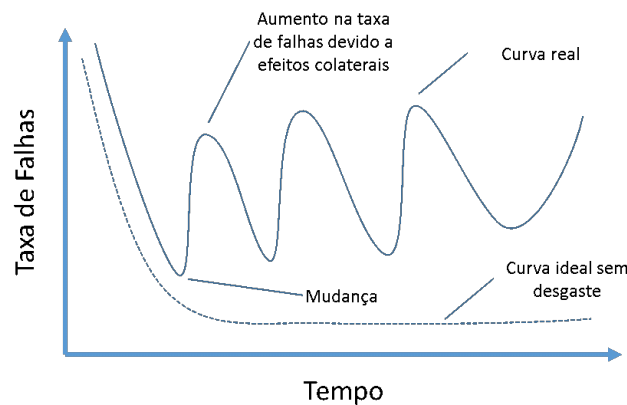


Figura 2.6.: Curva de falha do software. Fonte: (Pressman e Maxim, 2014)

Report” que mostra o estado dos projetos de software. As medidas básicas para determinar a resolução de um projeto são (The Standish Group, 2015):

- dentro do **orçamento**;
- dentro do **prazo**, e
- satisfatório em relação a **funcionalidade**.

Essa pesquisa divide os projetos, por resolução, em: **fracassos**, que foram cancelados ou não foram usados, **com problemas**, em uma ou duas das três medidas básicas, e **bem sucedidos**⁹.

Na versão de 2015, com uma base de 2011 a 2015, os percentuais mostraram números preocupantes, que podem ser vistos na Tabela 2.1, principalmente para projetos grandes.

Como podemos ver, ainda na Tabela 2.1, a taxa de fracassos aumenta muito com o tamanho, principalmente para projetos feitos com a metodologia mais tradicional, em Cascata. Além disso, a taxa de sucesso para projetos médios e grandes está abaixo de 30% mesmo para projetos feitos com a metodologia ágil, e menor que 10% para a metodologia tradicional. Isso é uma constatação da dificuldade de fazer projetos de software.

Uma atualização do CHAOS report de 2020, citada por Radek (2025) indica que ainda nessa época a taxa de sucesso era de apenas 31%, a taxa de projetos que acabaram excedendo *budget* ou prazo era de 50% e a taxa de cancelamentos de 19%.

⁹Os termos originais são *failed*, *challenged* e *successful*.

Tabela 2.1.: Resolução de projetos de software por tamanho e por método de desenvolvimento, para mais de 10.000 projetos. Fonte:(The Standish Group, 2015)

Tamanho	Método	Bem Sucedido	Com Problemas	Fracassos
Grande	Ágil	18%	59%	23%
	Cascata	3%	55%	42%
Médio	Ágil	27%	62%	11%
	Cascata	7%	68%	25%
Pequeno	Ágil	58%	38%	4%
	Cascata	44%	45%	11%

2020 foi o último ano do CHAOS Report porque a Standish Group decidiu encerrar a série com a edição intitulada “CHAOS 2020: Beyond Infinity”, propondo que o modelo tradicional de avaliação binária de sucesso e fracasso de projetos de software não era mais adequado às práticas modernas de desenvolvimento, especialmente com a ascensão de abordagens ágeis, ciclos iterativos contínuos e foco em fluxo de valor em vez de escopo fixo. A organização sugeriu que o futuro do desenvolvimento de software estaria mais alinhado com o conceito de *Infinite Flow* — um modelo no qual o trabalho é tratado como fluxo constante de entrega de valor, dificultando a aplicação dos critérios tradicionais de sucesso pontual utilizados pelos relatórios anteriores.

2.5. O que é Engenharia de Software

Engenharia de Software é “a aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software, i.e., a aplicação da engenharia ao software” (IEEE Computer Society, 2014a).

O termo foi criado em torno de 1968, perante a necessidade de melhorar o desenvolvimento de software, que desde o início se mostrou algo bastante difícil. Pesquisas mostram não só que muitos projetos de software falham, parcial ou totalmente, quanto o aumento drástico da probabilidade de fracasso de um projeto software com o aumento da complexidade do mesmo (The Standish Group, 1994).

Segundo o SWEBOK, a “Engenharia de Software é a profissão preocupada com a criação e manutenção de aplicações de software pela aplicação de tecnologias e práticas da Ciência da Computação, Gerência de Projetos, Engenharia, domínios de aplicação e outros campos”(IEEE Computer Society, 2014a).

Uma abordagem mais pragmática é pensar a Engenharia de Software como o estudo da formas e as próprias formas de desenvolver software de maneira custo-efetiva. Fornecer os maiores benefícios possíveis dentro de um conjunto de restrições é um dos principais objetivos da Engenharia que se aplica também a Engenharia de Software.

As metas da Engenharia de Software incluem atender o cliente que precisa do software, gerar valor para as organizações que produzem e utilizam o software e deixar satisfeitos, também, aqueles que produzem o software.

2.5.1. Uma Visão da Engenharia de Software

Pressman e Maxim (2014) construíram uma pirâmide de 5 níveis para explicar a Engenharia de Software, em uma abordagem técnica. Porém, é necessário mostrar também o que motiva essas técnicas, o que é parte significativa do problema da Engenharia de Software.

Desse modo, a Engenharia de Software pode ser vista por uma pirâmide de 7 níveis, Figura 2.7, onde a base é atender as **partes interessadas**, pela agregação de **valor**. Isso só pode ser feito por meio de produtos e serviços de **qualidade**, o que exige **processos**, onde são usados **métodos**, de acordo com **práticas**, que exigem **ferramentas**.



Figura 2.7.: Pirâmide que explica o escopo da Engenharia de Software.

2.6. Por que a Engenharia de Software é necessária?

Precisamos da Engenharia de Software porque não é possível fazer software “de qualquer jeito”. O Ciclo de Vida de um software, de qualquer tamanho, é um projeto que se inicia na descoberta do que se deve fazer e termina quando o software, depois de entregue e usado, é retirado do funcionamento. A fase de suporte e manutenção normalmente dura muito mais que a de desenvolvimento. Portanto, durante toda a vida do software é preciso usar técnicas típicas da engenharia, como a construção de modelos abstratos do software que deve ser criado, estimativas de prazo e custo, garantia da qualidade do produto, etc.

2.6.1. Diferenças para outras engenharias

Mas o que há de diferente entre Engenharia de Software e outras engenharias? A princípio, fazer software poderia ser considerado até mais fácil, já que na maioria das engenharias, após construirmos um artefato, temos que reproduzi-lo por um processo industrial. Software pode ser simplesmente copiado, com uma taxa baixíssima de falha na cópia e, mesmo em mídias que degradam, de fácil verificação.

Na prática, a Engenharia de Software se parece mais com a parte das engenharias que costumamos chamar de desenvolvimento do projeto, porém seguindo por um caminho de construção de um objeto único.

Em um projeto de software cada linha programada tem importância. A quantidade de linhas programadas, ou itens que devem ser analisados separadamente, e ainda a interação entre elas, seja por meio de código ou dados, é enorme. Assim, construir software é, de várias maneiras,

muito mais complexo que construir uma casa. Se um tijolo da casa não tem um pequeno pedaço, não há problema nenhum. Se um grão de pó de um elemento químico em um processo químico tem um peso inferior a média, o problema certamente também não existirá. Mas troque uma letra ¹⁰ em um programa de computador e você pode ter um programa que nem compila, ou funciona, porém, executa de forma absolutamente errada¹¹.

Outro problema, talvez o maior, é a dificuldade de saber o que é realmente necessário, devido aos problemas naturais de comunicação entre as pessoas, principalmente usuários e desenvolvedores. Além disso, muitas vezes não se sabe o que deve ser feito, mas sim o problema que deve ser resolvido, o que implica em processos de investigação ainda mais complicados¹².

2.7. Qual o Segredo do Sucesso de um Projeto de Software

Como vimos, desenvolver software é difícil. A área de Engenharia de Software, então, se preocupa em encontrar formas de melhorar esse desempenho. Segundo o mesmo The Standish Group (2015), se forem utilizadas Metodologias Ágeis, o índice de fracasso cai de 42% para 23% em projetos grandes, e o de sucesso sobe de 3% para 18%.

Esse mesmo relatório relata as razões de sucesso de projetos de software, apresentadas na Tabela 2.2. Observando a Tabela, é possível perceber que os principais fatores não estão ligados diretamente a Processos, Métodos e Ferramentas, porém eles precisam ser alcançados com o uso das técnicas de Engenharia de Software, principalmente o Envolvimento do Usuário e a Otimização, que começa com a gerência de escopo com base no valor de negócio (Hastie e Wojewoda, 2015)

Tabela 2.2.: Fatores de Sucesso de um Projeto de Software. Fonte:(The Standish Group, 2015)

Fator de Sucesso	Peso Relativo
Patrocínio Executivo	15
Maturidade Emocional	15
Envolvimento do Usuário	15
Otimização	15
Recursos Capacitados	10
Arquitetura Padrão	8
Processo Ágil	7
Execução Modesta	6
Experiência em Gerência de Projetos	5
Objetivos Claros de Negócio	4

Ainda analisando a Tabela 2.2 e usando conhecimentos de Engenharia de Software, podemos dizer que o envolvimento do usuário é melhorado com o uso de processos ágeis.

Mais recentemente, o *Standish Group* propôs três fatores de sucesso como determinantes para o êxito de projetos de software: Good Sponsor, Good Team e Good Place:

¹⁰A quantidade mínima e discreta de um programa escrito em uma linguagem de programação de terceira geração ou maior

¹¹Em um exemplo que aconteceu com o autor, dois espaços de uma indentação em Python fizeram que um programa gerasse os dados errados durante dias, até que o erro fosse descoberto.

¹²Isso também pode acontecer em projetos de Engenharia ou outras áreas, como Medicina

- *Good Sponsor* (bom patrocinador): é o coração do projeto — a pessoa que dá visão, autoridade e apoio contínuo. Melhorar as habilidades do patrocinador é o fator mais importante e, ao mesmo tempo, o mais fácil de aperfeiçoar, já que cada projeto tem apenas um patrocinador
- *Good Team* (boa equipe): a força motriz da entrega. Pequenas equipes com alto grau de autonomia, habilidades técnicas e colaborativas bem estruturadas são essenciais para alcançar os objetivos previstos
- *Good Place* (bom ambiente): o ecossistema de trabalho em torno do projeto, incluindo patrocinadores, equipe e demais colaboradores. Envolve suporte emocional, maturidade relacional, envolvimento de usuários e otimização de processos organizacionais

Esses três fatores são acompanhados pela teoria da **latência de decisão** (*decision latency*), o tempo que uma organização leva para tomar uma decisão necessária para o avanço de um projeto, que é identificada como a causa raiz dos insucessos nos projetos: projetos com latência decisória baixa têm até 3,5x mais chance de sucesso e gastam proporcionalmente menos orçamento nas tomadas de decisão.

2.8. O Corpo de Conhecimento da ES

Uma das publicações mais importantes da Engenharia de Software é o *Guide to the Software Engineering Body of Knowledge (SWEBOK(R))*. Esse guia, construído colaborativamente por dezenas de pesquisadores ao redor do mundo e mantido pela IEEE Computer Society, define os conhecimentos relativos a área, dividindo-os em 15 áreas do conhecimento(IEEE Computer Society, 2014a), como descritas na Figura 2.8.

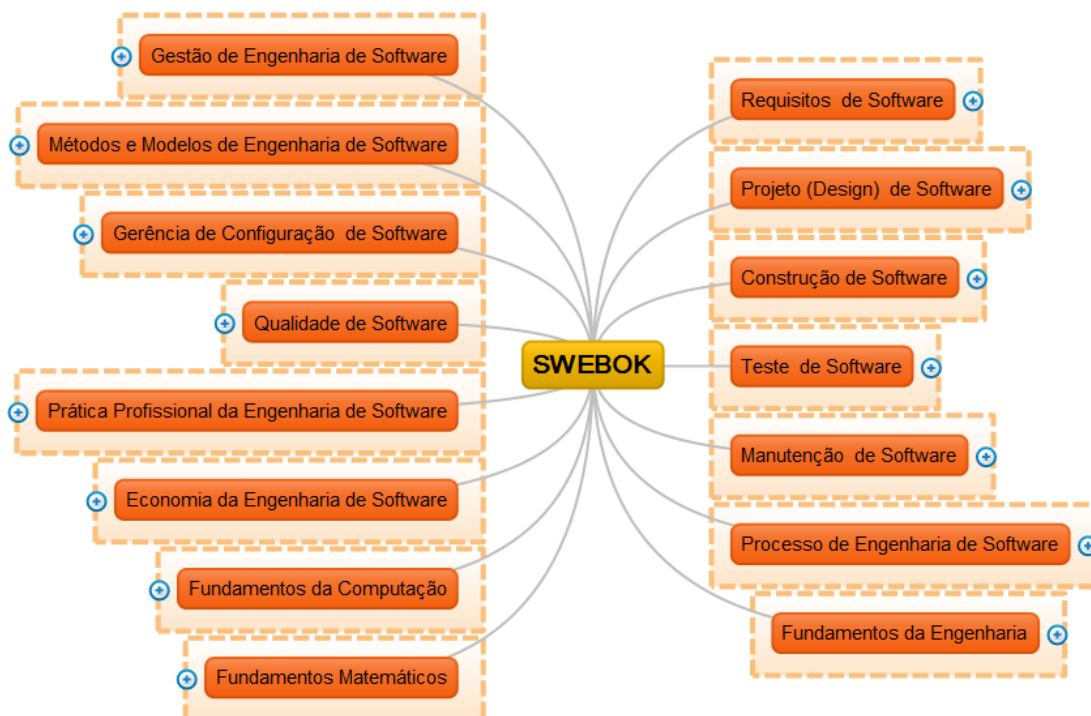


Figura 2.8.: Mind map para as áreas de conhecimento da Engenharia de Software:
Fonte:(IEEE Computer Society, 2014a)

Cada área do conhecimento é, por sua vez, subdividida em tópicos e subtópicos. O SWEBOK serve como definição do escopo do que é geralmente reconhecido como Engenharia de Software e ainda como referência primária para cada tópico, devido as citações que ele contém, além de possuir uma referência cruzada entre seus tópicos e as normas ISO/IEC a eles relacionadas.

Analizando criticamente, o SWEBOK é um registro, feito em um momento específico, do que se acordou, entre uma grande quantidade de pesquisadores, do que seriam os tópicos relacionados a Engenharia de Software. Com a evolução da própria Engenharia de Software, o projeto vai ficando desatualizado.

2.9. As normas da Engenharia de Software

Uma parte do conhecimento de Engenharia de Software foi normatizada por instituições nacionais e internacionais, principalmente a *International Standards Organizations* (ISO) e o *Institute of Electrical and Electronics Engineers* (IEEE)¹³. No Brasil, as normas são criadas, normalmente por tradução, pela Associação Brasileira de Normas Técnicas (ABNT) e são numeradas com o código NBR.

As normas geralmente se referem a conceitos estabelecidos, isto é, raramente há normas sobre inovações na Engenharia de Software, porém, nos últimos anos o processo de criação tem se tornado mais dinâmico.

De grande influência na indústria em geral são as normas de qualidade (família ISO 9000), que foram especializadas para software primeiro na família ISO 9216 e depois na família ISO 25000.

2.10. SEMAT

Um movimento interessante na Engenharia de Software é o SEMAT – *Software Engineering Method and Theory*. Fundado por Ivar Jacobson, Bertrand Meyer e Richard Soley em 2009, o SEMAT se propõe a “refundar a Engenharia de Software como uma disciplina rigorosa baseada em uma teoria geral da Engenharia de Software e um arcabouço de processo unificado”(Jacobson, Lawson e Ng, 2019; Jacobson, Ng et al., 2013).

Seus principais resultados são o Kernel, composto de Alphas, *Abstract Level Progress Health Attribute*, representações das coisas essenciais que a Engenharia de Software trabalha, Espaços de Atividade e Competências. As Figuras 2.9, 2.10 e 2.11 apresentam, respectivamente, uma representação dessa contribuição.

O SEMAT também define métodos e práticas. Um **método** “provê um guia para todas as coisas que você precisa fazer quanto está desenvolvendo e mantendo um software”(Jacobson, Lawson e Ng, 2019). Métodos são compostos por **práticas**, que são essas coisas que são feitas, ou mini-métodos. Práticas também podem ser compostas por outras práticas(Jacobson, Lawson e Ng, 2019). Segundo Jacobson, Lawson e Ng (2019), enquanto existem muitos métodos no mundo, o conjunto de prática é bem menor. Elas também são reusáveis, podendo aparecer em vários métodos.

¹³Pronunciado i-três-é ou *ai-Triple-i*

Ivar Jacobson inventou os Casos de Uso e tem forte impacto na Análise Orientada a Objetos. Bertrand Meyer inventou a linguagem Eiffel e Richard Soley foi líder da especificação CORBA. Este livro trata exatamente de

2.10.1. O Kernel Essencial

O Kernel Essencial é estruturado em 3 áreas: Cliente, Solução e Empreendimento. Dentro dessas áreas estão os Alphas, como mostra a figura a 2.9.] Os Alphas são as principais dimensões de um processo de software (Jacobson, McMahon e Racko, 2015).

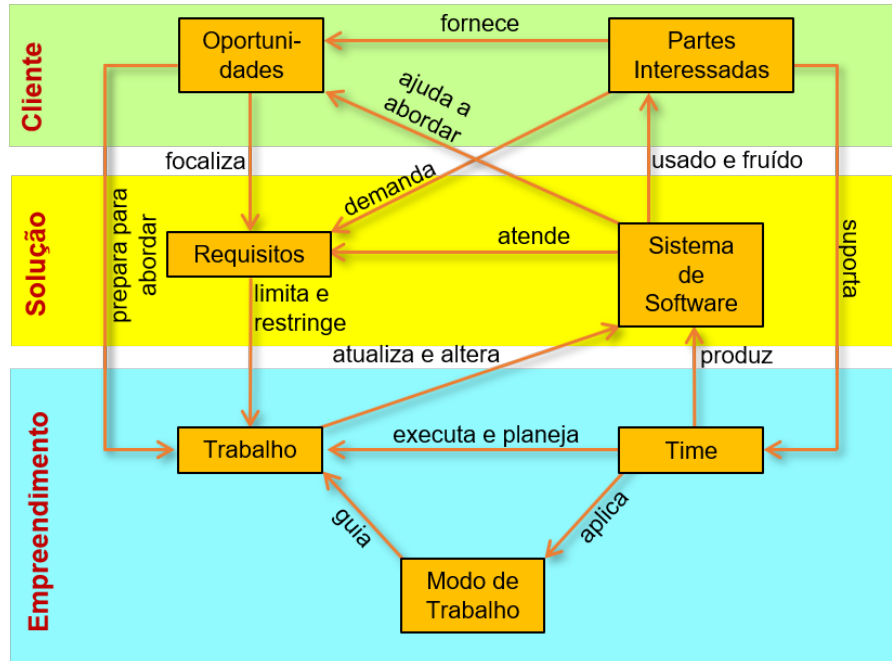


Figura 2.9.: O Kernel Essencial do SEMAT. Fonte: (Jacobson, Ng et al., 2013)

As Oportunidades do Kernel Essencial podem ser diretamente associadas ao conceito de Valor, já que são oportunidades de agregar valor as Partes Interessadas.

As **partes interessadas**, estudadas no Capítulo 3, são “pessoas, grupos ou organização que afetam ou são afetadas pelo sistema de software” (SEMAT Inc., 2019). São as partes interessadas que fornecem **oportunidades**, o “conjunto de circunstâncias que tornam apropriada o desenvolvimento ou mudança de um sistema de software”.

Essas **oportunidades** focalizam os **requisitos**, que são “o que o software precisa fazer para endereçar as oportunidades e satisfazer as partes interessadas” (SEMAT Inc., 2019). O **sistema de software** deve atender os requisitos, sendo o “produto primário do empreendimento de engenharia de software” (SEMAT Inc., 2019). O **time** é o “grupo de pessoas ativamente engajada no desenvolvimento, manutenção, entrega ou suporte de um sistema de software” (SEMAT Inc., 2019). É o time que executa e planeja o **trabalho**, pela aplicação de um ou mais **modos de trabalho**.

2. Introdução a Engenharia de Software

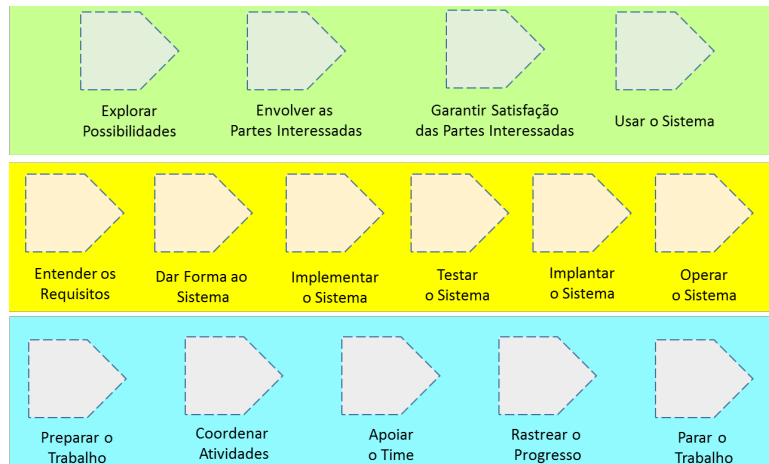


Figura 2.10.: Os Espaços de Atividades no Kernel do SEMAT. Fonte:(Jacobson, Ng et al., 2013)



Figura 2.11.: As Competências no Kernel do SEMAT. Fonte:(Jacobson, Ng et al., 2013)

2.11. Quem é o Engenheiro de Software

Não é fácil, hoje em dia, definir exatamente o que faz um Engenheiro de Software, porque a área é muito ampla, como se pode ver na Figura 2.8.

Uma carreira profissional na Engenharia de Software pode começar com funções ligadas a testes e controle de qualidade, ou a programação. A evolução técnica e pessoal do profissional vai levá-lo a cargos de maior responsabilidade técnica e gerencial, como cuidar da arquitetura de software ou gerenciar equipes de desenvolvimento. Com o tempo, deve subir mais na hierarquia e ser responsável por projetos ou mesmo portfólios de projetos.

A IEEE Computer Society lançou em 2014 um documento que fala sobre as competências esperadas de um Engenheiro de Software(IEEE Computer Society, 2014b). Ele divide as competências em cinco grandes áreas, que são detalhadas no documento:

1. Habilidades Técnicas
2. Habilidades e Atributos Comportamentais
3. Habilidades Cognitivas
4. Conhecimento Necessário
5. Disciplinas Relacionadas

Cada uma dessas habilidades possui atividades, que podem ser classificadas em 5 níveis de competência IEEE Computer Society (2014b):

1. **Técnico**, que segue instruções;
2. **Profissional (*Practitioner*) de Nível Inicial**, que faz atividades com supervisão;
3. **Profissional (*Practitioner*)**, que faz atividades com pouca ou nenhuma supervisão
4. **Líder Técnico**, que lidera equipes e indivíduos no desempenho das atividades, e
5. **Engenheiro de Software Sênior**, que serve de Engenheiro Chefe.

Além disso, se espera que todo Engenheiro de Software seja um instrutor e mentor de outros (IEEE Computer Society, 2014b).

2.12. Exercícios

Exercício 2.1:

Procure outra definição para Engenharia de Software. Discuta as diferenças entre essa definição e as dadas no texto.

Exercício 2.2:

Procure acidentes ou problemas causados por software funcionando incorretamente? Procure problemas na indústria espacial e médica, por exemplo.

Exercício 2.3:

Procure prejuízos financeiros causados por projetos de software que sofreram algum problema? Procure relatos sobre grandes projetos governamentais ou de empresas que foram interrompidos ou não tiveram sua funcionalidade completa devido a problemas no software.

Exercício 2.4:

Investigue prejuízos causados pelo mal uso de um software funcionando corretamente, por exemplo, quando planilhas Excel foram mal feitas. Ou acidentes de avião causados pelo erro de digitação. Explique como o software, mesmo funcionando corretamente, pode ter ajudado a causar o erro. Indique uma melhoria que poderia ter ajudado a alertar sobre o erro.

Exercício 2.5:

Procure um caso onde um símbolo ou espaço errado fez com que um programa não funcionasse de forma drástica.

Exercício 2.6:

Escolha um objeto ao seu redor que possua software e investigue o que esse software faz e onde ele está. Investigue o tipo de CPU usada por esse objeto.

Exercício 2.7:

Software também pode ser gerado para produzir erros em máquinas e equipamentos, passando a ser chamados de vírus ou cavalos de Tróia. Procure o relato de um vírus gerado intencionalmente para causar prejuízos específicos para uma organização, ou algo mais genérico que tenha provocado prejuízos a muitos (e talvez lucros a seus criadores).

Exercício 2.8:

Procure algumas descrições de empregos para Engenheiros de Software e faça um relato de suas atribuições. Avalie o grau de consistência dessas atribuições entre os empregos. Avalie também o que é mais técnico e o que é mais gerencial.

Exercício 2.9:

Procure o documento (IEEE Computer Society, 2014b) e entenda as habilidades desejadas de um Engenheiro de Software.

Partes Interessadas

Find the appropriate balance of competing claims by various groups of stakeholders. All claims deserve consideration but some claims are more important than others.

(Warren Bennis)

Conteúdo

3.1.	O Que São Partes Interessadas	28
3.2.	Partes Interessadas e Valor	31
3.3.	Gerência das Partes Interessadas	32
3.4.	A abordagem SEMAT para Partes Interessadas	33
3.5.	Caracterização da Parte Interessada	33
3.6.	Classificação das Partes Interessadas	35
3.7.	Análise das Partes Interessadas	38
3.8.	Representação das Partes Interessadas	39
3.9.	Engajamento das Partes Interessadas	39
3.10.	Matriz RACI	40
3.11.	Conclusão	41

Por que partes interessadas?

Projetos de software afetam e são afetados por pessoas e organizações, as partes interessadas. Entender como modelar e trabalhar com elas é essencial para conclusão satisfatório do projeto.

Todo projeto, inclusive projetos de software, não só atende a pessoas e organizações, compradores, clientes e usuários, mas também **afetam e são afetados** por outras pessoas e organizações de alguma forma. O somatório de todas essas pessoas, direta ou indiretamente ligadas ao projeto, é chamado de **partes interessadas**¹.

As partes interessadas são um Alpha do SEMAT, visto na Figura 3.1, no nível de Cliente (Jacobson, Ng et al., 2013). Neste modelo elas fornecem as oportunidades para um projeto de software, demandam requisitos, suportam o time de desenvolvimento e usam e fruem do sistema de software. Na verdade, elas tem posição basilar em todo projeto. Aqueles responsáveis por desenvolver o software trabalham para as partes interessadas.

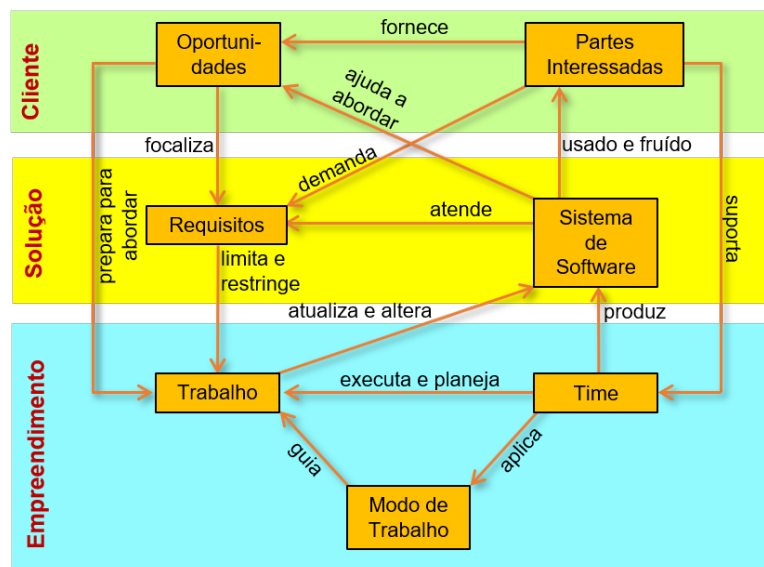


Figura 3.1.: O Kernel Essencial do SEMAT. Fonte: (Jacobson, Ng et al., 2013)

A influência das partes interessadas em um projeto tem um comportamento inverso ao do custo de mudanças, como ilustra a Figura 3.2. Ao longo do projeto, como o trabalho vai ficando pronto, uma mudança provocada pelas partes interessadas começa a custar muito mais, logo elas vão perdendo a capacidade de fazer alterações. Por outro lado, no início, quando o projeto ainda não fez escolhas e compromissos importantes que determinarão o seu futuro, e as mudanças custam pouco, as partes interessadas têm grande influência.

Deve ficar claro que o objetivo geral de qualquer projeto é **satisfazer as partes interessadas**.

As práticas ágeis, tentam diminuir, ao mesmo tempo, tanto a perda de influência das partes interessadas, quanto o aumento do custo de modificações. Isso é feito principalmente com os usuários e clientes mais próximos do projeto, ao longo do tempo. Mesmo assim, grandes mudanças têm um custo maior e podem ficar inviáveis.

¹O nome em inglês, **stakeholders**, aqueles que possuem algum interesse ou aposta no projeto, é também usado.

3. Partes Interessadas

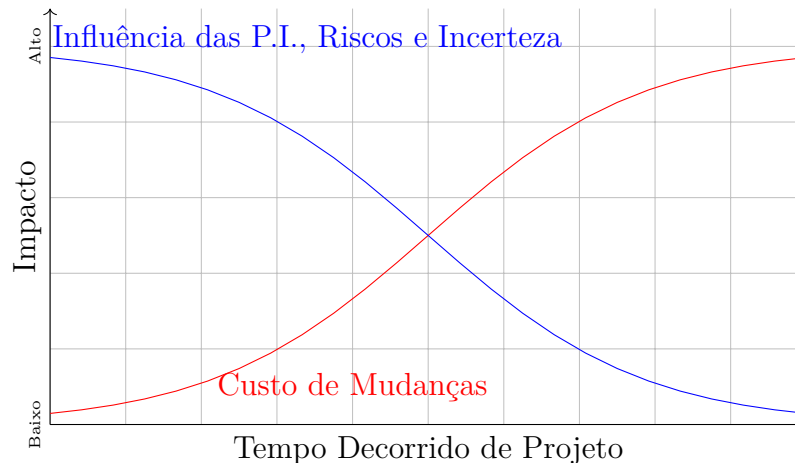


Figura 3.2.: O impacto das partes interessadas.

3.1. O Que São Partes Interessadas

Uma boa definição do que é uma parte interessada é fornecida pelo Guia PMBOK: “é um indivíduo, grupo ou organização que pode afetar, ser afetada ou sentir-se afetada por uma decisão, atividade ou resultado de um projeto. As partes interessadas do projeto podem ser internas ou externas ao projeto, e podem estar envolvidas ativamente ou passivamente, ou mesmo não estar cientes do projeto” (PMI, 2017). Também é possível, para facilitar uma análise, falar de outros tipos de parte interessada, não humanos, como um ecossistema ou o meio-ambiente.

O PMBOK (PMI, 2017) exemplifica várias partes interessadas em um projeto:

- equipe do projeto:
 - gerente do projeto;
 - equipe de gerência do projeto;
 - outros membros do projeto;
- patrocinador;
- clientes e usuários;
- fornecedores;
- parceiros comerciais;
- gerentes funcionais;
- gerente de portfólios;
- gerente de programas;
- escritório de projetos, e
- outras partes interessadas.

Esse mesmo guia avisa que “As partes interessadas do projeto podem ter um impacto positivo ou negativo no projeto, ou ser impactadas de forma positiva ou negativa pelo projeto.” (PMI, 2017).

O grupo dos que afetam e são afetados, com conhecimento do fato ou não, é bem maior que os envolvidos diretamente no projeto. Por exemplo, em um sistema que controla um radar, dedicado a aplicar multas de trânsito automaticamente, podem ser afetados os guardas de trânsito, que podem perder o emprego, os passageiros de todos os veículos e até todo o sistema de saúde do país, devido a diminuição da gravidade dos acidentes. Também podem ser afetados os moradores da região, com menos atropelamentos, ou com um trânsito mais lento que atrasa

3. Partes Interessadas

seus horários. Outro exemplo pode ser um sistema de marcação e monitoração do momento de atendimento de consultas médicas, onde os acompanhantes dos pacientes também são afetados.

Uma avaliação da Tabela 2.2 mostra a importância das partes interessadas para o sucesso de um projeto. Entre os 10 primeiros fatores de sucesso, 5 são ligados a pessoas com interesse no projeto. Entre os 4 primeiros, que estão em igualdade de condição, o envolvimento do Patrocínio Executivo e dos Usuários se mostra essencial.

⚠ Nunca diga “usuário”

Gladys S. W. Lam (2015) diz que um dos princípios básicos da análise de negócio é **nunca dizer usuário**. Segundo ele, usuários só existem na perspectiva de um sistema de TI. Nos contextos de negócio as pessoas não são usuárias, mas atores centrais do negócio. Na prática, usuário não significa muito em um mundo onde todos são usuários de sistemas de TI, então ao tratar as partes interessadas devemos usar o termo específico do negócio, como paciente, atendente, médico e enfermeira em um ambulatório, e correntistas, caixa e gerente em um banco.

Deve ficar claro que todo projeto se inicia por meio das partes interessadas, porém não necessariamente de todas. Descobrir todas as partes interessadas é uma das missões iniciais de um projeto. Na maioria das vezes os usuários, pelo menos em sua maioria, estão identificados no início do projeto. Outro caso comum é uma parte interessada advogar a necessidade do projeto para trazer melhorias para outra parte. É comum que software seja desenvolvido para melhorar o atendimento ao cliente da organização que está adquirindo o produto, obviamente a espera de melhorar o resultado global.

Entre as partes interessadas, o usuários do software sendo produzido são um grupo importante, que merece destaque especial em nossos estudos. A princípio são eles, possivelmente por meio de uma representação, que nos fornecerão os requisitos e aceitarão o produto.

3.1.1. Usuários

Desenvolvemos sistemas para serem usados. Na verdade, uma medida importante da qualidade de um sistema de informação e do sucesso do projeto que o construiu é quanto do sistema é realmente usado, ou seja, quanto do que foi feito é realmente útil. Logo, para descobrir o que deve ser feito (o quê), devemos primeiro entender para quem o sistema se destina (quem) e, principalmente, entender o motivo dessas pessoas (por que) quererem usar o sistema.

Portanto, um tipo especial de parte interessada é comumente chamada de **usuário**. Como diz o nome, um usuário sempre **usa** o sistema, só que o sistema pode ser usado de muitas formas. Esse nome pode ser entendido em duas formas. Na forma restrita, indica o **usuário final**, isto é, os que realmente usam o sistema dentro do escopo do seu objetivo, interagindo com suas interfaces, sejam por meio de telas ou relatórios impressos. Já na definição de forma ampla, todos aqueles que usam alguma parte do sistema, incluindo sua documentação e suas estruturas de dados, como os desenvolvedores, que usam o código do sistema, ou os responsáveis por sua operação, também são usuários. Entretanto, mesmo com uma interpretação ampla, algumas partes interessadas não são usuários, pois nunca usam alguma parte do sistema, mesmo que sejam afetadas por eles.

3. Partes Interessadas

Por exemplo, em um sistema que funciona para os vendedores de um fornecedor pães para restaurantes façam os pedidos, a parte interessada vendedor é um usuário final, porém o cliente do restaurante é afetado pela entrega, ou não, do pão, logo também é parte interessada.

Novamente, segundo o PMBOK, o objetivo de todo projeto é gerenciar a satisfação do usuário (PMI, 2017). Por isso é importante garantir o envolvimento de todas as partes interessadas. A forma de fazer isso é por meio de um processo de comunicação contínua, para “entender necessidades e expectativas, abordar as questões conforme elas ocorrem, administrar os interesses conflitantes e incentivar o engajamento apropriado das partes interessadas com as decisões e atividades do projeto” (PMI, 2017).

Devemos lembrar, porém, que alguns sistemas são feitos para atingir objetivos da organização, mesmo que não sejam os mesmos objetivos dos usuários. Por exemplo, qualquer sistema de vendas direta aos usuários deve ter como um dos seus objetivos aumentar o lucro da organização, o que não é, possivelmente, a vontade dos usuários. Do mesmo modo, muitos sistemas organizacionais foram criados para garantir a alta produtividade dos seus usuários, mesmo que em detrimento ao conforto ou interesses pessoais dos mesmos.

3.1.2. Perspectivas dos Usuários

As perspectivas básicas que encontramos em entrevistas e reuniões com usuários, são três, como ilustrado na Figura 3.3:

1. o usuário onisciente;
2. o usuário externo ao sistema,e
3. o usuário interno ao sistema.

O **usuário onisciente** fala do sistema como um todo, indicando normalmente coisas que ele deve, ou deverá, fazer. Ele vê tanto o sistema quanto seus usuários de uma perspectiva externa, conhecendo os mecanismos, de forma abstrata, tanto por dentro quanto por fora. Geralmente ele possui uma posição de gerência e muitas vezes é o patrocinador executivo do projeto. Algumas vezes já usou o sistema, mas é comum que não o use mais ou use apenas funções destinadas a gerentes, como relatórios, ou veja os resultados por meio de outros sistemas de gestão.

É possível que ele não saiba como o sistema funciona realmente, pois seu contato pode ser de outra época em relação a entrevista, desconhecendo então modificações e detalhes atuais. Ele normalmente exige funcionalidade gerencial do sistema. Esse usuário muitas vezes quer, e é capaz de ajudar a definir, uma nova abordagem de negócio para o sistema, o que pode entrar em contradição com usuários externos ou internos, que pensam mais sobre como o sistema funciona hoje. Tipicamente é um gerente ou diretor e tem algum poder sobre os usuários com outras perspectivas.

O **usuário externo** descreve o sistema como se o estivesse usando, e normalmente já usa uma versão do sistema. Exige funções do sistema, mas principalmente para atender seu nível de atuação, seja gerencial ou operacional. Ele pode apresentar algum nível de desconfiança, pois o novo sistema pode exigir um esforço para conseguir novos conhecimentos ou distribuir conhecimento que hoje apenas ele tem acesso, o que afeta o poder dele na organização. Além disso, ele conhece bem a entrada e a saída do sistema atual, mas não necessariamente os procedimentos e algoritmos, ou seja, como as contas são feitas. Esse usuário, principalmente quando mais próximo do nível operacional da organização, pode levar ao desenvolvimento de um sistema igual ao já existente, apenas com a correção dos defeitos, sem mudança na prática do negócio.

3. Partes Interessadas

O **usuário interno** descreve o sistema visto por dentro. Algumas vezes, não existe ainda um sistema plenamente informatizado, e é esse usuário que faz a parte de cálculos e tomadas de decisão. Ele conhece normalmente muito bem como o sistema atual funciona por dentro. Outras vezes ele é responsável pelo desenvolvimento e manutenção do sistema atual. É comum que esse tipo de usuário apresentar um risco ao desenvolvimento, porque ele é candidato a substituição pelo sistema novo, o que pode gerar desconfiança, franca hostilidade e mesmo sabotagem do projeto.

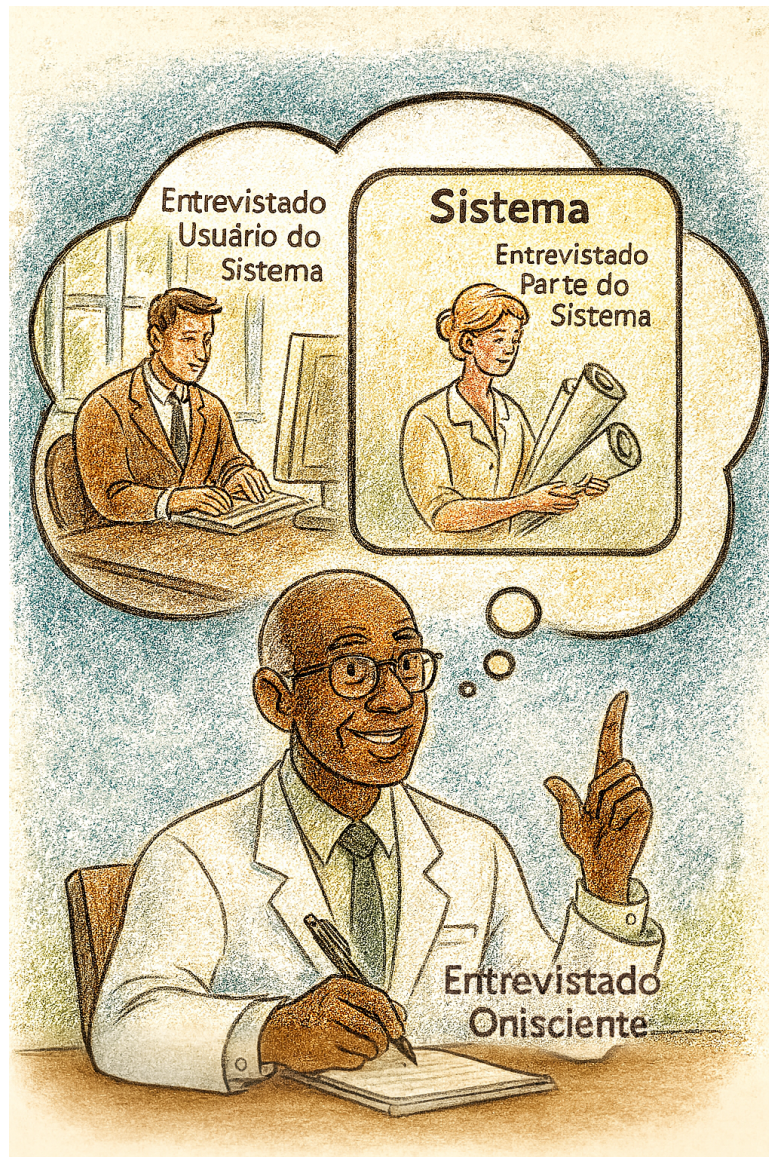


Figura 3.3.: 3 tipos comuns de usuários dos sistemas. Fonte:próprio autor

3.2. Partes Interessadas e Valor

Todo valor do projeto é entregue às partes interessadas. Na verdade, seria mais correto dizer que “todos os valores do projeto” são entregues às partes interessadas, já que um projeto pode ter valores diferentes de acordo com as partes interessadas, pois elas possuem interesses diferentes, como será tratado na Seção 3.5.1.

3. Partes Interessadas

Suponha o seguinte projeto de software: uma conjunto de serviços B2B para uma editora, que serão usados por livrarias *on line* que desejam comprar produtos *on demand* quando forem vendidos em suas lojas virtuais B2C. Esse software está planejado para ter efeito de melhorar o serviço das grandes livrarias *on line*, prejudicando, por consequência, as pequenas livrarias locais. Então, podemos concluir que ele fornece valor positivo para algumas partes interessadas:

- a editora;
- grandes livrarias *on line*;
- o leitor;
- o escritor;
- o serviço de entrega, e
- outros atores, como pessoal de operação do software, vendedores de equipamentos, etc.

Porém ele também fornece um valor negativo para partes interessadas que são afetadas, principalmente de forma indireta, como:

- livreiros e trabalhadores de livraria, que podem perder o emprego;
- proprietários que alugam lojas para livrarias;
- pessoas que querem comprar um presente pessoalmente em cima da hora, e
- pequenas livrarias locais.

É claro que no mercado esse padrão sempre se repetirá: favorecer um negócio sempre prejudica seus competidores, e com isso toda a sua cadeia de produção. Normalmente não há um problema legal ou ético com isso, mas é possível que em condições limites alguma lei de proteção à livre concorrência possa ser ferida.

3.3. Gerência das Partes Interessadas

A gerência das partes interessadas aparece em várias propostas da área de Gerência de Projeto e de Engenharia de Software.

Ela é uma atividade do PMBOK que inclui (PMI, 2017):

- “processos para identificar pessoas, grupos ou organizações que podem impactar ou ser impactadas pelo projeto”;
- “análise das expectativas das partes interessadas e seu impacto no projeto”, e
- “o desenvolvimento de estratégias apropriadas de gerência para efetivamente engajar as partes interessadas nas decisões e execução do projeto”.

Para isso é necessária a comunicação contínua com todas as partes interessadas, no intuito de entender e registrar todas as necessidades e expectativas. Além disso, é necessário abordar todas as questões e conflitos de interesse, que podem acontecer a qualquer momento do projeto, e ainda promover o engajamento apropriado nas decisões e atividades do projeto (PMI, 2017).

Essa gestão, então, vai se resumir em 4 partes (PMI, 2017):

1. identificar as partes interessadas;
2. planejar a gerência das partes interessadas;
3. gerenciar o engajamento das partes interessadas, e
4. monitorar o engajamento das partes interessadas.

3.4. A abordagem SEMAT para Partes Interessadas

O SEMAT considera que o trabalho com as partes interessadas deve evoluir em uma escala de 6 estados, do menos ao mais sofisticado (Jacobson, Ng et al., 2013):

1. reconhecer as partes interessadas;
2. criar uma representação para as partes interessadas;
3. envolver as partes interessadas;
4. fazer com que as partes interessadas concordem;
5. satisfazer as partes interessadas para a implantação, e
6. satisfazer as partes interessadas com o uso.

É importante entender, a partir desses estágios, em que ponto um projeto está em relação as partes interessadas. Essa avaliação pode ser feita a partir de *checklists* propostos pelo SEMAT.

3.5. Caracterização da Parte Interessada

Após a identificação das partes interessadas, continua-se o levantamento e caracterização.

Para isso, normalmente são respondidas, para cada parte interessada, algumas perguntas, como:

1. Nome
2. Posição na organização
3. Posição/Responsabilidade no projeto
4. Interesse no projeto (Stake)
5. Impacto no projeto
6. Benefícios recebidos do projeto
7. Colaboração necessária ao projeto (o que precisamos dessa parte interessada)?
8. Atitudes percebidas (riscos)

Essas perguntas variam de um autor da área para outra, porém sempre é necessário ter uma visão de como a parte interessada pode impactar o projeto, sendo apoiando, sendo criando obstáculos. Isso pode ser feito com práticas de classificação das partes interessadas.

3.5.1. Interesses e Objetivos

Como diz o próprio termo, todas as partes interessadas possuem, ou acreditam possuir, interesses no sistema, mesmo que não estejam cientes do mesmo, ou do seu planejamento. Os interesses têm relação com as expectativas, explícitas e implícitas, que o sistema, em operação, afete a elas, o seu domínio, ou o seu negócio, de alguma forma. Interesses são os benefícios que as partes interessadas esperam obter do sistema, ou, no caso de partes antagônicas, com a não existência do projeto.

Os usuários finais possuem também **objetivos**, i.e., eles usam o software para obter uma, ou mais, resposta planejada. Neste contexto, objetivos são requisitos funcionais que os usuário esperam encontrar no sistema.

A princípio, se deseja que o sistema seja construído para atingir seus objetivos de forma que todos os interesses sejam atendidos. Porém, pode haver incompatibilidades tanto entre os interesses quanto entre os objetivos das diversas partes interessadas. Tratar casos como esse já

3. Partes Interessadas

foi discutido inclusive como um processo de software específico, conhecido como WinWin (B. W. Boehm, 1988; B. Boehm e Kitapci, 2006).

É importante atentar para a diferença que existe entre **o uso que uma parte interessada fará do sistema** (o que), o seu **objetivo**, com o **interesse** que uma parte interessada tem no sistema, que é o **resultado esperado, ou expectativa** que ela tem. Enquanto todas as partes interessadas tem interesses, só algumas tem objetivos, e essas são as usuárias do sistema.

Estudar os interesses nos permite saber melhor como realizar os objetivos. Por exemplo, um cliente de web site de vendas de produtos tem como objetivos, ao longo de uma compra, várias ações, como escolher um produto, saber seu preço, pagar o produto. Porém ao longo de todo esse tempo, seu único interesse pode ser receber o produto. Mais do que comprar, o usuário quer recebe-lo em casa. Então toda a dinâmica do site, ou do software e logística do sistema por trás dele, pode ser construída voltada para esse interesse. Mesmo que quiséssemos atender um interesse menor, como “comprar o produto”, já poderíamos imaginar formas diferentes de implementar as ações correspondentes. Um exemplo disso no mundo real é o *one-click buy*, ou seja, comprar um produto com só um click do mouse e ter certeza de recebe-lo, ideia que foi inclusive patenteada pela Amazon².

Como documentação dos interesses e objetivos das partes interessadas, dois formatos são possíveis. O primeiro é uma tabela com três colunas: parte interessada, interesses e objetivos. Essa tabela tem como defeito não identificar claramente partes interessadas com interesses ou objetivos comuns. A vantagem é a facilidade de gerenciar e preencher, pois é um formulário muito simples.

Tabela 3.1.: Exemplo de formulário para registro do interesse e objetivo da parte interessada.

Parte Interessada	Interesses	Objetivos
Professor	Não perder seus livros	Registrar um emprés- timo
	Recuperar livros empre- stados	Registrar uma devolução
	Saber com quem está um livro	Registrar um livro
	Saber os livros que tem	Cobrar um livro atra- sado
Aluno		Registrar um aluno
	Pegar livro emprestado com facilidade	Registrar uma devolução
	Não ser cobrado por li- vro que devolveu	Receber recibo de devo- lução
Desenvolvedor	Fazer projeto final	

A segunda forma é uma matriz Partes Interessadas vs. Interesses e Objetivos. Dependendo do escopo do sistema ou da quantidade de partes interessadas, escolhe-se um ou outro para coluna ou linha. Nesse caso, a célula que mostra o cruzamento entre parte interessada e interesse ou objetivo pode conter uma avaliação qualquer, como o uso de palavras em uma escala de importância para aquele usuário: baixa, média e alta.

Um método semelhante a essa matriz, onde são dados pesos aos desejos de cada tipo de usuário é o QFD (*Quality Function Deployment*, chamado no Brasil de Desdobramento da Função Qualidade)(Franceschini, 2016; Kumar, 2020).

²O que causou muita polêmica na época. Essa patente já expirou.

3. Partes Interessadas

Tabela 3.2.: Exemplo de matriz de cruzamento para registro do interesse e objetivo da parte interessada.

		Professor	Aluno	Desenvolvedor
Interesses	Não perder seus livros	✓		
	Recuperar livros emprestados	✓		
	Saber com quem está um livro	✓		
	Saber os livros que tem	✓		
	Pegar livro emprestado com facilidade		✓	
	Não ser cobrado por livro que devolveu		✓	
	Fazer Projeto Final			✓
Objetivos	Registrar um empréstimo	✓		
	Registrar uma devolução	✓	✓	
	Registrar um livro	✓		
	Cobrar um livro atrasado	✓		
	Registrar um aluno	✓		
	Receber recibo de devolução		✓	

3.6. Classificação das Partes Interessadas

Muitos autores propõem classificar as partes interessadas de alguma forma, e propõem também o apoio de gráficos para entender melhor como gerenciá-las ao longo do projeto.

Um gráfico típico, mostrado na Figura relaciona o interesse da parte interessada com o poder, e indica que tipo de ação deve ser tomada na gerência de partes interessadas.

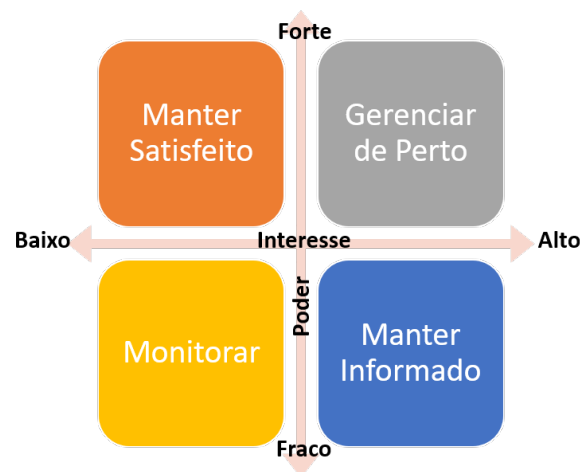


Figura 3.4.: Quadro para o tratamento das partes interessadas. Fonte: (PMI, 2017)

Essa classificação pode ser feita de várias formas, sendo as mais reconhecidas:

- Poder × Interesse;

3. Partes Interessadas

- Poder $\times$ Influência;
- Influência $\times$ Impacto, e
- Poder, Legitimidade e Urgência (modelo de saliência) (Mitchell, Agle e Wood, 1997).

Outra classificação feita é a **Matriz de Engajamento**. Nela se faz uma análise do comportamento atual do usuário em relação a um projeto e o comportamento desejado. São usadas cinco classes possíveis para descrever o engajamento do usuário:

1. desconhece;
2. resistente;
3. neutro;
4. apoiador, e
5. liderança.

O trabalho da equipe é identificar o comportamento atual e, por meio de ações positivas, levar as partes interessadas a um comportamento desejado.

Desconhece	Resistente	Neutro	Apoiador	Líder
Parte Interessada 1	C			D
Parte Interessada 1			C	D
Parte Interessada 1			CD	

Tabela 3.3.: Matriz de Engajamento: C significa comportamento corrente, D o comportamento desejado.

K. E. Wieggers e Beatty (2013), por sua vez, fazem uma classificação hierárquica das partes interessadas em:

- clientes, e
 - usuários diretos e indiretos, e
 - ◊ classes de usuários favorecidas, cuja satisfação está fortemente alinhada com os objetivos de negócio do projeto;
 - ◊ classes de usuários desfavorecidas, que não podem usar o produto, por motivos legais, de segurança ou outros, como estelionatários em um sistema bancário;
 - ◊ classes de usuários ignoradas, que não são relevantes ao projeto e
 - ◊ outras classes de usuário, que não se encaixam em nenhuma das outras, mas ainda fornecem requisitos para o projeto .
 - outros clientes.
- outras partes interessadas.

3.6.1. O que é poder?

O conceito de poder é amplamente discutido no contexto das ciências sociais e pode ser definido de várias maneiras. Dahl (1957) afirma que “A tem poder sobre B na medida em que pode conseguir que B faça algo que B não faria de outra forma”. Assim, o poder é a capacidade de influenciar outros para atingir objetivos específicos. A principal referência no tema é um artigo de French e Raven (1959), que foi revisado por Raven (1993) e Raven (2008).

De acordo com French e Raven (1959), o poder social refere-se à habilidade de um agente ou figura de poder de trazer mudanças na cognição, atitude ou comportamento de outra pessoa

3. Partes Interessadas

ou grupo. Esse poder pode ser exercido por meio de diferentes recursos disponíveis ao agente influenciador.

Há uma distinção importante entre poder e status (Magee e Galinsky, 2008). Enquanto o poder é baseado nos recursos controlados por um indivíduo, o status é conferido pela percepção de outros. Essas relações podem ser causais: o poder pode levar ao status, assim como o status pode resultar em poder. Contudo, status sem poder pode limitar a eficácia em contextos de troca, e poder sem status pode ser percebido como ilegítimo.

Finalmente, o poder pode ser classificado em categorias, como poder coercitivo, de recompensa, legítimo, de referência, de especialista e informacional (French e Raven, 1959). Cada uma dessas bases possui características e impactos distintos, influenciando o comportamento de diferentes maneiras. A seguir apresentamos uma pequena introdução a cada uma delas e a Tabela 3.4 com algumas propriedades.

- **Poder Coercitivo:** Baseado na capacidade de impor punições ou consequências negativas. Exemplos incluem advertências, demissões e sanções. Esse tipo de poder gera conformidade, mas pode provocar ressentimento e resistência. Sua eficácia depende da vigilância contínua (French e Raven, 1959).
- **Poder de Recompensa:** Derivado da capacidade de fornecer recompensas desejadas, como promoções, aumentos salariais e benefícios. Ele aumenta a motivação, mas pode levar à dependência excessiva de incentivos externos. Também é socialmente dependente e requer vigilância (French e Raven, 1959).
- **Poder Legítimo:** Fundado em normas culturais ou sociais que conferem autoridade formal. Exemplos incluem gerentes, diretores e líderes eleitos. Promove estabilidade, mas pode ser contestado se a autoridade for percebida como ilegítima. Não requer vigilância contínua (French e Raven, 1959). A legitimidade pode vir:
 - da posição ocupada,
 - da necessidade de retribuir algo (reciprocidade),
 - da equidade, i.e., da necessidade de conceder a cada um conforme o merecimento,
 - e da dependência, decorrente da dependência que um indivíduo tem da cooperação de outros.
- **Poder de Referência:** Baseado no carisma, admiração ou identificação com o líder. Exemplos incluem figuras públicas inspiradoras e líderes carismáticos. Fomenta lealdade, mas pode levar a decisões baseadas em emoção em vez de lógica. É socialmente dependente, mas não requer vigilância (French e Raven, 1959).
- **Poder de Especialista:** Originado do conhecimento ou habilidades específicas reconhecidas. Exemplos incluem consultores técnicos e especialistas em áreas específicas. Gera confiança e respeito, mas depende da relevância contínua do conhecimento. É socialmente dependente e não requer vigilância (French e Raven, 1959).
- **Poder Informacional:** Baseado na capacidade de convencer por meio de lógica, argumentos ou informações claras. Exemplos incluem explicações racionais de supervisores que levam subordinados a aceitarem mudanças. É socialmente independente e não exige vigilância (Raven, 2008)³.

Em especial, os poderes de coerção e recompensa podem vir de relações explícitas, consideradas impessoais, ou de relações implícitas, como a visão do subordinado sobre a opinião dos agentes de poder (Raven, 2008). Já os poderes de especialização, referência podem vir da imagem positiva ou negativa que o subordinado tem do agente de poder ou mesmo da visão que seus

³Raven explica que French não concordava que o Poder Informacional fosse um poder, mas seria só uma influência, logo não aparece como poder no artigo original de French e Raven (1959).

3. Partes Interessadas

pares têm sobre o primeiro. Finalmente, o poder de informação pode ser por meios diretos ou indiretos.

Tabela 3.4.: Bases do Poder Social - Resumo

Base do Poder	Categoria	Socialmente	Vigilância	Fator
Coercitivo	Formal	Dependente	Necessária	Ameaça
Recompensa	Formal	Dependente	Necessária	Recompensa
Referência	Informal	Independente	Desnecessária	Admiração
Especialista	Informal	Independente	Desnecessária	Confiança
Legítima	Formal	Independente	Desnecessária	Relações Formais
Informacional	Informal	Independente	Desnecessária	Convencimento

3.7. Análise das Partes Interessadas

A Análise das Partes Interessadas é uma atividade com o objetivo determinar quais são as partes interessadas, e determinar como são afetadas e como podem afetar o projeto, além de entender quem realmente “conta” em um projeto (Mitchell, Agle e Wood, 1997).

Normalmente análise das partes interessadas se inicia com a identificação das mesmas, como mostram PMI (2017) e Jacobson, Lawson e Ng (2019). Muitas são as formas de identificação, mas certamente a experiência do analista será muito importante, além de documentos formais do projeto.

É importante notar que algumas partes interessadas são essenciais ao sucesso do projeto de software. Por exemplo, deve haver um responsável final pelo financiamento do projeto, normalmente conhecido como patrocinador. Deve também existir pelo menos um grupo de usuários do futuro sistema, os usuários propriamente ditos.

Técnicas de levantamento podem incluir não só a leitura de documentos e a experiência de pessoas da área de negócio, mas também reuniões de *brainstorm* podem ser usadas para levantar o conjunto de partes interessadas a partir de um núcleo inicial de pessoas.

Existe uma base de análise para identificar partes interessadas (Mitchell, Agle e Wood, 1997) e determinar um grau de atenção que deve ser dado a elas, baseada no conceito de **saliência**, que é formada em 3 atributos: Poder, Legitimidade e Urgência. Esse modelo é conhecido como modelo de saliência ou modelo PLU.

Para Mitchell, Agle e Wood (1997), que separa poder e legitimidade, **poder** é a “capacidade de um ator fazer com que outro ator realize algo que não realizaria a princípio” (Mitchell, Agle e Wood, 1997). Ele vê o Poder apenas como baseado em coerção, i.e. o uso de força ou ameaça, em utilitarismo, i.e. o uso de incentivos, e normativo, por influências simbólicas, não reconhecendo as bases socialmente independentes previstas por French e Raven (1959) e Raven (2008).

Nessa abordagem, a **legitimidade** é a “percepção generalizada ou premissa que as ações de uma entidade são desejáveis, apropriadas dentro de algum sistema socialmente construído de normas, valores, crenças e definições” (Mitchell, Agle e Wood, 1997). Ela se baseia no indivíduo, na organização ou na sociedade.

3. Partes Interessadas

A **urgência** é “grau que a parte interessada reivindica de atenção imediata” (Mitchell, Agle e Wood, 1997) Ela se baseia na sensibilidade ao tempo ou na criticalidade, a importância da reivindicação ou do relacionamento com a parte interessada.

Finalmente, a **saliência** de uma parte interessada “é o grau que o gerente dá prioridade a partes interessadas que competem” (Mitchell, Agle e Wood, 1997). A análise da posse desses três fatores também leva a denominações como as apresentadas na Figura 3.5.



Figura 3.5.: Classificação das partes interessadas. Fonte: (Mitchell, Agle e Wood, 1997)

Mitchell, Agle e Wood (1997) afirma que partes interessadas que possuem apenas uma dessas características normalmente são deixadas de lado pelos gerentes, que aumentam o seu grau de engajamento com as que tem duas das características, até chegar ao grupo dos “definitivos”, que tem prioridade máxima.

3.8. Representação das Partes Interessadas

Algumas das partes interessadas são formadas por grupos de pessoas. Dependendo do tamanho do grupo, não é possível trazer todos eles para participar do projeto. Nesse caso é necessário criar uma representação para as partes interessadas (Jacobson, Lawson e Ng, 2019). Caberá a essa representação se envolver e tomar decisões em nome do grupo.

Em projetos ágeis é comum que exista um representante único de todos os usuários, que estabelece as demandas e prioridades do projeto, como o *Product Owner* (Rubin, 2013), porém isso não elimina a existência de todas as outras partes.

3.9. Engajamento das Partes Interessadas

O engajamento das partes interessadas se faz por meio de três processos (PMI, 2017):

- inclusão, aumentando o nível de participação das partes interessadas;

3. Partes Interessadas

- materialidade, pela identificação das questões mais importantes da organização e das partes interessadas, e
- responsividade, pela resposta da organização.

Esse engajamento pode ser obtido de várias formas, dependendo do nível de engajamento que é desejado. O tipo de engajamento pode ser classificado em:

- consulta, que é alcançado por meio de pesquisas, grupos de foco, entrevistas ou audiências privadas;
- diálogo, por meio de painéis, diálogos on line, mesas redondas, espaços de convivência e workshos;
- conflituoso, que pode ser resolvido com mediação, negociação ou outras técnicas especializadas;
- parceria, obtida com a negociação, gestão de contratos e acordos, e
- empoderamento, alcançada por meio de metodologias de decisão, reuniões de conselho, diagnóstico participativo e comunitário.

3.10. Matriz RACI

A participação das partes interessadas em cada atividade do projeto pode ser planejada por meio de uma Matriz RACI⁴.

A Matriz RACI, também discutida brevemente no Capítulo ??, faz o cruzamento entre as partes interessadas e alguma forma de entender o projeto, como atividades, casos de uso, histórias do usuário, entregáveis, etc. Nesta seção vamos usar o exemplo de entregáveis, sem perda de generalidade.

Para cada um dos entregáveis, deve ser definida, para cada uma das partes interessadas, uma ação, ou mais, em relação ao projeto. Estas ações formam o acrônimo RACI e significam:

- Responsável, que realiza a atividade;
- Aprovador, que aceita a atividade;
- Consultado, que fornece informações para a realização da atividade, e
- Informado, que deve ser informado sobre a atividade.

A Tabela 3.5 é um exemplo de Matriz RACI.

A Matriz RACI tem algumas regras:

1. Células podem conter mais de uma letra;
2. Apenas uma pessoa pode aprovar uma atividade, e
3. Pelo menos uma pessoa deve ser responsável por realizar uma atividade.

Além disso, duas análises devem ser feitas para verificar a Matriz RACI.

1. A Análise Vertical:

- alguma parte interessada tem um excesso de R's, sendo responsável por uma carga de trabalho excessiva?
- alguma parte interessada tem um excesso de A's, havendo pouca delegação ou responsabilidade excessiva?

⁴Apesar de ter uma preocupação em descobrir os autores de cada método proposto neste texto, a matriz RACI parece ser um mistério. Ela pode ser uma adaptação das *Responsability Charts*, propostas por Andersen, Grude e Haug (2009), originalmente na década de 80, ou uma adaptação de uma *Responsibility Assignment Matrix* que aparece citada em documentações da NASA já em 1977 (Facetation, 2015)

3. Partes Interessadas

	Diretor	Gerente	Caixa	Contador	Time Devel
Retirar Dinheiro	I	A	C	I	R
Depositar Dinheiro	I	A	C	I	R
Aprovar Empréstimo	AC	C	I	I	R
Cobrar Dívida	AC	C	I	C	R
Gerar Partidas Dobradas	I	I	I	CA	R

Tabela 3.5.: Exemplo de Matriz RACI

- uma parte interessada não tem nenhuma célula em branco, participando de todas as atividades do projeto?
2. A Análise Horizontal
- toda atividade tem pelo menos um responsável;
 - toda atividade tem um e apenas um aprovador;
 - atividades com mais de um responsável devem ser analisadas para duplicidade de trabalho;
 - atividades são realizadas e aprovadas em níveis hierárquicos corretos?
 - está correto onde o realizador é a mesma pessoa que o aprovador?
 - o número de pessoas sendo informada ou consultada é grande ou pequeno demais?

3.11. Conclusão

As partes interessadas são importantes por dois motivos principais: necessidade de identificar com quem levantar os requisitos e a necessidade do apoio delas durante o projeto.

Para isso um conjunto de técnicas está disponível. Algumas dessas técnicas, como a Matriz de Engajamento, podem causar problemas se divulgadas fora de um âmbito muito restrito no projeto, outras, como a Tabela RACI fazem parte de documentos ostensivos e precisam ser validadas pelas partes interessadas.

É importante que as partes interessadas sejam gerenciadas ao longo do projeto, seguindo procedimentos tradicionais ou ágeis, e o estado dessa gerência pode ser controlado tanto por técnicas tradicionais de gerência de projeto como por um modelo como o do SEMAT.

3.12. Exercícios

Exercício 3.1:

Descreva as partes interessadas do Sistema de Gestão Acadêmica da sua instituição de ensino.

Exercício 3.2:

3. *Partes Interessadas*

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita:

1. identifique todas as partes interessadas;
2. classifique-as quanto a:
 - a) poder $\times$ interesse;
 - b) poder $\times$ influência;
 - c) poder, legitimidade e urgência.
3. construa a Matriz de Engajamento.

O custo de uma coisa é a quantidade de vida que é necessária para ser trocada por aquilo imediatamente ou a longo prazo.

(Henry David Thoreau)

Conteúdo

4.1.	Conceituação Genérica de Valor	44
4.2.	Valor na Economia	44
4.3.	Valor em Marketing	46
4.4.	Priorização Baseada em Valor para o Cliente	51
4.5.	Valor Financeiro de Projetos	57
4.6.	Valor em Software	62
4.7.	Willingness to Pay e o Preço de Software	64
4.8.	Conclusão	65
4.9.	Exercícios	65

O objetivo de qualquer software é **entregar valor às partes interessadas**. O conceito de valor, porém, tem várias acepções. Mesmo que muitos métodos e autores se baseiem neste objetivo, raramente valor é definido. Assim, fornecedores e clientes se perguntam: o que é valor? Como posso medir valor?

A resposta pode depender da escolha de uma visão comum do significado de valor que depende de todas as partes interessadas do sistema, logo, é influenciada também por todo o contexto onde o projeto está inserido, incluindo o *background* das partes interessadas.

Este capítulo busca mostrar as várias facetas da ideia de valor. Começa com uma abordagem genérica e depois mostra alguns entendimentos da palavra valor na economia, no marketing e na área de software. Obviamente há alguma similaridade entre as definições, mas os diferentes

interesses entre as áreas também fornecem um conhecimento mais amplo do que é valor e de como as pessoas entendem o que é valor.

4.1. Conceituação Genérica de Valor

A palavra **valor** pode ser usada em vários sentidos (Cairncross, 1951, pg. 140):

- o **valor moral**, por exemplo, consideramos um *valor* a liberdade, a honestidade;
- o **valor estético**, por exemplo, damos *valor* a obra de Camões;
- o **valor em uso**, ou seja, ligado a utilidade, a como uma coisa atende as necessidades de alguém, como o valor da água, que é alto para a existência de vida, mesmo ela sendo barata;
- o **valor de troca**, por exemplo, o preço de uma casa a ser vendida, e
- o **valor ideal de troca**, o preço que um comprador imaginário pagaria pela casa.

Procuramos entender a acepção de valor com relação aos assuntos ligados aos negócios, em busca de entender **o que é valor para uma organização que está adquirindo ou desenvolvendo um software**. Isto não significa, porém, que a presença de atributos éticos e estéticos em um software não agregue valor, justamente o contrário. É possível, inclusive, que uma organização procure um produto que faça as mesmas funções que o produto que ela já possui, mas que seja mais agradável de usar, valor estético, ou garanta algumas propriedades ligadas ao valor ético, como privacidade.

Em todo caso, mesmo que a organização busque originalmente outro tipo de valor, no momento do projeto é mais provável que a interpretação dada ao conceito seja, na prática, ligada a fatores econômicos, financeiros ou legais, de acordo com a situação.

Da forma mais geral, é possível entender valor como **uma qualidade atribuída a algo ou alguém**, mas essa definição é muito ampla. Beleza, por exemplo, também é uma qualidade atribuída. Neste quadro em especial, valor é uma qualidade relacionada com a **utilidade que um produto ou serviço tem para os que o consomem ou usufruem**.

A princípio, valor econômico é “a importância que um indivíduo dá a determinado bem ou serviço, seja para uso pessoal ou organizacional, seja para a troca” (Holanda Ferreira, 1986).

Essa definição do Dicionário Aurélio é interessante para software, porque já diz que o valor é ligado a importância dada ao bem ou serviço que está sendo recebido para o uso, ou sendo produzido para a troca.

Nas próximas seções veremos definições de valor nas seguintes áreas:

- Economia;
- Marketing;
- Projetos, e
- Software.

4.2. Valor na Economia

As tentativas de definir algo que pode ser chamado de valor na Economia tem, historicamente, também relação com o entendimento do preço dos objetos e serviços.

4. Valor

Na Economia Clássica, John Smith, em 1776, primeiro reconhece que valor pode ter dois significados diferentes, um ligado a utilidade de um objeto particular, e outro ao poder de comprar outros bens. O primeiro seria o “valor de uso”, o segundo o “valor de troca” (Smith, 2003, Livro I, Cap. 4, para. 13)

Smith ainda discutiu valor, no sentido de preço, ou o preço relativo entre bens e serviços (Cairncross, 1951), com três abordagens (King e McLure, 2014), sendo que a terceira antecipou a teoria subjetiva de valor que apareceria mais tarde:

- o trabalho incorporado, que seria adequado às sociedades primitivas;
- a soma dos custos de produção, incluindo terra, capital e trabalho, mais adequado ao capitalismo, e
- a quantidade de trabalho e o incômodo de adquirir, ou que é poupado e pode ser imposto a outro, que é seu valor de troca (Strathern, 2003).

Podemos dizer que muitas empresas usam abordagens como essa para calcular o preço de seu software: calcular todos os custos de produção e somar um lucro, ou **markup**, sobre esse preço.

Também as organizações ou pessoas que estão pagando por um produto de software podem ser associadas à definição mais evoluída de Adam Smith: elas pagam para evitar algum incômodo.

Já em 1821, Ricardo (1996) propôs que para se saber o valor é necessário saber a utilidade, e que existiria um preço primário e natural, dado pela quantidade comparativa de trabalho necessário para a produção (King e McLure, 2014).

Novamente, a interpretação clássica de valor tem relação com a prática de desenvolvimento de software: utilidade para o usuário, custo de produção para o desenvolvedor.

4.2.1. Utilidade

Utilidade é o grau de satisfação que obtemos do consumo de bens e serviços (Krugman e Wells, 2013). Não é possível medir utilidade de forma prática, ou seja, dizer que algo tem 100 ou 2354 de utilidade para alguém. Porém, teoricamente, se discute a **função utilidade**, que é individual. Isso significa que cada pessoa tem uma satisfação diferente consumindo um produto, e não é possível comparar utilidade entre pessoas.

De acordo com o **princípio da diminuição da utilidade marginal**, cada unidade adicional de bens ou serviços adiciona menos à utilidade total do que a unidade prévia. Esse é um princípio genérico. A verdade é que existem produtos que têm utilidade marginal que aumenta, por exemplo, se você precisa de uma quantidade X para atingir um objetivo mínimo, a utilidade marginal não vai diminuir até você atingir X unidades, e pode até aumentar se for uma solução pior ter $X - 1$ unidades (Krugman e Wells, 2013). Um exemplo seria construir um muro de tamanho pré-determinado com tijolos, a utilidade dos tijolos não vai diminuir até atingir a quantidade necessária para o muro. Porém se você quer um muro que garanta privacidade, sem especificar a altura, a utilidade dos tijolos vai começar a diminuir quando certo tamanho é atingido, referente a uma privacidade necessária, até que a privacidade máxima seja alcançada e a utilidade marginal seja zero.

Software é um desses casos. Para um software funcionar, é necessária uma funcionalidade mínima que permita que ele comece a funcionar, porém após essa funcionalidade mínima, cada funcionalidade a mais vai seguir esse princípio, até o ponto que o cliente não mais vai querer pagar por uma funcionalidade, porque não vai valer a pena, já que o investimento vai ser maior que o retorno.

Utilidade é uma coisa importante no valor de produtos de software. Basicamente, em todas metodologias que usam o conceito de valor, o que se pode deduzir das explicações dadas é que **o mais útil em um certo contexto tem mais valor e deve ser priorizado**. Em software, utilidade pode ser um sinônimo de valor em alguns projetos.

4.2.2. Microeconomia Moderna e o Custo de Oportunidade

O **custo de oportunidade** indica “do que alguém deve desistir para obter o que deseja” (Greenlaw, Shapiro e Taylor, 2017), ou seja, indica a oportunidade perdida de se consumir ou usufruir outra coisa quando se consome ou usufrui de algo. É o “custo da próxima melhor alternativa” ao que está sendo consumido (Greenlaw, Shapiro e Taylor, 2017).

Segundo Krugman e Wells (2013), “o verdadeiro custo de algo é o seu custo de oportunidade”. Se alguém deseja um software, qual o verdadeiro custo disso? Isso envolve então o custo de oportunidade de uma segunda opção, por exemplo, não desenvolver o software e arcar com as consequências e investir o dinheiro.

Assim, novamente em software, o custo de oportunidade é um bom significado de valor. Perguntas que podem ser feitas, em caso de discussões de valor, estão ligadas alternativa de não fazer o produto, ou não corrigir um erro. E, no cálculo de um valor financeiro, o preço das outras ofertas que vão ser recebidas pelo cliente.

O conceito de valor na Economia, como tratado aqui, nos permite trabalhar com algumas ideias onde o homem é visto como um ser racional, que faz cálculos e toma a decisão de forma acertada baseada nesses cálculos. Porém o homem também é um animal emocional, e sua visão de valor envolve necessidades e desejos, como veremos na abordagem que o Marketing dá ao conceito de valor.

4.3. Valor em Marketing

O estudo de valor em marketing se inicia com o entendimento do que são **necessidades**, ou seja, “os requisitos básicos dos seres humanos”: água, comida, recreação, educação, etc. (Kotler e Keller, 2012). Necessidades são estados onde se sente uma privação (Kotler, Armstrong et al., 2017). Em uma empresa, comprar um software que emite nota fiscal segundo a legislação vigente é uma necessidade.

Necessidades se transformam em **desejos** “quando são direcionadas para objetivos específicos que podem satisfazê-las” (Kotler e Keller, 2012). Desejos “são a forma que as necessidades tomam quando moldadas pela cultura e personalidade individual” (Kotler, Armstrong et al., 2017).

Por exemplo, a fome gera uma necessidade de se alimentar, que pode se configurar como o desejo de um hambúrguer para um americano, ou um prato feito de arroz, feijão, bife e batata frita para um brasileiro.

Da mesma forma, dada uma necessidade de um tipo de software, uma organização pode desejar características específicas adicionais, como ser feita de código aberto.

As necessidades podem ser (Kotler e Keller, 2012):

- **declaradas**, que são ditas pelo cliente, mas não são necessariamente verdade;
- **reais**, que são o que o cliente realmente precisa;

4. Valor

- **não declaradas**, que são o que o cliente espera receber;
- **de algo mais** (*delight*), que são o que o cliente gostaria, e
- **secretas**, que são como o cliente deseja ser visto pela sociedade.

Kotler e Keller (2012) ainda lembram que responder apenas a necessidade declarada pode não ser o bastante, e isso é verdade especialmente **em projetos de software, onde muitas vezes o cliente nem mesmo sabe suas necessidades reais**.

Uma **demanda** é um “desejo por um determinado produto, suportado por uma capacidade de pagar”(Kotler e Keller, 2012). Muitos querem lagosta no almoço, mas só podem pagar por algo mais barato.

Em software, a questão financeira é importante. É possível cotar um projeto que descrito em um parágrafo como algo entre 30 mil reais e quatro milhões de reais, dependendo da extensão do produto e da capacidade de operação necessária, como servidores e previsão de milhões de usuários.

Uma demanda pode ter oito estados (Kotler e Keller, 2012) em relação a um, ou mais, produtos ou serviços:

1. **demanda negativa**, quando os consumidores não gostam do produto e podem até pagar para evitá-lo;
2. **demanda não existente**, quando os consumidores não sabem que o produto existe ou não tem interesse no mesmo;
3. **demanda latente**, quando os consumidores tem um necessidade grande que não é satisfeita por nenhum produto existente;
4. **demanda declinante**, quando os consumidores passam a comprar menos ou deixam de comprar um produto;
5. **demanda irregular**, quando existe uma variação na compra do produto, em pequena escala de tempo (horas) ou grandes (estações do ano, meses);
6. **demanda plena**, onde os consumidores estão comprando todos os produtos postos no mercado;
7. **demanda excessiva**, onde os consumidores gostariam de comprar mais produtos do que podem ser fornecidos, e
8. **demanda indesejada**, onde os consumidores são atraídos por produtos com consequências sociais indesejadas.

Compreendendo essa sequência que começa com a necessidade, passa para o desejo e chega a demanda, é possível então falar de valor.

Para o marketing, o valor é a “a somatória dos benefícios tangíveis e intangíveis proporcionados pelo produto subtraída da somatória dos custos financeiros e emocionais envolvidos na aquisição desse produto”(Kotler e Keller, 2013). O valor é então uma “combinação de qualidade, serviço e preço”(Kotler e Keller, 2013). As percepções de valor “aumentam com a qualidade e o serviço, mas diminuem com o preço”(Kotler e Keller, 2013).

Segundo Drucker (1974) o valor, para um cliente, é a satisfação de um desejo. Um dos objetivos do marketing é aumentar esse valor, gerando satisfação no cliente. A **satisfação** é então um “julgamento comparativo de uma pessoa sobre o desempenho percebido de um produto em relação as expectativas”(Kotler e Keller, 2013).

4.3.1. Os Elementos do Valor - B2C

Almquist, Senior e Bloch (2016) identificaram 30 elementos que fornecem valor para **consumidores**, i.e., pessoas, divididos em uma pirâmide¹ de quatro camadas². Essa pirâmide é apresentada a seguir em forma de lista.

- mais alta* • **Impacto social:** auto-transcendência (ajudar outros ou a sociedade de forma mais ampla).
- **Mudança de vida:** fornecer esperança, auto-atualização, motivação, herança para as próximas gerações, afiliação/pertencimento.
 - **Emocional:** reduzir ansiedade, recompensa pessoal, nostalgia, design/estética, representação de status ou aspirações, bem estar, valor terapêutico, diversão/entretenimento, aumentar a atração pessoal, fornecer acesso a outros itens de valor.
- mais baixa* • **Funcional:** poupar tempo, simplificar, fazer dinheiro, reduzir risco, organizar, integrar aspectos diferentes da vida, conectar com outras pessoas, reduzir esforço, evitar problemas, reduzir custo, qualidade, variedade de escolha, apelo sensorial, informar.



Figura 4.1.: Elementos do Valor B2C. Baseado em: Almquist, Senior e Bloch, 2016

Essa pirâmide pode ser comparada com a pirâmide de Maslow, apresentada na Figura 4.2, que mostra também as necessidades das pessoas em camadas cada vez mais abstratas. Sua aplicação é apropriada ao analisar valor em software que vai ser vendido para pessoas.

Os elementos podem ter importância diferente por pessoas e por indústria. Por exemplo, os elementos mais importantes para consumidores de *smartphones* são, em ordem decrescente (Almquist, Senior e Bloch, 2016): qualidade, reduzir esforço, variedade de escolha, organizar e conectar com outras pessoas³.

¹A pirâmide é uma construção usada para passar a ideia que os níveis inferiores são essenciais para os níveis superiores poderem ser alcançados

²Uma versão interativa pode ser acessada em <https://media.bain.com/elements-of-value/>, link válido em 9/2/2020

³Apesar de ser a missão original do *smartphone*, seu valor como conexão é só o quinto mais valorizado.

4. Valor

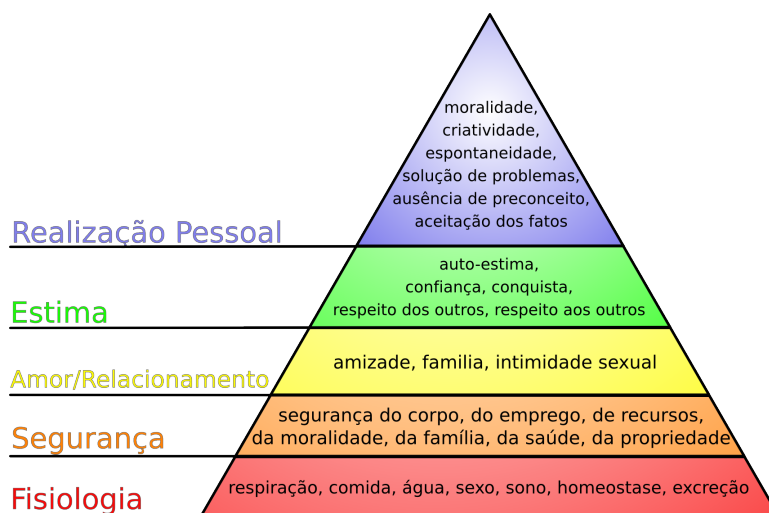


Figura 4.2.: Hierarquia das Necessidades de Maslow. Fonte: Wikipedia Commons por Felipe Sanchez (CC-BY-SA 3.0) e J. Finkelstein (GFDL)

4.3.2. Os Elementos do Valor - B2B

Mais tarde, Almquist, Cleghorn e Sherer (2018) desenvolveram modelo semelhante para negócios B2B, obtendo uma pirâmide um pouco mais complicada, com 40 elementos, onde há sub-níveis, representada na Figura 4.3 e na lista a seguir⁴. A importância de cada elemento novamente varia com a indústria. Essa segunda pirâmide é aplicável a software que vai ser vendido a organizações, porém leva em conta que as vendas são feitas para pessoas que participam das organizações. Os compromissos básicos são exigências para o vendedor.

mais alta • Valor inspiracional

- objetivo:
 - ◊ visão, ajuda o cliente a antecipar a direção do mercado;
 - ◊ esperança, dá aos clientes e usuários esperanças no futuro da empresa, e
 - ◊ responsabilidade social, ajuda o cliente a ser mais responsável socialmente.

• Valor individual:

- carreira:
 - ◊ expansão da rede (*network*), ajuda os usuários e colegas a expandir a rede profissional;
 - ◊ *marketability*, faz clientes e colegas mais *marketables* em seu campo, e
 - ◊ garantia de reputação, não atrapalha e pode aumentar a reputação do clientes no trabalho.
- pessoal:
 - ◊ design e estética, fornece produtos e serviços que são esteticamente agradáveis;
 - ◊ crescimento e desenvolvimento, ajuda usuários e colegas a crescer pessoalmente;
 - ◊ redução da ansiedade, ajuda clientes e outros na organização a se sentir mais seguros, e
 - ◊ diversão e vantagens, é agradável de interagir com ou dá recompensa de alguma forma.

• Valor da facilidade de fazer negócios:

- produtividade:

⁴Uma versão interativa pode ser acessada em <https://media.bain.com/b2b-eov/index.html>, link válido em 9/2/2020.

4. Valor



Figura 4.3.: Elementos do Valor B2B. Baseado em: Almquist, Cleghorn e Sherer, 2018

- ◇ poupar tempo;
- ◇ reduzir esforço;
- ◇ diminuir problemas;
- ◇ informação, e
- ◇ transparência, fornece uma visão clara da organização do cliente.
- operacional:
 - ◇ organização;
 - ◇ simplificação, reduz complexidade;
 - ◇ conexão, conecta organização e usuários com outros interna e externamente, e
 - ◇ integração, ajuda o cliente a integrar diversas facetas do negócio.
- acesso:
 - ◇ disponibilidade, garante que o bem ou serviço está disponível quando e onde necessário;
 - ◇ variedade, fornece uma variedade para escolha, e
 - ◇ configurabilidade, permite configurar o bem ou serviço de acordo com as necessidades do cliente.
- relacionamento:
 - ◇ responsividade, responde rápida e profissionalmente as necessidades da organização;
 - ◇ perícia, fornece *know-how* para o mercado ou indústria relevante;
 - ◇ compromisso, mostra que está compromissado com o sucesso do cliente;
 - ◇ estabilidade, é uma empresa(bem ou produto) estável no futuro previsível, e
 - ◇ ajuste cultural, se encaixa na cultura do cliente.
- estratégico:
 - ◇ redução de risco, protegendo o cliente;
 - ◇ alcance, permite o cliente operar em mais locais ou segmentos do mercado;

- ◇ flexibilidade, vai além dos bens ou padrões comuns para permitir customização, e
- ◇ qualidade de componentes, melhora a qualidade percebida dos produtos e serviços do cliente.

- **Valor funcional:**

- econômico:
 - ◇ redução de custos, e
 - ◇ aumento de receitas.
- desempenho:
 - ◇ qualidade do produto;
 - ◇ escalabilidade, e
 - ◇ inovação.

mais baixa ● **Compromissos básicos** (*table stakes*⁵)

- satisfazer especificações;
- preço aceitável;
- conformidade regulatória, e
- padrões éticos.

Ambas as pirâmides tentam ser exaustivas, mas foram criadas a partir de pesquisas de campo, o que quer dizer que podem existir outros elementos não detectados, porém isso não é esperado em geral.

Além disso, elas devem ser usadas em função dos elementos mais desejados em uma indústria ou cliente específico. Por exemplo, investigando a contribuição proporcional dos elementos para os clientes de infraestrutura de TI, com uma soma de 100%, (Almquist, Cleghorn e Sherer, 2018) encontraram que os três principais elementos são: qualidade do produto (7,8%), perícia (6,1%) e responsividade (5,5%). Poupar tempo, por exemplo, era décimo elemento na ordem e com proporção de apenas 3,0%. Na prática, a importância de cada elemento pode ser levantada com cada cliente.

4.3.3. Outras Técnicas

Tendo em vista que a finalidade do Marketing é engajar e gerenciar relações lucrativas com os clientes, é razoável que parte do trabalho de Marketing seja identificar o que é valor para o cliente, de forma que a organização possa atendê-lo com qualidade. Assim, muitas técnicas de Marketing se dedicam a isso e está além deste livro cobrir todas.

Outras técnicas interessantes provenientes do Marketing são o QFD (Franceschini, 2016), adequada tanto a software sob encomenda quanto a produtos a serem lançados no mercado, e a Estratégia do Oceano Azul (Kim e Mauborgne, 2005), adequada apenas a novos produtos a serem lançados no mercado.

4.4. Priorização Baseada em Valor para o Cliente

Em muito projetos é mais prático colocar os itens a serem realizados em uma lista priorizada do que dar um valor para cada item, sejam eles demandas do cliente, requisitos em português, casos de uso, histórias do usuário ou outros métodos.

⁵No linguajar de negócios em inglês, *table stakes* é o mínimo que você deve fazer para um mercado ou negócio específico.

A priorização é um processo para “determinar a importância relativa de informações”(IIBA, 2011). Ela permite várias abordagens:

- agrupamento, normalmente em poucas classes pré-determinadas;
- *ranking* ou ordenação;
- orçamento ou *time-boxing*, por meio da alocação de recursos finitos e
- negociação, em busca de um consenso.

4.4.1. Técnicas simples de ordenação

Algumas das técnicas que podem ser usadas para criar uma estimativa relativa de valor com clientes e outras partes interessadas são baseadas em criar uma pontuação ou separar os itens que têm o valor estimado em grupos de níveis de prioridade diferentes(Satpathy et al., 2016).

Métodos simples de priorização incluem:

- esquemas simples de pontuação, onde cada item do projeto analisado recebe um número entre uma variação pequena, como de 1 a 3 ou 1 a 5, ou uma rótulo como “Alto”, “Médio” ou “Baixo”, criando grupos ordenados;
- **dinheiro de brinquedo** ou **100-pontos** , onde é dada uma quantidade de dinheiro de brincadeira⁶, como 100 moedas, ou pontos, para o cliente associar aos itens do projeto, algumas vezes submetido a algumas regras, como não ter X itens com o mesmo valor (Lef-fingwell e Widrig, 1999), criando uma ordem parcial baseada na alocação de recursos, e
- ordenação de cartões contendo um item cada (J. Robertson e S. Robertson, 1998), mas também realizada em planilhas eletrônicas, ou em softwares como *Trello*.

4.4.2. Método MoSCoW

Na priorização **MoSCoW**, cada item é associado a um conceito entre “*Must Have*”, deve ter, “*Should Have*”, deveria ter, “*Could Have*”, poderia ter e “*Won’t Have*”, não irá ter (IIBA, 2011, p. 108), criando grupos.

Há uma diferença clara entre esse método e separar em grupos que determinam um grau de prioridade: no método MoSCoW os grupos tem um significado específico.

Entre os “*Must Have*” devem estar apenas os itens essenciais ao projeto, ou seja, aqueles que, não existindo, colocam o sucesso do projeto em risco. São fatores críticos do sucesso do projeto.

Nos dois grupos seguintes, “*Should Have*” e “*Could Have*”, cai a necessidade. Espera-se que contenham funcionalidades desejadas, mas que não sejam essenciais ao sucesso. No caso do “*Could Have*”, poderiam ser apenas melhorias eventuais ou mesmo o que é conhecido como *gold plating*, isto é, decoração que não soma realmente valor.

Finalmente os “*Won’t Have*” mostram requisitos que foram levantados, mas não são necessários ou mesmo podem causar problemas ao projeto.

Ao longo do tempo, principalmente em projetos iterativos, como os projetos ágeis, é comum um item mudar de grupo de uma iteração para outra. Por exemplo, quando todos os itens “*Must Have*” estão realizados, pode acontecer de outro itens passar a ser “*Must Have*”. Mesmo itens

⁶Facilmente obtida em uma caixa de Banco Imobiliário

que são indesejados (“*Won’t Have*”) em algum momento do projeto podem se tornar necessários, e o inverso também é verdadeiro.

Um exemplo de item indesejado que pode se tornar desejado em um projeto é a compra de um insumo ou equipamento muito caro, que passa a se tornar necessário em um certo ponto do projeto. Isso pode acontecer se a alternativa não atendeu às expectativas ou se o problema se tornou mais difícil do que planejado.

O método MoSCoW não é exclusivo da área de software, mas é parte importante de métodos de gestão de projetos e foi particularmente utilizado no método DSDM (Stapleton, 2003).

4.4.3. Análise de Kano

O modelo de Kano é um método para avaliar a reação emocional de clientes a características individuais de um produto. Kano detectou cinco respostas emocionais a essas características (Moorman, 2012):

- encantadores ou atrativos (*delighters*), “eu gosto disso”, que trazem satisfação se presentes, mas não trazem insatisfação se não estiverem presentes, normalmente porque são inesperadas e endereçam necessidades não reconhecidas;
- unidimensionais (*satisfiers*), que causam satisfação se estiverem presentes e insatisfação se não estiverem presentes;
- obrigatórios ou esperados (*dissatisfiers*), que causam insatisfação se não estiverem presentes, mas não causam satisfação se estiverem presentes;
- indiferentes (*indifferent*), ou
- indesejados.

Para classificar em uma dessas cinco respostas emocionais, são feitas duas perguntas ao cliente (Moorman, 2012):

- Como você vai se sentir se a funcionalidade estiver presente?
- Como você vai se sentir se a funcionalidade não estiver presente?

As respostas devem ser em uma escala ordinal:

- eu gosto disso;
- eu espero isso;
- eu sou neutro em relação a isso;
- eu posso tolerar isso, ou
- eu não gosto disso.

Finalmente, a classificação é feita cruzando as respostas na Tabela 4.1.

As categorias de Kano não são permanentes, e as características de um produto, ou tipo de produto, vão migrando de atrativas, para unidimensionais, e para obrigatórias, e podem até mesmo se tornar indesejadas. O tamanho do celular, por exemplo, sofreu essa mudança com o tempo.

J. Robertson e S. Robertson (2006) usam o método de Kano no seu método Volare. Nesse método o valor para o cliente é definido em duas perguntas são: quão satisfeito você se sentirá se esse requisito for implementado e quão insatisfeito você se sentirá se esse requisito não for implementado. segundo o casal, os graus de satisfação e insatisfação são a melhor mecanismo para saber que requisitos o cliente considera com maior valor. J. Robertson e S. Robertson,

4. Valor

Tabela 4.1.: Resolvendo a classe de Kano por meio do cruzamento de duas perguntas (Moorman, 2012)

		Questão Negativa				
Questão Positiva	Gosto	Gosto	Espero	Neutro	Tolero	Não Gosto
	Gosto	Conflito!	Atrativo	Atrativo	Atrativo	Unidimensional
	Espero	Indesejado	Indiferente	Indiferente	Indiferente	Obrigatório
	Neutro	Indesejado	Indiferente	Indiferente	Indiferente	Obrigatório
	Tolero	Indesejado	Indiferente	Indiferente	Indiferente	Obrigatório
	Não Gosto	Indesejado	Indesejado	Indesejado	Indesejado	Conflito!

porém, não se referem a Kano, mas a um autor que descreve suas técnicas. As notas dadas também são de 1 a 5.

O modelo de priorização do método Volare permite criar um gráfico de quatro quadrantes, como o da Figura 4.4. O mesmo princípio pode ser aplicado com as respostas do método de Kano.



Figura 4.4.

4.4.4. Prioridade e Consenso

Todas as técnicas explicadas nesta seção foram descritas como se o cliente tivesse uma opinião única, porém isto não é sempre verdade. Como projetos possuem muitas partes interessadas, cada parte interessada pode trazer uma visão diferente da realidade.

Para isso existem técnicas que visam atingir o consenso. A questão é que algumas técnicas atingem um ponto que não é considerado realmente consenso. No sentido estrito, consenso significa que todos concordaram com algo, porém algumas técnicas tentam resolver problemas de opiniões diferentes com votação, sendo possível ser uma votação por maioria simples, ou seja, metade dos votos mais um, ou por maioria qualificada, onde se exigem mais votos, como dois terços dos votos. Outras formas de eleição podem ser mais relacionadas com o conceito de consenso, como o voto em múltiplas rodadas, onde itens menos votados são eliminados em cada rodada. O voto em dois turnos é a versão mais simples desse método. Métodos de votação

preferencial tentam simular as múltiplas rodadas. Nesses métodos o eleitor ordena os candidatos de acordo com sua preferência, eliminando o candidato com menor preferência e recontando os votos, agora sem esse candidato, até que um candidato se eleja por maioria da primeira preferência não eliminada de cada eleitor (Adiel Teixeira de Almeida, 2019; Amy, 2000).

Algumas técnicas confundem o consenso com o consentimento. Consentir é concordar que a decisão foi tomada da melhor forma possível, mas não necessariamente concordar com ela.

Provavelmente a técnica mais famosa de atingir o consenso é o método de Delphi (Dalkey, 1967). Na sua versão original, eram usadas várias rodadas, feitas pelo correio ou meio a distância, onde especialistas deveriam avaliar uma afirmação dentro de um espectro linear, por exemplo, entre “não vai acontecer” e “muito provável”, ou entre uma data inicial e final de quando um evento era esperado. A cada rodada uma pessoa devia coletar todos os votos, fazer uma estatística, por exemplo mostrando a mediana e os quartis, e suas justificativas, e reenviar essa agregação para que todos os especialistas, a partir dos novos argumentos, refizessem sua avaliação. O Método de Delphi tem três características importantes: as opiniões são anônimas, o feedback é controlado e a resposta do grupo é estatística. A anonimização é importante para eliminar o efeito de dominância social de algumas pessoas, pela experiência ou por sua retórica poderosa (Helmer-Hirschberg, 1967). Existem variações do método de Delphi com objetivo específico, como o *Planning Poker*-(Cohn, 2005, p 56).

Também existem técnicas que permitem calcular o grau de consenso(Meskanen e Nurmi, 2006). Isso pode ser necessário, por exemplo, quando cada pessoa escolhe uma ordenação diferente. Nesse caso, havendo um valor específico que indique o consenso, é possível negociar até que ponto a discussão precisa continuar. Isso pode variar com o tipo de decisão, como por exemplo a discussão entre uma ordenação ou a escolha do melhor. T. J. Ross (2017, pags. 265–321) fornece métodos para ordenar e calcular diferentes tipos de consenso a partir de comparações par a par de preferência, mesmo que não sejam transitivas.

O **dot-voting** (Gray, Brown e Macanufo, 2010) é uma técnica típica do Design Thinking e dos métodos ágeis. Nessa técnica, os itens a serem priorizados são colocados em cartões em um quadro e cada participante recebe um número de votos, sendo 3 um número padrão na literatura, normalmente na forma de adesivos em forma de círculo (*dots*). Os participantes então colam seus adesivos, isto é, dão votos não identificados, nos cartões.

A técnica é muito usada, mas alguns defeitos já foram reconhecidos. Um deles é o efeito em cascata, um item já votado tende a chamar atenção e receber mais votos. Outro é que é comum o excesso de escolhas. Além disso, se os itens estiverem em níveis diferentes de uma hierarquia, é possível que um item menos desejado seja escolhido, porque o item mais desejado estava representado em duas possibilidades. Existem soluções, e várias delas vem do estudo do comportamento de eleitores em diferentes formas de eleições.

4.4.5. Usando mais de uma dimensão

Os métodos anteriores usavam apenas uma dimensão para determinar a prioridade, porém é possível utilizar outras dimensões. Um exemplo é usar uma dimensão que determine o benefício e outra que determine o custo, já que duas dimensões nos permitem novamente usar um diagrama 2D que pode ser dividido em quadrantes, sendo fácil de analisar. A dimensão que determina o benefício é sempre feita em função das partes interessadas principais, os clientes. Já a dimensão que determina o custo é melhor avaliada pelo equipe que executará o projeto, como a equipe de



Figura 4.5.: Gráfico com quadrantes para comparar prioridades usando benefício e custo.

desenvolvimento. Nesse caso, é necessário utilizar técnicas de predição, como o Planning Poker Cohn, 2004 ou Pontos de Função (IFPUG, 2010).

Definido o benefício e o custo, é possível analisar o resultado em um diagrama com 4 quadrantes, como o da Figura X.

Dada uma tabela com itens, onde tanto o benefício quanto o custo receberam um valor de 1 a 5, os itens podem ser posicionados no gráfico e uma ordem de prioridade pode ser determinada. 4.2. A Tabela 4.2 permite gerar o gráfico da Figura 4.6, onde as setas mostram uma sequência possível de priorização, AFHIEBJG, onde os itens C e D foram desprezados. Outra sequência possível seria AFHIEDJGC. Com essa visualização é possível melhorar a expectativa do cliente quanto a prazos de atendimento das suas reais necessidades.

Tabela 4.2.: Benefício e esforço para itens de um project backlog imaginário.

Funcionalidade	Benefício	Esforço
A	5	1
B	3	2
C	1	3
D	2	4
E	4	3
F	5	2
G	1	2
H	5	3
I	5	4
J	1	1

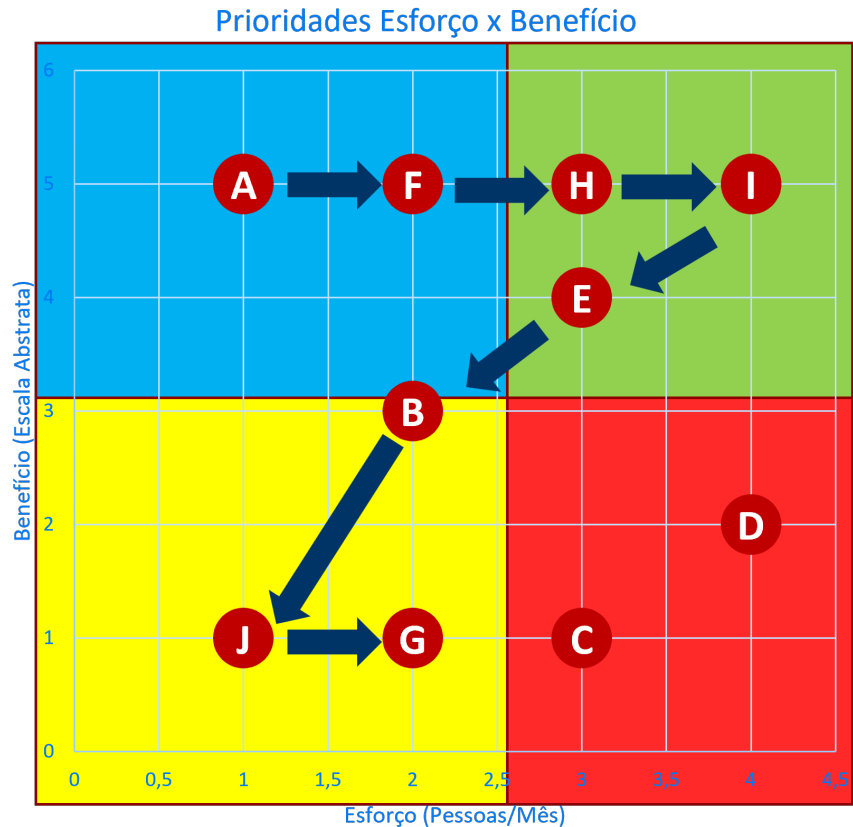


Figura 4.6.: Gráfico com quadrantes, determinados arbitrariamente pelo usuário, para os itens de projeto listados na Tabela 4.2. As setas mostram uma possível sequência de prioridade. Os itens C e D foram considerados desnecessários ao projeto.

4.4.6. Prioridade e Dependências

Suponha que um item A seja mais prioritário que um item B, porém A não possa ser usado sem que B esteja pronto. O que fazer?

Esse é um acontecimento comum, e mostra que há uma diferença entre as prioridades do negócio e as prioridades de um projeto que busca atender o negócio. Nesse caso, podem ser investigadas as seguintes soluções:

1. Alterar a forma como o software é desenvolvido, de forma que A funcione sem que B esteja pronto;
2. Dividir B em pedaços, de forma que apenas parte de B seja necessária para A funcionar;
3. Aceitar que B tem que ser feito antes de A.

4.5. Valor Financeiro de Projetos

Apesar de não ser o objetivo principal desse capítulo, é interessante observar as formas de calcular o valor financeiro de um projeto, ou subprojeto.

Antes de tratar das estimativas financeiras usadas em projetos, é importante conhecer os seguintes conceitos, como definidos por Dal Zot e Castro (2015):

4. Valor

- **principal** (P) é o capital inicial de um empréstimo ou uma aplicação financeira, também conhecido como **valor presente** (VP), **valor atual** (VA), **valor descontado**, ou por seu nome em inglês, *present value* (PV);
- **juro** (J) é a **remuneração** do capital emprestado, da parte de quem paga é uma despesa ou custo financeiro, da parte de quem recebe é um rendimento ou renda financeira;
- **montante** (S) é o **saldo**, ou **valor futuro** (VF), ou *future value* (FV), de um empréstimo ou de uma aplicação financeira. Pode ser também chamado de **valor de resgate**, ver equação 4.1.
- **prazo** (n) é o período de tempo que dura o empréstimo ou a aplicação financeira, podendo ser medido em dias, meses, anos...
- **prestação** ou **pagamento** (P,PGTO,PMT) se refere ao valor de pagamentos quando feitos em um número maior do que 1.
- **taxa de juros** (i) , ou simplesmente **taxa**, é o quociente entre juros e o principal, ver equação 4.2;

$$S = P + J \quad (4.1)$$

$$i = \frac{J}{P} \quad (4.2)$$

Analisando essas definições podemos detectar uma propriedade importante do dinheiro e que é a base da Matemática Financeira: o dinheiro muda de valor ao longo do tempo. Normalmente, como os cenários de inflação são muito mais comuns que os de deflação, R\$100,00 hoje compram mais que comprarão daqui a um ano e menos do que compravam um ano atrás. Por meio de cálculos como o do Valor Presente, como veremos no caso do Valor Presente Líquido, podemos comparar opções de investimentos ou empréstimos.

Calculadoras financeiras, como a HP-12C e o Excel permitem calcular facilmente o Valor Presente e o Valor Futuro com pagamentos, ou retiradas, fixas, em um período determinado, ao com uma taxa de juros fixas.

Por exemplo, se você tiver R\$10.000,00 por mês e puder aplicar em um projeto, deve procurar uma boa alternativa de investimento para poder saber se, financeiramente, vale a pena usar o dinheiro no projeto, ou se é melhor usar esse investimento. Com a fórmula do Valor Futuro (VF em português ou FV em inglês) esse cálculo pode ser facilmente feito no Excel, como mostrado na Figura 4.7.

Existem várias práticas que são utilizadas para calcular o valor financeiro de um projeto, entre elas (Satpathy et al., 2016):

- Custo Total de Propriedade (TCO - *Total Cost of Ownership*), que calcula todo custo de um serviço ou produto, incluindo a realização e a o uso do mesmo.
- Retorno do Investimento (ROI - *Return on Investment*), o método mais usado que é basicamente o lucro sobre o custo;
- Valor Presente Líquido - VPL (NPV - *Net Present Value*), onde se calcula a diferença entre a receita do projeto e seus custo ao longo do tempo, corrigido para o tempo presente, e
- Taxa Interna de Retorno - TIR (IRR - *Internal Rate of Return*) é a taxa de desconto de um investimento que torna o NPV zero para todos os fluxos de caixa de um projeto, não podendo ser calculado analiticamente, quanto maior for, mais desejável é o projeto (Hayes, 2019).

4. Valor

Investimento Alternativo	
Taxa de Juros	1,00%
Período	12
Pagamento	R\$ 10.000,00
Valor Presente	R\$ 0,00
Valor Futuro	R\$ 126.825,03

(a) Valores e formatação

Investimento Alternativo	
Taxa de Juros	0,01
Períodos	12
Pagamento	10000
Valor Presente	0
Valor Futuro	=FV(Taxa_de_Juros;Períodos;Pagamento;Valor_Presente;0)*-1

(b) Fórmulas

Figura 4.7.: Cálculo do valor futuro de um investimento de R\$10.000,00 por mês com uma taxa de juros de 1%.

Devemos ter cuidado com projetos com resultados subjetivos, que não podem ser medidos diretamente pelo valor financeiro. Essa questão não será discutida nesse livro.

4.5.1. Custo Total de Propriedade

Quando se analisa o custo de um sistema é normal falar de Custo Total de Propriedade, conhecido pela sigla em inglês TCO (Total Cost of Ownership). O TCO é o valor presente de todos os custos a serem feitos durante a vida de um sistema, produto, serviço ou equipamento (Anklesaria, 2008).

Por exemplo, se decidirmos trocar todo o parque computacional de uma empresa que usa Windows para Linux, mesmo que o custo do Linux seja zero, o TCO é bem alto, pois envolve o processo de troca, novos profissionais, treinamento, etc... Outro exemplo comum é o da compra de uma impressora. Seu TCO não envolve apenas o custo da impressora, mas também o custo do material consumível, quando uma certa produção é prevista. Por isso é que grandes empresas compram menos impressoras, porém impressoras maiores e mais caras, para baixar o TCO, já que o custo por impressão delas acaba saindo menor.

Para o software desenvolvido vale o mesmo conceito. Qual seu TCO? Envolve o preço pago pelo software mais tudo que vai ser pago para possibilitar a implantação e utilização do produto (instalação, cursos de treinamento, manutenção mensal, etc...). Espera-se, em uma decisão racional, que o TCO seja menor que o benefício trazido pelo sistema.

Um exemplo típico é a análise a ser feita na compra de uma impressora. Impressoras mais caras normalmente tem o preço por página impressa mais barato, porém só a partir de um ponto de uma quantidade grande de impressões.

A Tabela 4.3 e a Figura 4.8 mostram como isso acontece. Nelas vemos qual a melhor alternativa, considerando apenas o preço da impressora e do tipo de impressão, qual a impressão mais barata. É possível, por exemplo, somar o gasto de energia a essa conta.

4. Valor

Tabela 4.3.: Valores usados para calcular o TCO de uma impressora.

Impressora	Laser A	Ink A	Laser B
Preço	R\$ 880,00	R\$ 400,00	R\$ 2.100,00
Preço Toner	R\$ 480,00	R\$ 105,00	R\$ 539,00
Páginas Toner	1000	480	12.000

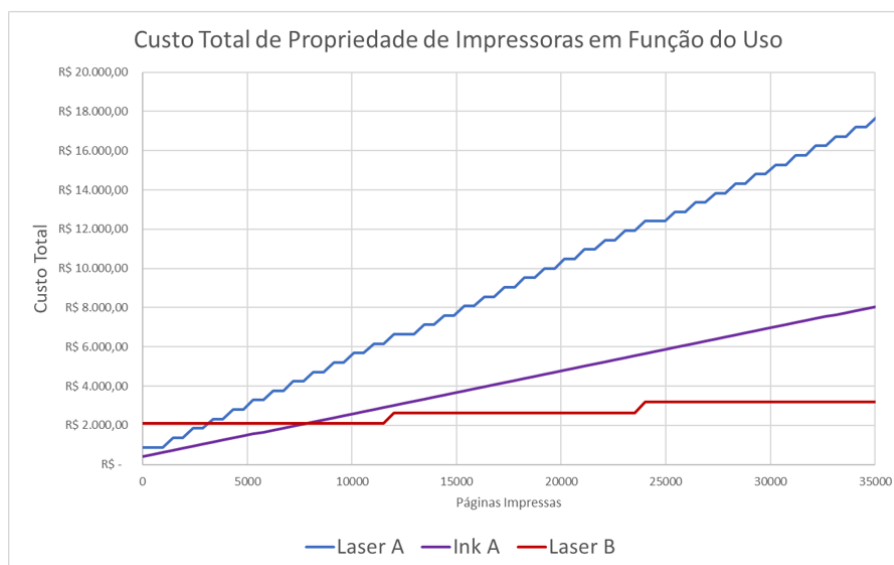


Figura 4.8.: Gráfico do TCO de impressoras em função de da quantidade de páginas impressas. Percebe-se que após aproximadamente 3500 páginas é melhor comprar uma impressora a laser mais cara, caso esse fosse o único fator de análise.

Anklesaria (2008) divide os custos em possíveis classes:

- **preço de compra**, e
- **custos internos**, que se dividem em
 - **custos de aquisição**, como transporte, inspeção, armazenagem;
 - **custos de uso**, como manutenção, garantias, e
 - **custos do fim de vida**, como custos para desmontar um projeto, etc.

4.5.2. Retorno do Investimento

O cálculo do ROI depende de haver uma previsão para a receita corrente R e para o custo C do projeto. Sua fórmula é:

$$ROI = \frac{R - C}{C} \quad (4.3)$$

O cálculo do ROI de um projeto que está previsto custar R\$ 500.000,00 e pretende obter como benefício um valor de R\$ 700.000,00 é:

$$ROI = \frac{700 - 500}{500} = \frac{200}{500} = 40\% \quad (4.4)$$

O ROI pode ser usado para comparações com taxas que o mercado financeiro paga por um investimento do mesmo valor. Assim, so valeria a pena fazer um projeto, financeiramente

4. Valor

falando, se ele resultasse em um retorno mais alto do que poderia ser obtido com o investimento em algum título do mercado.

Um cuidado ao analisar o ROI de um projeto é o fato de ser um valor percentual. Nesse caso, um projeto de R\$ 10.000,00 com um ROI de 150% pode parecer mais interessante que um projeto de R\$ 1.000.000,00 com um ROI de 1%, porém o segundo gera muito mais dinheiro que o primeiro.

4.5.3. Valor Presente Líquido

O Valor Presente Líquido é o valor atualizado para o presente de todos os gastos e ganhos de um projeto. Essa atualização é feita com uma taxa de desconto fixa. Isso significa que se você ganhar R\$ 100,00 daqui a um ano, e a inflação for de 5% ao ano, hoje isso tem o valor equivalente ao valor presente que, quando somado de 5%, resultasse em R\$100,00. A conta é $1/1,05$, que é aproximadamente R\$95,24.

A fórmula do VPL é(Kenton, 2019):

$$\text{VPL} = \sum_{t=1}^T \frac{C_t}{(1+i)^t} \quad (4.5)$$

onde C_t é o caixa líquido, receitas menos despesas, em um período t e i a taxa de desconto em um investimento alternativo, sendo T o número de períodos.

Esse cálculo é exemplificado na Tabela 4.4, onde usamos o método de calcular o saldo de cada período e atualizá-lo para o presente, com uma taxa fixa de 5%.

Período	Custo	Receita	Caixa	Valor Presente
0	R\$ 50.000,00	R\$ 0,00	-R\$ 50.000,00	-R\$ 50.000,00
1	R\$ 40.000,00	R\$ 52.500,00	R\$ 12.500,00	R\$ 11.904,76
2	R\$ 40.000,00	R\$ 52.000,00	R\$ 12.000,00	R\$ 10.884,35
3	R\$ 50.000,00	R\$ 61.500,00	R\$ 11.500,00	R\$ 9.934,13
4	R\$ 20.000,00	R\$ 31.000,00	R\$ 11.000,00	R\$ 9.049,73
5	R\$ 20.000,00	R\$ 30.500,00	R\$ 10.500,00	R\$ 8.227,02
6	R\$ 20.000,00	R\$ 30.000,00	R\$ 10.000,00	R\$ 7.462,15
7	R\$ 20.000,00	R\$ 29.500,00	R\$ 9.500,00	R\$ 6.751,47
Total				R\$14.213,63

Tabela 4.4.: Tabela de cálculo do Valor Presente Líquido, considerando saldos diferentes em cada período. Foi usada uma taxa de 5% a cada período. A fórmula para cada Valor Presente é $\text{Caixa}_i / (1 + \text{Taxa})^{\text{Período}}$

4.5.4. Taxa Interna de Retorno

A Taxa Interna de Retorno é um cálculo um pouco mais complicado, exigindo um processamento de tentativa e erro. Ela indica de quanto deveria ser a taxa de desconto para o VPL ser zero, isto é, o resultado do projeto não ser nem positivo, nem negativo. O processo de calculá-la é numérico, mas o Excel e calculadoras financeiras fornecem funções para isso.

4. Valor

A fórmula que relaciona o VPL com o TIR é (Hayes, 2019):

$$0 = \text{VPL} = \sum_{t=1}^T \frac{C_t}{(1 + \text{TIR})^t} - C_0 \quad (4.6)$$

onde C_0 é o investimento inicial total.

Quanto mais alto for o TIR, melhor será o resultado do projeto, sempre lembrando que como o cálculo é percentual, isso depende do valor do projeto. Uma maneira de entender isso é que você precisaria de uma aplicação melhor que a taxa para o projeto não valer a pena financeiramente.

4.6. Valor em Software

Conhecendo algumas definições na Economia e no Marketing, podemos compreender melhor outras definições que encontramos na Engenharia de Software.

Segundo Erdogmus, Favaro e Halling (2006), **valor** é a “diferença entre benefícios e custos de um bem, ajustados ao risco, em um certo momento de tempo”.

Ele é dirigido por valores individuais ou coletivos. As partes interessadas esperam obter algum **benefício**, seja ele tangível ou intangível, econômico ou social, monetário ou utilitário, ou ainda estético ou ético (Biffi et al., 2006).

Valor significa esse benefício final, que existe geralmente nos olhos do observados e admite múltiplas caracterizações (Biffi et al., 2006).

O risco, que ainda não tínhamos encontrado em outras definições, é importante, e é parte das técnicas de gestão de projeto modernas considerá-lo. O risco pode ser incluído inclusive indiretamente no custo total previsto de um projeto multiplicando sua probabilidade pelo seu impacto. Isso faz pouco sentido para um risco único, mas já faz sentido para o conjunto total de riscos de um projeto.

Assim, o valor de algo pode ser visto como uma fórmula matemática, de uma forma básica como:

$$\text{benefício} - \text{custos} \quad (4.7)$$

Ou, considerando os riscos:

$$(\text{benefício} - \text{custos}) \times \text{correção por risco} \quad (4.8)$$

A partir dessa interpretação, busca valor é buscar os maiores benefícios a um custo que valha a pena, com risco baixo. Esses benefícios devem ser identificados no início do projeto, e estar alinhados com o planejamento estratégico da empresa.

É importante frisar que o valor deve ser em relação a organização, não ao software propriamente dito. Assim, algumas funcionalidades que poderiam tornar o software muito mais poderoso podem, na prática, não ser úteis, não trazer benefícios, custar caro demais ou ter risco muito alto. E isso depende de cada projeto, ou cada aplicação do software.

Gane e Sarson (1979) introduziram um o acrônimo **IRACIS**, que identifica três formas de agregar valor (Gane e Sarson, 1979; Ruble, 1997):

- Aumentar Faturamento (*Improve Revenue*);
- Evitar Custos (*Avoid Costs*), e
- Melhorar Serviços (*Improve Services*).

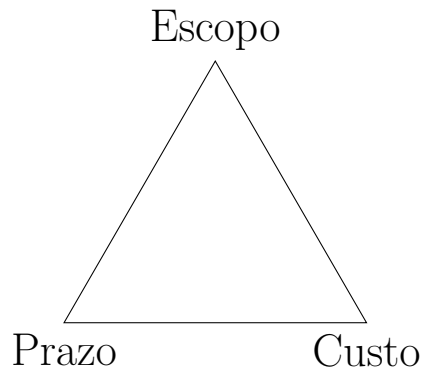


Figura 4.9.: O triângulo do valor para a maioria dos projetos.

4.6.1. Triângulo do Valor

É comum se falar, em projetos de software, que existem três parâmetros, que são qualidade (ou escopo), tempo e custo, formando um triângulo, mas o cliente só pode definir dois. Apesar de ser tratado como um brincadeira, é realmente impossível arbitrar esses três valores de forma independente. Ou seja, a afirmação “eu quero um software com a funcionalidade X em tempo Y e custo Z” não pode ser feita sem conhecimento de causa (BeSeen, 2015; Kerzner, 2017).

A princípio, isso pode ser entendido a partir do fato que Barry W. Boehm et al. (2000) mostraram que há uma relação entre a funcionalidade e qualidade desejada, e o esforço necessário para desenvolver o sistema, o que é óbvio. O que não era tão óbvio é que, além disso, dado um esforço, o tempo necessário para desenvolver o sistema também é determinado. Podem ser feitas certas modificações na previsão estatística, porém se forem feitas no sentido de diminuir o prazo necessário para a entrega do produto, elas implicam em risco maior. O assunto já foi tratado há muito anos por Brooks (1978) em seu livro adequadamente chamado *The Mythical Man-Month*.

Kerzner (2017) trata escopo, tempo e custo como restrições primárias que competem entre si em um projeto tradicional, e ainda apresenta restrições secundárias: reputação e imagem, risco, qualidade e valor. Já em projetos complexos, ainda segundo Kerzner, as restrições primárias passam a ser reputação e imagem, valor e qualidade, enquanto escopo, risco, custo e tempo passam a ser restrições secundárias. Podem aparecer também outros fatores, como segurança e estética, que aparece em parques de diversão, por exemplo.

4.6.2. Valor em Métodos Ágeis

É comum que métodos ágeis, como Scrum (Satpathy et al., 2016), proponham priorizar, de forma contínua, em ciclos curtos, os requisitos baseados no valor de negócio que é entregue aos clientes e usuários. O foco do Scrum, por exemplo, é tornar entregar valor, na forma de software executável, rapidamente, por meio de entregas incrementais a cada ciclo. Não há, porém, uma definição específica do significado da palavra valor, o que leva a necessidade de um entendimento maior do termo. Mesmo assim, cada história do usuário representa um valor a ser entregue ao cliente.

4.7. Willingness to Pay e o Preço de Software

Em geral, não faz parte do trabalho do analista de sistemas determinar o preço de um produto, porém essa é uma pergunta muito comum que é feita por jovens desenvolvedores aos mais experientes. O interessante é que, mesmo esses desenvolvedores experientes têm dificuldade de dar preço ao software a não ser pelo conceito de hora trabalhada. Esta seção apresenta um conceito da Economia relacionado a esse problema: “*willingness to pay*”

O conceito de *Willingness to Pay* (WTP), ou disposição a pagar, é amplamente utilizado em estudos de economia para determinar o preço máximo que um consumidor está disposto a desembolsar por um bem ou serviço. No contexto do desenvolvimento de software, compreender o WTP auxilia a fornecer uma base para decisões relacionadas ao preço de produtos, sejam eles pacotes comerciais, licenças ou soluções sob medida.

O WTP de um cliente por um software é influenciado por múltiplos fatores:

- **Valor percebido:** A percepção de como o software resolve problemas específicos ou melhora a produtividade. Estudos como os de Muñoz e Torres (2017) mostram que o valor percebido pode ser subjetivo e varia entre usuários individuais e empresas.
- **Características do cliente:** Empresas maiores podem estar dispostas a pagar mais por funcionalidades avançadas ou por soluções escaláveis (Nocera, 2019).
- **Alternativas disponíveis:** A existência de concorrentes com preços competitivos ou soluções *open source* pode reduzir o WTP (Pressman, 2020).
- **Sensibilidade ao preço:** Diferentes mercados demonstram diferentes sensibilidades ao preço. Por exemplo, Tang e Li (2020) observam que mercados emergentes frequentemente têm menor WTP.

Para estimar o WTP, empresas de software podem utilizar diversos métodos:

- **Pesquisas de mercado:** Questionários diretos para consumidores podem revelar o valor máximo que estariam dispostos a pagar (Anderson, 2014).
- **Análise de escolha:** Simulações onde consumidores escolhem entre diferentes pacotes com preços variados, como sugerido por Green e Srinivasan (2000).
- **Acompanhamento de comportamento:** Utilizar análises de comportamento real, como compras realizadas e preços pagos em promoções anteriores (A. Ross, 2018).

Embora o conceito de WTP seja valioso, existem desafios específicos no contexto de software:

- **Intangibilidade do software:** É difícil para os consumidores estimarem o valor de algo que não é físico (McConnell, 2020).
- **Atualizações e suporte:** O WTP pode ser afetado pela percepção de longo prazo sobre suporte técnico e manutenção (Nocera, 2019).
- **Diferenças culturais:** Mercados diferentes atribuem valores distintos às mesmas características (Tang e Li, 2020).

Empresas de software podem aplicar análises de WTP para:

1. Definir estratégias de preço diferenciadas para segmentos de mercado (*price discrimination*).
2. Otimizar pacotes e combinações de funcionalidades.
3. Planejar promoções e descontos sem comprometer margens de lucro (A. Ross, 2018).

4.8. Conclusão

Não existe uma ideia unificada do que é valor, principalmente quando o conceito é comparado entre áreas distintas.

Como os clientes do analista de sistemas, partes interessadas nos projetos, estão em várias áreas diferentes, mesmo em um só projeto, então podem interpretar valor de formas diferentes também.

Mesmo assim, **a principal função do desenvolvimento de software é produzir um software que tenha valor ao usuário**, seja esse valor calculado por uma fórmula ou um conceito mais qualitativo.

Para isso é importante definir o que é valor no início de qualquer projeto, de maneira consensual entre as partes interessadas. Isso é uma prática ágil que deve sempre ser adotada. Esse valor pode ser considerado de forma absoluta ou relativa.

A técnica MoSCoW (IIBA, 2011, p. 108), que dá valor de forma relativa, também é muito prática e pode ser associada a qualquer outra. A partir dela pode ser mais fácil fazer uma ordenação de uma grande lista de requisitos, por exemplo.

Já a técnica de Kano (Moorman, 2012; Sr., 2015), ou seu uso como proposto pelo Volare (J. Robertson e S. Robertson, 2006), permite trabalhar sobre a importância de cada item. Há, na verdade, uma relação entre o resultado da técnica de Kano e os grupos do método MoSCoW.

O uso dos Elementos de Valor (Almquist, Cleghorn e Sherer, 2018; Almquist, Senior e Bloch, 2016) pode ser uma solução apropriada para o processo de discussão com as partes interessadas em busca de definições mais precisas e completas do verdadeiro valor.

4.9. Exercícios

Exercício 4.1:

Procure outras definições de valor na Economia ou investigue mais sobre as definições dadas. Discuta a diferença entre essas definições e como elas mostram a evolução do entendimento do que é valor.

Exercício 4.2:

Busque outras definições de valor, principalmente de outras áreas que não foram citadas neste capítulo.

Exercício 4.3:

Vamos supor que Alice queira abrir um negócio. Para isso ela precisa de um software que permita registrar suas vendas. Sem esse software, por motivos legais, ela não pode vender. No mercado ela pode encontrar um software básico, que não atende todas as suas necessidades mas atende o requisito legal, por R\$ 100.000,00 por ano. Nesse caso, ela perderia a capacidade de mudar seus preços dinamicamente, o que faria com que ela deixasse de ganhar R\$ 240.000,00 por ano. Porém, a taxa de juros do mercado, se ela investir o dinheiro que usaria para o software novo é de 5% ao ano. Quanto ela pode pagar de aluguel, por ano, pelo software que atende completamente suas necessidades? **Solução:** A princípio ela pode pagar um valor que 5% seja

4. Valor

menor que R\$240.000, isto é R\$4.800.000,00. Se ela investir menos que isso vai ganhar mais no total, se investir mais que isso não vai ganhar tanto quanto ganharia se colocasse o dinheiro para render.

Exercício 4.4:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. Identifique qual pode ser o conceito de valor para o projeto que essa organização deseja.

Exercício 4.5:

Este trabalho foi inspirado no “Wake up in the morning game”⁷, adaptado para alunos de faculdade e um curso remoto.

- Listar atividades individualmente. (5 minutos)
 - Cada membro do grupo deve usar stick-notes para listar suas atividades de manhã, desde quando acordam até quando chegam à faculdade.
- Com o grupo, agrupar as atividades individuais em tópicos comuns. (5 minutos)
 - Usar stick-notes de outra cor ou tamanho.
 - Agrupar em função do objetivo.
- Ordenar as atividades no tempo, cronologicamente, da esquerda para direita. (5 minutos)
- Priorizar as os grupos de atividade segundo a técnica MoSCoW - de cima para baixo. (5 minutos)
- O que você faria se acordasse atrasado/tarde para uma reunião muito importante? (5 minutos)
 - Cada membro do grupo colocar estrelas nos stick-notes que não podem deixar de fazer.
 - Não tem limite de estrelas.
- Anotem onde o item estrelado muda a prioridade. (5 minutos)
 - Por exemplo, pode ser necessário pegar um táxi, o que antes seria considerado um “*Won't*”.
 - Pode não existir no seu caso.

⁷<https://www.agilesparks.com/blog/wake-up-in-the-morning-game/>

5W2H

I keep six honest serving-men
 (They taught me all I knew);
 Their names are What and Why
 and When
 And How and Where and Who.

(Rudyard Kipling)

Conteúdo

5.1.	As Perguntas Certas	68
5.2.	Usos do 5W2H	68
5.3.	Onde Está o Valor?	70
5.4.	Várias Perguntas para cada Palavra	71
5.5.	Exercícios	73

Este capítulo apresente uma técnica informal, porém extremamente útil, de descrição de fatos, coleta de informações, análise crítica análise de situação, ou registro de decisão conhecida como **5W2H**(Nakagawa, 2012).

A sigla 5W2H representa as iniciais de sete perguntas direcionadoras: "What"(O que será feito?), "Why"(Por que será feito?), "Where"(Onde será feito?), "When"(Quando será feito?), "Who"(Por quem será feito?), "How"(Como será feito?) e "How much"(Quanto custará fazer?).

Este método foi historicamente usado em várias disciplinas, incluindo engenharia de software, gestão de projetos e outras áreas que requerem planejamento estratégico e operacional. De tão simples, não existe uma referência formal, ou um método específico de como a usar, na área técnica, apesar de vários textos ou artigos a citarem ou proporem formas de usá-la e adaptá-la. Apesar de simples e informal, ela também é usada como base para vários modelos formais, de diferentes graus de complexidade, tanto de forma explícita como implícita.

5.1. As Perguntas Certas

Todo trabalho que vai ser feito precisa ser definido de alguma forma. Para isso é importante que se façam as perguntas certas, e que essas perguntas levem a uma definição precisa, ou seja, não ambígua, do que vai ser feito e quais as condições em que será aceito. Também, quando se descreve algo, é necessário que as informações dadas sobre o que está sendo descrito sejam suficientemente completas para serem úteis. Isso é verdade para a criação de notícias, histórias, acordos de ações a serem tomadas no final das reuniões e para a análise de sistemas, onde estamos na prática descrevendo um sistema de forma a permitir sua implementação.

Um quantidade grande de textos, desde daqueles que tentam ensinar crianças a contar histórias até aqueles que ajudam a profissionais a determinar uma arquitetura de informação empresarial, indica que é necessário responder as seguintes perguntas, conhecidas como 5W2H em inglês:

- Por que? (*Why*)
- O que? (*What*)
- Quem? (*Who*)
- Quando? (*When*)
- Onde? (*Where*)
- Como? (*How*)
- Por quanto? (*How Much*)

Existem variações. Uma lista mais básica só possui 6, exclui o “Por quanto?” (5WH). Listas maiores incluem termos como “ganhos” ou *Wins*, “Quantos”, e variações de “Quem” que são usadas em inglês, como “Whose”, em português “De quem?”, e “Whom”, que é uma versão passiva. Na prática, essa lista de 7 é a mais conhecida na área de negócios, onde os valores financeiros são sempre relevantes, e a versão 5WH, em áreas como jornalismo e educação.

A fonte original destas perguntas é o estudo da Ética, e mais tarde da Retórica, na Grécia antiga, e foram chamadas originalmente de **sete circunstâncias** por Aristóteles. Em latim as perguntas eram: *quis, quid, quando, ubi, cur, quem ad modum e quibus adminiculis*. A tradução mais próxima seria: quem, o que, quando, onde, por que, como e por que meios (Fedotov, 2019; Sloan, 2010).

Por que essas perguntas são boas? Primeiro, quando usadas realmente como perguntas em uma entrevista, são perguntas abertas, que não incluem a resposta dentro delas e obrigam quem responde a responder sem usar apenas uma resposta do tipo “sim” ou “não”. Também são perguntas que, a princípio, não induzem uma resposta. Por exemplo, perguntamos “Quem merece ganhar o prêmio?” em vez de “Alice merece ganhar o prêmio?”, deixando para o entrevistado decidir sem ser influenciado pelo nome de Alice na pergunta. Além disso, as perguntas, reunidas, mostram um grande quadro informacional sobre um tema.

5.2. Usos do 5W2H

Um das áreas de utilização, por exemplo, é o jornalismo. Se uma notícia vai ser dada, é importante saber responder, no texto da notícia, todas essas perguntas, ou a notícia estará incompleta.

Um exemplo é o seguinte primeiro parágrafo de uma notícia:

BRASÍLIA — Um dia depois de vetar a proibição para que as companhias aéreas cobrem pelo despacho de bagagens, o presidente Jair Bolsonaro justificou a decisão, afirmando, nesta terça-feira,

5. 5W2H

que “empresas menores alegavam que seria um empecilho” à operação no país e disse que não pretende enviar outra medida ao Congresso Nacional para restringir a cobrança apenas para as chamadas “low cost”, de baixo custo. (Extra, 2019)

As perguntas 5W2H para essa notícia estão respondidas na Tabela 5.1.

Pergunta	Resposta
Onde	Brasília
Quando	Um dia depois de vetar nesta terça-feira
Quem	Jair Bolsonaro
O que	vetar a proibição para que as companhias aéreas cobrem pelo despacho de bagagens
Por que	empresas menores alegavam que seria um empecilho à operação no país

Tabela 5.1.: Perguntas e respostas a partir da notícia.

Outro uso é para registrar decisões de uma reunião. Nesse caso se constrói, em uma folha de papel ou quadro branco, um quadro onde os itens de decisão são descritos e, para cada um deve ser registrado:

- O que deve ser feito?
- Quem é o responsável?
- Quando deve ser feito/entregue?
- Onde deve ser feito/entregue?
- Por que deve ser feito?
- Como vai ser feito?
- Quanto vai ser gasto?

Possivelmente, em alguns usos, podem não fazer sentido responder algumas perguntas. Por exemplo, pode não ser adequado responder onde algo vai ser feito se sempre será feito no escritório, ou como será feito se é algo óbvio.

Também é importante notar que a pergunta “Por que” vai ser feito, mesmo que pareça desnecessária, é um guia importante para a forma como algo vai ser realizado. Isto é, para um mesmo “o quê”, vários “como” diferentes podem ser possíveis tendo em visto o motivo da entrega. Ainda mais importante, para alguns autores, é que as pessoas saibam por que estão fazendo suas tarefas (Flores, 2012).

Na análise de sistemas, e na análise de requisitos, as perguntas mais respondidas são ligadas a **quem** deseja **o que** e **por que** deseja. Perguntas adicionais, feitas em algumas práticas ou que tem que ser respondidas nas fases iniciais de um projeto de software são, por exemplo: para **quando** deseja, **por quanto** deseja, **como** deseja que seja feito e **onde** deseja que funcione. Várias outras perguntas podem ser feitas e que também vão influenciar o projeto, como **quem** paga, **quem** vai ser afetado, até **quando** vai funcionar, etc.

5.2.1. O *Framework de Zachman*

O *Framework de Zachman* é uma metodologia de descrição de uma arquitetura de sistemas de informação baseado na técnica 5WH, i.e., não incluindo custos (Sowa e Zachman, 1992; Zachman, 1987). Várias das técnicas que usaremos nesse livro geram artefatos que pertencem

ao *Framework de Zachman*. Ele é um dos principais exemplos do uso direto da técnica 5WH em Engenharia de Software.

Sua principal caracterização é o quadro da Figura 5.1.

ENTERPRISE ARCHITECTURE - A FRAMEWORK™							
	DATA <i>What</i>	FUNCTION <i>How</i>	NETWORK <i>Where</i>	PEOPLE <i>Who</i>	TIME <i>When</i>	MOTIVATION <i>Why</i>	
SCOPE (CONTEXTUAL)	List of Things Important to the Business 	List of Processes the Business Performs 	List of Locations in which the Business Operates 	List of Organizations Important to the Business 	List of Events/Cycles Significant to the Business 	List of Business Goals/Strategies 	SCOPE (CONTEXTUAL)
Planner	ENTITY = Class of Business Thing	Process = Class of Business Process	Node = Major Business Location	People = Major Organization Unit	Time = Major Business Event/Cycle	Ends/Mean = Major Business Goal/Strategy	Planner
BUSINESS MODEL (CONCEPTUAL)	e.g. Semantic Model  Ent = Business Entity Rein = Business Relationship	e.g. Business Process Model  Proc = Business Process I/O = Business Resources	e.g. Business Logistics System  Node = Business Location Link = Business Linkage	e.g. Work Flow Model  People = Organization Unit Work = Work Product	e.g. Master Schedule  Time = Business Event Cycle = Business Cycle	e.g. Business Plan  End = Business Objective Means = Business Strategy	BUSINESS MODEL (CONCEPTUAL)
Owner							Owner
SYSTEM MODEL (LOGICAL)	e.g. Logical Data Model  Ent = Data Entity Rein = Data Relationship	e.g. Application Architecture  Proc = Application Function I/O = User Views	e.g. Distributed System Architecture  Node = I/S Function (Processor, Storage, etc) Link = Line Characteristics	e.g. Human Interface Architecture  People = Role Work = Deliverable	e.g. Processing Structure  Time = System Event Cycle = Processing Cycle	e.g. Business Rule Model  End = Structural Assertion Means = Action Assertion	SYSTEM MODEL (LOGICAL)
Designer							Designer
TECHNOLOGY MODEL (PHYSICAL)	e.g. Physical Data Model  Ent = Segment/Table/etc. Rein = Pointer/Key/etc.	e.g. System Design  Proc = Computer Function I/O = Data Elements/Sets	e.g. Technology Architecture  Node = Hardware/Systems Software Link = Line Specifications	e.g. Presentation Architecture  People = User Work = Screen Format	e.g. Control Structure  Time = Execute Cycle = Component Cycle	e.g. Rule Design  End = Condition Means = Action	TECHNOLOGY MODEL (PHYSICAL)
Builder							Builder
DETAILED REPRESENTATIONS (OUT-OF-CONTEXT)	e.g. Data Definition  Ent = Field Rein = Address	e.g. Program  Proc = Language Statement I/O = Control Block	e.g. Network Architecture  Node = Address Link = Protocol	e.g. Security Architecture  People = Identity Work = Job	e.g. Timing Definition  Time = Interrupt Cycle = Machine Cycle	e.g. Rule Specification  End = Sub-condition Means = Step	DETAILED REPRESENTATIONS (OUT-OF-CONTEXT)
Sub-Contractor							Sub-Contractor
FUNCTIONING ENTERPRISE	e.g. DATA	e.g. FUNCTION	e.g. NETWORK	e.g. ORGANIZATION	e.g. SCHEDULE	e.g. STRATEGY	FUNCTIONING ENTERPRISE

© 1986 - 2005 John A. Zachman, Zachman International

Figura 5.1.: O Framework de Zachman fornece uma visão da arquitetura empresarial a partir de 6 dimensões definidas pelo 5WH. Reprinted with permission by John A. Zachman, Zachman International.

5.3. Onde Está o Valor?

Tendo em vista que a proposta é **desenvolver software tendo em vista entregar valor ao cliente**, onde está o valor no método 5W2H?

Normalmente a resposta é razoavelmente simples: o valor está normalmente caracterizado, de alguma forma na resposta a pergunta “por que?” feita a um requisito, pois a justificativa de um requisito indica o motivo pelo qual ele é importante.

É possível que, para descobrir o verdadeiro valor, seja necessário fazer essa pergunta mais de uma vez, isto é, perguntar por que a resposta para a pergunta “por que” foi a dada, e então perguntar de novo até que se alcance uma resposta que realmente caracterize o valor. Essa técnica é conhecida como **5 Por Ques** (Serrat, 2017).

Algumas vezes também é possível que a resposta necessite de perguntas do tipo *O que*, ampliando o conceito tanto do “por que” quanto do “o que”. Por exemplo, se em algum momento a resposta for “preciso dessa função por causa de uma lei”, a próxima pergunta poderia ser “O que (ou qual) é essa lei?”.

Perguntar por que sucessivamente ou perguntar o que sucessivamente, ou misturar as duas seqüências, é uma prática chamada *Squeeze and Stretch* Higgins (1994).

5.4. Várias Perguntas para cada Palavra

Algumas perguntas podem ser feitas mais de uma vez. Por exemplo, “Quem?” podem se referir ao agente ou ao paciente de uma ação, ou a um observador. A pergunta “Quando?” pode se referir ao quando começou ou quando acabou o evento, ou quando ele foi planejado.

Cabe ao profissional entender como usar essas possibilidades ao seu favor. As subseções a seguir mostram algumas opções que podem, e na maioria das vezes devem, ser investigadas em um projeto, sem esgotar as possibilidades.

Além disso, como as práticas de desenvolvimento de software tratada nesse livro podem ser usadas desde o início do projeto, para servir de especificação do mesmo, várias perguntas podem ser feitas não só sobre o produto, mas também sobre o projeto.

5.4.1. Perguntas sobre o que

Descobrir o que é a tarefa principal da análise. Basicamente todo este livro fala sobre isso.

Normalmente as perguntas do tipo “o quê” caracterizam mais o produto da análise que o projeto propriamente dito.

- O que será feito?
- O que não será feito?
- O que é necessário?
- O que é suficiente?
- O que é insuficiente?

5.4.2. Perguntas sobre quem

As perguntas sobre “quem” são muito importantes em várias fases do projeto. No início porque precisamos identificar as partes interessadas (Capítulo 3). Depois porque cada funcionalidade do sistema vai atender a algum usuário.

Uma das técnicas muito usadas em gerência de projetos é, para cada atividade, classificar as partes interessadas de acordo com suas responsabilidades, criando uma matriz conhecida como Matriz RACI (PMI, 2017). Essa matriz contém pacotes de trabalho ou atividades nas linhas e partes interessadas nas colunas, sendo que cada célula deve indicar se a parte interessada realiza, aprova, é consultada ou é informada.

- Quem deve realizar?
- Quem deve aprovar?
- Quem deve ser consultado?
- Quem deve ser informado?
- Quem deve estar envolvido?
- Quem pode ser afetado?
- Quem pode se beneficiar?
- Quem pode ser prejudicado?

- Quem deve verificar?
- Quem deve validar?
- Quem deve homologar?
- Quem deve pagar?
- Quem deve usar?
- Quem deve operar?
- Quem deve manter?

5.4.3. Perguntas sobre quando

Geralmente as perguntas do tipo “quando” tem relação com prazos. Elas podem se referenciar tanto ao negócio quanto ao projeto, ou seja, existem tempos do negócio, como o prazo para entregar um relatório, e tempos do projeto, como o prazo para entregar uma função no software.

- Quando deve ficar pronto?
- Quando deve iniciar a execução?
- Quando vai acontecer?
- Quando será tarde demais?
- Quando os recursos estarão disponíveis?
- Quando devem ser as entregas?
- Quando deve ser aprovado?

5.4.4. Perguntas sobre onde

Novamente esse tipo de pergunta tem relação com diferentes assuntos. Um é o local do mundo real, por exemplo, um software pode ser usado em uma fábrica, mas não no escritório de uma organização. Outro é em relação a infraestrutura de TI, o software pode rodar na nuvem, em um servidor da empresa ou em celulares. Finalmente, em relação ao projeto, um software pode ser feito em um laboratório ou por uma equipe espalhada pelo mundo.

- Onde será feito?
- Onde será testado?
- Onde será homologado?
- Onde será instalado?
- Onde será utilizado?
- Onde será mantido?

5.4.5. Perguntas sobre como

Geralmente a fase de Análise, objetivo deste livro, não discute muito o “como”, sendo mais preocupada com o “o quê”, sendo o “como” mais tratado na fase de projeto. Mesmo assim, várias perguntas desse tipo já tem que ser respondidas pela análise, tanto em relação ao produto como ao projeto de construí-lo.

- Como será feito?
- Como será utilizado?
- Como será comercializado?
- Como será corrigido no futuro?
- Como sua operação será gerenciada?

5.4.6. Perguntas sobre por que

Apesar de parecer que os porquês não influenciam a execução do projeto, a verdade é que considerá-los traz uma mudança importante na visão que o analista vai ter da solução. Os porquês são na verdade a justificativa, definem valor e são a orientação do caminho a ser seguido a cada decisão.

- Por que aconteceu?
- Por que será feito?
- Por que foi pedido?
- Por que foi aprovado?
- Por que será aprovado?
- Por que existem resistências?
- Por que alguém é a favor?
- Por que alguém é contra?
- Por que a funcionalidade é desejada?
- Por que o requisito foi pedido?

5.4.7. Perguntas sobre quanto

Não existe projeto sem orçamento. Mesmo que você vá trabalhar para si, ou de graça, há um custo em horas e outros recursos necessários. Por isso essas perguntas são normalmente respondidas quando todas as outras já foram respondidas, ou, ao contrário, são respondidas antes porque vão servir de limites ao projeto, limitando as respostas aceitáveis para as outras perguntas.

- Quanto custará fazer?
- Quanto custaria não fazer?
- Quanto será obtido em benefício quando feito?
- Quanto será necessário do recurso X¹?
- Quanto tempo será necessário?

5.5. Exercícios

Exercício 5.1:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da visita, analise as respostas dadas as perguntas 5W2H e veja se elas são adequadas. Escreva uma lista de perguntas adicionais, sempre no formato 5W2H a serem feitas para cada personagem.

¹ pessoas, funções no projeto, equipamentos, etc.

Qualidade

Quality is everyone's responsibility.

(William Edwards Demming)

Conteúdo

6.1.	Tipos de Qualidade	77
6.2.	Dimensões da Qualidade	77
6.3.	Qualidade é Sempre por um Ponto de Vista	82
6.4.	Qualidade de Produto e Qualidade de Processo	82
6.5.	Falhas e Defeitos	83
6.6.	Quanto Custa a Qualidade	83
6.7.	Gestão da Qualidade	84
6.8.	Exercícios	85

Toda as áreas da Engenharia de Software têm como pedra basilar a Qualidade. Tratada de forma específica, a **Qualidade de Software** é uma das áreas do conhecimento do SWE-BOK(IEEE Computer Society, 2014a), onde é dividida conceitualmente de acordo com o mapa mental mostrado na Figura 6.1.

Pressman e Maxim (2014), por exemplo, tentam representar isso com um diagrama piramidal que é de certa forma base para seu livro de Engenharia de Software. Recriado na Figura 6.2, o diagrama mostra como o foco em qualidade suporta os Processos de Software, que por sua vez suportam os Métodos que suportam as Ferramentas.

Como já está claro o que é software, é importante também entender o que quer dizer qualidade. Qualidade é um área de conhecimento que aparece, por necessidade, em muitas outras áreas. Um dos principais autores que discute qualidade, Juran, apresenta duas definições, a primeira sendo “a característica dos produtos que atendem as necessidades dos clientes e, assim, proporcionar a satisfação do mesmo” (Defeo e Juran, 2010), e a segunda “a ausência de defeitos” (Defeo e Juran,

6. Qualidade

2010). A primeira é uma definição focada no uso, a segunda focada no produto em si. Essas duas abordagens são comuns e adotadas na ISO 25010, a norma internacional de qualidade de software (ISO, 2011).

Várias são as perspectivas de entender a palavra qualidade, o que leva os autores muitas vezes optar por mais de uma definição. O SWEBOK, por exemplo, diz que o termo Qualidade de Software pode se referir a “as características desejadas de produtos de software, a extensão em que um produto de software em particular possui essas características e aos processos, ferramentas e técnicas que são usadas para garantir essas características” (IEEE Computer Society, 2014a).

A International Standard Organization (ISO) também tem grande preocupação, em suas normas, com a questão da qualidade. As normas genéricas de qualidade estão na família ISO 9000. Na norma ISO 9000:2015 **qualidade** é definida, de forma bem geral, como “o grau em que um conjunto de características cumpre os requisitos” (ISO, 2015b).

Especificamente sobre Qualidade de Software, existiam as famílias de normas **ISO 9126** e que foram substituídas pelas normas **ISO 25000**, que definem qualidade como “a capacidade do produto de software em satisfazer as necessidades implícitas e explícitas quando usado em condições específicas”(ISO, 2001, 2011).

Segundo o SWEBOK, a área de conhecimento Qualidade de Software trata das questões estáticas de qualidade, enquanto a área Teste de Software trata das questões dinâmicas. A maioria dos autores não vê essa divisão, pelo menos não de maneira tão forte, e a maioria dos congressos sobre Qualidade de Software possui muitos artigos sobre Testes. Na verdade, é possível dizer que os autores do SWEBOK quiseram chamar a atenção da área de Teste de Software colocando ela a parte, mas sem deixar de referenciar que o objetivo dos testes é alcançar qualidade(IEEE Computer Society, 2014a). Neste texto as áreas de Qualidade e Teste serão vistas em momentos distintos.

6. Qualidade

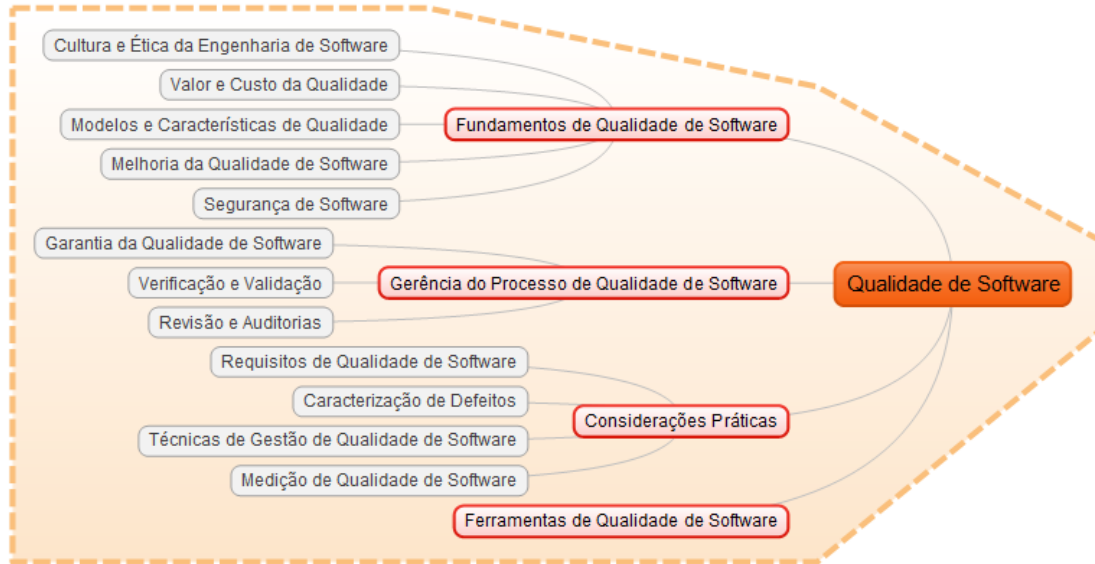


Figura 6.1.: Mapa Mental da organização da área do conhecimento Qualidade de Software no SWEBOK 3.0. Fonte:(IEEE Computer Society, 2014a)

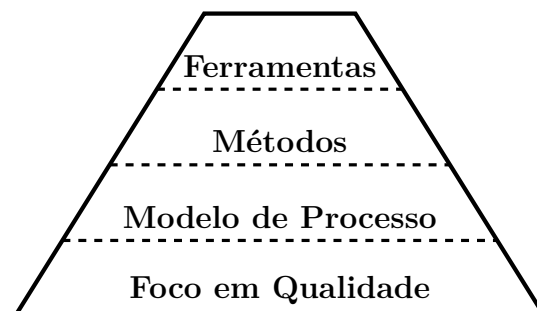


Figura 6.2.: A Qualidade é a pedra basilar da Engenharia de Software. Fonte:(Pressman e Maxim, 2014)

6.1. Tipos de Qualidade

Garvin (1984) apresenta 5 significados diferentes que damos a qualidade:

1. **Abordagem transcendental:** onde a qualidade é um atributo global que não pode ser definida precisamente. É a “qualidade sem um nome”, que é reconhecida pela experiência total que uma pessoa tem com o que está sendo avaliado. Seria um conceito primitivo, que não pode ser definido. Um problema dessa abordagem é justamente essa indefinição formal, pois a análise detalhada e objetiva não é possível, sendo uma visão subjetiva e pessoal. Alguns autores usam o acrônimo **QWAN**, para “Quality Without a Name”.
2. **Abordagem baseada no produto:** nesse caso, a definição de qualidade supõe que ela pode ser precisamente definida e medida, a partir de propriedades possuídas pelo produto, pela presença de uma quantidade de certo atributo, como quantidade de gordura no leite. Essa abordagem permite construções da qualidade em diferentes dimensões que devem ser agregadas para um resultado final. Um problema dessa abordagem é da importância de cada dimensão. Nesse caso, é fácil avaliar cada dimensão separadamente, por meio de critérios objetivos, mas é mais difícil de agregar todas essas avaliações. Além disso, supõe que a qualidade não está ligada a experiência, uso ou usufruto do produto, mas sim diretamente a característica do mesmo.
3. **Abordagem baseada no usuário:** onde o que importa é o atendimento as necessidades ou desejos de diferentes usuários. Um problema dessa abordagem é a dificuldade de agregar a visão de vários usuários. Desse modo, mesmo que o objeto não cumpra exatamente a especificação, ainda pode atender o usuário.
4. **Abordagem baseada na fabricação:** onde o que importa é a conformidade com os requisitos definidos para o produto, ou seja, a ausência de desvio em relação a uma especificação. Muito aplicável a processos fabris onde certas propriedades podem ser medidas diretamente e produtos podem ser recusados. Um problema dessa abordagem é que apesar de entregar produtos dentro da especificação, não questiona se a especificação atende o cliente.
5. **Abordagem baseada em valor:** vê a qualidade em função de custos e preços, em busca de produtos que forneçam desempenho a um preço aceitável ou conformidade a um custo aceitável.

6.2. Dimensões da Qualidade

Qualidade é um conceito multidimensional. Um exemplo bastante simples que podemos usar no Brasil é a avaliação das escolas de samba. Decidir qual a melhor escola é como decidir qual a que apresentou a melhor qualidade. Existem prêmios, como o Estandarte de Ouro, que são dados de uma forma holística, pelo voto na melhor escola. Porém, na competição oficial, o resultado é avaliado em vários quesitos: Enredo, Harmonia, Bateria, etc. Assim, para calcular a melhor escola são usadas várias dimensões de qualidade. Garvin (1984), ao mesmo tempo que descrevia as visões de qualidade, propôs também 8 dimensões básicas para um arcabouço para pensar a qualidade de um produto: desempenhos, capacidades, confiabilidade, conformidade, durabilidade, manutenibilidade, estéticas e qualidade percebida.

A visão dimensional evoluiu. Hoje as normas de qualidade ISO e a maioria dos autores adotam a visão de qualidade com dimensões. Isso normalmente também acaba resultando em uma visão hierárquica da qualidade de software. Porém modelos iniciais para Qualidade de Software, como o de McCall, que aparece na Figura 6.3, não eram hierárquicos (McCall,

6. Qualidade

Richards e Walters, 1977). De certa maneira, os modelos hierárquicos são mais simples que os não hierárquicos, porém podem perder certa representatividade. As abordagens atuais também tentam ser, pelo menos minimamente, exaustiva, isto é, apresentam a lista completa de dimensões de qualidade esperadas para um tipo de produto, como Software. Mesmo assim, é claro que produtos específicos podem exigir especializações dos modelos genéricos de qualidade, por exemplo, um modelo de qualidade para software educacional pode exigir mais dimensões ou detalhamento das já existentes para software.

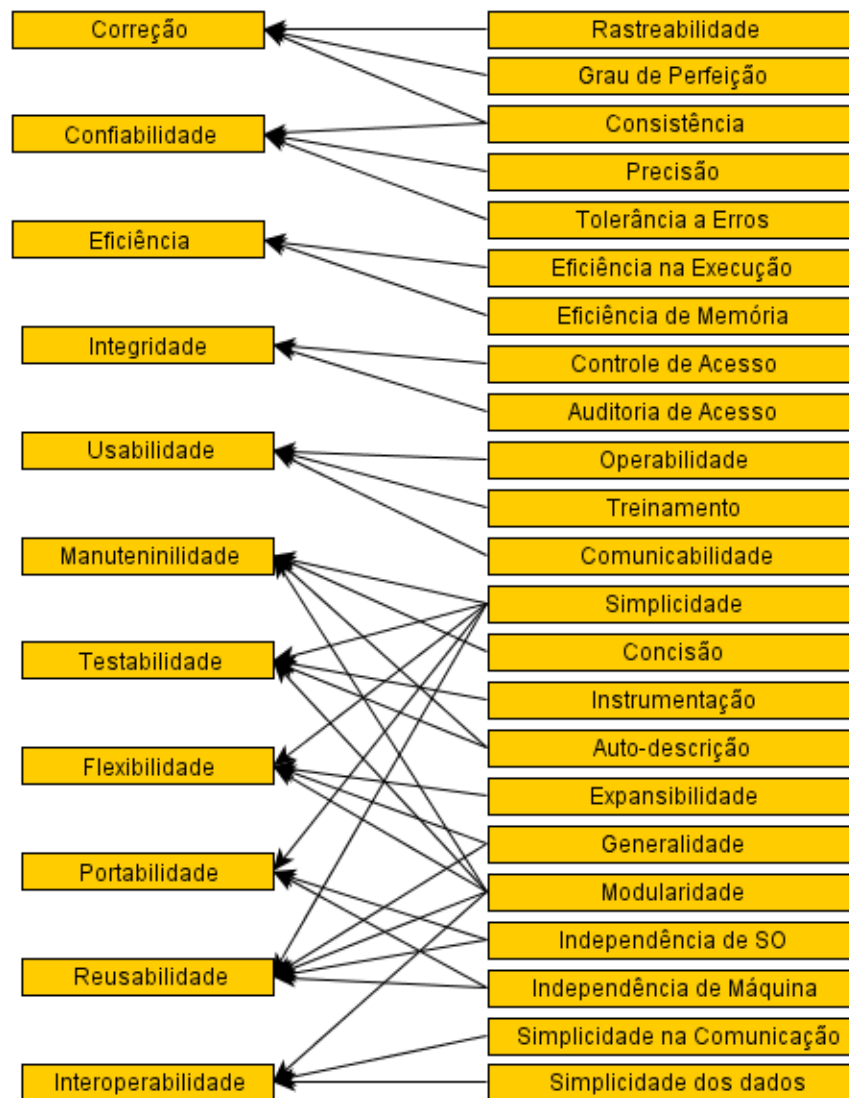


Figura 6.3.: Modelo de Qualidade de McCall. Fonte:(McCall, Richards e Walters, 1977)

Um modelo de qualidade não é composto apenas pela organização hierárquica de suas dimensões de qualidade, mas também por um forma de pensar que gera o modelo. O Modelo Fuzzy de Qualidade de Software, descrito na Figura 6.4, por exemplo, discute a organização hierárquica a partir de objetivos de software, e dois tipos de relações, as lógicas, que relacionam conceitos, e as quantitativas, que relacionam as formas de agregação do conceito. Além disso, define que existe um nível mensurável na hierarquia de dimensões, o dos critérios, que precisam ter processo de avaliação definido(Belchior, 1997).

6. *Qualidade*

Já o modelo de qualidade da norma **ISO 25010**, atualizado no momento da escrita deste texto, é um modelo hierárquico dividido em duas partes, Qualidade do Produto e Qualidade em Uso, representados, respectivamente, pelos mapas mentais das Figuras 6.5 e 6.6(ISO, 2011).

6. Qualidade

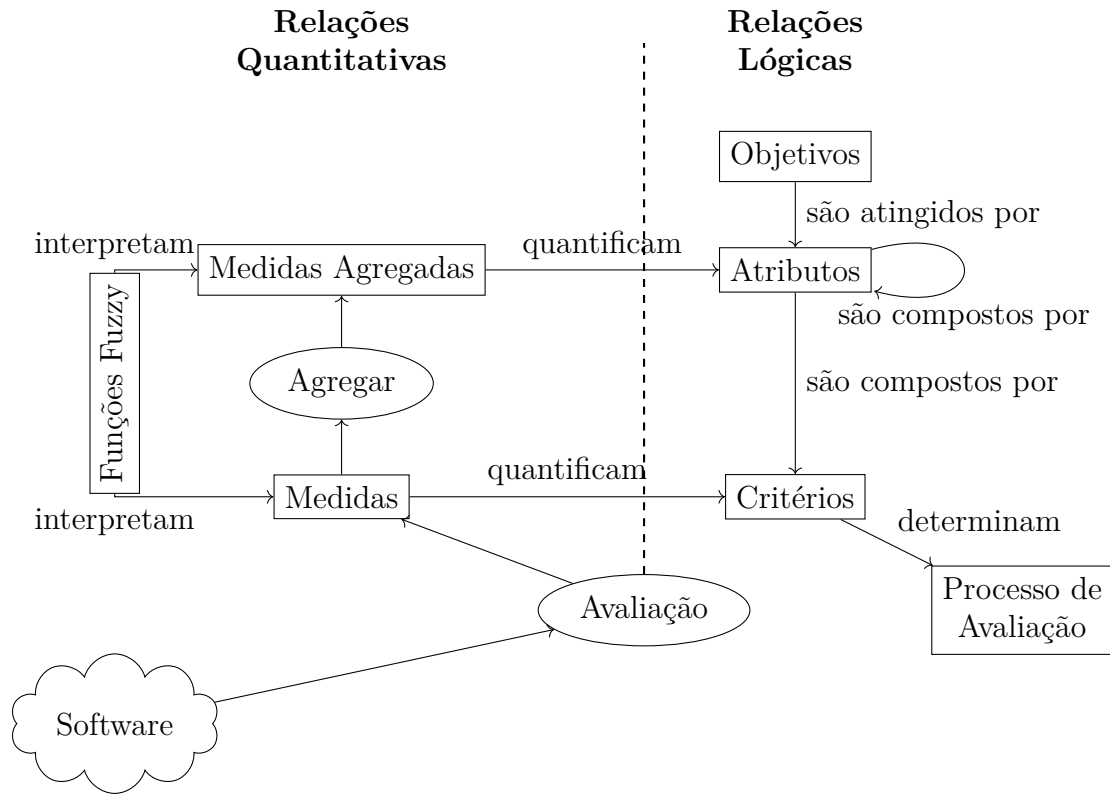


Figura 6.4.: O Modelo Fuzzy de Qualidade de Software. Fonte:(Belchior, 1997)

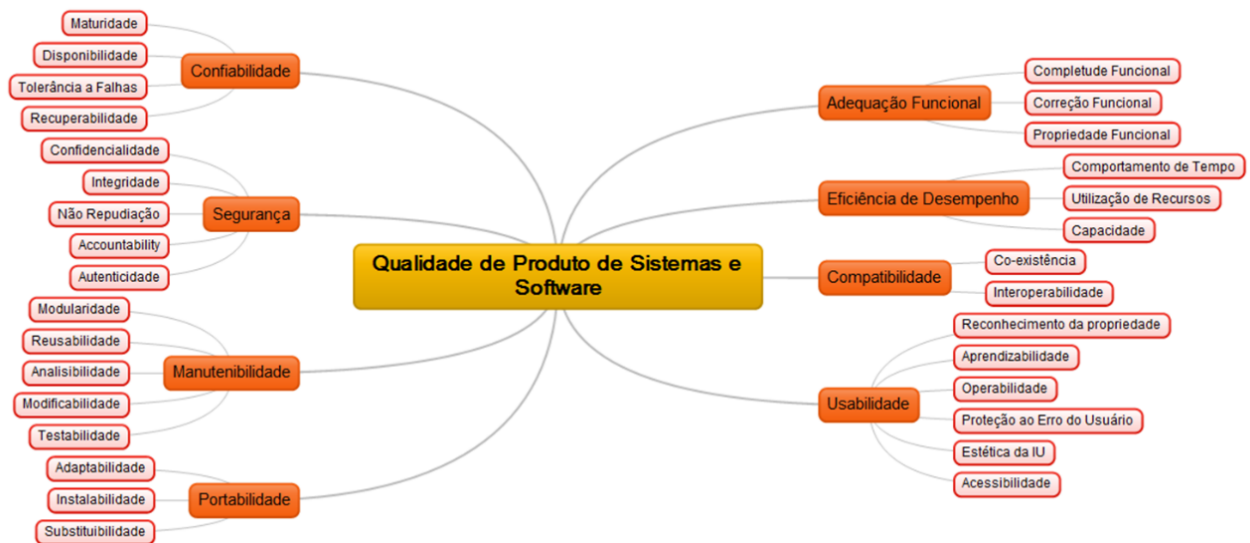


Figura 6.5.: Modelo de Qualidade de Produto de Software da norma ISO 25010. Fonte:(ISO, 2011)

6. Qualidade

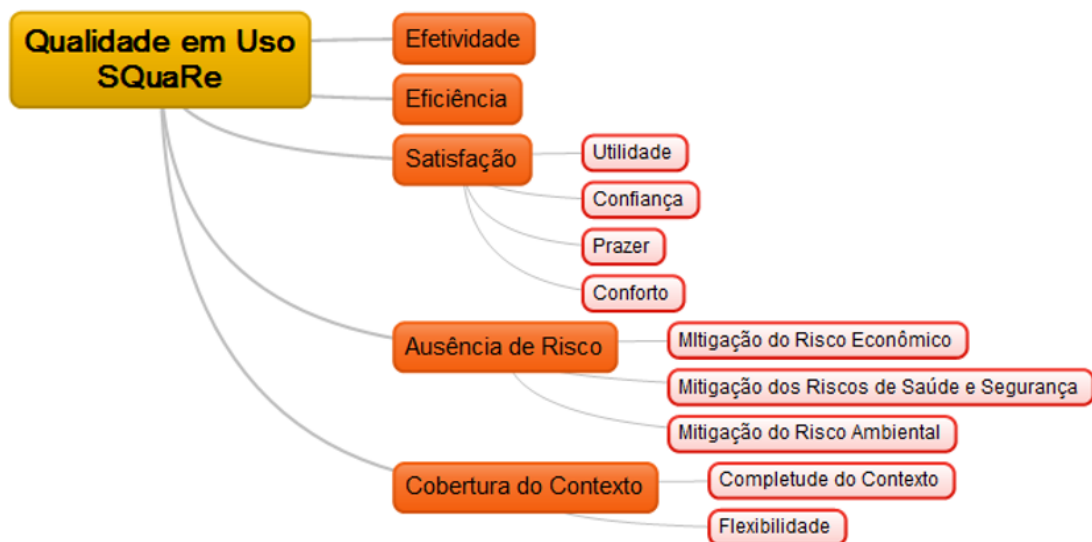


Figura 6.6.: Modelo de Qualidade em uso de Software da norma ISO 25010. Fonte:(ISO, 2011)

6.3. Qualidade é Sempre por um Ponto de Vista

É importante notar que qualidade é um atributo associado a algo (objeto, processo, serviço) por uma pessoa. Assim, diferentes pessoas podem ter diferentes visões de qualidade.

Outra analogia que podemos dar é a da escolha de um lanche por duas pessoas. Vamos supor que Alice e Bob desejam escolher um lanche para a tarde. Eles resolvem então discutir o que esperam do lanche e descobrem 3 dimensões de qualidade: sabor, ser saudável, e praticidade.

O problema de escolher o melhor lanche, isto é, o que tem mais qualidade, é que Alice e Bob tem visões diferentes da importância dessas dimensões. Bob acha o sabor mais importante, enquanto Alice acha que fazer bem a saúde é mais importante.

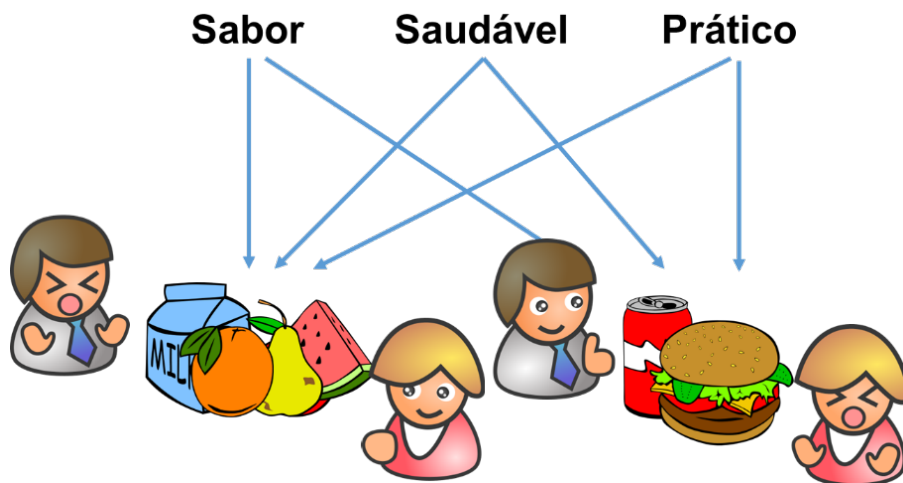


Figura 6.7.: Diferentes pontos de vista implicam em diferentes avaliações de qualidade.

Fonte: próprio autor

Da mesma forma, pessoas e organizações diferentes tem visões diferentes de que dimensões são importantes na qualidade de software. Em uma pesquisa realizada na COPPE, por exemplo, verificou-se que a questão mais importante de qualidade de software financeiro é a precisão, composta de correção e acurácia, com a nota “extremamente alta”. Basicamente isso significa que ao construir software financeiro é extremamente importante que as contas estejam corretas. Já a independência de ambiente foi considerada entre “baixa” e “muito baixa”. Imagine agora outro tipo de software, que deve funcionar em qualquer celular, nesse caso a independência de ambiente certamente seria mais necessária (Belchior, 1997) .

6.4. Qualidade de Produto e Qualidade de Processo

Uma das principais preocupações da Engenharia de Software é com a qualidade do processo de desenvolvimento de software. Essa preocupação vem de uma premissa básicas dos estudos de qualidade: a qualidade do processo é determinante da qualidade do produto.

Essa preocupação é bastante válida, pois tem relação com o importante problema de como garantir a qualidade do produto. A resposta seria: garantindo que o processo que o produz é de qualidade.

Baseado nessa premissa, o Departamento de Defesa Americano solicitou ao Software Engineering Institute da Universidade Carnegie-Mellow para desenvolver um modelo de maturidade de

processo que pudesse ser usado como uma medida da qualidade do processo de desenvolvimento de software de uma empresa a ser contratada. Isso resultou no **Capability Maturity Model for Software (CMM)** que mais tarde levou a criação do **Capability Maturity Model Integration (CMMI)**.

O **CMMI** é uma “coleção de melhores práticas que ajudam organizações a melhorar seus processos”(CMMI Product Team, 2010). Na prática é um modelo que indica que práticas são necessárias para atingir um processo de alta qualidade. Ele permite que organizações sejam avaliadas e recebam certificações sobre a maturidade de seus processos.

Baseado na ideia do CMMI, o Programa Brasileiro de Qualidade e Produtividade Software decidiu apoiar uma iniciativa similar, denominada MPS.Br, que se apoia no CMMI e em outras normas internacionais.

Além disso, existem normas ISO relacionadas ao processo de software, como por exemplo as normas: ISO 12207:2017 (ISO, 2017), ISO 15504 (ISO, 2004) e ISO 33001:2015 (ISO, 2015a).

6.5. Falhas e Defeitos

Falta de qualidade está normalmente associada a falhas na fase operacional. Para discutir essas falhas é necessário ter um vocabulário comum. Esse vocabulário é (também) definido na norma *IEEE Recommended Practice on Software Reliability*.

Erro (Error) Uma ação humana que produziu um resultado incorreto, como o software conter uma falta. Exemplos incluem omissões, interpretações erradas de requisitos do usuário ou especificação de software, tradução incorreta ou a omissão de um requisito na especificação de projeto.

Defeito (Defect) Um problema que, se não corrigido, pode causar uma aplicação a falhar ou produzir um resultado incorreto.

Falta (Fault) (A) Um defeito no código que pode causar uma ou mais falhas, ou (B) a manifestação de um erro no software.

Falha (Failure) (A) A incapacidade de um sistema ou componente de sistema de desempenhar uma função requisitada dentro dos limites especificados, ou (B) a terminação da habilidade de um produto de desempenhar uma função requisitada ou sua incapacidade de desempenhar dentro dos limites previamente especificados. (C) A saída da operação do programa de dentro dos seus requisitos. Uma *falha* pode ser produzida quando uma falta é encontrada e resulta na perda dos serviços especificados.

Essa definição nos permite entender a ideia que humanos fazem erros, que levam a defeitos e faltas, que causam falhas.

6.6. Quanto Custa a Qualidade

Qualidade não é de graça. Para alcançá-la é preciso investir recursos, e se não alcançá-la, será necessário pagar para consertar os defeitos. A experiência mostra que esse investimento compensa, principalmente para grandes projetos.

O SWEBOK (IEEE Computer Society, 2014a) indica os seguintes custos de Qualidade:

- Prevenção
 - Melhoria de Processo

6. Qualidade

- Qualidade da Infra Estrutura
- Treinamento
- Auditoria
- Revisões Gerenciais
- Avaliação (*Appraisal*)
 - Achando Defeitos
 - ◊ Revisões
 - ◊ Testes
- Falhas Internas
 - Corrigir defeitos encontrados na avaliação
- Falhas Externas
 - Corrigir defeitos encontrados após o lançamento

C. Jones e Bonsignour (2012) apresenta uma tabela de custos típicos de software por tamanho (em Pontos de Função) e nível de qualidade, replicada na Tabela 6.1. Fica claro que o custo total de desenvolvimento de software é menor quando a qualidade é maior, o que justifica o investimento em qualidade.

Tabela 6.1.: Custos típicos de software por tamanho e nível de qualidade. Fonte:(C. Jones e Bonsignour, 2012)

Pontos de Função	Qualidade		
	Baixa	Média	Alta
10	6,875	6,250	5,938
100	88,561	78,721	74,785
1000	1,039,889	920,256	846,636
10000	23,925,127	23,804,458	18,724,012
100000	507,767,782	433,989,557	381,910,810

O Custo da Qualidade leva a um dilema: qual o investimento que deve ser feito? Se não há nenhum investimento, ninguém vai querer usar o software, se o investimento é em direção a uma qualidade perfeita, o custo pode ser altíssimo. Então é necessário encontrar o ponto mágico, onde é bom o suficiente para não ser rejeitado, mas não tão perfeito que seja inviável(Venners, 2003).

Isso leva há algumas organizações e autores defender o conceito de *Good enough software*.

6.7. Gestão da Qualidade

Tendo em vista que Qualidade tem um custo e precisa ser alcançada de forma ativa, então é necessários haver um processo de gestão da qualidade.

Existem várias normas sobre o assunto, tanto em relação a qualidade em geral como a qualidade específica de software, incluindo gestão e garantia de qualidade.

A norma ISO 12207 (ISO, 2017) propõe um Processo de Gestão de Qualidade, cujo objetivo é “garantir que produtos, serviços e implementações de processos do ciclo de vida atinjam objetivos de qualidade organizacionais e alcancem a satisfação do consumidor”. Os resultados desse processo são:

- definição das políticas e procedimentos de gestão de qualidade;

6. Qualidade

- definição os objetivos organizacionais de qualidade;
- definição das responsabilidades e autoridades para a gerência de qualidade;
- monitoração do estado de satisfação do consumidor, e
- ações apropriadas são tomadas quando os objetivos de qualidade não são alcançados.

Segundo a norma ISO 9000-2015 (ISO, 2015b), um Sistema de Gestão de Qualidade (SGQ) “compreende atividades pelas quais a organização identifica seus objetivos e determina os processos e recursos necessários para alcançar os resultados necessários”.

O princípios da Gestão de Qualidade são:

Foco no Cliente Atender às necessidades e empenhar-se em exceder as expectativas

Liderança Líderes estabelecem unidade de propósito, direcionamento e criam condições para as pessoas engajadas alcançar os objetivos

Engajamento das pessoas Pessoas competentes, com poder e engajadas são essenciais para aumentar a capacidade da organização em criar e entregar valor

Abordagem de processo Resultados consistentes e previsíveis são alcançados de forma mais eficaz e eficiente

Melhoria Foco contínuo na melhoria

Tomada de decisão com base em evidência Decisões com base na análise e avaliação de dados e informações são mais propensas a produzir resultados desejados

Gestão de relacionamento Para o sucesso sustentado, as organizações gerenciam seus relacionamentos com as Partes Interessadas.

6.8. Exercícios

Exercício 6.1:

Escolha um produto de software qualquer e tente fazer uma análise de sua qualidade.

Exercício 6.2:

Discuta as diferenças de qualidade esperadas entre um jogo vendido em cartucho para um console, um jogo vendido para download no PC e um jogo para *smartphones*.

Processo de Software

If you don't know where you are, a map won't help.

(Watts Humphrey)

Conteúdo

7.1.	Processos de Software segundo a ISO/IEC/IEEE 12207:2017	87
7.2.	O Modelo Genérico de Pressman	88
7.3.	Cascata versus Protótipos	88
7.4.	Iteração e Incrementos	91
7.5.	Processos Prescritivos	93
7.6.	Processos Ágeis	94
7.7.	Importância da Estruturação de Processos Organizacionais	98
7.8.	O Capability Maturity Model Integration (CMMI)	98
7.9.	O Modelo MPS.BR para Software	99

O desenvolvimento de software é uma atividade complexa que exige planejamento, organização e repetibilidade. Para isso, utilizamos as noções como Ciclo de Vida e Processo de Software.

O **ciclo de vida** de software refere-se ao conjunto de etapas pelas quais um sistema de software passa, desde a concepção inicial até seu descarte (ISO, 2017). Ele contempla todas as atividades envolvidas na criação, implantação, uso, manutenção e eventual substituição do software. As etapas típicas incluem: concepção, definição de requisitos, projeto, implementação, testes, implantação, operação, manutenção e descarte.

Qualquer que seja um software, ele terá um ciclo de vida. No entanto, para que as etapas desse ciclo sejam gerenciadas de forma eficiente, previsível e reproduzível, é necessário adotar uma representação estruturada de como essas etapas se organizam e se inter-relacionam ao longo do tempo. Essa representação é fornecida pelos **modelos de ciclo de vida**, que servem

como guias conceituais para o planejamento e a execução dos projetos de software. A escolha de um modelo influencia diretamente aspectos como documentação, ritmo de entrega, controle de mudanças e envolvimento do cliente. Modelos como o cascata, o incremental e o espiral descrevem diferentes formas de organizar a progressão do desenvolvimento, estabelecendo marcos, fases e dependências.

Ao definirmos um modelo de ciclo de vida, estamos descrevendo a sequência lógica de estágios pelos quais um software deve passar. No entanto, para que esses modelos possam ser implementados de maneira eficaz, é necessário definir como cada uma dessas etapas será executada na prática. É nesse ponto que entra o conceito de processo de software.

O processo de software determina, com clareza, como cada etapa do modelo será executada, quem será responsável por cada atividade, quais produtos serão gerados e quais critérios serão utilizados para avaliar sua qualidade. Ele é composto por atividades técnicas e gerenciais, práticas bem estabelecidas, papéis atribuídos aos participantes, produtos de trabalho esperados, critérios de qualidade e ferramentas de apoio. Sua principal finalidade é fornecer uma base para conduzir projetos de forma previsível, garantindo uniformidade, padronização e a possibilidade de melhoria contínua com base na experiência acumulada. Podemos entender que um processo é uma receita de como fazer um projeto, como um template onde todas as atividades e produtos estão definidos, em algum nível de abstração.

O Primeiro Processo de Software Descrito

Possivelmente, a primeira descrição de um processo de software foi feita por Benington (1956). Ele trabalhava no MIT Lincoln Laboratory como responsável pelo desenvolvimento de software do projeto SAGE (Semi-Automatic Ground Environment), um sistema de defesa aérea em tempo real patrocinado pela Força Aérea dos Estados Unidos. Na apresentação original de 1956, Benington descreveu uma abordagem estruturada para o desenvolvimento de grandes programas computacionais, com fases bem definidas: planejamento operacional, especificações da máquina, especificações operacionais, especificações do programa, especificações de codificação, teste de parâmetros (similar ao que chamamos hoje de teste de unidade), teste de *assembly*, *shakedown* (teste operacional) e avaliação do sistema.

7.1. Processos de Software segundo a ISO/IEC/IEEE 12207:2017

A norma ISO/IEC/IEEE 12207:2017 estabelece um conjunto abrangente de processos organizados para apoiar o ciclo de vida de sistemas e software. Ela define um modelo de referência comum, aplicável a organizações que adquirem, fornecem, desenvolvem, operam, mantêm ou descartam produtos de software. Os processos são organizados em quatro grupos principais (ISO, 2017):

- Processos de Acordo (*Agreement Process*), com dois processos, Aquisição e Fornecimento.
- Processos Organizacionais Habilitadores de Projeto (*Organizational Project-Enabling Processes*), com cinco processos, Gestão de Modelo de Ciclo de Vida, Gestão de Infraestrutura, Gestão de Recursos Humanos, Gestão de Qualidade e Gestão de Conhecimento.
- Processos de Gestão Técnica (*Technical Management Processes*), com oito processos, Planejamento de Projeto, Controle e Avaliação de Projeto, Gestão de Decisão, Gestão

de Risco, Gestão de Configuração, Gestão da Informação, Mensuração e Garantia de Qualidade

- Processos Técnicos (*Technical Processes*), com 14 processos, Análise de Negócio ou Missão, Necessidades das Partes Interessadas e Definição de Requisitos, Definição de Requisitos de Sistema/Software. Definição de Arquitetura, Definição de Projeto (*Design*), Análise de Sistemas, Implementação, Integração, Verificação, Transição, Validação, Operação, Manutenção e Descarte (*Disposal*).

Esses processos são genéricos o suficiente para serem aplicados a projetos de diferentes tamanhos e domínios, mas detalhados o bastante para oferecerem guias operacionais claros. Eles ajudam a garantir que os projetos estejam alinhados com os objetivos organizacionais, que os produtos atendam aos requisitos técnicos e de qualidade, e que o desenvolvimento ocorra de forma sustentável e controlada.

Cada uma dessas atividades pode ser detalhada em tarefas específicas, com artefatos de entrada e saída, critérios de aceitação, recursos e responsabilidades bem definidos. A norma ISO 12207:2017 não prescreve uma metodologia específica para conduzir essas atividades, permitindo que sejam adaptadas conforme o contexto, desde que respeitados os objetivos e os resultados esperados.

7.2. O Modelo Genérico de Pressman

Pressman (2020) propõe o entendimento do desenvolvimento de software como cinco grandes atividades:

- **Comunicação:** envolve o levantamento e a compreensão das necessidades das partes interessadas, por meio de atividades como entrevistas, reuniões e análise de documentos, garantindo que os requisitos sejam claros e compartilhados entre todos os envolvidos.
- **Planejamento:** consiste na definição das tarefas, recursos, cronograma e riscos do projeto, estabelecendo um roteiro realista e controlável para a execução das atividades de desenvolvimento.
- **Modelagem:** abrange a criação de representações — textuais, gráficas ou formais — do sistema, incluindo modelos de requisitos, de arquitetura e de design detalhado, que servem como base para a construção do software.
- **Construção:** refere-se à implementação do código-fonte e à realização de testes, integrando componentes de forma incremental e assegurando que o produto atenda às especificações definidas na modelagem.
- **Implantação:** trata da disponibilização do software ao usuário final, incluindo atividades de entrega, instalação, configuração, treinamento e suporte inicial, visando garantir a aceitação e o uso eficaz do sistema.

7.3. Cascata versus Protótipos

A história da engenharia de software nas décadas de 1970 a 1990 foi marcada por um intenso debate entre dois modos distintos de pensar o desenvolvimento de sistemas: de um lado, os processos que hoje chamamos de prescritivos, representados principalmente pelo modelo em cascata e seus sucessores que buscavam resolver os seus problemas típicos; de outro, os processos baseados em protótipos, que mais tarde inspirariam os métodos ágeis.

O modelo em cascata ganhou força principalmente por sua afinidade com práticas consolidadas da engenharia tradicional, como as utilizadas na indústria aeroespacial, de defesa e grandes obras de infraestrutura. Nessas áreas, o controle rígido sobre requisitos, documentação e fases bem definidas era considerado essencial para o sucesso. A formalização do modelo cascata foi consolidada por meio de normas militares e contratos governamentais, como os utilizados pela OTAN (NATO) e pelo Departamento de Defesa dos Estados Unidos, que buscavam previsibilidade, controle de custos e responsabilização clara. Empresas de grande porte e agências governamentais, portanto, tornaram-se seus principais defensores. Além de se basearem fortemente em etapas compartimentadas que podiam ser entregues, e cobradas, durante projetos, também eram exigidas uma grande quantidade de documentação, tanto como entregas quanto como comprovação da execução de atividades. Podemos dizer que o foco desses projetos, o conceito de sucesso, era cumprir o combinado.

Em contrapartida, profissionais atuando em ambientes mais dinâmicos, como laboratórios de pesquisa, *startups* de tecnologia e departamentos de informática de empresas comerciais — começaram a propor abordagens mais flexíveis. Nesse contexto, surgem os processos baseados em protótipos. Esses processos rejeitavam a rigidez documental e a separação estrita entre fases. Em vez disso, propunham construir versões simplificadas ou parciais do sistema desde os estágios iniciais do projeto, para validar hipóteses, esclarecer requisitos e reduzir riscos. Essa linha de pensamento foi abraçada principalmente por desenvolvedores preocupados com a usabilidade, adaptação ao cliente e evolução rápida dos sistemas. Podemos dizer que o foco desses projetos, o conceito de sucesso, era entregar o valor possível aos clientes dentro de limitações de prazo e custo.

A controvérsia entre essas abordagens não era apenas técnica, mas também cultural e organizacional. Enquanto os processos prescritivos prezavam pela disciplina, controle e planejamento de longo prazo, os defensores da prototipação valorizavam a experimentação, o *feedback* constante e a adaptação contínua. Essa tensão foi se manifestando ao longo das décadas em debates sobre como lidar com mudanças, como planejar testes, como documentar o sistema e, principalmente, como garantir que o produto final refletisse o que o usuário realmente precisava — e não apenas o que estava no contrato original.

Com o tempo, os limites entre as abordagens foram se tornando mais difusos. A experiência prática demonstrou que os modelos prescritivos precisavam incorporar protótipos e mecanismos de iteração e adaptação, enquanto os modelos centrados em protótipos tiveram que lidar com problemas de escalabilidade, qualidade, documentação adequada e manutenção. Essa convergência culminou, nos anos 2000, com a consolidação dos princípios de iteração e incrementalidade dos processos.

Marcos importantes foram o surgimento do Ciclo de Vida Espiral (B. W. Boehm, 1988), do RUP, o processo unificado da Rational (Jacobson, Booch e Rumbaugh, 1999; Kruchten, 1998) e que buscava integrar todos esses conceitos aos métodos prescritivos, e no surgimento dos métodos ágeis (Beck, Beedle et al., 2001), que buscam descrever processos mais adaptativos com uma linguagem de processo, com inspiração em técnicas como *just in time* e *kanban*.

7.3.1. Processo em Cascata

O modelo em cascata, ou modelo linear, é um dos primeiros e mais conhecidos modelos de processo de software, implementado em várias normas de contratação da NATO e do governo americano, o que ajudou sua divulgação. Ele organiza o desenvolvimento em fases sequenciais bem definidas: levantamento de requisitos, análise, projeto, implementação, testes, implantação

e manutenção. A ideia central é que cada fase seja concluída, e aprovada pelo cliente, antes do início da seguinte, com entregas documentais servindo de transição.

Apesar de sua simplicidade e compatibilidade com modelos tradicionais de engenharia, o modelo em cascata apresenta sérias limitações. Sua principal desvantagem é a rigidez: o processo não lida bem com mudanças de requisitos e não incorpora mecanismos eficazes de feedback entre as etapas. Como consequência, o produto de software só era entregue ao cliente ao final de todo o processo, dificultando correções e ajustes ao longo do desenvolvimento.

Ainda assim, o modelo em cascata continua presente em muitos contextos, especialmente em contratos governamentais ou ambientes fortemente documentais, porém adaptado com estratégias trazidas de processos baseados em protótipos.

Também devemos chamar atenção que parte da grande adoção do processo em cascata, mesmo hoje em dia, vem do fato dele se adaptar facilmente as estruturas organizacionais e métodos tradicionais de gestão de projeto. Os papéis, como analista de sistema, projetista, programador, testador, representam funções bem determinadas, e têm habilidades específicas que permitem uma contratação eficiente. Já a gestão tem marcos de controle e pode medir se a produtividade está de acordo com o planejado. O problema, porém, é que o desenvolvimento de software não se encaixa bem nesses métodos, a não ser em condições específicas, como quando conhecemos muito bem o que desejamos, temos uma equipe bem treinada tecnicamente em computação e no negócio e o escopo está bem delimitado. Esse é um quadro raro de se encontrar no desenvolvimento de software.

É interessante notar que o artigo que propõe o Processo em Cascata foi proposto no meio de um artigo de Royce (1970) que analisava como software deveria ser desenvolvido e deixava claro que o processo linear usado em várias áreas de projeto não era adequado para o desenvolvimento de software. Ele cita, por exemplo, que as interações de design não estão confinadas aos passos do projeto, exigindo voltar atrás, algumas vezes muitos passos, o que é tem impacto profundo sobre o planejamento, e hoje sabemos que vai gerar atrasos e custos adicionais.

7.3.2. Prototipagem

A prototipagem surgiu como uma resposta prática às limitações do modelo cascata, e as necessidades de entregar rapidamente software funcionando às áreas fim das organizações. Ela enfatiza a construção rápida de artefatos experimentais, ou de uso momentâneo, os protótipos, para eliciar requisitos, validar funcionalidades e antecipar interações com o usuário.

O uso de protótipos desde de cedo no projeto facilita o entendimento do sistema por parte dos clientes e reduz drasticamente os riscos de construção de um software que não atenda às expectativas.

Ao longo do tempo foram descritas diversas abordagens para o desenvolvimento de software com protótipos, como, os descartáveis, os evolucionários, os operacionais (Urban, 1992) e outros.

Protótipos descartáveis são construídos rapidamente com o objetivo principal de esclarecer requisitos ou explorar opções de interface. Após cumprirem sua função, esses protótipos são descartados e não reaproveitados no desenvolvimento final, uma vez que não seguem padrões de qualidade adequados para produção. Sua vantagem está na agilidade e baixo custo de experimentação, mas exigem cuidado para não serem erroneamente incorporados ao produto final.

Protótipos evolucionários, por sua vez, são construídos com a intenção de serem refinados iterativamente. Eles começam como versões simplificadas do sistema, mas são progressivamente expandidos e melhorados até se tornarem o produto final. Essa abordagem é útil quando os requisitos não estão completamente definidos desde o início, permitindo que o sistema evolua a partir da colaboração com os usuários e da aprendizagem ao longo do projeto.

Já os protótipos operacionais são versões do sistema utilizadas temporariamente no ambiente real de operação, seja para avaliação, validação de desempenho ou apoio a transições complexas. Embora ainda não sejam o produto final, possuem um grau maior de qualidade e robustez, podendo ser utilizados em paralelo ao sistema antigo ou como base para continuidade do desenvolvimento. No entanto, é fundamental que sua função transitória esteja clara, pois o uso prolongado de protótipos operacionais sem revisão pode gerar problemas técnicos, de segurança e manutenção (Davis, 1992).

Por outro lado, protótipos mal utilizados podem induzir a decisões prematuras ou servir de base inadequada para implementação definitiva. Além disso, a prototipagem exige cuidados com a gestão da expectativa das partes interessadas e com o descarte apropriado dos artefatos não finais.

Um dos efeitos reconhecidos dos protótipos e sua adoção como se fossem o produto final, mesmo quando feitos com baixa qualidade. Isso leva a produtos com falhas constantes e alto custo de manutenção.

7.4. Iteração e Incrementos

A principal característica dos processos modernos, sejam eles prescritivos ou ágeis, é incorporar de alguma forma dois conceitos que, ao longo do tempo, ficaram claros como essenciais para o desenvolvimento com sucesso de software.

O primeiro conceito é a **iteração**. As iterações são uma característica do processo. O software precisa ser feito em ciclos, onde a cada ciclo aumenta o entendimento do que é necessário de produto sendo construído, e avança o produto. Cada iteração pode buscar cobrir um objetivo diferente, como definir as principais funções, descobrir a viabilidade técnica, etc. Por exemplo, no RUP temos várias iterações que estão dentro de quatro fases: concepção, elaboração, construção e transição. Essas iterações buscam alcançar, dentro da fase, os objetivos específicos.

Durante as iterações, o produto sofre **incrementos**. Incrementos melhoram o produto, adicionando valor. Os incrementos podem estar ligados a novas funcionalidades ou a melhoria de uma funcionalidade existentes.

No paradigma atual iterações que geram incrementos, softwares são construídos por partes, com foco no valor entregue ao cliente, na necessidade imediata, em necessidades de implementação e de forma a garantir que as partes interessadas entendam em que direção o projeto segue, e se essa é a direção certa.

Um processo interativo e incremental vai possuir as duas características simultaneamente. A cada ciclo de iteração os incrementos podem tanto trazer novas funcionalidades quanto aprofundar o detalhamento de uma atividade.



Figura 7.1.: Desenho representativo de iterações em uma imagem, aumentando o detalhe a cada passo.



Figura 7.2.: Desenho representativo de incrementos em uma imagem.

7.5. Processos Prescritivos

Processos prescritivos de software propõem estruturas detalhadas, com atividades, papéis e entregáveis bem definidos, com o intuito de guiar a execução de projetos de maneira sistemática e controlada. São geralmente baseados em uma extensiva documentação, em ciclos bem definidos e em métricas de controle e qualidade.

Entre os modelos prescritivos clássicos estão o modelo cascata, o modelo em V, o modelo incremental, o modelo espiral e o modelo transformacional. Cada um deles responde a necessidades diferentes, como controle rígido, entrega incremental ou mitigação de riscos.

7.5.1. RUP

O *Rational Unified Process* (RUP) é um modelo de processo de software prescritivo desenvolvido pela Rational Software (posteriormente adquirida pela IBM), que se destaca por combinar estruturação rigorosa com flexibilidade (Kruchten, 1998). Ele organiza o desenvolvimento em quatro fases principais, cada uma composta por uma ou mais iterações, nas quais são produzidos incrementos do produto. O RUP é orientado por casos de uso, centrado na arquitetura e enfatiza a modelagem, a verificação contínua e o gerenciamento de riscos.

As fases do RUP são:

- Concepção (*Inception*)** Fase inicial cujo objetivo é estabelecer a visão geral do sistema, identificar o escopo, os principais casos de uso, riscos iniciais e estimativas preliminares de custo e prazo. Ao final, deve haver um *Business Case* claro que justifique o projeto.
- Elaboração (*Elaboration*)** Fase de definição e estabilização da arquitetura do sistema. Inclui a captura detalhada dos requisitos mais críticos, a mitigação de riscos técnicos relevantes e a criação de uma arquitetura executável inicial. É nesta fase que se estabelecem as bases para a construção.
- Construção (*Construction*)** Fase dedicada à implementação e ao teste do sistema completo, com entregas incrementais de funcionalidades. O foco está na produtividade e na qualidade, desenvolvendo o produto de forma iterativa até que esteja pronto para implantação.
- Transição (*Transition*)** Fase de disponibilização do sistema aos usuários finais, incluindo implantação, treinamento, correção de defeitos residuais e ajustes com base no feedback. O objetivo é garantir a aceitação do produto.

Disciplinas do RUP

Além das fases, o RUP define **disciplinas** — conjuntos de atividades relacionadas a uma área específica do desenvolvimento. As disciplinas se estendem por todas as fases, com intensidade variável:

- **Modelagem de Negócio (*Business Modeling*)**: compreensão dos processos e objetivos do negócio, estabelecendo o contexto no qual o sistema operará.
- **Requisitos (*Requirements*)**: captura, análise e gerenciamento dos requisitos do sistema, preferencialmente representados por casos de uso.
- **Análise e Design (*Analysis and Design*)**: definição da arquitetura e detalhamento da solução técnica para atender aos requisitos.
- **Implementação (*Implementation*)**: codificação, integração e construção do sistema.

- **Testes** (*Test*): verificação e validação do sistema, assegurando que ele atende aos requisitos e está livre de defeitos críticos.
- **Implantação** (*Deployment*): entrega do sistema ao ambiente de produção e suporte inicial.

O RUP também inclui disciplinas de apoio (*supporting disciplines*), que sustentam a execução técnica:

- **Gerenciamento de Configuração e Mudança** (*Configuration and Change Management*): controle de versões, artefatos e solicitações de mudança.
- **Gerenciamento de Projeto** (*Project Management*): planejamento, acompanhamento e controle do progresso do projeto.
- **Ambiente** (*Environment*): configuração e manutenção do ambiente de desenvolvimento, incluindo ferramentas e processos de suporte.

Essa estrutura faz do RUP um *framework* adaptável: cada organização pode selecionar e ajustar as disciplinas e artefatos a serem utilizados, de acordo com o porte, o domínio e os objetivos do projeto (Jacobson, Booch e Rumbaugh, 1999; Kruchten, 1998). Há inclusive uma ferramenta que permite descrever o projeto e receber um processo adequado.

O RUP hoje é chamado apenas de Processo Unificado.

7.6. Processos Ágeis

Processos ágeis surgiram no início dos anos 2000 como uma reação à rigidez dos modelos prescritivos de desenvolvimento de software. A principal ideia é a entrega frequente de software funcional. Ao invés de valorizar documentação exaustiva e planejamento detalhado, os métodos ágeis priorizam indivíduos, interação, comunicação, simplicidade e resposta rápida (Beck, Beedle et al., 2001; Highsmith, 2002).

A filosofia ágil se manifesta por meio de métodos como *Extreme Programming* (XP), Scrum, Lean Software Development e Kanban. Todos compartilham valores e princípios comuns, conforme delineado no Manifesto Ágil.

7.6.1. O Manifesto Ágil

O Manifesto Ágil foi redigido em fevereiro de 2001, durante um encontro realizado no *The Lodge at Snowbird*, um resort em Utah, Estados Unidos. O evento reuniu 17 profissionais influentes no campo do desenvolvimento de software que defendiam diferentes abordagens que viriam compor o espectro de métodos ágeis. Em comum estavam unidos pela insatisfação com os métodos prescritivos vigentes e por experiências de sucesso com processos muito mais leves (Beck, Beedle et al., 2001).

O manifesto estabelece quatro valores fundamentais:

- Indivíduos e interações mais que processos e ferramentas;
- Software em funcionamento mais que documentação abrangente;
- Colaboração com o cliente mais que negociação de contratos;
- Responder a mudanças mais que seguir um plano.

Além disso, apresenta 12 princípios que enfatizam a entrega contínua de valor, a comunicação face a face, o ritmo sustentável, a simplicidade e a auto-organização das equipes. Esses princípios formam a base conceitual que continua a orientar a prática ágil em diferentes contextos e setores.

A importância histórica do Manifesto Ágil reside não apenas no impacto imediato sobre a indústria de software, mas também no fato de ter estabelecido um paradigma cultural que valoriza a adaptabilidade, a colaboração e a entrega de valor contínuo ao cliente (Denning, 2018; Highsmith, 2002).

7.6.2. Evolução da Agilidade

Nas duas décadas desde o lançamento do Manifesto Ágil, a agilidade expandiu-se para além do desenvolvimento de software, influenciando áreas como gestão de produtos, marketing, operações e até educação (Denning, 2018). Modelos como *Scaled Agile Framework* (SAFe), *Large Scale Scrum* (LeSS) e *Disciplined Agile Delivery* (DAD) surgiram para aplicar princípios ágeis em contextos organizacionais complexos e distribuídos.

Além disso, a integração com práticas de DevOps e Entrega Contínua (*Continuous Delivery*) permitiu ciclos de entrega ainda mais curtos e feedback mais rápido do cliente, consolidando um ecossistema de práticas voltadas à adaptabilidade e à melhoria contínua.

7.6.3. Críticas aos Processos Ágeis

Apesar do sucesso e ampla adoção, a agilidade enfrenta críticas. Uma das principais refere-se ao fenômeno conhecido como *agile in name only* (AInO), em que organizações adotam rituais e terminologia ágil sem absorver seus princípios fundamentais (Hoda, Noble e Marshall, 2017). Outra crítica recorrente é a tendência de engessamento ao escalar práticas ágeis, levando à criação de frameworks complexos que contradizem a simplicidade originalmente defendida pelo manifesto (M. Poppendieck e T. Poppendieck, 2011).

Há ainda debates sobre o uso da agilidade em projetos de missão crítica ou regulados, onde a flexibilidade excessiva pode entrar em conflito com requisitos rígidos de conformidade e segurança. Em alguns casos, a pressão por entregas rápidas levou à redução inadequada de atividades de engenharia, comprometendo a qualidade.

7.6.4. XP

O *Extreme Programming* (XP) é uma metodologia ágil proposta por Kent Beck no final da década de 1990 e formalizada no livro *Extreme Programming Explained: Embrace Change* (beck1999xp; beck2004xp2). Concebido inicialmente no contexto de um projeto da Chrysler Comprehensive Compensation (C3), o XP foi projetado para lidar com ambientes de requisitos voláteis, com forte ênfase na comunicação, simplicidade e feedback rápido.

Extreme Programming (XP) é uma metodologia ágil centrada em boas práticas de desenvolvimento e na colaboração intensiva com o cliente. Ela enfatiza a simplicidade no design, feedback constante e qualidade técnica através de práticas como programação em par (*pair programming*), desenvolvimento orientado a testes (*TDD*), integração contínua e propriedade coletiva do código. O cliente participa ativamente, definindo as funcionalidades mais prioritárias por meio de histórias de usuário, que são continuamente refinadas e testadas.

O processo assume que os requisitos mudam com frequência e que a melhor forma de enfrentá-los é com uma equipe motivada, comunicação intensa e feedback rápido. O ciclo de desenvolvimento é curto e iterativo, permitindo correções e ajustes frequentes com base na interação entre desenvolvedores e clientes.

Beck propôs um conjunto de práticas integradas que, combinadas, maximizam a adaptabilidade e a confiabilidade do software. As doze práticas originais do XP são:

- **Planejamento de Jogo** (*Planning Game*): colaboração entre clientes e desenvolvedores para definir o escopo e priorizar funcionalidades.
- **Pequenas Entregas** (*Small Releases*): disponibilização frequente de versões funcionais do software, entregando valor contínuo.
- **Metáfora do Sistema** (*System Metaphor*): uso de uma metáfora compartilhada para descrever o funcionamento global do sistema, facilitando o entendimento comum.
- **Design Simples** (*Simple Design*): desenvolvimento da solução mais simples possível que atenda aos requisitos atuais.
- **Testes** (*Testing*): uso extensivo de testes automatizados de unidade e de aceitação para validar continuamente o sistema.
- **Refatoração** (*Refactoring*): melhoria contínua do código sem alteração de comportamento, visando legibilidade e manutenibilidade.
- **Programação em Par** (*Pair Programming*): dois desenvolvedores trabalham juntos no mesmo código, promovendo qualidade e compartilhamento de conhecimento.
- **Propriedade Coletiva do Código** (*Collective Code Ownership*): qualquer desenvolvedor pode modificar qualquer parte do código a qualquer momento.
- **Integração Contínua** (*Continuous Integration*): integração frequente das alterações no repositório principal, com execução automática de testes.
- **Semana de 40 Horas** (*40-Hour Week*): manutenção de um ritmo de trabalho sustentável, evitando sobrecarga e fadiga.
- **Cliente no Local** (*On-Site Customer*): presença constante de um representante do cliente para esclarecer dúvidas e validar decisões.
- **Padrões de Código** (*Coding Standards*): adoção de convenções e padrões de codificação compartilhados para garantir consistência.

Essas práticas são interdependentes: a ausência de uma pode comprometer a eficácia das demais. O XP enfatiza que a disciplina técnica é tão importante quanto a colaboração e a gestão do processo, reforçando que a agilidade só é sustentável com bases sólidas de engenharia (beck2004xp2).

7.6.5. Scrum

Scrum é um *framework* ágil de gerenciamento de trabalho, inicialmente concebido para o desenvolvimento de software, mas hoje aplicado em diversas áreas. Baseia-se em ciclos curtos de desenvolvimento, denominados *sprints*, com duração típica de duas a quatro semanas, nos quais a equipe transforma um conjunto de itens priorizados em um incremento funcional do produto (schwaber2020scrum). O foco é entregar valor continuamente, adaptando o processo conforme as necessidades evoluem.

O trabalho no Scrum é guiado por três pilares fundamentais:

- **Transparência**: todos os aspectos relevantes do processo devem ser visíveis aos envolvidos, garantindo entendimento comum.

- **Inspeção:** acompanhamento frequente dos artefatos e progresso para detectar desvios ou problemas.
- **Adaptação:** ajustes imediatos ao processo ou ao produto sempre que forem detectadas inconsistências ou oportunidades de melhoria.

O Scrum define três papéis principais, cada um com responsabilidades específicas:

- **Product Owner:** responsável por maximizar o valor do produto resultante do trabalho da equipe. Mantém e prioriza o *Product Backlog*, representando as necessidades do cliente e das partes interessadas.
- **Scrum Master:** atua como facilitador e guardião do framework Scrum. Remove impedimentos, promove entendimento dos princípios ágeis e auxilia a equipe a melhorar continuamente.
- **Time de Desenvolvimento (*Developers*):** grupo multifuncional e auto-organizado que constrói o incremento de produto a cada sprint. É responsável por transformar os itens selecionados do backlog em funcionalidades prontas para uso.

O ciclo de trabalho é estruturado em eventos com propósito e duração definidos:

- **Sprint Planning (*Planejamento da Sprint*):** reunião no início da sprint para definir o objetivo e selecionar os itens do *Product Backlog* que serão trabalhados.
- **Daily Scrum (*Reunião Diária*):** breve encontro diário (até 15 minutos) para sincronizar atividades, inspecionar o progresso e planejar o trabalho das próximas 24 horas.
- **Sprint Review (*Revisão da Sprint*):** reunião no final da sprint para apresentar o incremento ao cliente e coletar feedback, permitindo adaptação do backlog.
- **Sprint Retrospective (*Retrospectiva da Sprint*):** reunião para avaliar o processo de trabalho, identificar melhorias e planejar ações para aumentar a eficácia da equipe.

O resultado de cada sprint é um **Incremento**, que é uma versão potencialmente utilizável e integrada do produto. Cada incremento deve estar pronto para ser entregue e atender à *Definição de Pronto (*Definition of Done*)* acordada pela equipe.

O Scrum não prescreve práticas técnicas específicas de desenvolvimento ou gestão de qualidade. Ele define a estrutura mínima de papéis, eventos e artefatos, mas deixa a cargo da equipe a escolha das práticas a adotar. Assim, métodos como programação em par, integração contínua, TDD ou gestão visual de fluxo podem ser incorporados gradualmente conforme as necessidades, a experiência e a maturidade da equipe aumentam (**schwaber2020scrum**).

7.6.6. Continuous Delivery

Continuous Delivery (CD) é uma prática que busca tornar o software continuamente pronto para implantação, com confiança e estabilidade. Baseado em automação extensiva de testes, integração contínua e pipelines de entrega, CD permite que equipes liberem versões novas com frequência, sem a necessidade de ciclos longos de preparação.

Ao adotar CD, os times reduzem o tempo entre a implementação de uma funcionalidade e sua disponibilidade para uso, o que favorece feedback rápido, experimentação e menor acúmulo de dívida técnica. Essa prática está intimamente ligada ao movimento DevOps, que integra desenvolvimento e operações para promover entrega rápida e confiável em produção.

CD também está associado a um compromisso com a qualidade: testes automatizados em diferentes níveis (unidade, integração, sistema) são essenciais, assim como revisões de código e

monitoramento de sistemas em produção. Dessa forma, a entrega contínua contribui para um processo de desenvolvimento mais seguro, responsivo e eficiente.

7.7. Importância da Estruturação de Processos Organizacionais

Em organizações onde não há processos formais definidos, em que cada projeto depende de esforços heroicos individuais, conforme apontam tanto o CMMI (CMMI Product Team, 2010) quanto o MPS.br (**mps**), observa-se uma variabilidade significativa entre o que foi previsto e o que foi efetivamente realizado, em termos de prazos, custos e funcionalidades entregues. Essa incerteza deriva da ausência de repetibilidade e de controle sistemático no desenvolvimento de software.

Por outro lado, organizações que adotam processos estruturados tendem a apresentar uma variabilidade muito menor, pois suas atividades são planejadas, monitoradas e controladas de forma sistemática, promovendo previsibilidade e desempenho consistente. Evidências mostram que, à medida que a maturidade de processo aumenta, as variações de custo e cronograma tendem a diminuir.

A adoção de um processo, mesmo que básico, normalmente melhora o desempenho dos projetos. Basicamente “fazer algo é sempre melhor do que não fazer nada”, uma vez que mesmo ações iniciais de padronização tendem a reduzir custos e a melhorar a satisfação das partes interessadas.

Entretanto, é fundamental reconhecer que implementar um novo processo constitui um projeto de mudança e carrega seus próprios riscos. A introdução de atividades, responsabilidades e competências inéditas pode ter impacto negativo se não for planejada e apoiada adequadamente. Por essa razão, projetos de mudança devem começar por iniciativas de menor risco, evitando somar instabilidades ao projeto principal.

Além disso, quanto menor a complexidade do processo implementado inicialmente, menores os riscos associados. Mesmo processos complexos devem ser introduzidos de forma incremental, permitindo adaptação organizacional e aprendizado contínuo.

Modelos como o CMMI e o MPS.br oferecem roteiros claros para essa evolução gradual. O CMMI, com seus cinco níveis de maturidade, orienta o crescimento progressivo do controle e da capacidade organizacional (CMMI Product Team, 2010). Analogamente, o MPS.br, com seus níveis de maturidade de G a A, foi concebido para tornar essa evolução exequível e sustentável, especialmente para empresas de pequeno e médio porte no contexto brasileiro (Montoni, Rocha e Weber, 2009).

7.8. O Capability Maturity Model Integration (CMMI)

O *Capability Maturity Model Integration* (CMMI) é um modelo de referência para a melhoria de processos, desenvolvido originalmente pelo *Software Engineering Institute* (SEI) da Carnegie Mellon University, e atualmente mantido pelo *CMMI Institute* sob gestão da ISACA (**chrissis2011cmmi**; **forrester2001cmmi**). Seu objetivo principal é fornecer às organizações um guia estruturado para avaliar, desenvolver e melhorar suas capacidades de processos, aumentando a previsibilidade, a qualidade e a eficiência na entrega de produtos e serviços.

O CMMI surgiu como evolução e integração de diversos modelos de maturidade preexistentes, incluindo o *Capability Maturity Model for Software* (SW-CMM), modelos para engenharia de sistemas e para desenvolvimento integrado de produtos e processos. Ao unificar essas abordagens, o CMMI criou um framework aplicável a diferentes domínios, como desenvolvimento (*Development*), aquisição (*Acquisition*) e serviços (*Services*) (chrissis2011cmmi).

O modelo organiza práticas recomendadas em áreas de processo (*Process Areas*), que cobrem dimensões como gestão de requisitos, planejamento de projetos, monitoramento e controle, garantia de qualidade, gestão de riscos, medição e análise, entre outras. Essas práticas são estruturadas em níveis de maturidade (no modelo por estágios) ou níveis de capacidade (no modelo contínuo), permitindo que a organização progrida de maneira incremental:

- **Nível 1 — Inicial:** Processos imprevisíveis e reativos, dependentes de esforços individuais.
- **Nível 2 — Gerenciado:** Processos de gerenciamento de projetos estabelecidos para planejar e acompanhar desempenho.
- **Nível 3 — Definido:** Processos padronizados e documentados em toda a organização.
- **Nível 4 — Quantitativamente Gerenciado:** Uso de métricas e controles quantitativos para gerenciar qualidade e desempenho.
- **Nível 5 — Em Otimização:** Foco em melhoria contínua baseada em inovações e análises quantitativas.

A adoção do CMMI está associada a benefícios como redução de variabilidade entre o planejado e o realizado, melhoria da qualidade do produto, maior previsibilidade de cronogramas e custos, e aumento da satisfação do cliente (goldenson2003benefits; subramanyam2004impact). No entanto, sua implementação demanda investimento significativo em treinamento, adaptação de processos e mudança cultural, devendo ser tratada como um projeto de transformação organizacional.

7.9. O Modelo MPS.BR para Software

O *Modelo de Melhoria de Processo do Software Brasileiro* (MPS.BR) é um programa nacional coordenado pela Softex, criado em dezembro de 2003 com apoio de órgãos governamentais, com o propósito de fomentar a competitividade das organizações brasileiras por meio da melhoria de seus processos de software, serviços e gestão de pessoas, especialmente em micro, pequenas e médias empresas (SOFTEX_MPSBR).

Na sua versão mais recente, aplicada desde 1º de março de 2023, o modelo de referência para software — **MR-MPS-SW:2023** — redefine os requisitos, processos, competências e níveis de maturidade conforme normas internacionais reconhecidas (ISO/IEC 12207, ISO/IEC 33000), além de manter compatibilidade com o CMMI-DEV (SOFTEX_GuiaGeral2024; Promove_MPSBR).

O MR-MPS-SW apresenta sete níveis de maturidade, dispostos em ordem crescente de sofisticação e institucionalização dos processos:

- **G – Parcialmente Gerenciado:** inclui processos básicos como Gerência de Projeto, Engenharia de Requisitos e Treinamento, com capacidade mínima de planejamento, execução e monitoração (Promove_MPSBR).
- **F – Gerenciado**
- **E – Parcialmente Definido**
- **D – Definido**

- C – Completamente Definido
- B – Gerenciado Quantitativamente
- A – Em Otimização

Esses níveis combinam processos de projeto e processos organizacionais com atributos de capacidade progressivamente mais elevados, conforme previsto no modelo internacional de avaliação de processos (**Promove_MPSBR**).

O MR-MPS-SW é composto por diversas áreas, incluindo Gerência de Projeto, Engenharia de Requisitos, Construção do Produto, e outros processos organizacionais, todos descritos no *Guia Geral MPS de Software:2024* (**SOFTEX_GuiaGeral2024**). A conformidade com padrões como ISO/IEC 12207 e ISO/IEC 33000 garante a aderência às melhores práticas internacionais, com flexibilidade adaptada ao contexto nacional empresarial (**Promove_MPSBR**).

Como consequência dessa estrutura fundamentada, o modelo oferece às organizações brasileiros um programa incremental e acessível de melhoria de processos, facilitando adoção sustentada, comparável ao CMMI, mas com maior adequação às realidades locais.

Além do MPS-SW foram desenvolvidos o MPS-SV, para serviços, e o MPS-RH, para gestão de pessoas.

Parte II.

Especificando o Software

Software Project Canvas

If I had eight hours to chop down a tree, I'd spend six sharpening my axe.

(Abraham Lincoln)

Conteúdo

8.1.	O que são Canvas	103
8.2.	Projetos, Agilidade e Canvas	104
8.3.	O <i>Software Project Canvas</i>	107
8.4.	A Metodologia do <i>Software Project Canvas</i>	114
8.5.	Conclusão	119

Esse capítulo apresenta o Software Project Canvas¹, representado na Figura 8.1, uma ferramenta ágil cujo objetivo é facilitar a colaboração das partes interessadas na discussão e definição inicial de um projeto de software. Esta é uma contribuição original deste livro.

O *Software Project Canvas* foi motivado por certas dificuldades encontradas ao ensinar e usar o *Project Model Canvas* (Finocchio Jr., 2013) no contexto de projetos de Engenharia de Software. Após identificar essas dificuldades, foram feitas propostas de correção, e variações do modelo foram avaliadas por alguns especialistas e iniciantes na área.

¹Este documento e arquivos com o Software Project Canvas podem ser encontrados em <https://github.com/xexeo/Software-Project-Canvas>



Figura 8.1.: O Software Project Canvas. Imagem do autor

8.1. O que são Canvas

Canvas são ferramentas colaborativas que servem de ponto focal em uma atividade onde algo vai ser controlado, decidido ou planejado. São compostos de um quadro branco dividido em várias áreas, cada uma representando uma pergunta, assunto ou tópico em discussão, onde são colocados *post-its* contendo a informação desejada.

Alguns autores, como Finocchio Jr. (2013), associam os canvas a neurociência, tanto por usar de características de agrupamento e proximidade, que auxiliam a compreensão dos conceitos, como por apoiarem uma atividade colaborativa que tende a criar engajamento nos membros da reunião em torno do assunto.

O primeiro canvas de grande impacto na indústria foi o *Business Model Canvas*, proposto por Osterwalder, Pigneur e Clark (2010), apresentado ainda vazio na Figura 8.2. Ele é destinado ao planejamento de um modelo de negócio para um empreendimento. Nele, cada área receberá, em uma ordem pré-estabelecida e ao longo de uma reunião facilitada, *post-its* que dão respostas às perguntas indicadas nos quadros. Por exemplo, no quadro “Customer Segments” pode aparecer um *post-it* onde está escrito “adolescentes praticantes de Skate”.

A partir do sucesso do *Business Model Canvas*, outros canvas foram propostos por pesquisadores, consultores ou pessoas interessadas em algum problema específico. No Brasil, Finocchio Jr. propôs, com bastante sucesso, o *Project Model Canvas* (Finocchio Jr., 2013), que serviu de inspiração para nossa proposta.

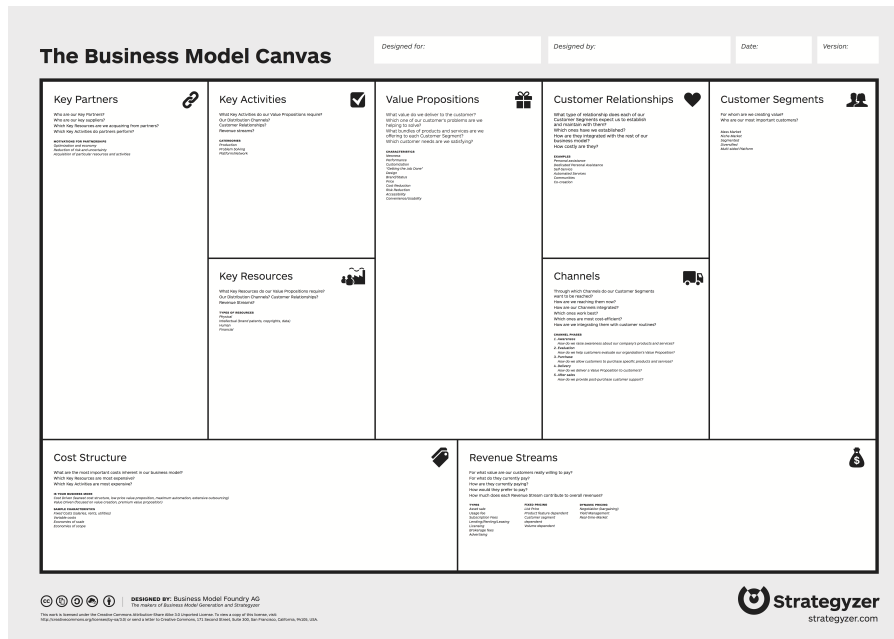


Figura 8.2.: O Business Model Canvas. Fonte: Osterwalder, Pigneur e Clark (2010)

8.2. Projetos, Agilidade e Canvas

A Gestão de Projetos é uma área de conhecimento essencial ao sucesso de um projeto. Segundo Kerzner (2017), um projeto tem sucesso se acaba no prazo, dentro do orçamento, com o desempenho especificado, aceito pelos clientes e usuários, com mudanças mínimas no escopo acordadas entre as partes interessadas, sem causar distúrbio ao fluxo de trabalho da organização e, quando desejado, sem mudar a cultura da organização. Além disso, é necessário satisfazer as partes interessadas (PMI, 2017). Para alcançar todos esses objetivos é necessário planejar, monitorar e controlar o projeto ao longo de sua realização.

Ao longo dos anos, a área de gestão de projetos desenvolveu processos eficazes e reconhecidos, sendo que muitos foram consolidados em padrões ou livros de referência (Kerzner, 2017; PMI, 2017). A Engenharia de Software, disciplina relativamente nova em relação as outras disciplinas onde esses processos foram desenvolvidos, foi criada com o intuito de aplicar as práticas de Engenharia ao desenvolvimento de software, incluindo as práticas de gestão (IEEE Computer Society, 2014a; Naur e Randell, 1969; Pressman e Maxim, 2019).

Com a experiência de desenvolver software sob a pressão do mundo de negócios, algumas dessas práticas, ou seus princípios, começaram a ser contestadas por não serem adequadas ao desenvolvimento de software, exigirem uma carga muito alta de trabalho adicional (*overhead* alto), ou exigirem muitos documentos. Isso resultou no fortalecimento dos defensores de processos de desenvolvimento de software mais leves, capazes de reagir rápido a mudanças, que acabaram sendo nomeados métodos ágeis ou enxutos. Essa tendência acabou por afetar toda a indústria, como mostra a ampla adoção dos métodos ágeis (digital.ai, 2022).

Entre os princípios que não funcionaram na Engenharia de Software está o da execução de projetos de forma estritamente sequencial, onde uma fase é realizada de forma completa e fornece subsídios para a próxima fase, sendo a entrega feita, em sua totalidade, no final do projeto, o que é conhecido como Processo em Cascata. Atualmente, há ampla convicção que é essencial usar processos iterativos e incrementais, onde as entregas são pequenas e feitas ao



Figura 8.3.: Um quadro Scrum/Kanban muito simples. Fonte: por Jess Iasovski, CC BY-SA 3.0

longo do projeto, principalmente devido a sua maior taxa de sucesso (Pressman e Maxim, 2019; The Standish Group, 2015).

Uma prática comum dos métodos ágeis é o uso de quadros brancos com áreas delimitadas por assunto, indicando, de forma transparente a todos, alguma informação controlada por meio do quadro. O mais famoso desses quadros é provavelmente o a adaptação, para o processo *Scrum* (Satpathy et al., 2016), do *Kanban*, apresentado em uma forma simplificada na Figura 8.3. O Kanban é um método de produção que pode ser considerado um predecessor dos métodos ágeis e faz parte de um conjunto de práticas utilizadas na Toyota e que iniciou uma revolução de conceitos de produção e qualidade, alterando o até então paradigma *Fordista* de produção (Gross e McInnis, 2003). Outros métodos adotaram quadros semelhantes, como o SEMAT (Jacobson, Lawson e Ng, 2019), que usa cartas pré-definidas posicionadas no quadro para indicar o estado de um processo.

Na Figura 8.3 também é possível notar que o quadro não precisa ser feito de forma impressa, podendo ser simplesmente desenhado em um quadro branco, ou mesmo na parede com fitas, e como os *post-its* são usados, nesse caso para indicar tarefas a fazer, sendo feitas e prontas.

8.2.1. O Project Model Canvas

O *Project Model Canvas* (Finocchio Jr., 2013), apresentado na Figura 8.4, é um canvas destinado ao planejamento inicial de um projeto, baseado em uma metodologia de 4 grandes etapas, descrita na Figura 8.4. Ele possui 12 quadros que são preenchidos passo a passo na etapa de Concepção, aprimorados nas fases de Integração e Revisão, e depois tem seu resultado comunicado as partes interessadas na fase de Compartilhamento. Sendo destinado a qualquer tipo de projeto, pode ser também aplicado a projetos de software, porém não necessariamente de forma perfeita.

Esse canvas é uma abordagem típica de projetos ágeis ou enxutos, aplicada ao planejamento de projetos de qualquer tipo. Isso é, simultaneamente, uma vantagem e uma desvantagem do *Project Model Canvas*, já que, sendo geral, pode ser aplicado com sucesso em qualquer área, mas

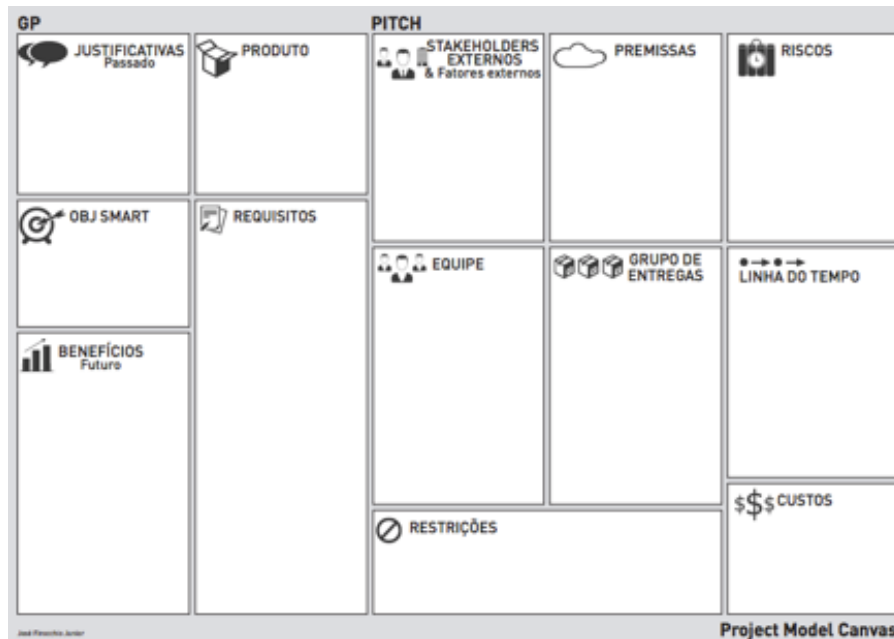


Figura 8.4.: O processo de planejamento usando o Project Model Canvas. Fonte: Finocchio Jr. (2013)

não sendo específico, acaba por ter algumas incompatibilidades com projetos de áreas específicas, como, e em especial, a Engenharia de Software.

Projetos de software têm características diferentes de projetos tradicionais que afetam diretamente o planejamento. Entre elas, podemos citar: a dificuldade de medir ou prever o tamanho da entrega, medido em esforço ou em prazoB. Boehm, Abts e Brad Clark; Cohn; Pressman e Maxim, a necessidade de aprendizado da equipe em relação ao domínio da aplicação ao longo do projeto e a má-adequação às formas tradicionais e hierárquicas de equipe (Constantine, 1993). Essas e outras características que diferenciam o projeto de desenvolvimento de software afetam diretamente a capacidade de planejamento, exigindo estratégias diferenciadas em relação as estratégias tradicionais.

Ao ensinar e usar o *Project Model Canvas* para o planejamento de projetos de desenvolvimento de software, também podem ser percebidas algumas dificuldades dos alunos e praticantes com os quadros, listadas a seguir.

1. A dificuldade de separar o *pitch* do produto e do objetivo SMART, e depois separar este do produto. Essas duas primeiras dificuldades advêm do fato de ser realmente difícil diferenciar o produto específico sendo criado do objetivo para o qual é criado. Isso, porém, pode não ser necessário no projeto de software, já que justificativas, benefícios e requisitos cumprem bem esse papel.
2. A limitação a apenas um objetivo SMART para o software, quando é comum softwares atenderem várias funcionalidades que são normalmente criadas para atender objetivos;
3. A dificuldade de separar requisitos não-funcionais de restrições;
4. A não separação dos requisitos entre funcionais e não funcionais, tradicional na área de software, e importante no projeto de software, o que afeta diretamente a capacidade de estimar os custos do software, já que a prática mais frequente, formalizada nos Pontos de Função (IFPUG, 2012), implica em calcular a funcionalidade entregue e depois melhorar a estimativa a partir de exigências feitas pelos requisitos não funcionais.

5. Um excesso de esforço na separação entre grupos de entregas e linha do tempo, que ainda muitas vezes obriga a re-arranjar entregas ou requisitos, quando as práticas mais atuais, por exemplo baseadas em *road-maps* do produto, tomam essa decisão de forma simultânea.
6. A dificuldade de definir custos, já que basicamente todo custo de um projeto de software vem da mão de obra, ou seja, do esforço necessário para desenvolvê-lo, geralmente medido em pessoas-mês ou pessoas-hora.

Todos esses problemas levam a uma ineficiência no processo de ensino e de uso do *Project Model Canvas*, já que o tempo é gasto, repetidas vezes, na elucidação de conceitos ou restrições que não são essenciais ao projeto de software.

Para resolver essas dificuldades, este artigo propõe o uso do *Software Project Canvas*. Além de ser criado especificamente para projetos de software, ele trata os problemas detectados no *Project Model Canvas* da seguinte forma:

- elimina o *pitch*, substituindo-o pelo produto de software a ser criado;
- não apresente o objetivo SMART único, considerando que o objetivo é entregar o software;
- trata objetivos e metas com uma abordagem aceita na Engenharia de Software (Ruble, 1997);
- unifica no processo o tratamento de restrições e requisitos não-funcionais, para facilitar o processo de trabalho;
- separa os requisitos funcionais e não funcionais, os últimos com adição das restrições, em áreas diferentes do canvas, atendendo a prática comum da Engenharia de Software (Pressman e Maxim, 2019);
- unifica entregas e linha do tempo em uma caixa, onde as entregas já devem vir com uma data, ou será criado um roadmap do produto, típica de gestão ágil, e
- transforma todo o cálculo do custo em uma previsão do esforço (Cohn, 2005).

Como resultado, o processo é transformado de 12 para 9 passos, já que mesmo em áreas separadas no canvas os requisitos são todos levantados no mesmo momento, facilitando sua explicação. Além disso, são evitadas muitas discussões desnecessárias, como onde colocar algo que tanto pode ser visto como uma restrição ou como um requisito não-funcional, por diferentes partes interessadas.

8.3. O Software Project Canvas

O Software Project Canvas utiliza 10 quadros, que serão descritos nesta seção. Para cada quadro foi buscado explicar o que ele trata, métodos que podem ser usados para auxiliar seu preenchimento e alguns exemplos. Os quadros se organizam em 6 grupos que respondem às perguntas 5W2H (Nakagawa, 2012): *Why*, *What*, *Who*, *When*, *How* e *How Much*. Não é necessário, em geral, reponder a pergunta *Where* para projetos de software.

- Por que: o que justifica o projeto, os Problemas e Oportunidades, e os Objetivos e Metas;
- O que: o que é o projeto, indicado pelos Requisitos Funcionais, Requisitos Não-Funcionais e Restrições;
- Quem: indicado pelas Partes Interessadas e a Equipe;
- Como: indicado pelos Subsídios e Premissas;
- Quando: indicado pelas Entregas, e
- Quanto: indicado pelos Riscos e Esforço.

8.3.1. Problemas e Oportunidades

Todo projeto se inicia porque há algum problema a ser resolvido ou uma oportunidade a ser aproveitada (Kerzner, 2017; OMG, 2018). Ou seja, se deseja uma mudança no estado da organização, incluindo o lançamento de produtos ou serviços para o mercado. Nas discussões sobre projetos de software, os problemas logo aparecem na forma de descontentamento com um sistema atual, informatizado ou não, que funciona com falhas, erros ou não-conformidades, ou uma expectativa para um produto novo.

Um problema é uma diferença entre um estado percebido e um estado desejado, que deverá ser tratada pelo projeto. São dificuldades a serem resolvidas, desvios de um padrão, regulamento ou lei, falta de capacidade de realizar uma tarefa, etc.

Problemas trazem prejuízo, sejam eles financeiros ou de outra forma, como impacto na imagem da empresa. Quando esses prejuízos podem ser medidos de forma quantitativa, como o tempo de atendimento, deve ser indicada a forma de medi-los, de maneira que isso possa ser usado para verificar os benefícios que o projeto trará. Quando forem prejuízos mais qualitativos, como percepção dos clientes, também se deve buscar formas de avaliar o impacto do projeto.

Os problemas podem ser divididos de várias formas. Uma delas é entender que existem problemas por não fazer algo, problemas funcionais, ou por não fazer algo como deve ser feito, problemas operacionais. Ambos têm efeitos no negócio, que são problemas de negócio, ou sintomas. Um problema de negócio pode ser a existência de longas filas em um restaurante, que pode ser causado por um problema operacional, a demora no cálculo da conta. Outro problema de negócio pode ser a dificuldade de entrar em contato com os clientes, que pode ser causado por um problema funcional, como a não existência de um cadastro de clientes.

Problemas podem ser descobertos e priorizados com uso de método de Pareto. É possível, porém, que o método de Pareto detecte apenas os sintomas. Por exemplo, se usarmos o método de Pareto para analisar as reclamações feitas em um restaurante, podemos descobrir que a maioria é sobre o excesso de tempo de espera na fila. Porém, esse não é o verdadeiro problema, mas sim um sintoma causado por, entre outras coisas, a demora em fechar uma conta. Para identificar as causas-raiz dos problemas, ou seja, o verdadeiro problema que deve ser resolvido, podem ser usadas técnicas como o Diagrama de Espinha de Peixe, ou os 5 Porquês (Gray, Brown e Macanufo, 2010). Existem outras técnicas disponíveis, principalmente nas literaturas de Controle de Qualidade em geral, Qualidade de Software, Gestão e Marketing.

Já as oportunidades são percepções que algo pode ser feito para que a organização tenha uma vantagem em relação ao estado atual. Oportunidades podem ser levantadas de várias formas, incluindo o uso de *brainstorming* (Tracy, 2015).

Problemas e oportunidades podem ser priorizados por meio de técnicas como MoSCoW (Stapleton, 2003) e dot-voting (Gray, Brown e Macanufo, 2010).

Exemplos de problemas: documentos perdidos, perda de vendas ao longo do processo, fila excessiva, reclamações dos clientes, reclamações dos fornecedores, muitos atrasos nas entregas, produtos ou serviços que dão prejuízo, etc.

Exemplos de oportunidades: novo produto vai ser lançado, dados disponíveis para *data mining*, invenção de um novo serviço, detecção de uma nova necessidade dos clientes, etc.

8.3.2. Objetivos e Metas

Nesta seção do *Software Project Canvas* serão apresentadas as melhorias que o sistema traz ao negócio. Há certa confusão dos termos objetivos e metas, então é importante definir esses termos para o *Software Project Canvas*. Outro termo que pode ser usado é benefício.

Objetivos são mudanças que desejamos na organização. Esses objetivos devem ser SMART, isto é:

- **S**pecíficos,
- **M**ensuráveis,
- **A**lcançáveis,
- **R**ealistas ou **R**ealizáveis, e
- delimitados no **T**empo.

Um exemplo de objetivo SMART simples é: construir um muro (S), de 2 metros de altura (M), com tijolos (A), a um preço de R\$2000,00 (R), em um mês(T).

Em Engenharia de Software, nem sempre é fácil determinar alguns desses fatores, como a medida que indica que o projeto está completo, mas é necessário se esforçar para isso, já que é preciso, em algum momento, se certificar que o projeto alcançou seus requisitos mais básicos.

Metas são resultados que esperamos quando houver a mudança trazida pelo projeto, ou seja, os benefícios trazidos ao negócio por termos alcançados os objetivos. Metas precisam de medidas, subjetivas ou objetivas, também chamadas de qualitativas ou quantitativas, associadas a elas. Por exemplo, o muro pode ter sido colocado para diminuir a quantidade de pessoas que entra em um local, o que pode ser contado, ou para aumentar a sensação de segurança dos moradores, o que é um sentimento subjetivo. Cada objetivo deve ter pelo menos uma meta, e cada meta uma forma de avaliar (Ruble, 1997). Todo objetivo, e suas metas, deve endereçar um item que aparece na justificativa (Finocchio Jr., 2013).

Uma das várias formas de buscar benefícios é usar a heurística IRACIS² (Gane e Sarson, 1979; Ruble, 1997), que os divide em 3 tipos: aumentar receitas, evitar custos e melhorar serviços.

Exemplo de objetivo: detecção de necessidade de repor o estoque, com a meta diminuir reclamações sobre falta de item no estoque e as métricas: número de reclamações que um item não estava no estoque, e número de vendas perdidas porque o item não estava no estoque.

8.3.3. Requisitos Funcionais, Não-Funcionais e Restrições

Segundo a norma ISO/IEC/IEEE 24765:2017, “um requisito é uma condição ou capacidade necessária para uma parte interessada resolver um problema ou atingir um objetivo.”, ou, ainda segundo a mesma norma, “Ele deve ser possuído ou alcançado por um sistema, componente de um sistema para satisfazer um contrato, padrão, especificação ou outro documento formalmente imposto” (IEEE, 2017b).

Requisitos podem ser divididos em dois grandes grupos: funcionais ou não-funcionais. Restrições são tipos especiais de requisitos não-funcionais impostos ao projeto. Requisitos funcionais dizem que funcionalidade o software entrega, na prática, que funcionalidades o usuário exige do software. Já requisitos não-funcionais especificam como essas funcionalidades devem acontecer ou desempenhar. Requisitos funcionais são, em geral, independentes de tecnologias McMenamin e Palmer, 1984.

²Increase Revenue, Avoid Costs and Improve Service

Os Requisitos não-funcionais, funcionais e as restrições podem ser levantados no mesmo passo, apesar de haver uma separação no quadro. Isso facilita a discussão do que deve ser entregue, pois não é necessário dizer “guarde esse requisito para o próximo passo”.

8.3.4. Requisitos Funcionais

Requisitos funcionais dizem o que o software deve fazer. Recomendamos que isso seja feito na forma de histórias do usuário, cujo formato padrão é uma frase do tipo “Como gerente, eu quero listar os vendedores e suas vendas, para determinar os melhores vendedores da empresa” (Cohn, 2004).

Outra forma possível é usar objetivos do usuário, ou seja, os nomes de casos de uso, como: listar melhores vendedores (Cockburn, 2000).

É importante usar uma estratégia para não deixar a lista de requisitos funcionais muito longa. A primeira é usar casos de uso ou história de usuário de nível de abstração alto, ou seja, usar uma estratégia *top-down* no desenvolvimento dessa lista. Dessa forma, os requisitos poderiam ser semelhantes aos encontrados em uma história do usuário de alto nível (Jacobson, Spence e Bittner, 2011b), em uma DFD de nível 0 (Gane e Sarson, 1979), ou ainda em diagrama de casos de uso do nível pipa ou nuvem (Cockburn, 2000). O outro é listar apenas os mais importantes. Para isso, pode ser usada a técnica de priorização MoSCoW (IIBA, 2011; Stapleton, 2003) para eliminar os requisitos não adequados à fase inicial.

Exemplos de requisitos funcionais são:

- no caso de usar casos de uso: comprar livros, enviar mensagem, relatar ausência de item no estoque, e
- no caso de histórias do usuário: como gerente eu quero listar os vendedores e suas vendas para descobrir os melhores vendedores, como leitor eu quero listar os novos lançamentos para poder decidir que livro vou comprar.

8.3.5. Requisitos Não-Funcionais e Restrições

Requisitos não-funcionais e restrições falam da forma como os requisitos funcionais vão ser alcançados. Apesar de podermos considerar que os requisitos não-funcionais são desejados e as restrições são impostas, é difícil para um pessoa não-treinada fazer essa separação de forma rápida, então não nos preocupamos em fazê-la agora. Em outra fase do processo de criação do *Software Project Canvas*, as restrições, ou requisitos não-funcionais, que inviabilizam o trabalho, podem ser eliminadas ou amenizadas.

Estes requisitos podem ser de vários tipos e conhecer uma taxonomia pode ajudar a levantá-los. Sommerville (2015) apresenta uma hierarquia de requisitos não funcionais que foi adaptada na Figura 8.5.

Restrições limitam o trabalho realizado e devem ser bem específicas, quantificadas e ainda indicar por quem são impostas.

Os requisitos não-funcionais podem ser associados a todos ou apenas a alguns requisitos funcionais. Se for o último caso, é interessante fazer essa indicação.

Exemplo de requisito não-funcional: mensagens devem ser na forma de e-mail, o envio de um e-mail deve demorar no máximo 2 minutos, a senha deve seguir as regras de segurança definidas

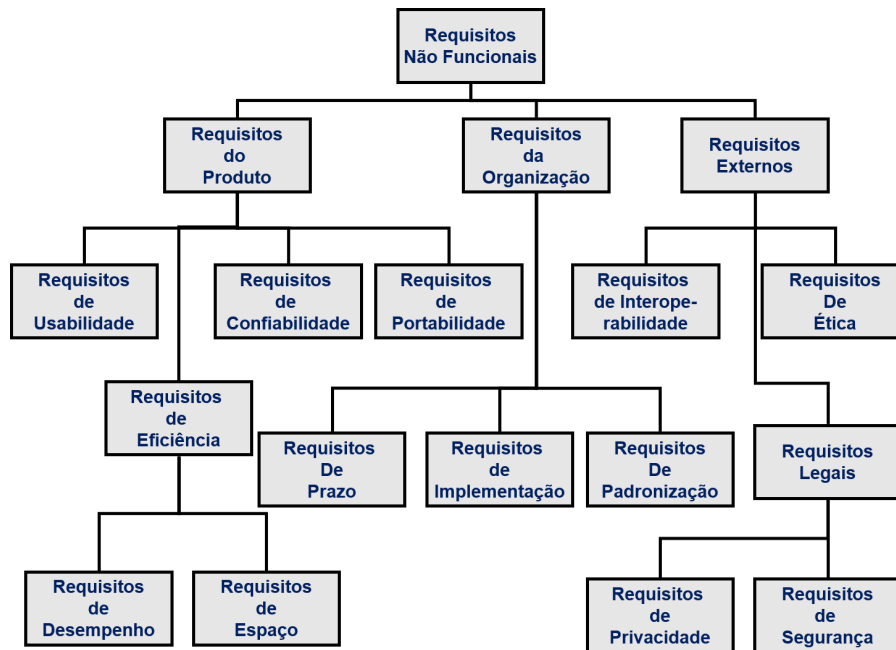


Figura 8.5.: Taxonomia de Requisitos Não Funcionais. Fonte: Sommerville (2015)

pela empresa. Exemplos de restrições: a linguagem de programação só pode ser Java, o banco de dados usado pelo projeto tem que ser Oracle 10.

8.3.6. Partes Interessadas

Uma parte interessada (P.I.), ou *stakeholders*, é “um indivíduo, grupo ou organização que pode afetar, ser afetado, ou perceber que será afetado, por uma decisão, atividade, ou resultado, parcial ou final de um projeto” (PMI, 2017).

As partes interessadas podem ter opiniões diferentes em relação a como projeto vai afetá-las positiva ou negativamente, seja por sua participação ou interesse, quanto a percepções, corretas ou incorretas. Esse efeito pode ser causado tanto pelo projeto como um todo, quanto por partes específicas do mesmo. Algumas serão consideradas *champions* ou evangelistas do projeto, outras podem ser neutras, e ainda outras abertamente, ou secretamente, contra o projeto.

Identificar as todas as partes interessadas é o passo inicial dos processos de gestão das partes interessadas (Jacobson, Lawson e Ng, 2019; PMI, 2017), com o objetivo de garantir a satisfação das mesmas, e com isso o sucesso do projeto. No SEMAT, por exemplo, identificar as partes interessadas é o primeiro estado para o Alpha *Stakeholders*, seguido da escolha de representantes (Jacobson, Lawson e Ng, 2019). Os estados seguintes são nomeados: envolvidas, em acordo, satisfeita para a implantação e satisfeita com o uso (Jacobson, Lawson e Ng, 2019).

Nessa parte do *Software Project Canvas* são listadas todas as partes interessadas do projeto. Devem ser consideradas especialmente três tipos de P.I., os usuários finais, o patrocinador e o governo, como órgão legislador a que o projeto pode estar submetido. Para listá-las, pode ser necessário o uso de técnicas como *brainstorming* (Tracy, 2015) ou *writestorming* (Gray, Brown e Macanufo, 2010).

Outras partes interessadas comuns de encontrarmos são: clientes dos usuários, fornecedores dos usuários, futuros operadores do software, sistemas que podem ser acessados ou acessar o

software, e seus mantenedores, organizações e suas partes que podem ter algum interesse no resultado do software.

É importante que sejam usados, sempre que possível, termos ligado ao negócio. Assim, um médico não tem clientes, mas pacientes, enquanto um banco tem correntistas.

Fatores externos fora do controle do projeto também podem ser listados aqui. Não é incomum que projetos de software sofram o impacto do valor do dólar, por exemplo.

Por conveniência do processo de definição do projeto de software, a equipe de desenvolvimento, que é, em geral, uma parte interessada, não deve ser considerada nesse momento.

Exemplos de partes interessadas são: vendedores, compradores, pacientes, patrocinador, diretoria de operações, etc.

8.3.7. Equipe

Nesse quadro é indicada a equipe necessária para o desenvolvimento do projeto. Cada *post-it* deve indicar um papel a ser tomado pelos membros da equipe. Quando for necessário mais de uma pessoa, isso deve ser indicado no mesmo *post-it*. Possivelmente, algumas pessoas podem estar nomeadas no mesmo *post-it*, ou em outro, de outra cor, possivelmente menor, colocado sobre o primeiro.

Alguns papéis são esperados Finocchio Jr., como o gerente do projeto e um ou mais programadores. Alguns papéis também são esperados em funções do processo de desenvolvimento de software sendo usado. Por exemplo, se é usado o Scrum, esperamos encontrar o Scrum Master, o Product Owner, que pode, por outro lado, ser listado como parte interessada, e o Time, que será formado de várias pessoas.

8.3.8. Subsídios e Premissas

Nesta seção do *Software Project Canvas* deve aparecer tudo que é necessário para que o projeto tenha sucesso. Tipicamente, em projetos de software, subsídios e premissas estão ligados ao acesso da equipe às Partes Interessadas, e a participação das mesmas quando necessário.

Também é comum que existam premissas ligadas a disponibilidade da infraestrutura necessária para o sucesso do projeto. Em muitos projetos também é necessário tratar aqui a disponibilidade financeira.

Premissas e subsídios são garantidas pelas partes interessadas, em favor da equipe. Caso uma parte interessada não cumpra seu compromisso em garantir a premissa, ocorre um risco do projeto. Assim, a análise de risco deve considerar todas as premissas feitas.

Exemplo de premissas ou subsídios: os computadores novos serão entregues dia 15 de maio, os computadores possuirão uma placa gráfica compatível com a NVidia 3060, serão contratados cinco novos programadores, o patrocinador disponibilizará 4 horas por semana para o projeto.

8.3.9. Entregas

Um projeto de software moderno é feito de forma iterativa e incremental (Pressman e Maxim, 2019; Rubin, 2013), o que implica em várias pequenas entregas, que se iniciam com protótipos,

provas de conceito ou um produto mínimo viável, e se seguem de versões ou *releases* que fornecem cada vez mais valor aos seus usuários.

Essas entregas devem ser previstas, sendo que as entregas a curto prazo devem ser previstas com mais detalhe de que as de longo prazo, já que os métodos de desenvolvimento de software usados na indústria se tornaram, na sua maior parte, adaptativos (digital.ai, 2022).

Em cada entrega é entregue um conjunto de funcionalidades, ou seja, são cumpridos requisitos funcionais. Também são garantidos os requisitos não funcionais e as restrições. Assim, as entregas devem ser construídas a partir desses quadros anteriores, levando em consideração a equipe prevista.

É possível que as entregas sejam descritas de forma a implementar parcialmente vários requisitos de forma parcial, como o que é proposto por Jacobson, Spence e Bittner (2011b). Por exemplo, um requisito que diga “Como professor eu quero enviar mensagens para alunos para dar avisos sobre a próxima aula”, pode em uma entrega enviar e-mails, na seguinte incluir mensagens por WhatsApp, e assim por diante. Nesse caso, deve haver cuidado na descrição das entregas, para deixar claro que os requisitos não são cumpridos integralmente.

Entregas também podem ser desenvolvidas, mais tarde, ou como resultado intermediário para gerar o *Software Project Canvas*, por meio de uma Estrutura Analítica de Projeto (EAP), também conhecida pelo nome em inglês, que faz mais sentido, *Work Breakdown Structure (WBS)*. Nesse caso, deve ser seguida a abordagem de criar uma hierarquia de entregas, e não de processos.

É importante que as entregas contenham prazos, que indicam o compromisso inicial da Equipe para satisfazer as Partes Interessadas.

Exemplos de entregas: lista de livros, seleção e compra - em 40 dias; relatórios de acompanhamento - em mais 20 dias, sistema de recomendação - em 60 dias;

8.3.10. Riscos

Riscos são incertezas que podem influenciar o resultado do projeto. Atualmente riscos são divididos em ameaças e oportunidades, isto é, existem riscos negativos e positivos.

É importante para a gestão do projeto gerenciar o risco, para isso devem ser conhecidos sua causa, isto é, porque ele acontece, sua probabilidade de ocorrer e seu impacto. Com isso é possível calcular o efeito do risco no projeto (Kerzner, 2017; PMI, 2017).

Os riscos podem ser aceitos desprezados, por terem um impacto baixo, aceitos e gerenciados, mitigados, ou, ainda, eliminados ou transferidos. Essas ações foram listadas do que deve ser feito para os riscos de impacto mais baixo até o mais alto.

Todo projeto tem riscos gerais e específicos. Riscos gerais são geralmente reconhecidos pela comunidade. Um risco geral típico de projeto de software é a perda de um membro importante da equipe, ou que possui um conhecimento único.

Toda premissa é candidata a gerar um risco, que deverá ser analisado. Existem categorias reconhecidas, como riscos técnicos, externos, organizacionais ou de gerenciamento (PMI, 2017). Para os riscos gerais podemos encontrar *checklists* a serem seguidas (Thomsett, 2002). Já para os riscos específicos é necessário analisar atentivamente o projeto, suas premissas, complexidades e outros fatores.

Existem várias heurísticas para tratar o risco, mas a princípio, para cada risco levantado deve ser estimada sua probabilidade e seu impacto. O produto dos dois implica em uma reação. Finocchio Jr. (2013) sugere usar duas escalas de 1 a 10 para avaliar a probabilidade e o impacto, fazendo o produto e obtendo as recomendações segundo a Tabela.

Probabilidade × Impacto	Ação
1 a 3	Aceitar passivamente
4 a 10	Aceitar ativamente, com um plano de resposta
11 a 36	Mitigar o risco
37 ou mais	Eliminar ou transferir o risco

Tabela 8.1.: Ações recomendadas por (Finocchio Jr., 2013) de acordo grau de importância do risco

Exemplo de riscos: não conseguir contratar cinco novos funcionários com a habilidade necessária (probabilidade 7, efeito 8), servidor atual não aguentar a carga do novo sistema.

8.3.11. Esforço

Nesse quadro deve ser estimado o esforço para desenvolver o software em homens-mês ou homem hora. No momento da reunião onde o canvas é usado, não é possível ainda usar uma técnica fortemente quantitativa para isso, pois não se espera que seja possível nem estimar o tamanho real do software, então isso deve ser feito a partir da experiência do grupo, com comparações com projetos anteriores.

Técnicas adequadas nesse ponto são variações do método de Delphi (Dalkey, 1967), como o *Planning Poker* (Cohn, 2005), ou uso de heurísticas como buscar previsões pessimistas (P), médias (M) e otimistas (O) e calcular uma média ponderada ($M_p = \frac{P+4 \times M+O}{6}$) (PMI, 2017).

Outra possibilidade é usar técnicas rápidas de Contagem de Pontos de Função (IFPUG, 2010).

8.4. A Metodologia do *Software Project Canvas*

A metodologia utilizada do *Software Project Canvas* possui 4 grandes passos, sendo inspirada na metodologia proposta pelo *Project Model Canvas*. Ele está descrita em BPMN nas Figuras 8.6 e 8.7

8.4.1. Conceber Projeto

A principal atividade criativa da metodologia é Conceber o Projeto. Isso é feito em uma, ou mais, reuniões facilitadas, em que o Canvas é usado como foco de colaboração. Os passos dessa reunião são descritos na Figura 8.7, porém esse segundo processo deve ser entendido de forma genérica, já que não há proibição de se voltar a um passo anterior para melhorar, apagar ou propor novos itens.

Essas reuniões devem contar com representantes da equipe e das partes interessadas, incluindo o patrocinador, clientes, e membros do escritório de projetos ou assemelhados, se essa função

8. Software Project Canvas

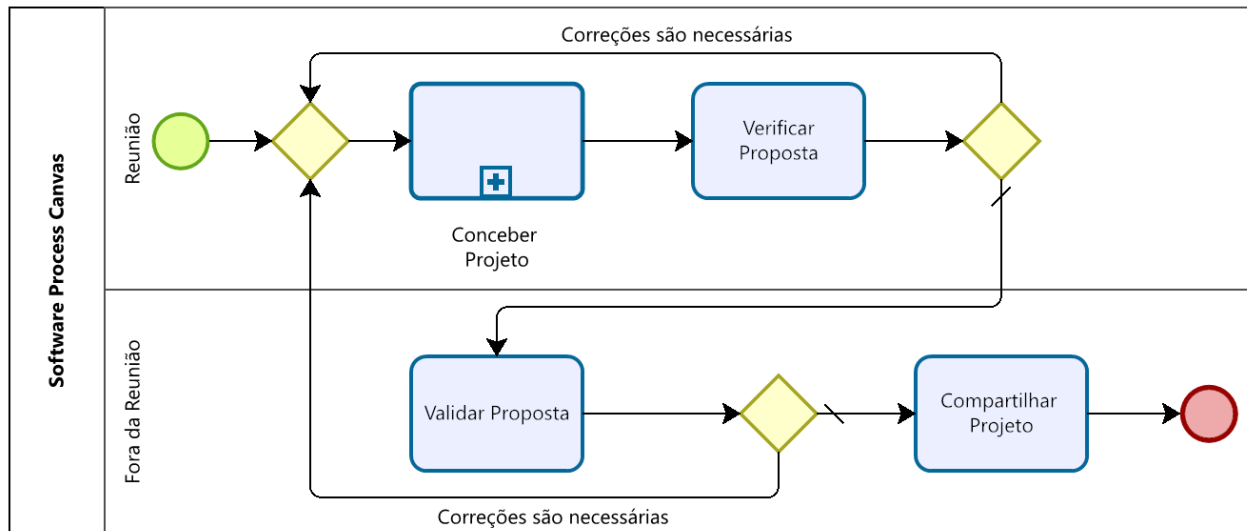


Figura 8.6.: Descrição do processo de criação do Software Project Canvas. Fonte: imagem do autor

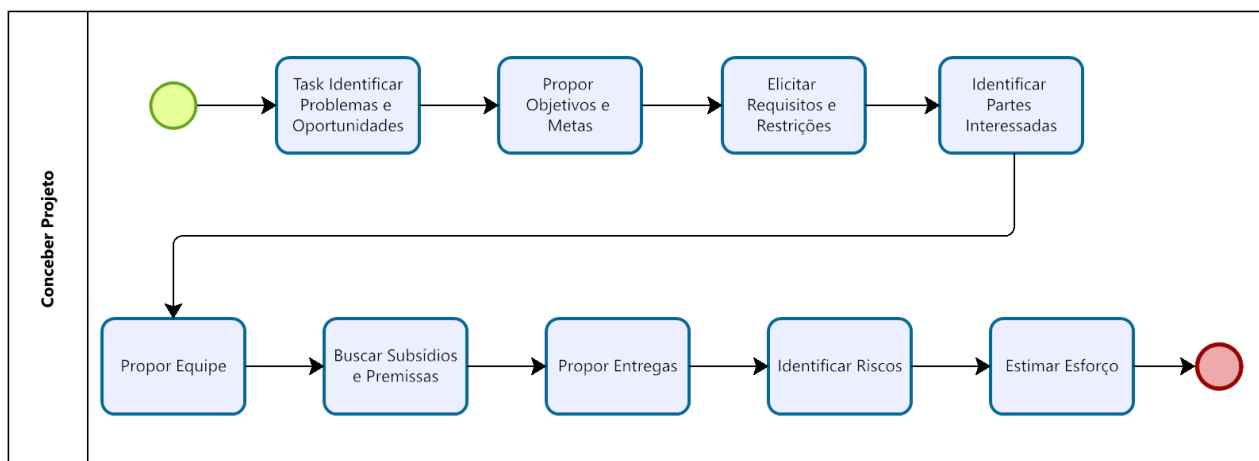


Figura 8.7.: Descrição do sub-processo Conceber Projeto, do processo de criação do Software Project Canvas. Fonte: imagem do autor

existir na organização. É essencial que existam pessoas com a compreensão da gestão de projetos de desenvolvimento software para liderar a reunião, explicar conceitos e trazer sua experiência nas propostas e estimativas.

Durante a reunião, cada passo é focado em preencher um quadro, e para isso deve ser usada uma estratégia em diamante, onde em um primeiro momento deve ser usada uma prática de divergência, ou geração criativa de ideias, normalmente o *brainstorming* (Tracy, 2015) ou técnica derivada, ou assemelhada, e em um segundo momento uma técnica de convergência, seja ela de a busca de consenso, priorização ou votação, como por exemplo MoSCoW (Stapleton, 2003) ou *dot-voting* (Gray, Brown e Macanuf, 2010).

Os quadros devem ser inicialmente preenchidos sem limites, já que é necessário estimular a criatividade durante o momento de divergência. Já durante o momento onde se faz a convergência eles devem ser agrupados em ideias semelhantes e postos em ordem de prioridade, urgência ou importância. Além disso, é necessário garantir que todos os *post-its* estejam em um mesmo nível

de abstração, se necessário agrupando vários em um só, ou separando um em vários. Em cada passo, então, são colocados, modificados e retirados *post-its* no canvas, até que os participantes estejam satisfeitos.

É importante notar que a metodologia faz, em um só passo, todo o levantamento de requisitos funcionais, não-funcionais, e restrições de alto nível. Isso foi decidido pelo fato de, ao usar outras metodologias, ser comum que um item desses tópicos seja proposto enquanto outro tópico é feito. Unificando esse momento, não é necessário parar para discutir questões de ordem da reunião.

É possível que durante um quadro, como por exemplo “Premissas”, se descubra a necessidade de incluir um novo *post-it* em um quadro anterior, por exemplo, uma nova parte interessada. Isso é totalmente permitido, não sendo necessário refazer todo o trabalho do quadro anterior, porém devemos verificar com atenção esse novo item no próximo passo.

Ao longo da Concepção, é importante que os participantes da reunião estejam atentos às regras que serão usadas na verificação, principalmente no momento de convergência, para garantir desde início a qualidade do quadro.

É possível também usar *post-its* com cores e tamanhos diferentes para dar significados específicos. Por exemplo, podem ser usados *post-its* vermelhos para chamar atenção de itens que foram colocados em um quadro após eles já terem sido resolvidos em uma primeira passagem.

8.4.2. Concepção Iterativa e Incremental

É possível realizar a concepção do processo de forma iterativa e incremental, mesmo que em apenas uma seção. Para isso, se inicia cada iteração com um problema ou oportunidade, normalmente o mais importante, e se passa por todos os quadros atendendo apenas aquele problema, e mais nenhum outro. Ao terminar a iteração, deve ser analisado se é razoável fazer outra iteração ou o projeto já está definido.

Durante cada iteração pode ser necessário adaptar o resultado das iterações anteriores para atender os requisitos da nova iteração. Por isso é importante manter alguma memória ao final de cada iteração, por exemplo fotografando o quadro, ou o reproduzindo rapidamente em uma ferramenta digital, como Miro ou mesmo o Power Point.

Mesmo que a próxima iteração do projeto vá resolver apenas o primeiro problema, é interessante já começar com alguns problemas, e suas soluções subsequentes, determinados, levando a criação de um *road-map* do produto. Técnica similar é feita no Scrum, onde é mantido o Product Backlog e escolhido apenas um Sprint Backlog para realizar a cada iteração (Satpathy et al., 2016).

Se o quadro é mantido fixo na parede ao longo do projeto, pode ser revisado também a cada iteração.

8.4.3. Ordens Alternativas de Preenchimento

Apesar de ser comum planejar um projeto na ordem inicial Por que, O que e Quem, definindo o contexto para o qual as outras perguntas vão ser respondidas, é possível fazê-lo em outra ordem.

O Software Project Canvas não impõe, então, que seja usada apenas a ordem proposta, e ainda oferece canvas pré-prontos para duas ordens alternativas:

- iniciar com as Partes Interessadas, encontrar os Problemas e Benefícios que elas tem e só depois projetar o que será o projeto, e
- iniciar com o porquê, descobrir as partes interessadas e então passar para os requisitos.

Essas ordens alternativas representam situações diferentes na vida real.

Na nossa proposta inicial, apresentada na Figura 8.1 supomos que a organização tem problemas ou oportunidades, e a partir daí o projeto é criado e, então, olhando esse quadro, são determinadas todas as partes interessadas.

Na segunda proposta, primeiro foi identificado que algumas partes interessadas estão descontentes, ou precisam de algo. É provável que estas partes interessadas estejam inclusive presente na reunião, e começar por identificá-las pode ajudar a procurar os problemas e seguir com o projeto.

Finalmente, na terceira proposta, imaginamos que os problemas foram identificados, e logo depois identificamos quem está envolvido com esses problemas.

Outra situação comum é termos como demanda inicial o "O quê". Nesse caso, não é recomendável começar pelos requisitos, mas sim pelos Problemas ou pelas Partes interessadas, para garantir uma melhor elicitação dos verdadeiros requisitos do projeto.

8.4.4. Verificar Proposta

A segunda atividade do processo tem como objetivo verificar a correção do *Software Project Canvas* que será apresentado para validação na organização e com as outras partes interessadas.

Nessa fase, os participantes da reunião passam a se preocupar com a correção local e global do canvas, buscando garantir que ele apresenta um projeto coeso, racional e viável.

Para isso, além de verificações *ad-hoc*, é possível seguir um *check-list* apresentado a seguir.

- Verificações para os *post-its*
 - Todo *post-it* é claro e pode ser entendido por qualquer leitor ou parte interessada?
 - Todo *post-it* está no quadro a que deve pertencer?
- Verificações para os quadros
 - Os quadros estão organizados de forma que grupos de *post-its* representem ideias semelhantes e que a importância ou prioridade de cada *post-it* dentro do quadro ou assunto esteja representada?
 - Todos os *post-its* dentro de um quadro estão em um mesmo nível de abstração?
 - Existem, em um quadro, elementos que são um sub-grupo de outro elemento no quadro. Por exemplo, uma parte interessada pode ser "governo federal" e outra "Ministerio da Educação". Isso é realmente desejado?
- Verificações para o canvas
 - Todo o canvas está no mesmo nível de abstração?
 - A linguagem é consistente entre os quadros e *post-its*?
 - Todo problema e oportunidade está atendido por meio de um objetivo ou meta?
 - Todo objetivo, e meta, busca atender um problema ou oportunidade?
 - Toda meta tem uma métrica?
 - Os requisitos funcionais e não-funcionais atendem o Por Que do projeto?
 - As partes interessadas estão envolvidas com algum por que ou algum requisito ou restrição do projeto?
 - Requisitos e entregas deixam as partes interessadas satisfeitas?

8. Software Project Canvas



(a) Forma alternativa onde o projeto se inicia com as partes interessadas. Fonte: imagem do autor



(b) Forma alternativa onde o projeto se inicia com os problemas e as partes interessadas. Fonte: imagem do autor.

Figura 8.8.: Desenhos alternativos, que levam a processos alternativos para a fase de Conceber Projeto.

- Os subsídios e premissas são suportados por alguma parte interessada?
- A equipe consegue atender os requisitos e cumprir as entregas?
- As entregas atendem o porquê do projeto?
- Todo risco está associado a alguma premissa, entrega, requisito ou restrição?
- Toda premissa tem um risco, ou o risco foi pelo menos avaliado e considerado nulo?
- As estimativas são razoáveis para o projeto e deixam as partes interessadas satisfeitas?

8.4.5. Validação

Ao completar a verificação do canvas, é necessário validar o projeto com outras partes da organização, principalmente, com as partes interessadas interessadas que possuem, simultaneamente, poder, urgência e legitimidade sobre o projeto (Mitchell, Agle e Wood, 1997). Para isso, deve ser feito um documento inicial com a proposta do projeto, que pode ser uma cópia do próprio canvas.

Fora da reunião, então, o projeto é validado e possíveis *feedbacks* podem ser trazidos ao grupo original para melhorar, refinar, modificar ou mesmo abandonar o projeto, caso tenham sido encontradas falhas irremediáveis.

Este método não define uma forma fixa de se fazer a validação. Inclusive, grupos diferentes em uma mesma organização podem usar formas diferentes de validação. Podem ser usados reuniões, workshops, questionários, entrevistas, etc.

O resultado dessa validação também não é definido em formato, e depende muito do tipo e da qualidade do *feedback* recebido. A partir do resultado, o grupo responsável por lançar o projeto pode decidir não fazer nada, fazer pequenos ajustes ou voltar a fase de concepção.

8.4.6. Compartilhar Projeto

Na última fase do planejamento o projeto é compartilhado com a organização. Novamente, a forma de fazer isso depende do projeto, da organização e da parte interessadas sendo atingida. Pode ser necessário criar um plano de projeto formal para aprovação final em uma reunião anual, ou criar documentos mais ligados a métodos ágeis, como uma visão do projeto ou um *backlog* do produto para equipe de desenvolvimento, ou uma notícia para o jornal de divulgação interna.

8.5. Conclusão

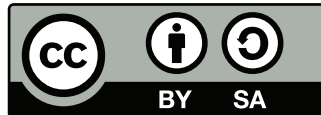
Este artigo apresenta o *Software Project Canvas* um novo canvas para o planejamento ágil de um projeto de software. Ele foi criado para ser usado em substituição a outros canvas destinados ao planejamento inicial de projetos em geral, que são menos adaptados a linguagem e a prática dos desenvolvedores de software. Para a sua criação foram observados 6 dificuldades encontradas ao ensinar ou utilizar o *Project Model Canvas*, que serviu de fonte de inspiração. As adaptações feitas visam facilitar a explicação e o uso, além de evitar perda de tempo com discussões que não adiantam a definição do projeto.

No futuro será feita uma avaliação do uso do canvas em sala de aula e em projetos de software, a fim de validar as modificações feitas.

Agradecimentos

Agradeço a Yasmin pelo design final do *Software Model Canvas* e aos alunos, como Eduardo Mangeli e Marcus Parreira, que fizeram críticas e sugestões.

Licença



Software Model Canvas Copyright © 2021 Geraldo Xexéo está licenciado com uma Licença Creative Commons - Atribuição-CompartilhaIgual 4.0 Internacional.

Baseado no trabalho disponível em:

<https://github.com/xexeo/Software-Project-Canvas>.

Podem estar disponíveis autorizações adicionais às concedidas no âmbito desta licença com [email:xexeo@ufrj.br](mailto:xexeo@ufrj.br).

O texto completo da licença pode ser obtido em: <https://creativecommons.org/licenses/by-sa/4.0/>

Modelagem de Processos de Negócio

The problem is that at a lot of big companies, process becomes a substitute for thinking. You're encouraged to behave like a little gear in a complex machine. Frankly, it allows you to keep people who aren't that smart, who aren't that creative.

(Elon Musk)

Conteúdo

9.1.	Processos de Negócio	122
9.2.	Exemplos de Processos	123
9.3.	Representação de Processos de Negócio	124
9.4.	Por que Entender os Processos de Negócio	124
9.5.	Exercícios	124

Por que processos de negócio?

Os sistemas de informação executam dentro das organizações para atender processos de negócio, logo é precisos entendê-los para melhor atender as partes interessadas.

Cada vez mais a Modelagem de Processos de Negócio vem sendo usada como uma ferramenta principal ou de auxílio do processo de desenvolvimento de software, incluindo data warehouses e aplicações relacionadas, visando o levantamento completo dos requisitos do sistema.

9.1. Processos de Negócio

Uma das informações importante para o conhecimento de uma organização é entender o que ela faz e como ela faz para atingir seus objetivos de negócio.

Normalmente, isso é obtido por meio da descrição de seus processos de negócio.

Um **processo** “é uma coleção de tarefas inter relacionadas que são iniciadas em resposta a um evento, e que tem como objetivo alcançar um resultado específico para o consumidor do processo”(Scheel et al., 2015).

Processos aparecem em todos os lugares, e podem ser documentados de maneira mais ou menos formal. Um processo simples que usamos no dia a dia é o de fazer um café expresso em uma máquina de cápsulas. Os passos desse processo podem ser descritos como:

1. ligar a máquina de expresso;
2. escolher a cápsula de café;
3. colocar água na máquina;
4. colocar a cápsula na máquina
5. colocar a xícara no local indicado
6. apertar o botão de fazer café
7. remover a xícara

É possível notar que mesmo um processo simples que é feito no dia a dia apresenta dificuldades de descrição. Na lista anterior, por exemplo, não se discute o que fazer se a máquina não estiver na tomada, ou se uma cápsula já está na máquina, ou se já tem água na máquina. Isso significa que uma descrição de um processo simples de forma completa pode exigir um cuidado muito grande com detalhes, se tornando, no final, muito complexa.

Todas as organizações realizam processos. Esses processos podem ter nomes como “Atender Pedido do Cliente” ou “Fabricar Peça”.

A **modelagem de processos** demonstra como funciona a organização, passo a passo, no seu dia a dia. A partir dela pode ser possível levantar os pontos a serem automatizados de um processo e como os processos realmente realizados diferem dos processos normatizados da organização.

Um **processo de negócio** “uma coleção de tarefas e atividades (operações e ações de negócio) consistindo de empregados, materiais, máquinas, sistemas e métodos, que são estruturadas de forma a projetar, criar e entregas um produto ou serviço ao consumidor”(Scheel et al., 2015).

Processos de negócio são sempre descritos a partir de um ponto de vista que indica a profundidade com que são tratados. Um processo de “Calcular o Preço Final”, por exemplo, tanto pode ser visto apenas como uma atividade única, como por várias etapas que passam por partes diferentes da organização, como ainda por um algoritmo detalhado.

Entre as características principais de processos de negócio é possível citar que eles sempre são sequências de passos, sempre respondem a uma necessidade ou demanda que define seu início e sempre chegam a um resultados específico e mensurável, seja ele o sucesso ou o fracasso do atendimento a necessidade ou demanda original.

A **Modelagem de Processos de Negócio** tem como objetivo entender os processos que ocorrem na organização, listá-los, descrevê-los por meio de uma representação adequado, indicando o passo a passo em um nível de abstração adequado ao propósito e, também, verificar se as representações correspondem a realidade.

Linguagens de descrição de processo de negócio, como ARIS e BPMN, se preocupam em fornecer uma abstração razoável para a descrição de processos. Normalmente essa abstração permite descrições tanto de um ponto de vista muito amplo, com a supressão da maioria dos detalhes, que visa mais entender o que é feito, como um ponto de vista razoavelmente detalhado, que permite ao leitor entender realmente como o processo é executado na organização, ou seja, como algo é feito. Cabe ao modelador do processo escolher e manter esse nível de abstração ao longo do trabalho.

9.2. Exemplos de Processos

É interessante ver alguns exemplos de processos sendo descrito na forma de uma narrativa simples que não inclui alternativas.

O processo de entrega de uma pizza pode ser descrito como:

1. O atendente recebe o pedido de pizza e bebidas
2. O pedido da pizza passa para o pizzaiolo, que prepara as pizzas e coloca na fila para assar
3. O pizzaiolo coloca as pizzas do pedido para assar
4. Quando as pizzas estão prontas, o pizzaiolo retira e passa as pizzas para seu ajudante colocar em caixas
5. Se foram pedidas bebidas, o atendente junta as bebidas ao pedido e o separa
6. O entregar pega o pedido completo e passa no caixa para pegar o troco
7. O entregador entrega a pizza
8. O entregador cobra a pizza do cliente
9. O entregador traz o dinheiro para o caixa

Já uma loja de construção pode ter o seguinte processo para atender um pedido:

1. O vendedor recebe o pedido
2. O vendedor levanta o preço de cada produto
3. O vendedor confirma com o cliente os produtos que vão ser comprados
4. O vendedor envia a lista de produtos vendidos para o caixa
5. Enquanto o caixa faz a cobrança, O vendedor separa os produtos desejados
6. Enquanto o vendedor separa o pedido, o caixa cobra o pedido do cliente
7. Enquanto o vendedor separa o pedido, O caixa emite a nota fiscal
8. O vendedor junta a nota fiscal ao pedido
9. O vendedor entrega o pedido ao cliente

O que essas duas descrições permitem entender sobre processos de negócio, e mesmo processos em geral?

Primeiro, elas descrevem um passo a passo do que é feito. Todo processo é um passo a passo para realizar algo. Segundo, elas começam com um evento. Todo processo tem que ter um motivo para começar e deve atender uma necessidade definida por esse motivo.

Outra característica importante: os passos de um processo levam a organização a chegar mais perto do resultado desejado.

Além disso, no processo da pizzeria existe um passo opcional, enquanto no processo da loja de ferragens existem passos paralelos. Tanto passos opcionais como paralelos são comuns em processos de negócio.

Finalmente, os dois são descrições parciais do funcionamento do negócio.

9.3. Representação de Processos de Negócio

Várias são as formas de representar processos de negócio. É possível dividir essas formas em dois grupos básicos: formas textuais e formas gráficas.

Entre as formas textuais encontramos os Casos de Uso, as narrativas formais ou informais, linguagens declarativas e outras.

Entre as formas gráficas, podemos encontrar formas comuns mas hoje consideradas ultrapassadas, como fluxogramas e diagramas IDEF0 e IDEF3, formas usadas na academia mas com pouco uso na área de negócios, como Redes de Petri, e formas atuais em uso no mercado, como ARIS, Diagramas de Atividade e a principal ferramenta da atualidade: os diagramas BPMN.

9.4. Por que Entender os Processos de Negócio

Normalmente sistemas de informação estão inseridos e automatizam processos de negócio. Eles atuam garantindo que aconteçam, ajudando na sua gerência, monitoração e controle nos vários níveis da organização.

Durante os processos de negócio dados primitivos são criados e armazenados. São os sistemas de informação que tratam esses dados, e possibilitam gerar informações adicionais, derivadas a partir deles.

Assim, entender os processos de negócio facilita a entender o ambiente dos sistemas de informação e seus requisitos.

9.5. Exercícios

Exercício 9.1:

Descreva o processo de fazer o café expresso em uma máquina de cápsulas da forma mais detalhada possível, considerando as seguintes condições iniciais:

- a máquina de café está em cima do balcão, desligada da tomada e sem água;
- as xícaras estão no armário;
- as cápsulas de café ainda estão fechadas em sua caixa com 10 unidades, na despensa;
- as portas do armário e da despensa estão fechadas;
- é possível pegar água no filtro.

Risk comes from not knowing what you're doing.

(Warren Buffett)

Conteúdo

10.1.	Usos de BPMN	126
10.2.	Participantes	126
10.3.	Símbolos básicos	127
10.4.	Atividades	127
10.5.	Eventos	128
10.6.	<i>Gateways</i>	132
10.7.	Fluxos	134
10.8.	Alguns Exemplos	134
10.9.	Eventos Anexados a Atividades	135
10.10.	Exercícios	135

Por que BPMN?

BPMN é o padrão *de facto* da modelagem de processos, ferramenta essencial para qualquer projeto de sistema de informação para uma organização

BPMN, sigla para *Business Process Model and Notation*, é uma linguagem gráfica para a modelagem de processos de negócio padronizada pela OMG e que está se tornando o padrão *de facto* da indústria.

A Figura 10.1 mostra um exemplo razoavelmente simples de um processo descrito em BPMN. Nesse diagrama podemos ver um processo que se inicia na tarefa “Receber Pedido de Crédito” e

continua na tarefa “Aprovar Pedido de Crédito”. Nesse momento encontramos um *gateway*, que nesse caso é uma decisão onde apenas um caminho pode ser seguido (o losango sem símbolo), e caso o pedido não tenha sido aprovado, seguimos para a tarefa “Avisar Resultado”, ou, caso o pedido tenha sido aprovado, seguimos para a tarefa “Efetuar Transação de Crédito” e então também para a tarefa “Avisar Resultado”, que sempre leva ao fim do processo.

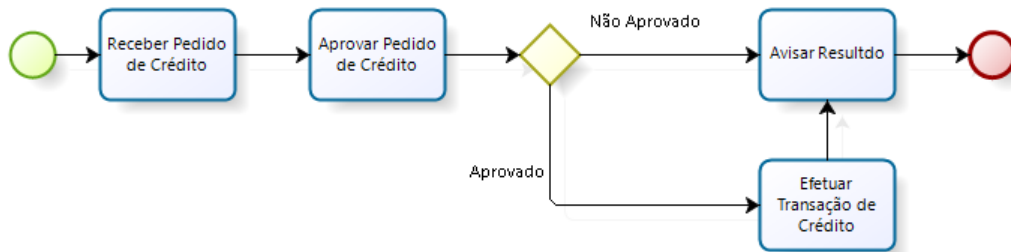


Figura 10.1.: Um exemplo de processo descrito em BPMN. Fonte: imagem do autor

10.1. Usos de BPMN

BPMN foi criado para unificar a representação de processos de negócio, tendo em vista a grande quantidade de linguagens existentes. Além disso, ele mira permitir não só a descrição de processos, mas também sua simulação e execução.

BPMN pode ser usado para criar **diagramas de processo**, **diagramas de colaboração** e **diagramas de coreografia** (OMG, 2013)(OMG, 2013).

Processos, ou orquestrações, podem ser privados ou públicos. Processos privados ainda podem ser executáveis ou não. Processos internos também são chamados de *workflows*, ou *orquestrações* no caso de serem suportados por *web services*(OMG, 2013), e descrevem processos dentro de organizações.

Um processo executável é modelado para ser executado, enquanto processos não executáveis são criados para documentação do comportamento de um processo de negócio da organização (OMG, 2013).

Um processo é **público** quando mostra a interação entre um processo privado e outro processo, ou participante, e sua representação só inclui as atividades usadas para se comunicar com outros participantes (OMG, 2013).

Uma **colaboração** “mostra as interações entre duas ou mais entidades de negócio”(OMG, 2013).

Uma **coreografia** é a “definição de um comportamento esperado, basicamente um contrato procedural, entre participantes interagindo”(OMG, 2013).

Este terá foco maior na descrição de processos privados e não tratará coreografias. Ele também não tem preocupação com a capacidade de executar os processos que apresenta.

10.2. Participantes

Participantes representam organizações ou papéis que participam em uma colaboração. São normalmente responsáveis pela execução de um processo dentro de uma pool (OMG, 2013).

10.3. Símbolos básicos

BPMN representa processos como uma sequência de tarefas, por meio de um fluxo de controle herdeiro do conceito de fluxograma, controlado por *gateways* e eventos.

Para isso são usados quatro 4 símbolos básicos:

1. **atividades**, representadasa por caixas de bordas arredondadas;
2. **eventos**, representados por círculos;
3. **gateways**, representados por losangos, e
4. **fluxos**, representados por setas.

Além disso possui alguns símbolos adicionais para representar objetos de dados, armazenamento de dados, e mensagens, anotações e grupos. Também é possível criar novos símbolos representando artefatos, contanto que não sejam similares aos já existentes. Finalmente, processos podem ser organizados em *pools* e raias.

10.4. Atividades

As **atividades** são o trabalho que as organizações fazem. Se incluídas em um processo, passam a ser chamadas de **tarefas** ou **subprocessos**. Um processo é composto por um fluxo de atividades que podem ser genéricas ou possuir significado específico. Elas são representadas por caixas de cantos arredondadas e possuem variações descritas pela espessura da caixa, por símbolos no canto superior esquerdo e por símbolos na sua parte de baixo.

Atividades podem ser na verdade chamadas a outros processos ou tarefas (*call activities*) e representam um ponto onde processos ou tarefas globais, como sub-processos, são usados dentro de um processo. Elas são identificados por ter a borda mais grossa.

Atividades são **sempre** nomeadas no formato:

<verbo no infinitivo><objeto>

como em “coletar resíduos” ou “aprovar pedido”.

Tarefas podem ser mais detalhadamente especificadas, por uma marca no canto superior esquerdo, como (OMG, 2013):

- tarefa de envio, onde é enviada uma mensagem a um participante externo;
- tarefa de recepção, que espera a chegada de uma mensagem de um participante externo, muitas vezes usada para iniciar um processo;
- tarefa de *script*, que é executado por um *engine* de processo de negócio;
- tarefa de regra de negócio, que indica o uso de um *engine* que executa regras de negócio;
- tarefa de serviço, que usam um tipo de serviço, como um *web service* ou uma aplicação automatizadas;
- tarefa manual, feitas de forma totalmente manual, ou
- tarefa de usuário, feitas por uma pessoa com auxílio de uma aplicação.

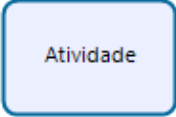
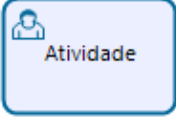
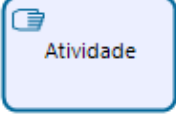
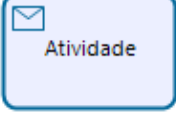
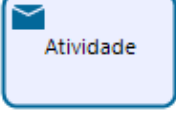
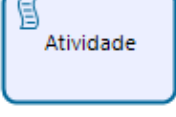
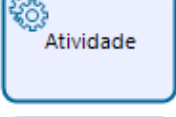
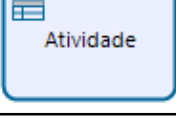
Além disso, marcadores na base de tarefas e processos podem indicar (OMG, 2013):

- um subprocesso colapsado, por meio de uma caixa com um símbolo de soma;
- um *loop*, por meio de uma seta circular;
- múltiplas instâncias em sequência, por meio de 3 barras horizontais;
- múltiplas instâncias em paralelo, por meio de 3 barras verticais;

- uma compensação, por meio de dois triângulo ligados voltados para a esquerda, e
- *ad-hoc*, por meio de um til, apenas para subprocessos;

Também existem símbolos para subprocessos.

Tabela 10.1.: Símbolos para os eventos que podem aparecer no início de um processo.

Significado	Símbolo	Explicação
Simple		atividade simples
Usuário		atividade feita por um usuário
Manual		atividade feita manualmente
Recebimento		atividade que recebe
Envia		atividade que envia
Script		atividade feita por um script
Serviço		atividade feita por um serviço
Regra de Negócio		atividade que segue regra de negócio

10.5. Eventos

Eventos aparecem em três momentos:

1. **iniciais**, que indicam que um processo vai iniciar, e tem a borda simples;
2. **intermediários**, que afetam o fluxo de processo, e tem uma borda dupla, e
3. **fim**, que determinam onde um processo termina, que tem uma borda grossa.

Os eventos podem estar no fluxo do processo ou ligados diretamente a uma atividade. Nesse caso ele podem interromper ou não a atividade.

O círculo do evento possui duas possíveis bordas e um marcador interno, que pode ser preenchido ou não preenchido. As bordas dão indicação da localização do evento no processo.

No caso de um evento ligado a uma tarefa, as bordas normais indicam que o evento interrompem a atividade, e bordas tracejadas indicam que o evento não interrompe.

Eventos podem ser sem tipo, ou especificados detalhadamente em subtipos. Existem vários subespecificações de eventos, mas nem todas podem ser aplicadas a todos os momentos em que o evento pode aparecer (OMG, 2013):

- **cancelamento**, reagindo a cancelamentos ou ativando cancelamentos;
- **compensação**, ativando ou tratando compensações;
- **condição**, reagindo a mudanças em condições do negócio ou relacionadas com as regras de negócio;
- **erro**, capturando ou lançando erros com nomes, semelhantes a exceções em linguagens de programação;
- **escalamento**, indicando a passagem do processo para um nível maior de responsabilidade;
- **ligação**, simplesmente ligando dois pontos como conectores, por exemplo em páginas diferentes;
- **mensagem**, indicando o recebimento ou envio;
- **múltiplo**, capturando um de vários eventos;
- **múltiplo e paralelo**, capturando vários eventos;
- **sinal**, enviados e recebidos através de vários processos e possivelmente tratados múltiplas vezes;
- **temporização**, indicando eventos cíclicos, momentos específicos, prazos e períodos ou faixas de tempo, e
- **terminação**, ativando a terminação final e imediata do processo.

É importante diferenciar um sinal de uma mensagem. Mensagens sempre tem claramente definidas quem as enviou e quem as recebe. Um sinal pode ser recebido por mais de um processo.

O subtipo é especificado por um desenho no interior do círculo que representa o evento. Se o desenho for escuro ou preenchido, significa que o evento está lançando seu sinal, seu erro, ou outra coisa. Se o símbolo for vazado, desenhado apenas com linhas, significa que está capturando.

10.5.1. Eventos Iniciais

Os eventos iniciais possíveis são apenas o simples e mais seis tipos, como listado na tabela 10.2. Esses eventos nunca emitem nada, só capturam mensagens, sinais e múltiplos eventos de início, ou seja, respondem a gatilhos (OMG, 2013).

Tabela 10.2.: Símbolos para os eventos que podem aparecer no início de um processo.

Significado	Início	Explicação
Simple		evento simples de início
Mensagem		processo se inicia quando recebe uma mensagem
Temporizador		processo se inicia em um certo momento, ou uma data específica, ou outra questão relacionada a tempo, como um prazo
Condicional		processo se inicia quando uma condição ou regra de negócio se torna verdade
Sinal		processo se inicia com um sinal recebido
Múltiplo		processo se inicia quando um de vários eventos acontecem, bastando um acontecer
Múltiplo Paralelo		processo se inicia quando vários eventos acontecem

10.5.2. Eventos Finais

Os eventos finais podem ser simples ou de outros 8 subtipos. Eventos finais nunca recebem ou capturam, sempre enviam algo, ou seja, eles criam resultados (OMG, 2013). A Tabela 10.3 mostra os tipos de eventos possíveis no final de um fluxo de processo. É importante notar que os eventos normalmente só terminam o seu fluxo de processo, a não ser o evento de terminação que termina todos os fluxos.

Tabela 10.3.: Símbolos para os eventos que podem aparecer no fim de um fluxo de processo.

Significado	Início	Explicação
Simple		evento simples de fim
Mensagem		processo termina enviando uma mensagem
Sinal		processo termina enviando um sinal
Múltiplo		processo termina enviando vários eventos
Compensação		processo termina e lança compensação
Escalamento		processo termina e escala para nível superior
Erro		processo termina e avisa de erro
Cancelamento		processo termina e cancela
Terminação		processo termina completamente

10.5.3. Eventos Intermediários

Os eventos intermediários são o simples e mais 13, sendo que 7 recebem mensagens ou esperam algo acontecer e 6 enviam alguma comunicação para outros eventos. Em geral eles podem responder a gatilhos ou gerar resultados (OMG, 2013). A Tabela 10.4 mostra todos os eventos intermediários possíveis.

Tabela 10.4.: Eventos que podem aparecer no meio de um processo.

Significado	Simbolo de Recepção	Símbolo de Envio	Explicação
Simples			evento simples intermediário
Tempo			evento ativado por questão temporal
Mensagem			processo recebe ou envia uma mensagem
Sinal			processo recebe ou envia um sinal
Condição			condição acontece
Múltiplo			processo recebe ou envia vários eventos
Múltiplo			processo recebe vários eventos paralelos
Compensação			processo envia compensação
Escalamento			processo escala para nível superior

10.6. Gateways

Gateways servem para indicar o controle do fluxo do processo e são indicadas por um losango. No diagrama, elas podem dividir um fluxo em vários ou juntar vários fluxos em um único (OMG, 2013).

Um *gateway*, como mostra também a Tabela 10.5 pode ser de 6 tipos (OMG, 2013):

1. exclusivo;
2. inclusivo;
3. paralelo;
4. baseado em evento;
5. exclusivo baseado em eventos
6. paralelo baseado em eventos, e
7. complexo.

Um **gateway exclusivo** representa uma decisão de seguir um entre vários caminhos, ou de chegar a um lugar por um caminho entre muitos. É desenhada como um losango vazio, ou com um x dentro dele, conhecido como *gateway* exclusivo. O fluxo de saída pode ser marcado com uma barra diagonal para dizer que é o caminho *default*. Podemos associar o significado do *gateway* com uma instrução de *if* ou *case* de uma linguagem de programação. A Figura 10.1 mostra um gateway exclusivo sendo usado para indicar um caminho caso o pedido de crédito seja aprovado e outro caso não seja aprovado. Ela também demonstra o comportamento *default* do BPMN, que é considerar que se vários fluxos que entram em uma tarefa, apenas um precisa chegar a ela para ela iniciar¹.

¹Esse comportamento é o contrário do ARIS-EPC, onde todos os fluxos devem chegar a ela.

Tabela 10.5.: Símbolos para os gateway.

Significado	Símbolo	Explicação
Exclusivo		escolhe um entre os caminhos
Inclusivo		escolhe alguns entre os caminhos
Paralelo		inicia vários fluxos simultaneamente
Baseado em eventos		inicia o fluxo do primeiro evento que acontece
Exclusivo Baseado em eventos		cada ocorrência de um evento subsequente ativa uma nova instância
Paralelo Baseado em eventos		inicia quando todos os eventos subsequentes ocorrem
Complexo		indica uma regra complexa de decisão

Já um **gateway paralelo**, desenhado com um losango com um símbolo de uma cruz dentro dele, indica que um fluxo deve ser convertido em vários, que deverão ser seguidos em paralelo, ou vários fluxos devem ser combinados em um, sendo necessário que todos os caminhos cheguem ao *gateway* para poder continuar o processo. Eles são equivalentes as operações de *fork* ou *join*. A Figura 10.2 mostra um exemplo de uso desse *gateway*.

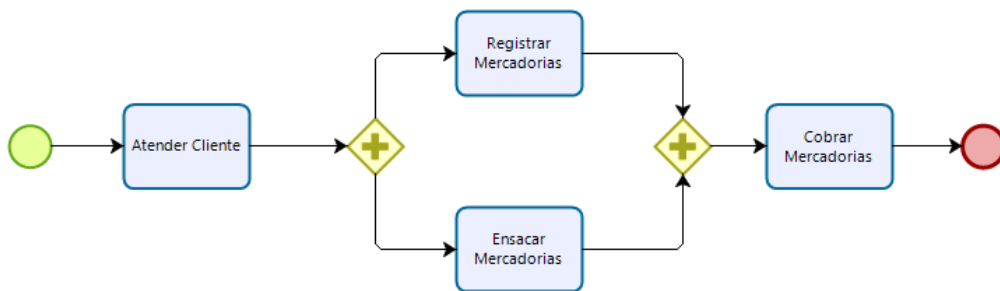


Figura 10.2.: Exemplo de uso de um evento paralelo em BPMN. Após receber o cliente em um caixa o registro de mercadorias e o ensacamento é feito em paralelo, só depois que ambos acabam se faz a cobrança ao cliente. Fonte: do autor.

Temos um **gateway inclusivo** se houver um círculo dentro do losango, temos um caso onde um ou mais caminhos podem ser seguidos e todos devem ser avaliados.

O **gateway baseado em eventos**, o **gateway paralelo baseado em eventos**, e **gateway exclusivo baseado em eventos** são *gateways* bem interessantes. Todos eles depende da existência de eventos nos fluxos subsequentes.

Por exemplo, com o baseado em eventos, depois dele podemos ter dois fluxos, um com um evento que espera uma mensagem e outro que espera passar um prazo, indicando que a mensagem tem um prazo para chegar. Já com o paralelo baseado em eventos podemos seguir vários fluxos, cada fluxo tendo seu início quando o evento ocorre.

Finalmente o **gateway complexo**, que possui um asterisco dentro de si, significa uma comportamento complexo de condições ou sincronização.

10.7. Fluxos

Existem três tipos de fluxos (OMG, 2013):

1. **fluxo de sequência normal**, que indicam a ordem das atividades, e não iniciam em um evento intermediário, são linhas contínuas com uma seta indicando a direção;
2. **fluxo não controlado**, é um fluxo que não é afetado por condições e não passa em um *gateway*,
3. **fluxo condicional**, que pode ter uma expressão de condição que determina se vai ser seguido. Se sai de uma atividade, deve ter um pequeno losango no seu início. Caso saia de um *gateway* é uma linha comum.
4. **fluxo default**, que é usado em *gateways* exclusivos ou inclusivos e mostram o caminho *default* a ser seguido se todas as outras condições nos outros fluxos saindo desse *gateway* forem falsas.
5. **fluxo de mensagem**, que indica o envio de uma mensagem entre dois participantes, indicados por uma linha tracejada que se inicia em um círculo e acaba em uma seta aberta;
6. **fluxo de exceção**, que se inicia em um evento intermediário anexado a um processo, e
7. associações, que ligam elementos do processo a informações, representados por linhas pontilhadas, e não são fluxos de controle.

10.8. Alguns Exemplos

A Figura 10.3 descreve, em BPMN, o atendimento a uma ordem de serviço. Ela representa o mesmo processo descrito em ARIS EPC na Figura ??.

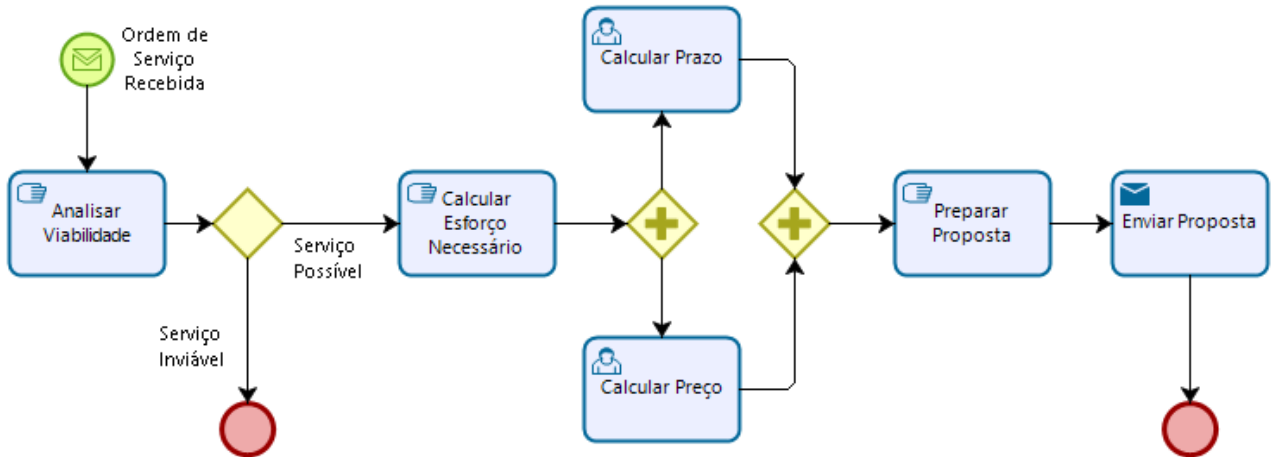


Figura 10.3.: Exemplo do atendimento da ordem de serviço em BPMN, também apresentado na Figura para Aris EPC. Fonte: do autor

10.9. Eventos Anexados a Atividades

Eventos iniciais e intermediários podem ser associados diretamente a uma atividade. Isso é feito colocando o evento sobre a borda da atividade.

Esses tipos de eventos podem interromper as atividades ou não. Caso não interrompam, seu círculos devem ser tracejados. Compensações e erros sempre interrompem a atividade.

10.10. Exercícios

Exercício 10.1:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita desenhe todos os processos que você identificar, usando para isso o método ARIS-EPCe.

Exercício 10.2:

Mostre as diferenças do que você conseguiu ou não representar com o EPCe e o BPMN.

Oportunidades e Problemas

A wise man will make more
opportunities than he finds.

(Francis Bacon)

Conteúdo

11.1.	Oportunidades	137
11.2.	Problemas	137
11.3.	Tipos de Problemas	138
11.4.	Identificando Problemas	138
11.5.	Caracterização do Problema	147
11.6.	Oportunidade e 5W2H	147
11.7.	Propondo Oportunidades ao Cliente	148
11.8.	Oportunidade e Valor	149
11.9.	Exercícios	149

Por que oportunidades e problemas?

Um projeto de software sempre tem como objetivo aproveitar oportunidades ou resolver problemas. Esses termos, porém, devem ser usados de forma mais precisa para que possamos entender os verdadeiros objetivos do projeto, de acordo com as partes interessadas.

Neste capítulo serão discutidas algumas técnicas para descobrir, caracterizar e classificar oportunidades e problemas. As oportunidades são fornecidas pelas partes interessadas, discutidas no Capítulo 3.

11.1. Oportunidades

Um **oportunidade** é uma circunstância que torna apropriado o desenvolvimento de um sistema de software(OMG, 2018). Elas são “uma chance de fazer algo, incluindo consertar um problema existente”(Jacobson, Lawson e Ng, 2019).

Uma oportunidade pode ser uma combinação de problemas, sugestões, percepções de mercado ou diretrizes. Ela articula a razão de criação de um sistema novo ou da modificação de um sistema(OMG, 2018). Ruble (1997), que separa oportunidades de problemas, afirma que enquanto problemas sempre indicam que algo está quebrado, em uma oportunidade isso não é necessário.

As oportunidades devem ser entendidas pela equipe de desenvolvimento de forma a gerar os requisitos. Elas justificam e dão foco aos requisitos(OMG, 2018).

As principais oportunidades se originam das partes interessadas que não são a equipe, porém a equipe de desenvolvimento pode propor também oportunidades, ou reconhecer problemas, sendo necessário então ter o acordo das partes interessadas de que devem ser consideradas no projeto. Um cuidado especial deve ser tomado quando a equipe propõe uma oportunidade, que é a tendência de dar importância excessiva a tecnologia, o que pode gerar um requisito falsoMcMenamin e Palmer (1984).

As oportunidades permitem:

- identificar e motivar as partes interessadas;
- entender o valor que o sistema de software oferece as partes interessadas;
- entender por que o software está sendo desenvolvido;
- entender como o sucesso da implantação do sistema de software vai ser julgado e avaliado, e
- garantir que o sistema de software aborda efetivamente as necessidades de todas as partes interessadas.

É muito importante avaliar continuamente a viabilidade das oportunidades. Jacobson, Lawson e Ng (2019) chamam a atenção para o fato que é necessário:

- confirmar que a oportunidade atende uma **necessidade verdadeira** do cliente;
- tomar decisões sobre que soluções são mais apropriadas para atender a oportunidade, e
- ter a consciência que os benefícios podem demorar algum tempo para ser visíveis para os clientes.

Uma necessidade é verdadeira se o sistema não vai cumprir sua finalidade caso a necessidade não seja atendida(McMenamin e Palmer, 1984).

Oportunidades, positivas ou negativas, são grandes motivadores dos clientes. Se elas não existissem, não haveria necessidade de implementar um novo sistema. A verdadeira motivação de desenvolver o sistema é atender a essas oportunidades, principalmente aqueles que mais afetam o negócio. No *Project Model Canvas* as oportunidades aparecem já no primeiro módulo (Por que), no quadro de justificativas.

11.2. Problemas

Um problema é um estado onde existe uma dificuldade que deve ser resolvida, como o desvio de um padrão aceitável, **uma diferença entre o comportamento desejado e o percebido**

ou ainda uma causa, real ou possível, de um ou mais eventos que trouxeram ou podem trazer algum prejuízo.

Problemas incomodam as partes interessadas, trazendo prejuízos financeiros, afetando negativamente a imagem da organização, atrapalhando a produção ou os processos, colocando em risco alguma atividade, e mesmo incomodando apenas psicologicamente.

É comum que os problemas apareçam muito rapidamente nas conversas entre as partes interessadas e a equipe de análise, pois provavelmente estão entre os motivos que causaram a necessidade de um novo desenvolvimento de software, porém não necessariamente as partes interessadas têm conhecimento da causa raiz do problema, pois o que as afeta mais diretamente são os sintomas.

Por exemplo, um restaurante pode estar faturando pouco, mesmo com filas na porta, sendo esse o problema que o cliente quer resolver, procurando um novo sistema de pedidos e fechamento de conta. A causa real, porém, pode ser uma cozinha mal planejada, um salão mal aproveitado ou garçons mal treinados. Nesse caso, um novo sistema dificilmente resolveria o problema.

Deve-se tomar cuidado para não procurar uma solução sem que o problema esteja claramente determinado. Por isso é importante descobrir, para cada problema, a sua **causa raiz**. Fazê-lo resulta na continuação do problema e do provável abandono da solução.

Além disso, problemas devem ser definidos em termos de comportamento, não em termos de atitudes ou sentimentos. Para um problema estar realmente descrito, isso deve ser feito na forma de um comportamento que acontece no momento e que é indesejado, colocado em comparação com um comportamento desejado. Segundo Blanchard e S. Johnson (1983), se não há um comportamento desejado não há um problema, mas uma reclamação, e ainda é necessário definir o problema.

11.3. Tipos de Problemas

Podemos identificar três tipos básicos de problemas que são descritos pelos clientes:

- **problemas funcionais**, que ocorrem quando um sistema não permite uma funcionalidade que o cliente necessita ou deseja;
- **problemas operacionais**, que ocorrem quando uma funcionalidade funciona de forma errada, e
- **problemas de negócio**, que estão relacionados a manutenção do negócio propriamente dito.

Esses três tipos de problemas se relacionam. Um problema de negócio, por exemplo, pode ser um sintoma da falta de uma funcionalidade adicional no sistema ou de um problema em uma funcionalidade no sistema atual.

11.4. Identificando Problemas

Para analisar o problema, é necessário que os usuários:

- Concordem com a definição do problema;
- Entendam as causas do problema, já que o problema por trás do problema pode ser mais importante, sendo que o problema visto inicialmente é apenas um sintoma;

11. Oportunidades e Problemas

- identifiquem as partes interessadas e usuários;
- definam a fronteira do sistema de solução. e
- identifiquem as restrições impostas à solução

Algumas práticas para identificar problemas são:

- verificar os resultados obtidos atualmente nos processos e compará-los com objetivos da organização ou padrões do mercado;
- observar o comportamento dos colaboradores;
- levantar a opinião de fornecedores, clientes e vendedores, representantes ou distribuidores, da organização.

Quanto aos resultados obtidos, uma análise do processo da organização pode procurar atividades que:

- contém erros;
- são feitas vagarosamente;
- não são mais feitas como definidas em documentos da organização, e
- não são completadas.

Quanto aos colaboradores, é importante verificar se:

- estão desestimulados;
- não conseguem descrever claramente suas responsabilidades e objetivos, e
- pedem demissão com muita frequência.

Além da visão interna da organização, é interessante olhar para os parceiros externos. A partir deles podemos investigar:

- reclamações;
- sugestões de melhorias;
- queda nas vendas, e
- vendas com perdas.

A identificação de problemas às vezes não é fácil se não há um exemplo a comparar. Como saber se seu serviço é lento sem saber como é o serviço da concorrência? Existem práticas específicas para isso, como a compra de relatórios feitos por consultorias especializadas, *benchmarks* (Stapenhurst, 2009), etc.

Quando um problema é identificado, é importante que todos concordem que esse é o problema a ser resolvido pelo projeto. No Capítulo ?? é apresentado um método prático para iniciar um projeto com esse acordo, o *Project Model Canvas*.

Para estabelecer o problema, é interessante criar uma sentença apropriada, como a seguinte:

O problema de <descrição do problema>, afeta <partes interessadas afetadas> e resulta em <impacto nas partes interessadas>. A solução <indicar a solução> Trará os benefícios de <lista dos principais benefícios>.

11.4.1. A Análise de Pareto

Ao lidar com uma quantidade de problemas detectados em um sistema funcionando, é importante determinar os principais problemas, aqueles que tem mais impacto, de modo a dar foco aos esforços e obter melhores resultados.

Uma das técnicas mais conhecidas para isso é a Análise de Pareto.

O **Princípio de Pareto** diz que 20% dos problemas causam 80% dos efeitos. Essa divisão não é real, mas é uma forma de indicar que poucas causas são responsáveis pelos maiores impactos e que é necessário colocar os problemas em perspectiva.

Então, segundo o Princípio de Pareto, devemos calcular o impacto de cada tipo de erro em nosso sistema, selecionando os de maior impacto para resolver.

Devemos tomar cuidado, porém, porque o resultado que obteremos depende dos dados que tivermos¹. A seguir são apresentados dois exemplos, baseados na contagem e custo de erros obtidos em um sistema fictício, onde no primeiro apenas são contados apenas os erros por tipo, chegando a uma análise errada do que é importante, e no segundo é calculado o impacto de cada tipo.

11.4.2. Calculando a Quantidade

Para tratar problemas de acordo com a técnica de Pareto, **supondo que todo problema tem o mesmo efeito**, é necessário seguir os seguintes passos:

1. dividir os problemas em categorias;
2. levantar o número de ocorrências por categoria;
3. classificar as categorias em ordem decrescente por quantidade de ocorrências, como na Tabela 11.1;
4. calcular o total de ocorrências;
5. calcular a porcentagem de cada categoria;
6. determinar uma escala;
7. colocar as categorias como colunas, ordenadamente, a maior a esquerda, e
8. traçar as curvas de porcentagem acumulada, como na Figura 11.1.

A Tabela 11.1 apresenta o primeiro resultado útil, isto é, a tabela ordenada em ordem decrescente da quantidade de ocorrência por tipo de problema, para problemas encontrados em um sistema fictício.

Tabela 11.1.: Uma contagem de tipos de erros para uso da técnica de Pareto

Problema	Ocorrências
Requisito Original Errado	35
Erro de Programação	25
Erro de Projeto	21
Erro de Entrada de Dados	18
Erro de Formato	13
Problema de Segurança	12
Mudança no Negócio	10
Outros (1 Ocorrência)	7
Mudança na Legislação	5
Falta de Rede	3
Falta de Energia	1

¹Os americanos tem um ditado: *Garbage in, garbage out.*

11. Oportunidades e Problemas

A partir da Tabela 11.1 é possível gerar o gráfico da Figura 11.1, que já permite comparar a importância dos problemas, porém ainda considerando que todos os problemas tem o mesmo impacto, o que é raro acontecer.

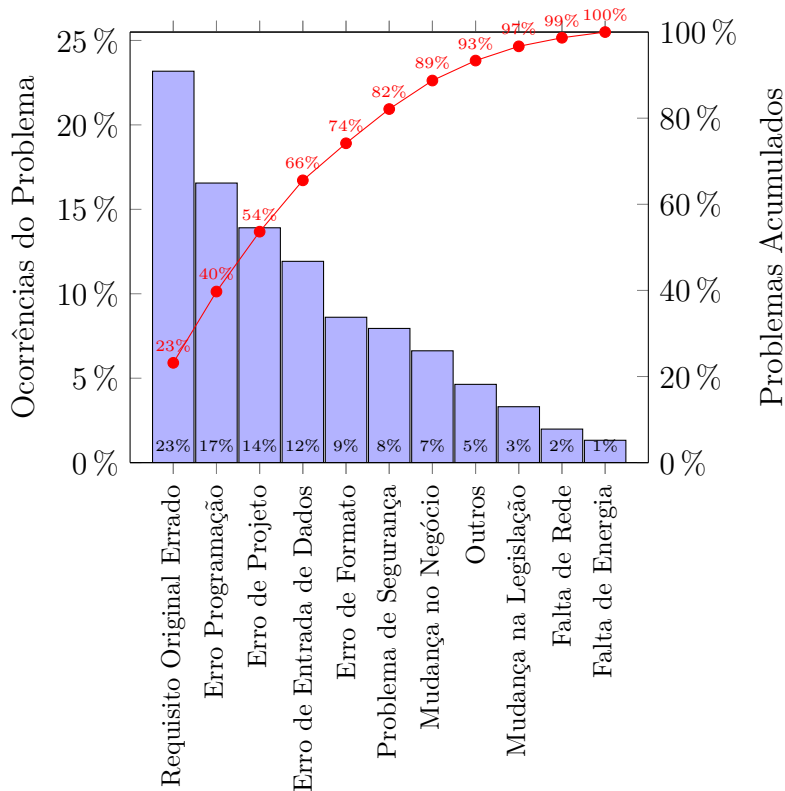


Figura 11.1.: Gráfico com a quantidade de problemas.

A análise da Tabela 11.1 e do Figura 11.1 levaria a crer que 82% dos problemas seriam causados por 6 tipos: Requisitos Originais Errados, Erro de Programação, Erro de Projeto, Erro de Entrada de Dados, Erro de Formato e Problema de Segurança. Na prática, não houve um atendimento do Princípio de Pareto, isso porque estamos apenas contando, e não calculando o impacto. No mundo real, é difícil que a premissa que foi feita nesse cálculo seja verdade, dificilmente todo problema tem o mesmo impacto.

11.4.3. Usando o Impacto dos Problemas

A análise anterior pode ser muito melhor se soubermos o impacto de cada problema, seja ele o custo financeiro ou apenas um valor relativo. Dessa forma podemos melhorar a Técnica de Pareto. Vamos supor que os problemas detectados no sistema tenham um custo médio relativo como descrito na Tabela 11.2;

11. Oportunidades e Problemas

Tabela 11.2.: Custo médio relativo dos problemas

Problema	Ocorrências
Requisito Original Errado	100
Mudança no Negócio	80
Mudança na Legislação	80
Erro de Projeto	10
Problema de Segurança	10
Outros (1 Ocorrência)	10
Erro de Programação	2
Erro de Entrada de Dados	1
Erro de Formato	1
Falta de Rede	1
Falta de Energia	1

Quando os problemas tem impacto diferente, é necessário multiplicar a quantidade de problemas de um tipo pelo custo médio do problema, gerando uma tabela de impacto dos erros, como na Tabela 11.3, e o gráfico ficaria como na Figura 11.2.

O novo passo a passo, e o mais aconselhado, para aplicar o Princípio de Pareto com a inclusão do impacto de cada tipo de problema é:

1. dividir os problemas em categorias;
2. levantar o número de acontecimentos por categoria, como na Tabela 11.1;
3. levantar o custo médio por categoria, como na Tabela 11.2;
4. multiplicar o custo médio pelo número de acontecimentos;
5. classificar as categorias em ordem decrescente por custo médio, como na Tabela 11.3;
6. calcular o custo total dos problemas;
7. calcular a porcentagem de cada categoria;
8. determinar uma escala;
9. colocar as categorias como colunas, ordenadamente, a maior a esquerda, e
10. traçar as curvas de porcentagem acumulada, como na Figura 11.2.

Tabela 11.3.: Custo total relativo

Problema	Ocorrências
Requisito Original Errado	3500
Mudança no Negócio	2000
Mudança na Legislação	1680
Erro de Projeto	180
Problema de Segurança	130
Outros (1 Ocorrência)	120
Erro de Programação	20
Erro de Entrada de Dados	7
Erro de Formato	5
Falta de Rede	3
Falta de Energia	2

11. Oportunidades e Problemas

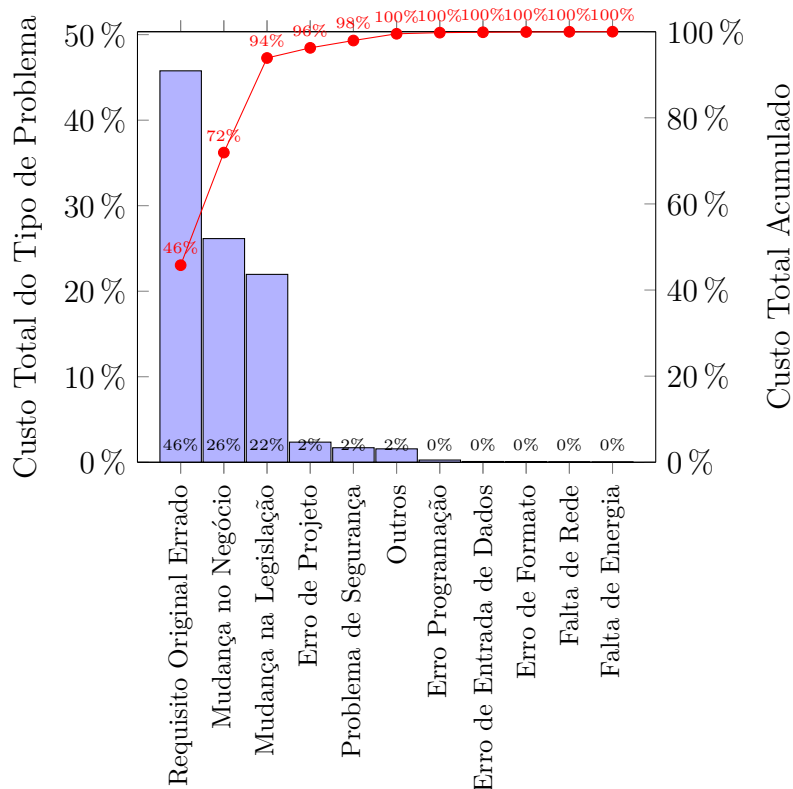


Figura 11.2.: Pareto com custo

Analisando esses resultados vemos a importância de calcular não só a quantidade de erros, mas também seu custo, para melhor priorizar os problemas.

Na Figura 11.2 fica claro que apenas três, ou mesmo dois, problemas precisam atenção, sendo que o primeiro problema é bem mais importante que os outros. Nesse caso, o Princípio de Pareto é atendido.

11.4.4. Buscando Causas

Um problema é gerado por várias causas, que podem também ser geradas por outras causas. Descobrir a causa raiz de um problema é um processo de investigação que pode ser documentado em um **Diagrama de Espinha de Peixe**, com o da Figura 11.3, também conhecido como **Diagrama de Causa Raiz**.

Esse diagrama é simples de desenhar ou construir, mas certamente esconde toda uma dificuldade de levantar corretamente as informações que revela. Como muitas propostas de análise, o processo de construção, e o aprendizado dele decorrente, é muitas vezes mais importante que o registro completo final.

O Diagrama de Espinha de Peixe indica as causas para um problema de maneira hierárquica. Para cada causa pode haver outras causas, e assim sucessivamente. O objetivo é a partir de um problema achar as causas raiz.

Ele parte da premissa que normalmente problemas são percebidos por um sintoma, e que o verdadeiro problema está escondido em uma camada de problemas sucessivos.

Problemas do Restaurante

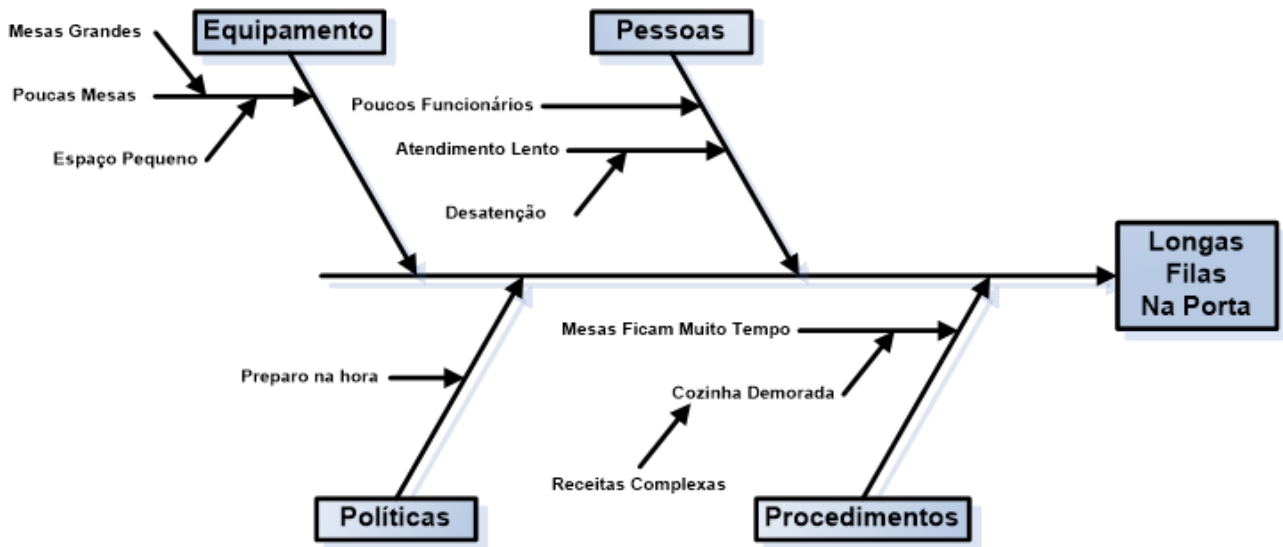


Figura 11.3.: Um diagrama de Espinha de Peixe

O processo de criação é mais produtivo se feito por um conjunto de pessoas, em uma reunião semelhante a um *brainstorm*. Nesse caso é importante que todos concordem com o efeito ou problema inicial a ser discutido. Sucessivamente devem ser feitas perguntas sobre “por que” o problema ocorre, ou seja, quais suas causas.

Cada linha de causalidade deve ser seguida até sua causa raiz. Isto é, a prática mais recomendada é trabalhar em profundidade, com perguntas sucessivas sobre por que algo acontece. Isso é similar a técnica dos 5 porquês, usada da mesma forma.

Apesar de no início o diagrama poder ser construído de forma livre, é interessante organizá-lo de alguma maneira para a apresentação. Ramos lotados podem ser divididos, ramos quase vazios podem ser unificados. Também é necessário ter uma noção da importância das causas, porque não é necessário gastar muito tempo em uma causa que não é realmente impactante para o problema.

A construção pode ser feita em um quadro branco ou com um projetor. É importante que todos os participantes tenham acesso completo ao quadro.

O diagrama de Figura 11.3 mostra a investigação de causas para o fato de um restaurante ter longas filas na porta. Pode ser visto que algumas causas conseguiram uma hierarquia de causa raiz mais longa do que outras. Isso não é um problema.

Algumas ferramentas permitem trabalhar com diagramas diferentes mas com a mesma finalidade. Ferramentas de *mind mapping*, por exemplo, permitem não só criar diagramas equivalentes, como permitem também trabalhar com resumos das figuras, como mostrado nas Figuras 11.4 e 11.5

Também é possível usar o diagrama como uma ferramenta pessoal de auxílio a investigação, ou para apoiar a técnicas dos 5 Porquês. Na Figura 11.6, o diagrama é usado para ilustrar o seguinte raciocínio.

1. Tirei uma nota baixa
2. Por que?

11. Oportunidades e Problemas



Figura 11.4.: Um diagrama de Espinha de Peixe resumido, construído com uma ferramenta de *mind map*.



Figura 11.5.: Um diagrama de Espinha de Peixe expandido, construído com uma ferramenta de *mind map*.

3. Porque a prova estava difícil.
4. Por que?
5. Porque não estudei.
6. Por que?
7. Por que não me organizei.
8. Por que?
9. Acho que não há um motivo para isso.
10. Há outro motivo para tirar uma nota baixa?
11. Sim, não fiz a AD, preparatória para a prova.
12. Por que?
13. Não tive tempo.
14. Por que?
15. Novamente, não me organizei.
16. Por que?
17. Acho que novamente não tenho uma explicação para isso.

11.4.5. Organização do Diagrama Espinha de Peixe

O Diagrama de Espinha de Peixe pode ser organizado com as causas agrupadas em categorias. Existem pelo menos 3 organizações reconhecidas: para indústrias, para serviços e administração, e para Gestão de Qualidade Total.

As classes para indústrias são:

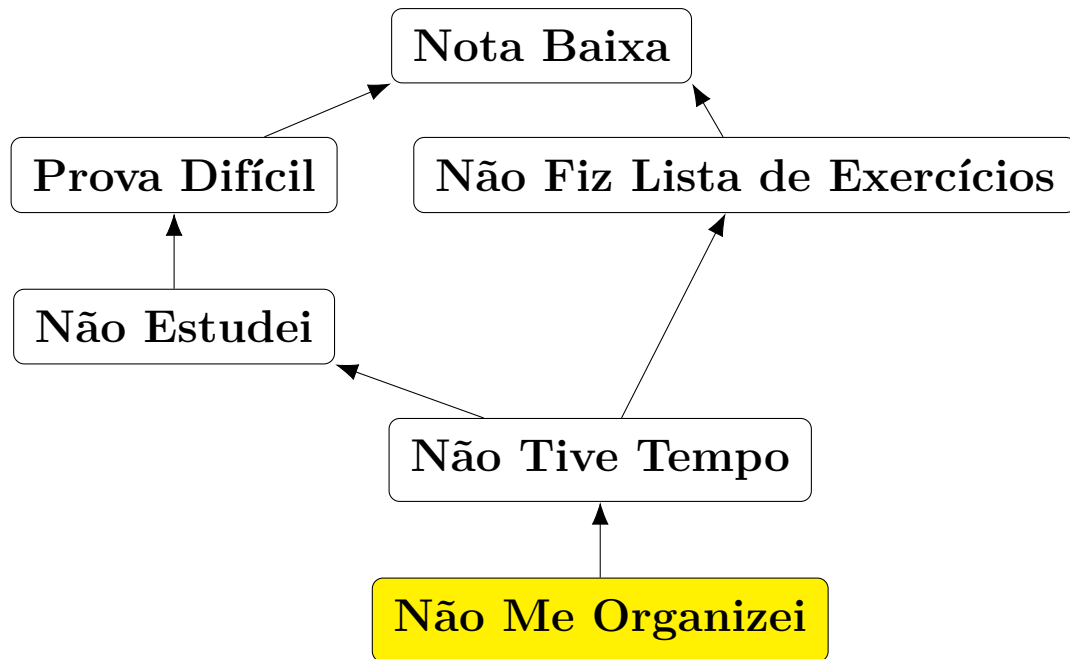


Figura 11.6.: Um Diagrama de Causa Raiz vertical, onde a causa raiz está marcada.

- pessoal;
- métodos;
- materiais, e
- máquinas.

As classes para serviços e administração são:

- pessoal;
- equipamentos;
- procedimentos, e
- políticas.

As classes para Gestão de Qualidade Total (TQM, *Total Quality Management* são:

- ambiente;
- gestão;
- pessoas;
- materiais;
- equipamento, e
- processo.

Finalmente, o método 6-sigma, sugere usar 6 categorias, chegando a chamar o diagrama de 6-M, por usar nomes que começam por M:

- métodos;
- máquinas;
- materiais;
- mão-de-obra;
- meio-ambiente, e
- medida.

Alguns autores ainda incluem outros M, como *management*, usando a tradução forçada de manejo administrativo, e meios financeiros.

Todas essas categorias podem ser usadas nas reuniões como inspiração para a busca de causas, fazendo perguntas como “Existe alguma causa de gestão para esse problema?”.

Mesmo usando essas técnicas, podemos encontrar fatores que não se alinham as categorias pré-determinadas, ou criam dúvidas sobre em que categoria colocar. Um dos motivos é que elas olham para dentro da organização, e o problema pode ser fora. Pessoas que não querem se vacinar, por exemplo, não seriam encontradas usando apenas o 6M, mas seriam encontradas usando o “Pessoas” em serviço. O importante é entender que essas categorias estão aí para ajudar e não para atrapalhar. Elas nos lembram de onde procurar problemas, e não são uma camisa de força. Se ficar em dúvida de onde colocar algo, coloque em qualquer lugar.

11.5. Caracterização do Problema

Identificado o problema é necessário caracterizá-lo claramente. Para isso, é necessário identificar:

- o problema;
- quem é afetado pelo problema;
- seu efeito no negócio e nas partes interessadas;
- o impacto financeiro;
- suas causas e sua causa raiz;
- seu causador, seja ele um processo, um dado, um profissional, etc.;
- pelo menos uma sugestão de solução, e
- os benefícios que uma solução pode trazer.

Alguns desses dados podem ter sido levantados de forma aproximada na identificação do problema. Nesta fase é importante detalhá-los e documentá-los.

11.6. Oportunidade e 5W2H

As oportunidades, ou problemas, bem identificadas devem responder as perguntas 5W2H. Perguntas comuns a serem respondidas são:

- Por que são necessárias? Por que acontecem?
- O que elas são? Qual a causa raiz? Qual o efeito na empresa?
- A quem atendem? A quem incomodam?
- Quando elas ocorrem, quando elas devem ser aproveitadas?
- Quanto elas custam? Quanto elas permitem lucrar?
- Onde ocorrem? Onde podem ser resolvidas ou aproveitadas?

11.6.1. Buscando Oportunidades com as Partes Interessadas

A melhor maneira de levantar oportunidades é entender simultaneamente: o que o cliente deseja, o que está funcionando agora, quais os problemas atuais e quais oportunidades existem para um novo sistema.

As entrevistas iniciais para levantamento de necessidades de informação são geralmente feitas com executivos. Um bom conjunto inicial de questões que podem ser feitas é apresentados na lista a seguir (Gillenson e Goldberg, 1984).

- Quais seus objetivos?
- Quais suas responsabilidades?
- Que medidas você usa?
- Que informações você precisa?
- Quais são seus problemas de negócio?
- Que mudanças você vê no futuro (que vão impactar a infra-estrutura do seu negócio)?
- Quais são os fatores críticos de sucesso?
- Qual a informação mais útil que você recebe?
- Como você classificaria as informações que recebe quanto à adequação, validade, duração, consistência, custo, volume, etc.?

Alguns pontos podem ser notados sobre essas perguntas. A pergunta sobre objetivos muitas vezes pode não ser muito bem respondida, assim a segunda pergunta, sobre responsabilidades, permite uma resposta mais prática e mais fácil de ser dada.

A terceira pergunta visa buscar uma compreensão sobre os dados importantes para o entrevistado, abrindo uma sequência de perguntas sobre o tema.

A quinta pergunta, sobre os problemas de negócio, é a mais importante do conjunto e deve ser dada a maior atenção e quantidade de tempo disponível.

É interessante que o problema seja estruturado em um formato causa-efeito, por exemplo: "por causa da falta de dinheiro, o resultado é que não é possível comprar peças de reposição". Também é importante verificar se a causa primordial do problema é um processo ou um dado.

Além disso, o entrevistador deve tentar obter alguma informação de valor (financeiro) sobre o impacto do problema, seja em valores absolutos ou relativos, objetivos ou subjetivos.

11.7. Propondo Oportunidades ao Cliente

O analista de sistemas, com sua experiência tecnológica e de negócio, também pode oferecer oportunidades aos seus clientes.

Uma **oportunidade tecnológica**, como diz o nome, é uma tecnologia específica a ser usada na implementação e que oferecerá alguma vantagem ao cliente. Por exemplo, ao implantar um novo sistema de estoque podemos oferecer a entrada e saída de produtos pelo uso de código de barras. A oportunidade tecnológica não altera a funcionalidade que o cliente necessita, registrar a entrada de um item no estoque, mas sim sua forma de implementação.

Algumas vezes também podemos oferecer **oportunidades de negócio**.

Devemos ter muito cuidado ao propor oportunidades, pois podemos estar criando falsos requisitos, discutidos no Capítulo 12. Um exemplo típico de falso requisito, que costuma ser proposto por analistas, é a proliferação de relatórios em sistemas que na prática usam apenas alguns.

Uma oportunidade de negócio deve ser exaustivamente discutida com o cliente de forma a ficar claro que ela realmente traz benefícios, que esses benefícios são consideráveis, que o risco do projeto não aumenta muito e que ela será realmente utilizada.

Um exemplo que podemos dar é, em um controle de estoque, a funcionalidade de prever que produtos estão próximos de sua data de vencimento. Essa não é uma função "normal" de sistemas de estoque. Exige um custo adicional não só de desenvolvimento, mas também de operação, como identificar a data de vencimento, controlar diferentes datas para um produto,

etc. Em muitos contextos, essa função pode parecer interessante mas ser, na prática, inútil. Em uma oficina mecânica, por exemplo, ela pode ser totalmente inútil. Em um grande mercado, ela pode ser bastante útil. Porém em um pequeno mercado, onde o proprietário tem na verdade um controle mental do estoque e precisa do sistema de estoque apenas para fazer um controle legal ou financeiro, essa funcionalidade pode parecer útil a princípio, mas nunca ser usado no dia a dia.

11.8. Oportunidade e Valor

As oportunidades devem produzir resultados úteis para a empresa, como acelerar processos, eliminar passos desnecessários, reduzir erros na entrada e saída de dados, aumentar a integração entre sistemas, aumentar a satisfação do usuário e facilitar a interação com os parceiros externos da organização (fornecedores, representantes e clientes)(K. E. Kendall e J. E. Kendall, 2013).

A prática IRACIS ajuda a buscar oportunidades, ou a analisar se algo é ou não uma oportunidade, que é tratada na Seção 4.6, busca encontrar valor por meio de aumentar as receitas, diminuir os custos e melhorar os serviços.

Já os elementos de valor, tratados na SubSeção 4.3.1, permitem também buscar ou analisar outras formas de valor, tanto para pessoas físicas, em sistemas B2C, como para pessoas jurídicas, em ambientes B2B.

Claramente, uma oportunidade (problema) precisa de um valor associado a ela para que se possa reivindicar o seu aproveitamento (solução). Quanto mais este valor puder ser resumido em um valor financeiro, mas fácil é verificar se vale a pena investir em um sistema de software para aproveitá-la (resolvê-lo). Calcular o impacto financeiro de uma oportunidade ou problema é responsabilidade da área de negócio e não do analista de sistemas. Porém, cabe ao analista analisar se o benefício pretendido é factível com o projeto sendo proposto.

Para fazer a decisão financeira final podem ser usadas as técnicas ROI, IRR e NPV apresentadas na Seção 4.5.

11.9. Exercícios

Exercício 11.1:

Associe as características que devem ser levantadas para um problema às perguntas 5W2H.

Exercício 11.2:

Analise o local onde você estuda e os serviços de software fornecidos a você. Faça uma lista de problemas e oportunidades. Discuta as causas raiz dos problemas.

Exercício 11.3:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita liste os problemas e oportunidades.

Requisitos

The hardest part of the software task is arriving at a complete and consistent specification, and much of the essence of building a program is in fact the debugging of the specification.

(Frederick P. Brooks)

Conteúdo

12.1.	Definição Formal	151
12.2.	Requisitos x Benefícios	154
12.3.	Tipos de Requisitos	154
12.4.	O Papel dos Requisitos no Projeto	156
12.5.	Engenharia de Requisitos	157
12.6.	Elicitação de Requisitos	157
12.7.	Níveis de Abstração dos Requisitos	161
12.8.	O Risco de Requisitos Falsos	161
12.9.	Características de um Bom Requisito	162
12.10.	Atributos dos Requisitos	163
12.11.	Priorizando Requisitos	165
12.12.	Requisitos Mudam com o Tempo	166
12.13.	Requisitos e Necessidades	166
12.14.	Requisitos de Acordo com Normas IEEE	167
12.15.	A Visão do Novo Sistema	170

12.16.	Descrevendo Requisitos	170
12.17.	Exercício	172

Por que requisitos?

Levantar e especificar os requisitos são a principal função da análise de sistemas, seja qual for a metodologia usada.

Para construir qualquer coisa é preciso saber o que vai ser feito, de maneira a atender da melhor forma possível as demandas das partes interessadas. No caso de software, ou de sistemas em geral, este conhecimento normalmente é registrado na forma de uma **especificação de requisitos**. Em uma interpretação restrita, requisito de software é uma exigência feita ao software no início do projeto.

Como projetos de software normalmente são desenvolvidos em fases, sejam elas sequenciais ou cíclicas, é comum que uma fase anterior construa artefatos de conhecimento que permitem definir melhor o que será feito na próxima fase. Todo artefato criado em uma fase de projeto que define de alguma forma o que é preciso ser feito em uma fase posterior de um projeto pode ser chamado também de requisito¹. Assim, em uma interpretação ampla, qualquer especificação, realizada em uma fase qualquer do processo de software, é um requisito para a próxima fase, já que ela tem que ser cumprida.

Essa ambiguidade, entre a interpretação ampla e restrita, normalmente é resolvida pelo contexto. Outros termos também são adicionados para qualificar o termo requisito e ajudar na interpretação do seu significado exato. Por exemplo, requisitos do usuário é o termo usado normalmente para significar requisitos levantados pelo usuário, com ou sem ajuda de analistas de sistemas, em uma fase anterior, ou no início, da análise de sistemas.

12.1. Definição Formal

Segundo a norma *ISO/IEC/IEEE International Standard - Systems and software engineering – Vocabulary*(IEEE, 2010), um **requisito** é:

- uma condição ou capacidade necessária para um usuário resolver um problema ou atingir um objetivo;
- uma condição ou capacidade que precisa ser possuída ou alcançada por um sistema, produto, resultado, serviço ou componente para satisfazer um acordo, contrato, padrão, especificação ou outro documento formalmente imposto, ou
- Uma representação documental de uma condição ou capacidade no sentido das duas definições anteriores.

Esta definição, múltipla, permite o uso da interpretação ampla de requisitos.

Sommerville e Sawyer (1997) definem requisitos como

uma especificação do que deve ser implementado. Eles são descrições de como o sistema deve se comportar, ou de uma propriedade ou atributo do sistema. Eles podem ser uma restrição ao processo de desenvolvimento do sistema.

¹O termo em inglês é *requirement*, o que levou a algumas traduções usarem o termo requerimento, que é errado em português.

Esses autores também afirmam que “não existe uma melhor forma de escrever requisitos”, isto é, cada projeto pode precisar de uma forma específica de acordo com condições como assunto, cultura organizacional, questões de segurança, etc. Por exemplo, é razoavelmente mais caro fazer especificações em linguagens formais, exigindo pessoal com mais formação, software especializado, etc. Além disso, especificações formais são difíceis de serem validadas ou negociadas com o cliente, logo raramente são usadas em desenvolvimento de sistemas comerciais. Porém são usadas em especificações de sistemas que precisam de níveis adicionais de garantias de segurança e robustez, como o *TCAS* (*Traffic Alert and Collision Avoidance System*) e outros (Craig, Gerhart e Ralston, 1993).

McMenamin e Palmer (1984), em um trabalho um pouco esquecido mas importante, dizem que “uma especificação de requisitos deve conter todos os requisitos verdadeiros e nada além desses. Ainda segundo eles, “um requisito é falso se o sistema for capaz de cumprir sua finalidade sem a implementação do requisito” (McMenamin e Palmer, 1984)². Esses requisitos falsos podem ser criados de forma arbitrária pelos analistas de sistema, ou por influência da tecnologia. A metodologia da Análise Essencial, proposta por eles, trata de requisitos de forma bastante rigorosa, e busca os **requisitos essenciais**, que ainda são requisitos entre fundamentais, que atendem às verdadeiras demandas dos usuários, e custodiais, que tratam dos dados que permitem executar os requisitos fundamentais. Em todo caso, todo requisito essencial é totalmente ausente de tecnologia e supõe que o computador é uma máquina perfeita, de velocidade e memória infinita (McMenamin e Palmer, 1984). A Análise Essencial de McMenamin e Palmer foi adotada por Yourdon no livro *Modern Structured Analysis*, e a influência de ambos é sentida até hoje nas práticas de análise de sistemas, mesmo que escondidas por novos nomes.

12.1.1. Exemplos de Requisitos

Como visto, a noção de requisito pode ser muito ampla. Eles podem ser encontrados em várias formas, dependendo dos métodos e práticas usados em cada projeto de software específico.

Na sua forma mais tradicional, um requisito é apenas uma sentença que define uma exigência feita ao sistema, na perspectiva do usuário, escrita em linguagem corrente, como em:

“O sistema deverá permitir que o correntista verifique o saldo de sua conta”.

Em casos de requisitos escrito em linguagem natural, muito encontrados em documentos conhecidos como **RFP**, **Request for Proposal**, a forma mais correta é que a exigência seja sempre feita ao sistema sendo especificado, nunca ao usuário, no caso o correntista, porém não é incomum encontrar RFPs contendo erros de perspectiva, já que muitas vezes são construídas por pessoas leigas em relação a área de Engenharia de Software.

Muitas formas já foram propostas para elicitar ou analisar requisitos, ou para fazer uma parte desse trabalho. Algumas deram forma:

- eventos essenciais (McMenamin e Palmer, 1984);
- diagramas de fluxo de dados (DFD) (Gane e Sarson, 1979);
- mini especificações (Gane e Sarson, 1979);
- diagramas de contexto (Gane e Sarson, 1979; K. E. Wiegers e Beatty, 2013);
- cartões volare, como o da Figura 12.1 (J. Robertson e S. Robertson, 2006);
- especificações formais (Sommerville, 2015);
- casos de uso (Jacobson, Christerson et al., 1992; Jacobson, Spence e Bittner, 2011a);

²Na tradução brasileira é usado o termo *requerimento*, que mais tarde foi abandonado

12. Requisitos

- diagramas de caso de uso, como o da Figura 12.2 (Jacobson, Christerson et al., 1992; OMG, 2017);
- mapas mentais, como o da Figura 12.3;
- descrições em linguagem natural e hierárquicas (K. E. Wiegers e Beatty, 2013)
- árvores de funcionalidades (K. E. Wiegers e Beatty, 2013), e
- histórias do usuário (Cohn, 2004).

Requirement #: 75	Requirement Type: 9	Event/Use Case #: 7,9
Description: O projeto deverá registrar todas as estradas que foram consertadas		
Rationale: Estar apto a agendar estradas sem conserto e avisar de potenciais perigos		
Originator: José da Silva, Engenheiro Chefe		
Fit Criterion: As estradas consertadas ou não registradas devem estar de acordo com os logs		
Customer Satisfaction: 3	Customer Dissatisfaction: 5	
Priority:	Conflicts:	
Supporting Materials:		
History: Criado 16/2/2017	Volere Copyright © Atlantic Systems Guild	

Figura 12.1.: Um cartão Volere.

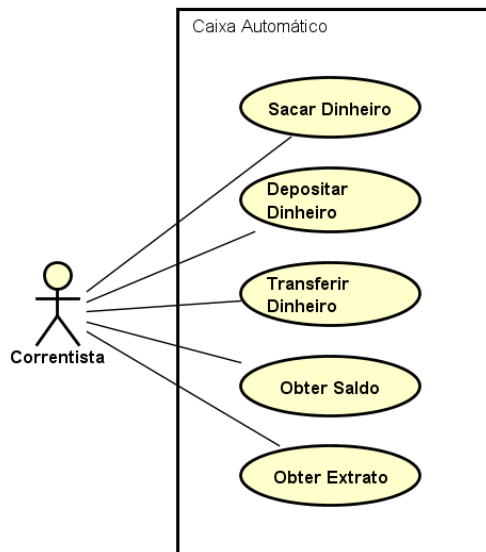


Figura 12.2.: Exemplo de um diagrama de casos de uso

As duas formas mais em voga no mercado em 2020 serão apresentadas nos capítulos a seguir: histórias do usuário e casos de uso.

12. Requisitos

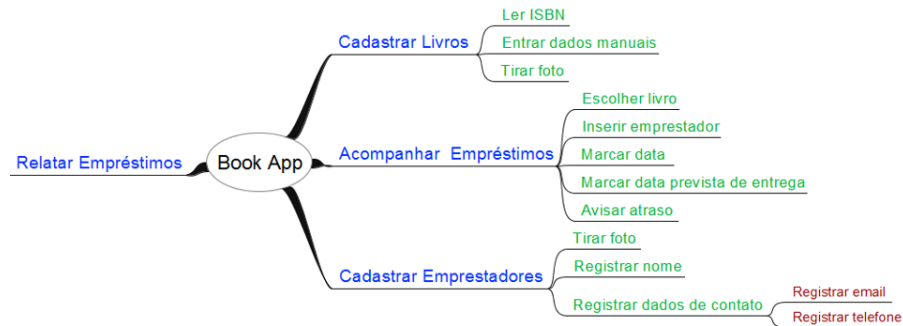


Figura 12.3.: Exemplo de um mapa mental.

12.2. Requisitos x Benefícios

No Capítulo 8 apresentamos brevemente requisitos e benefícios. Um benefício típico seria “diminuir os erros no preenchimento dos pedidos”, enquanto um requisito típico seria “o sistema deve oferecer ao comprador a lista de produtos disponíveis na loja”.

Algumas pessoas podem pensar que um benefício também é algo que se pede ao software, logo também é um requisito. Isto está errado. Um requisito é sempre uma demanda determinada diretamente para o produto ou serviço sendo especificado, enquanto um benefício é uma mudança esperada no negócio quando o produto ou serviço estiver em operação. Não se pode testar um benefício, por exemplo, a não ser que haja, para ele, um requisito específico em paralelo. Um exemplo disso seria um benefício como “permitir compra com cartão de crédito” e um requisito na forma “o sistema deverá permitir que o cliente pague com cartão de crédito”.

12.3. Tipos de Requisitos

Talvez a melhor definição de requisitos funcionais venha

Vários autores dividem os requisitos de formas diferentes, porém quase todos concordam que existem dois tipos básicos de requisitos: os **requisitos funcionais** e os **requisitos não funcionais**. Além disso é comum que um tipo de requisito seja colocado a parte, as **restrições**.

12.3.1. Requisitos Funcionais

Um requisito funcional representa algo que o sistema deve fazer, ou seja, uma função esperada do sistema que agregue algum valor a seus usuários. Exemplos típicos incluem a emissão de relatórios e a realização e manutenção de cadastros.

Um requisito funcional tradicional é escrito de uma forma específica, exigindo que o sistema cumpra uma finalidade, como em “o sistema deve permitir que um aluno solicite a emissão do seu histórico escolar”.

Também é possível que um requisito funcional seja uma especificação mais detalhada, impondo limites ou exigências a outro requisito, com em: “O CR será calculado a partir da média ponderada das notas em cada curso, sendo que o peso de cada nota será a quantidade de créditos acreditada ao curso.”

Normalmente, um requisito funcional não deve tratar de tecnologia. Por exemplo, o requisito “o sistema deverá enviar um e-mail para cada participante um dia antes da data da reunião” não é puramente funcional, porque “e-mail” é uma tecnologia. Uma descrição funcional pura deveria ser “o sistema deverá enviar um aviso para cada participante um dia antes da reunião”.

12.3.2. Requisitos não funcionais

Requisitos não funcionais falam da forma como os requisitos funcionais devem ser alcançados. Eles definem propriedades e restrições do sistema. Muitos requisitos não funcionais são também requisitos de qualidade, como exigências de desempenho e robustez. Outros são restrições ou exigências de uso de uma ou outra tecnologia.

Muitas vezes não é só difícil descobrir quais são os requisitos não-funcionais, mas também produzir uma especificação do sistema que possa ser cumprida em custo razoável e prazo hábil de forma a atender os usuários. Um exemplo típico seriam dois requisitos não funcionais que, genericamente, são opostos: velocidade e transportabilidade. Ora, para fazer um software muito veloz você precisa adaptá-lo especificamente para o ambiente onde ele está funcionando. Para fazer um software transportável, você precisa implementá-lo de forma a funcionar no maior número possível de ambientes. Normalmente esses dois requisitos se relacionam de forma inversa e para implementá-los simultaneamente é necessário um grande investimento de recursos.

Alguns requisitos não funcionais, quando negados, geram um “anti-requisito”, algo que nunca seria pedido por um cliente. Por exemplo, um requisito não funcional comum é que o software seja “confiável”. Ninguém pediria para ser construído um software “não confiável”, porém diferentes clientes estão interessados em diferentes níveis de confiabilidade e diferentes formas de certificar esse nível.

Deve ser levado em conta que quanto mais esforço é colocado para se alcançar um requisito, maior é o custo agregado ao projeto e isso servirá como referência para o cliente escolher o grau de confiabilidade que deseja. Por isso um requisito deve ser verificável. O requisito “robusto”, por exemplo, pode ser medido por meio do **MTBF** sistema.

(Sommerville, 2015) mostra uma pequena taxonomia dos tipos de requisitos não-funcionais mais importantes, apresentada na Figura 12.4.

MTBF é o tempo médio entre falhas.

12.3.3. Restrições

É difícil a discussão para decidir se algo é um requisito ou uma restrição. Essa divisão é muitas vezes artificial e feita a partir de um ponto de vista específico. Por exemplo, se uma especificação obriga o sistema a ser desenvolvido em Java, isso é uma restrição ou um requisito?

Uma maneira de separar restrições de requisitos é analisar se a demanda vem da vontade dos clientes ou usuário diretos, ou é algo obrigatório para eles, como uma lei ou uma regra organizacional que limita as possibilidades do projeto. Ou seja, se o sistema deve ser feito em Java por uma obrigação qualquer, e existem soluções mais fáceis, talvez seja melhor considerá-lo uma restrição.

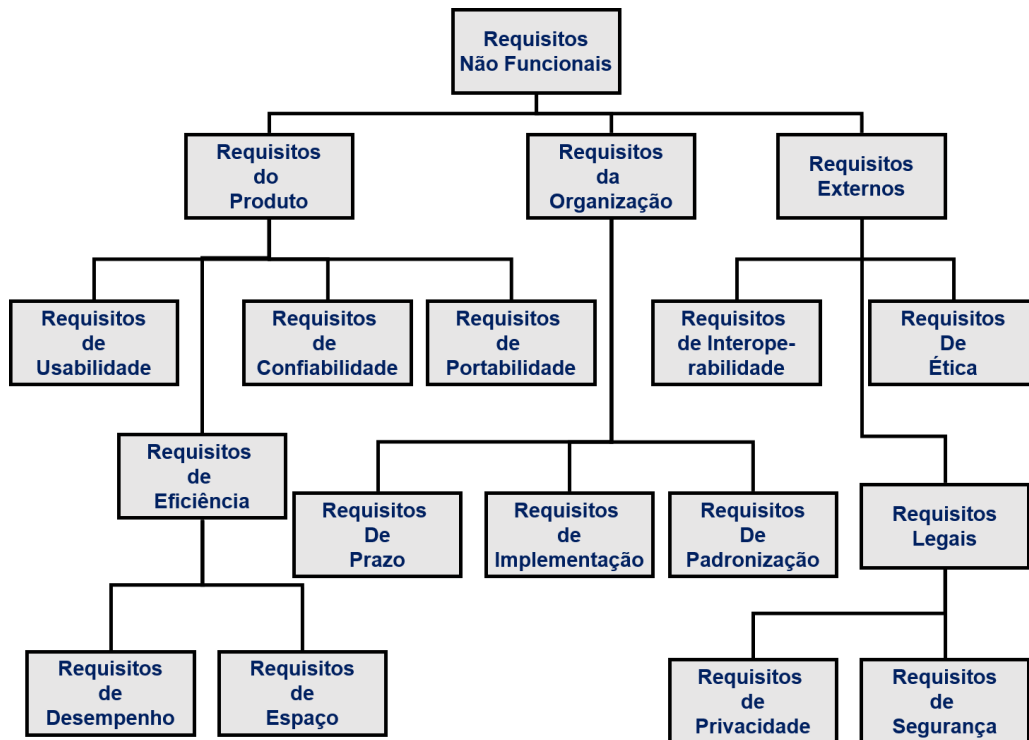


Figura 12.4.: Pequena taxonomia de requisitos não funcionais. Fonte: (Sommerville, 2015)

12.3.4. Outros tipos de requisitos

A literatura de requisitos é ampla e propõe muitas divisões para os requisitos, como requisitos de interface, requisitos de ambiente e etc. Essas divisões são úteis para organização de documentos de especificação de requisitos.

Requisitos de Informação representam a informação que o cliente deseja obter do sistema. Requisitos de informação também são atendidos por eventos. Muitas vezes o cliente expressa requisitos de informação de forma funcional. Outras vezes o cliente está preocupado em conseguir uma informação, mas não sabe como fazê-lo na forma de um requisito funcional. Em todo caso, sempre nos preocuparemos em levantar todos os requisitos de informação, pois eles representam as respostas fundamentais do sistema.

Requisitos de interface são exigências de como o sistema deverá interagir com outros sistemas ou humanos. Também são comuns na maioria das especificações formais. **Requisitos de desempenho** especifica requisitos numéricos estáticos e dinâmicos que são determinados para o sistema ou para a interação com o usuário(IEEE, 1998).

12.4. O Papel dos Requisitos no Projeto

J. Robertson e S. Robertson (2006) mostram o papel dos requisitos em um projeto de software de acordo com a Figura 12.5. O processo de Elicitação de Requisitos parte das Necessidades do Usuário e gera requisitos tanto para a Modelagem de Sistema (principalmente funcionais) quanto para o Projeto de Sistema (principalmente não funcionais). Ele também recebe feedback dessas e outras fases que podem levar a necessidade de correção dos requisitos. Modelos gerados pela Modelagem de Sistemas auxiliam na busca de novos requisitos, por exemplo, permitindo entender que informação está faltando para construir um modelo de dados correto.

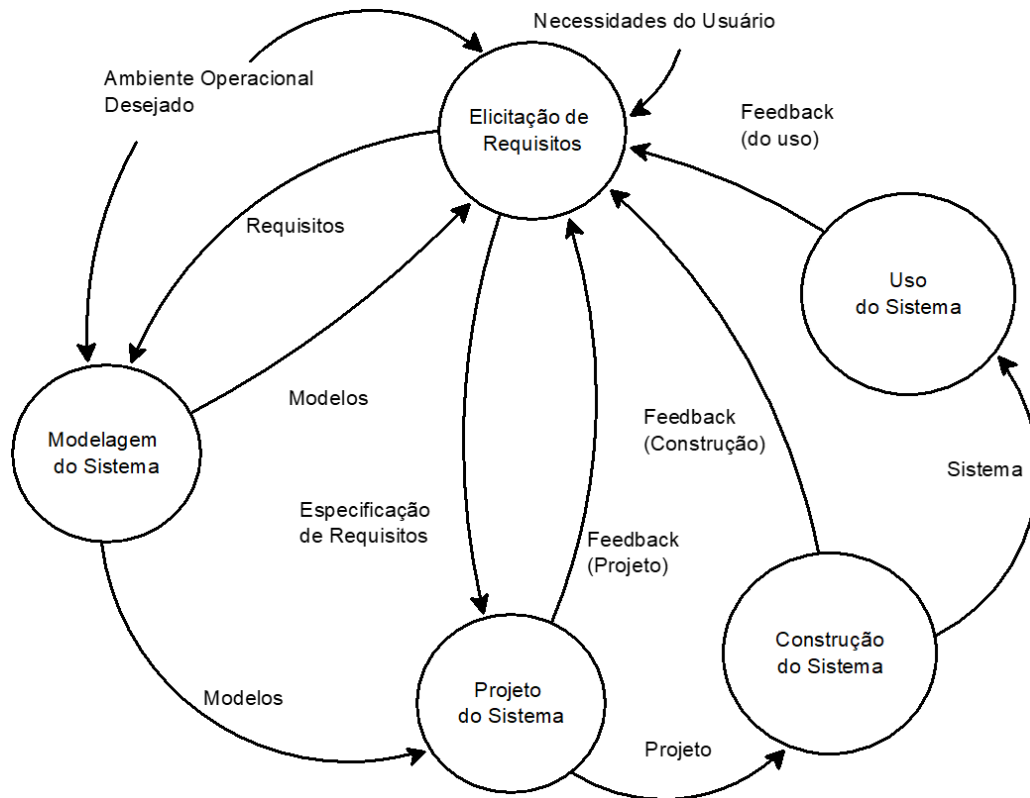


Figura 12.5.: O papel dos requisitos no desenvolvimento do sistema. Fonte:(J. Robertson e S. Robertson, 2006)

12.5. Engenharia de Requisitos

A Engenharia de Requisitos é a disciplina relacionada com o trabalho com requisitos, e pode ser dividida em duas grandes áreas: Desenvolvimento de Requisitos e Gerência de Requisitos, sendo que a última não é tratada neste texto. O Desenvolvimento de Requisitos, por sua vez, pode ser dividido em (K. E. Wiegers e Beatty, 2013):

- Elicitação, que cobre todas as atividades de descoberta dos requisitos;
- Análise, que cobre o entendimento mais rico e preciso de cada requisito;
- Especificação, que trata da representação e armazenamento dos conhecimentos coletados de forma persistente e bem organizada, e
- Validação, que trata da confirmação que os requisitos estão corretos e vão permitir uma solução que satisfaz as partes interessadas.

12.6. Elicitação de Requisitos

A **elicitación** é o levantamento, registro e validação (Christel e Kand, 1992) das expectativas dos diversos interessados no sistema, seguido da consolidação e validação dessas expectativas em requisitos formais. Contemplar essas diferentes visões implica projetar interesses divergentes e conciliá-los.

Esse trabalho investigativo de descoberta dos requisitos, é feito com dois processos gerais: entrevistas ou reuniões, ou observação. A diferença básica é que no primeiro tipo de elicitação são feitas perguntas aos clientes, enquanto na segunda o analista observa o ambiente de trabalho

12. Requisitos

da organização. O entendimento gerado pelo processo de observação pode exigir entrevistas ou reuniões adicionais para confirmação dos achados. Existem muitas práticas específicas para ambos os processos, como entrevistas fechadas, entrevistas por meio de questionários, reuniões de vários formatos, observação direta, observação de cenas gravadas, análise de documentos, atuação no papel do usuário, etc. (Sommerville, 2015).

Já K. E. Wiegers e Beatty (2013) sugerem 8 técnicas de elicitação e indicam em que tipos de projetos são adequadas, como mostrado na Tabela 12.1.

Tabela 12.1.: Adequação das formas de elicitação propostas por K. E. Wiegers e Beatty (2013) e alguns tipos de projeto.

TIPOS DE PROJETO	TÉCNICAS DE ELICITAÇÃO							
	entrevistas	workshops	grupos de foco	observações	questionários	análise da interface do sistema	análise da interface do sistema	análise da documentação
software para o mercado consumidor	✓		✓		✓			
software interno da organização	✓	✓	✓	✓		✓		✓
troca de um sistema existente	✓	✓		✓		✓	✓	✓
melhoria de um sistema existente	✓	✓				✓	✓	✓
aplicação nova	✓	✓				✓		
pacote de software	✓	✓		✓		✓		✓
sistemas embarcados	✓	✓				✓		✓
partes interessadas geograficamente distribuídas	✓	✓			✓			

Para levantar requisitos é necessária a interação entre aqueles que conhecem os requisitos ou as necessidades dos usuários e outras partes interessadas e os desenvolvedores. É um processo iterativo de comunicação, onde, por aproximações sucessivas, o desenvolvedor constrói um modelo aceito pelos usuários, como ilustrado na Figura 12.6.

O mesmo acontece na investigação que um analista tem que fazer com o cliente em busca do requisito. Inicialmente o cliente será ambíguo, generalista e paulatinamente, com a ajuda do analista, deverá chegar a uma especificação precisa do que deseja.

Uma receita geral para o levantamento de requisitos pode ser dada em 5 passos:

1. Identificar quem fornecerá os requisitos
2. Levantar lista de desejos para estas pessoas
3. Refinar cada lista de desejos até que seja auto-consistente
4. Integrar as listas de desejos de forma que sejam consistentes
5. Determinar os requisitos não funcionais.

12. Requisitos

Apesar de tudo, não é uma tarefa fácil levantar os requisitos. Muitos problemas podem acontecer. É comum que os clientes não saibam exatamente o que querem ou não saibam articular propriamente suas ideias, especialmente quando o desenvolvedor não possui o linguajar típico da área de aplicação (jargão). Muitas vezes, também, os desenvolvedores pensam entender melhor do problema que seus clientes, propondo supostas “melhorias” que afetam custo e aumento o risco dos sistemas propostos.

12.6.1. Processo de Elicitação de Requisitos

Dentro do Processo de Desenvolvimento de Software existe um Processo de Elicitação de Requisitos. Christel e Kand (1992) sugeriu o processo descrito na Figura 12.7 e na Tabela 12.2:

Tabela 12.2.: Processo de Elicitação de Requisitos segundo Christel e Kand (1992).

Tarefas da Elicitação de Requisitos		
Atividade	Tarefas orientadas ao usuário	Tarefas orientadas ao desenvolvedor
Busca de Fatos	• Identificar grupos relevantes através de múltiplos níveis da organização. • Determinar os contextos operacional e do problema, incluindo a definição dos modos operacionais, objetivos e cenários de missão como apropriados. • Identificar sistemas similares. • Realizar análise de contexto.	• Identificar especialistas do domínio da aplicação e de desenvolvimento. • Identificar modelo de domínio e modelo de arquitetura. • Conduzir pesquisas tecnológicas, para mais tarde fazer estudo de viabilidade e análise de risco. • Identificar custos e restrições à implementação impostas pelo patrocinador.
Coleta e Classificação dos Requisitos	Levantar a lista de desejos de cada grupo.	Classificar a lista de acordo com funcionais, não funcionais, restrições de ambiente, restrições de projeto e ainda de acordo com as partições definidas pelo modelo de domínio e pelo paradigma de desenvolvimento.
Racionalização e Avaliação	Responder questões da forma “Por que você precisa de X?”, a partir de raciocínio abstrato. Isso auxilia a transformar o raciocínio das questões sobre “como?” para as questões sobre “o quê?”.	Realizar uma análise de riscos, investigando técnicas, custos, prazos e incluindo análise de custos e benefícios e viabilidade baseado na disponibilidade da tecnologia.
Priorização	Determinar funcionalidades críticas para a missão.	Priorizar requisitos baseados em custo e dependência. Estudar como o sistema pode ser implementado de forma incremental, investigando modelos arquiteturais apropriados.
Integração e Validação	• Resolver a maior quantidade possível de pontos em aberto. • Validar que os requisitos estão concordando com os objetivos iniciais. • Obter autorização e verificação para passar ao próximo passo de desenvolvimento (e.g. a demonstração e a validação).	Resolver conflitos e verificar consistência.

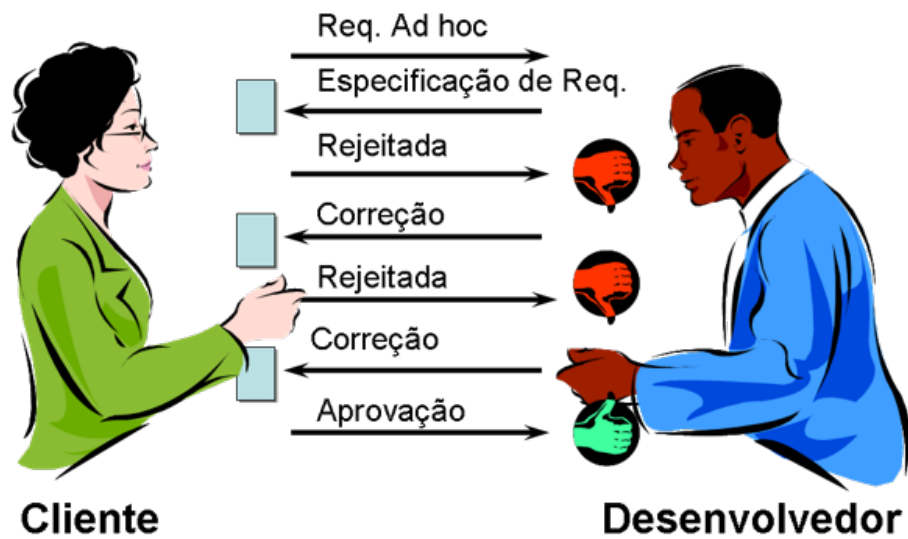


Figura 12.6.: A eliciação de requisitos é um processo iterativo onde, por aproximações sucessivas, o desenvolvedor consegue a aprovação de uma especificação de requisitos.

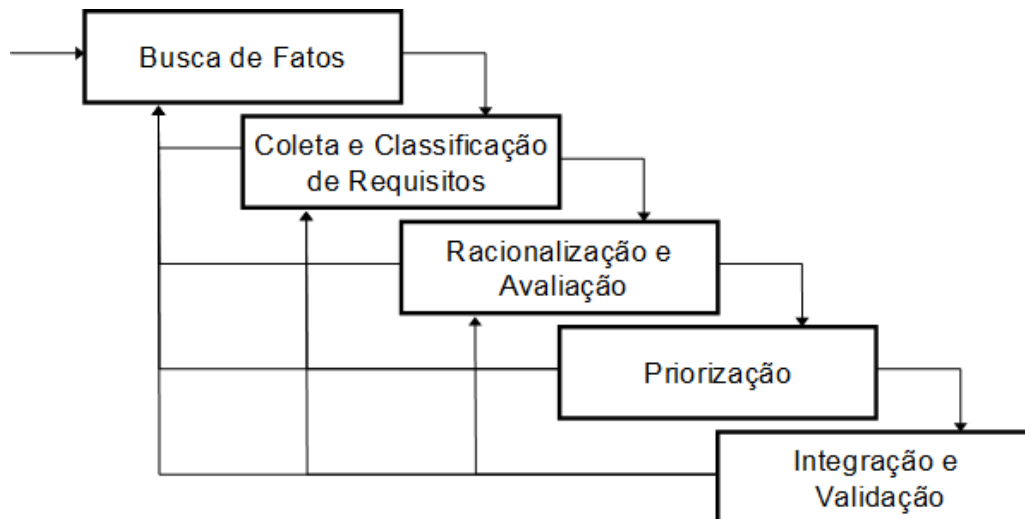


Figura 12.7.: O processo de eliciação de requisitos, adaptado de Christel e Kand (1992).

12.7. Níveis de Abstração dos Requisitos

Qualquer que seja o sistema, existem várias formas de entendê-lo. Similarmente, existem vários níveis de abstração de requisitos em que eles podem ser definidos, dependendo da visão necessária naquele instante. Alguns níveis de abstração reconhecidos para o início do projeto são (K. E. Wiegers e Beatty, 2013), na sequência em que normalmente aparecem:

1. **requisito do negócio** descrição de alto nível do objetivo do projeto que justifica a necessidade do projeto para o negócio, com visão dirigida para os objetivos do negócio;
2. **requisito do usuário** é algum comportamento ou característica que o usuário deseja do software ou o sistema como um todo; o que o usuário quer. São escritos pelo próprio usuário ou levantados por um analista de sistemas que consulta o usuário.
3. **requisito do sistema** é algum comportamento ou característica exigido do sistema como um todo, incluindo hardware e software. O comportamento desejado do sistema. São normalmente levantados por engenheiros ou analistas de sistemas, refinando os requisitos dos usuários e os transformando em termos de engenharia.
4. **requisito do software** é algum comportamento ou característica que é exigido do software. São normalmente levantados por analistas de sistemas. O objetivo da análise é capturar todos os requisitos para o software sendo desenvolvido e apenas aqueles requisitos verdadeiros.

12.8. O Risco de Requisitos Falsos

Uma especificação de software deve conter todos os requisitos e apenas requisitos verdadeiros (McMenamin e Palmer, 1984).

É óbvio que a falta de requisitos implica que o sistema não atenderá seus clientes, não atingindo o valor desejado. Já o requisito falso, ou seja, aquele que não é necessário para que o sistema cumpra sua finalidade (McMenamin e Palmer, 1984), implica em custos e riscos maiores ao projeto.

Segundo McMenamin e Palmer (1984) há dois tipos de requisitos falsos em uma especificação: os tecnológicos e os arbitrários.

Uma das formas do requisito falso tecnológico acontecer é pela transmissão de uma tecnologia de um sistema já existente para outro, quando ela não devia mais existir. Por exemplo, um sistema antigo que precisa avisar os usuários que algo aconteceu pode enviar avisos por *pager*, fax, SMS e correio eletrônico. Em uma nova versão, se adiciona o envio por *Whatsapp* e *Messenger*, mas não se remove o envio por *pager* e fax, tecnologia que não são mais usadas.

A segunda forma é pela especificação antecipada de um requisito tecnológico. Requisitos tecnológicos falsos tendem a levar o sistema novo em uma direção de desenvolvimento que pode não ser a mais adequada (McMenamin e Palmer, 1984). Um exemplo seria em um projeto de longo prazo especificar já no início do projeto que deveria ser usado um *framework* específico de software, sem avaliar se o *framework* atende todas as capacidades exigidas. Ao longo do projeto, por exemplo, pode ser descoberto que é necessário que o software seja um web site responsivo, tecnologia que não necessariamente estaria coberta pelo *framework*.

Os requisitos falsos arbitrários aparecem quando o analista inclui um requisito por vontade própria ou por causa do uso de uma técnica de modelagem (McMenamin e Palmer, 1984).

Requisitos falsos tem dois efeitos no desenvolvimento de um projeto de software: eles podem esconder os requisitos verdadeiros e eles aumentam o tamanho do sistema, gerando custos e riscos maiores.

12.9. Características de um Bom Requisito

Um requisito, ou melhor, seu registro, deve ter as seguintes características (IEEE, 2018):

necessário, significando que, se retirado, haverá uma deficiência no sistema, isto é, ele não atenderá plenamente as expectativas do usuário;

- em especial, não devem existir requisitos do tipo “seria interessante ter”³, ou o requisito é necessário ou é dispensável;
- deve ser levado em conta que cada requisito aumenta a complexidade, o custo e o risco do projeto, logo, não podem ser introduzidos de forma espúria;

apropriado, a intenção e a quantidade de detalhe é apropriada ao nível de abstração da especificação e a entidade a que se refere;

não-ambíguo, possuindo uma e apenas uma interpretação, sendo muito importante prestar atenção na linguagem sendo utilizada;

completo, ou seja, não precisa ser explicado ou aumentado, garantindo capacidade suficiente do sistema;

consistente, não contradizendo ou mesmo duplicando outro requisito e utilizando os termos da mesma forma que outros requisitos;

singular, ou seja, sendo apenas um requisito e não uma composição de requisitos;

alcançável, ou seja, realizável a um custo definido por uma ou mais partes interessadas no sistema ;

rastreável, de forma que sua fonte ou origem possa ser encontrada;

verificável, não sendo vago ou geral e sendo quantificado de uma maneira que permita a verificação na forma de inspeção, análise, demonstração ou teste, e

correto, sendo uma representação adequada, e

conforme, de acordo com um padrão e estilo aprovados, quando aplicável.

Além desses fatores, quando um requisito for funcional, deverá ser também **independente da implementação**, ou seja, o requisito define o que deve ser feito, mas não como.

Um requisito mal elaborado pode ser transformado em um requisito por meio de seu detalhamento. Por exemplo, o requisito “O sistema deverá responder rapidamente” tem grande ambiguidade. O que é “rapidamente”? Em que contexto deve ser entendido? Esse requisito pode ser melhorado para “O tempo de resposta do sistema deverá ser de 2 segundos”. Esta forma, apesar de mais detalhada, gera uma dúvida: se o sistema for mais rápido do que isso, o tempo de resposta deve ser aumentado até chegar a 2 segundos? Queremos exatamente dois segundos? Podemos melhorar então para “O tempo de resposta do sistema deverá ser de no máximo de 2 segundos”. Já está bem melhor, mas isso se aplica a todas as partes do sistema? Alguns relatórios, por exemplo, não podem tomar mais tempo. O requisito é alcançável? Então podemos melhorar para dois requisitos “O sistema deverá buscar manter o tempo de resposta menor do que 2 segundos em todas suas funcionalidades” e “O sistema deverá avisar ao usuário de qualquer operação cujo tempo de resposta previsto for maior do que 2 segundos”.

³Isso está em desacordo com a proposta de priorização da técnica MoSCoW

12.9.1. Efeitos de Requisitos com Problemas

A importância de obter requisitos corretos pode ser compreendida rapidamente se imaginarmos que um requisito, quanto mais básico e importante ele é, mais partes do sistema vai afetar.

Basicamente, cada requisito do negócio vai gerar um ou mais requisitos de usuário, e cada requisito do usuário vai gerar um ou mais requisitos do sistema, que vão, por sua vez, gerar outros requisitos com mais detalhes, e isso vai se repetindo a cada fase do projeto de software. O efeito, com a evolução do projeto, é que cada requisito inicial representa não só um compromisso, mas um fator crítico em uma sequência de decisões que são tomadas ao longo do projeto e influenciam várias de suas partes.

Por exemplo, se um requisito for “O sistema deverá enviar uma mensagem de felicitações no dia do aniversário do usuário”, será necessário ter um campo com a data de aniversário no banco de dados, campos em telas que permitem entrar e corrigir a data de aniversário, uma função para mandar a mensagem, possivelmente uma tela, ou campo em uma tela existente, para cadastrar essa informação, etc. Várias partes do código então se referem a um só requisito.

Assim, os erros de requisito afetam drasticamente todo o projeto. Dados empíricos indicam que um problema nos requisitos, se detectado nas fases finais do projeto, pode custar 100 vezes mais para ser corrigido do que se for detectado na fase de elicitação de requisitos. Mesmo que essa detecção se faça mais cedo, em uma fase intermediária, esse custo ainda seria 10 vezes maior.

Um requisito que não caiba em uma das características de qualidade citadas será propenso a defeitos e aumentará o risco do projeto.

Se não for necessário, gastaremos esforços em sua definição e implementação que seriam mais bem gastos tratando de outros requisitos. Além disso, cada requisito adicional aumenta o risco do projeto como um todo, ainda mais se a adição for dispensável.

Se for ambíguo, corremos alto risco de implementá-lo errado, desagradando o cliente.

Se não for verificável, não poderemos testar de maneira inequívoca se está corretamente atendido, possibilitando discussões sobre sua realização.

Se não for completo, faltará ao projeto alguma coisa que o usuário necessita. Se não for consistente, o projeto chegará a encruzilhadas ou talvez se torne impossível.

12.10. Atributos dos Requisitos

A norma IEEE 29148 diz que requisitos bem formados devem ter incluir um conjunto de atributos descritivos que permitam entendê-los e gerenciá-los, e dá como exemplo(IEEE, 2018):

- identificação;
- versão;
- proprietário, isto é, quem mantém o atributo;
- prioridade para as partes interessadas;
- justificativa;
- risco;
- dificuldade, e
- tipo.

Veremos que a maioria desses atributos são normalmente informados nos métodos atuais, como histórias do usuário ou casos de uso, porém normalmente o risco não é tratado requisito por requisito, sendo normalmente resultado de outra atividade de gerência de projeto ou análise de sistemas.

12.10.1. Volere e o *snowcard*

O Cartão Volere ou *snowcard* (J. Robertson e S. Robertson, 2006) é uma forma de registrar requisitos de software que atende parcialmente esta especificação, foi apresentado na Figura 12.1. O registro de requisitos em cartões possibilita que realização de algumas atividades interativas sejam realizadas com facilidade, como a ordenação por prioridade. O cartão apresentado na Figura 12.1, contém os seguintes campos:

- **número identificador**, o para facilitar a discussão, identificamos todos os requisitos unicamente;
- **tipo**, classificando-o como funcional, não funcional, ou outra classificação mais detalhada usada na organização;
- **evento**, história do usuário, caso de uso, etc. que o atende;
- **descrição**, uma sentença que descreve o requisito;
- **justificativa**;
- **donte do requisito**, a pessoa, grupo, organização, documento ou outra que o origem para o requisito;
- **critério de aceitação**, uma medida que possa ser usada para garantir que o requisito foi alcançado;
- **satisfação do usuário**, um grau, de 1 (nenhum interesse) a 5 (extremamente satisfeito), por exemplo, indicando a satisfação do cliente se esse requisito for alcançado;
- **insatisfação do usuário**, um grau, de 1 (nenhum interesse) a 5 (extremamente insatisfeito), por exemplo, indicando a satisfação do cliente se esse requisito não for alcançado;
- **dependências**, referências a outros requisitos que dependem de alguma forma desse requisito;
- **conflitos**, referência aos requisitos que de alguma forma conflitam com esse
- **material de apoio**, listagem de material de apoio para atender esse requisito, e
- **histórico**, focumentação da criação e das mudanças efetuadas.

Um diferencial do método Volere é que ele associa o valor a duas perguntas: a satisfação com a implementação do requisito e a insatisfação com sua ausência no produto. Isso permite classificar cada requisito em um de quatro quadrantes, de acordo com a nota que recebem: crítico, decoração, conflituooso ou dispensável (não fazer). A Figura 12.8 demonstra essa ideia.

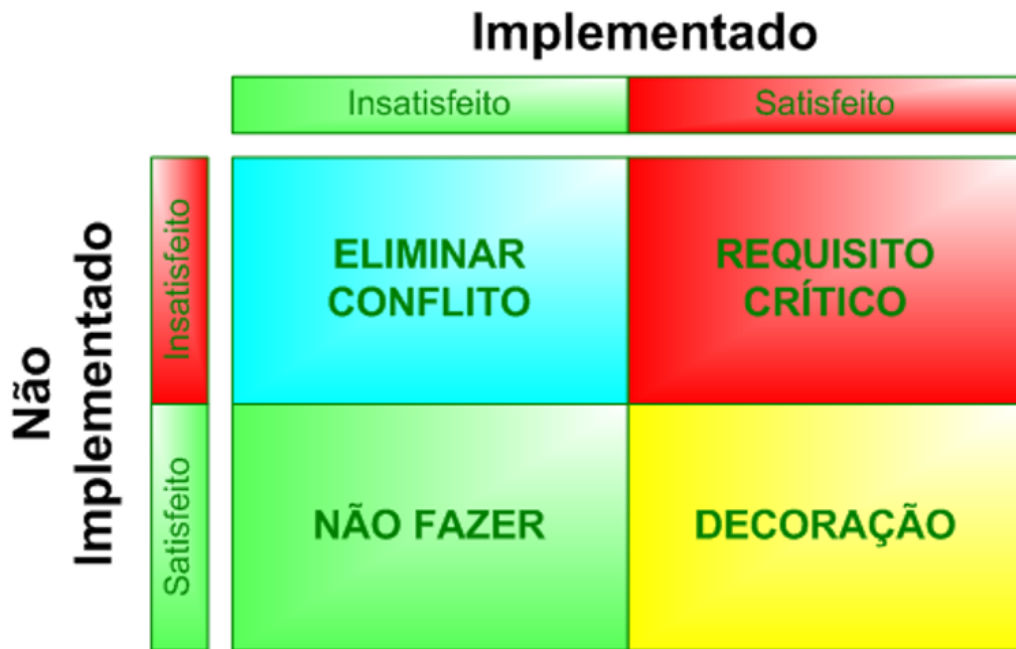


Figura 12.8.: Cada requisito se posicionará em algum ponto desse gráfico, de acordo com as notas de avaliação.

12.11. Priorizando Requisitos

Existe uma tendência grande de o sistema crescer muito durante a análise. Principalmente se entrevistamos um grande número de pessoas, existe uma facilidade natural que elas têm para propor novas funcionalidades para um sistema que ainda não existe, por imaginarem alguma utilidade nessas funções propostas. Assim, muitas vezes nos vemos envolvidos com uma quantidade de requisitos tão grande que é óbvio que o sistema a ser feito não poderá ser entregue no prazo ou pelo custo combinado, ou que se pensava em combinar.

Nesse caso, algumas técnicas podem ser utilizadas para caracterizar o que deve ser realmente feito ou, pelo menos, em que ordem as coisas devem ser feitas. A priorização de requisitos é atividade essencial em qualquer projeto e natural em métodos ágeis que usam o *product backlog*.

A primeira técnica disponível é associar a cada requisito do sistema uma importância. Uma escala de três ou cinco valores é adequada para isso, como em: “Imprescindível para o sucesso do sistema”, “Funcionalidade Importante, mas podemos esperar algum tempo”, “Ajudaria ter, mas é possível viver sem essa funcionalidade”, “Benefícios mínimos”, “Desnecessário”.

A segunda técnica disponível é planejar o sistema para ser entregue em várias **versões**, mesmo que nem todas as versões estejam incluídas nesse contrato, e pedir para o cliente determinar que funcionalidades devem aparecer em cada versão. Nesse caso pode ser interessante dar um peso ou custo para cada requisito, de modo que o cliente possa “controlar seus gastos”.

Uma terceira técnica disponível é dar uma **moeda virtual** para o cliente, por exemplo, 1000 dinheiros, e pedir para ele distribuir quanto pagaria por cada função, priorizando no desenvolvimento aquelas funções que o cliente decidir pagar mais.

Uma quarta técnica é conhecida como **MoSCoW**, onde cada item proposto é associado a um conceito do grupo “*Must Have*”, precisa ter, “*Should Have*”, devia ter, “*Could Have*”, podia ter e “*Won't Have*”, não vai ter (IIBA, 2011), criando grupos com significados específicos;

A técnica **Volere** também separa os requisitos dentro de seu quadro de importância, como visto na Figura 12.8.

Todas essas técnicas, porém, ficam dependentes de uma outra priorização importante dos requisitos: a priorização por dependência.

Devem ser levados em conta os vários fatores que influenciam nessa determinação de prioridades, entre eles os citados por (Volare, 2020):

- Diminuir o custo da implementação
- Valor para o comprador
- Tempo para implementar
- Facilidade técnica de implementar
- Facilidade do negócio para implementar
- Valor para o negócio
- Obrigação por alguma autoridade externa

12.12. Requisitos Mudam com o Tempo

Também é importante perceber que os requisitos de um software mudam com o tempo. Essas mudanças ocorrem porque o ambiente em que o software reside também muda. Os países alteram suas leis, as organizações alteram suas práticas, a tecnologia evolui, os usuários exigem novas funcionalidades até então não imaginadas ou desnecessárias.

Caper Jones (2005), afirmou que a taxa de mudanças de requisitos para um software comercial em desenvolvimento chega a 3,5% ao mês, e para um software para Web, chega a 4,0% ao mês. Considerando o tempo médio de execução desses e outros tipos projetos, ele chega a valores médios de 2,58% de mudanças ao mês, 14 meses em média de execução e 32,33% de mudança total nos requisitos. É importante notar, porém, que todas essas afirmações são baseadas em cálculos com Pontos de Função para manutenção, onde pequenas alterações podem causar grandes efeitos. Além disso, modificações ao longo do projeto podem representar não uma mudança do requisito verdadeiro, mas sim uma compreensão melhor do que é esse requisito. Mesmo assim, uma mudança corretiva em um requisito tem o mesmo impacto de uma mudança por novas necessidades.

Esse fato tem efeitos em nossa prática. O primeiro é que devemos estar preparados para a mudança, pois ela vai acontecer. O segundo é que quanto maior o tempo de duração do projeto, maior a quantidade de mudanças de requisitos, o que aumenta ainda mais o risco, que já é afetado pelo projeto ser longo e provavelmente complexo.

Assim, não só é importante conhecer os requisitos, mas também conhecer qual a sua estabilidade. Requisitos pouco estáveis devem ser tratados de forma diferente dos requisitos mais estáveis. Requisitos mais críticos devem ser tratados de forma diferente dos requisitos opcionais. Isso não tem relação com o software apenas, mas sim com atender o negócio.

12.13. Requisitos e Necessidades

Requisitos são originários de necessidades das partes interessadas. Enquanto os requisitos vivem no “mundo das soluções”, as necessidades vivem no “mundo dos problemas”. Os requisitos indicam as características que devem ser observadas no sistema, que atendem as necessidades.

Voltando ao discutido no Capítulo 4, as necessidades causam desejos, que por sua vez causam demandas. Os desejos são influenciados pelo ambiente, as demandas pela capacidade financeira do comprador. Assim, se uma organização tem como necessidade tornar mais rápido seu processo de vendas, baixando de 15 minutos para 3 minutos o tempo de concluir uma venda, como estratégia de negócios ela pode desejar desde um aplicativo para celular para seus vendedores usarem na loja, como faz a Apple, ou uma loja toda automatizada onde o que você coloca no carrinho, ou tira do carrinho, é identificado por câmeras e posto, ou removido, automaticamente de sua conta, o que acontece no mercado da Amazon. Porém, o que ela pode pagar? Pode ser que apenas uma nova versão do seu software de caixa registradora, de forma que ele seja mais rápido. Os requisitos virão dessa demanda.

Como é visto no Capítulo 11, necessidades podem ser provenientes de problemas, e um problema é uma diferença, vista com uma perspectiva comportamental, entre uma situação percebida e uma situação desejada. No momento, nos basta a noção que o problema incomoda o usuário (ou a parte interessada) ao ponto dele considerar necessário investir alguma quantia ou esforço de forma a evitar que o problema aconteça.

É comum ouvirmos o ditado “Em time que está ganhando, não se mexe”. Quando somos chamados para desenvolver um sistema, devemos imaginar imediatamente que, se alguém deseja mexer em sua organização, então é porque “não está ganhando”, pelo menos da maneira que supõe possível. Faz pouco sentido imaginar que alguém iria fazer um investimento de risco, e sempre há risco no desenvolvimento de software, sem que haja uma necessidade clara. Se isso for identificado, talvez seja mais adequado cancelar o projeto. Não se deve esquecer o adágio contrário: “em time que está vencendo que se mexe”. Essa frase representa a ideia que para continuar “ganhando” é necessário estar em constante evolução, e isso pode ser também uma motivação importante para um projeto de software.

Muitas vezes o analista se depara com a necessidade de, antes de definir requisitos, definir propriamente qual o problema a ser tratado. Para isso, deve fazer com que os clientes cheguem a um acordo sobre qual é o verdadeiro problema, ou o problema mais importante. Faz pouco sentido construir um sistema se o verdadeiro problema de negócios não for endereçado pela solução planejada. Isso é tratado no Capítulo 11

Um sistema pequeno certamente produzirá a solução para uma pequena quantidade de problemas, um sistema grande certamente atingirá uma grande quantidade de problemas. O ponto em questão é que deve existir alguma não conformidade, seja ela atual ou futura, para valer a pena o investimento, e o risco, de tentar um sistema novo.

12.14. Requisitos de Acordo com Normas IEEE

A IEEE tem um conjunto de normas que tratam de requisitos. Esta seção trata de alguns dos assuntos levantados por essas normas.

Segundo a norma IEEE 24765:2010 (IEEE, 2018) requisitos devem dizer o que (*what*) é desejado, e não como (*how*), sem incluir nenhuma decisão de projeto nos seus níveis mais abstratos.

A linguagem da especificação de requisitos deve evitar termos gerais e abstratos, ou que geram requisitos difíceis de verificar. Devem ser evitados(IEEE, 2018):

- adjetivos e superlativos, como “bom” ou “melhor”;
- linguagem subjetiva, como “amigável ao usuário” ou “custo eficiente”;

- pronomes vagos, como “aquele”;
- termos ambíguos como advérbios e adjetivos, como “quase sempre” ou “mínimo”;
- termos lógicos que em português podem ser confusos, como “e” ou “ou”;
- termos não verificáveis, como “fornece suporte”;
- frases comparativas ambíguas, como “melhor que” ou “de melhor qualidade”;
- termos que não são um compromisso definitivo, como “se possível”
- termos que implicam totalidade, como “nunca” ou “sempre”, pois são difíceis ou impossíveis de verificar, e
- referências incompletas, como falta de uma versão ao especificar um software a ser usado.

Ainda segundo a norma IEEE 29148 (IEEE, 2018) é comum estipular o significado uso de palavras que definem obrigatoriedade, futuro ou proibição⁴. A norma ainda diz que requisitos ágeis, como histórias do usuário, não precisam seguir essa regra.

A norma IEEE 29148 (IEEE, 2018) apresenta uma sintaxe para os requisitos com a forma:

[condição][sujeito][ação][objeto][restrição]

onde:

- **condição** é uma regra para ativação do requisito, seja ela o acontecimento de um evento ou um fato que se torna realidade;
- **sujeito** descreve um ator, normalmente o próprio sistema sendo especificado;
- **ação** descreve a ação principal do requisito;
- **objeto** descreve o objeto da ação, e
- **restrição** limita a forma como a ação funciona.

As sintaxes de exemplo apresentadas para requisitos na norma IEEE 29148 são(IEEE, 2018) são:

- **[sujeito] [ação] [objeto] [restrição a ação]**, como em “O sistema acadêmico [sujeito] deve calcular [ação] a aprovação de cada aluno [objeto] segundo as regras do curso específico [restrição]”, ou
- **[condição] [sujeito] [ação] [objeto] [restrição a ação]**, como em “Ao fim do período [condição] o sistema acadêmico [sujeito] deve calcular [ação] a aprovação de cada aluno [objeto] segundo as regras do curso específico [restrição]”.

12.14.1. Requisitos Funcionais na Norma IEEE

Segundo a norma IEEE 830:1998 (IEEE, 1998) os requisitos funcionais devem “definir as ações fundamentais que devem acontecer no software ao aceitar e processar as entradas e no processamento e geração das saídas”. Normalmente, devem ser listados com uma sentença que se inicia com “O sistema deve”⁵. No contexto dessa norma, os requisitos funcionais incluem(IEEE, 1998):

- validações de entradas;
- sequência exata de operações;
- respostas a situações anormais, como sobrecarga, falhas de comunicação e tratamento e recuperação de erros;
- efeitos dos parâmetros, e

⁴Em inglês são usadas palavras como “*shall*”, “*must*”, “*should*” e “*shall not*”.

⁵Em inglês, “The system shall”

- relacionamentos das saídas com as entradas, incluindo sequências de entradas e saídas e fórmulas de conversão de entradas para saídas.

Exemplos de requisitos funcionais seguindo essa norma são:

- o sistema acadêmico deve permitir que um aluno solicite seu boletim;
- o sistema acadêmico deve permitir que um professor dê as notas dos alunos para as turmas em que está alocado, e
- o sistema acadêmico deve permitir a um coordenador incluir um aluno em uma turma apesar de ser contra as regras de inclusão.

Para evitar confusões entre um sistema funcionando hoje em dia e um sistema sendo especificado para o futuro, é possível colocar o verbo deve no futuro, ficando o início do requisito “O sistema deverá”.

12.14.2. O Documento de Especificação de Requisitos

Uma especificação de requisitos de sistema é um documento que reúne os requisitos definidos e aceitos por todos os interessados naquele sistema.

Os assuntos que devem ser tratados em uma especificação de requisitos são, segundo a norma IEEE *ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering* (IEEE, 1998) são:

- funcionalidade, ou o que o software tem que fazer?
- interfaces externas, com quem ou com que sistemas o software interage?
- desempenho, qual a velocidade, disponibilidade, tempo de resposta das várias funções, etc.?
- atributos (de qualidade), quais são as considerações de portabilidade, correção, manutenibilidade, segurança, etc.?
- restrições de projeto impostas a implementação, como padrões a serem seguindo, políticas de integridade da base de dados, limites dos recursos disponíveis, ambiente de operação, etc.?

Segundo o mesmo padrão, um sumário adequado a uma especificação de requisitos seria(IEEE, 2018):

1. Introdução
 - a) Objetivo do Sistema
 - b) Escopo do Sistema
 - c) Visão Geral do Sistema
 - i. Contexto do Sistema
 - ii. Funções do Sistemas
 - iii. Características dos Usuários
 - d) Definições
2. Referencias
3. Requisitos do Sistema
 - a) Requisitos Funcionais
 - b) Requisitos de Usabilidade
 - c) Requisitos de Desempenho
 - d) Requisitos de Interface
 - i. Requisitos de Interface Externa
 - ii. Requisitos de Interface Interna

- e) Operações do Sistema
 - f) Modos e Estados do Sistema
 - g) Características Físicas
 - h) Condições Ambientais
 - i) Requisitos de Segurança
 - j) Requisitos de Gerência de Informação
 - k) Requisitos Regulatórios e Legais
 - l) Requisitos de Sustentação do Ciclo de Vida do Sistema
 - m) Requisitos de Empacotamento, Trato, Envio e Transporte
4. Verificação (paralelo as subseções da seção 3)
5. Apêndices
- a) Suposições e Dependências
 - b) Acrônimos e Abreviações

12.15. A Visão do Novo Sistema

Dar uma visão do sistema sendo proposto é uma maneira de definir requisitos de altíssimo nível com facilidade de entendimento para as partes interessadas.

A Visão do Sistema é uma descrição de como vai funcionar o novo sistema em uma descrição simples, em linguagem corrente. Essa visão deve ser escrita com forte apoio do cliente, senão pelo próprio cliente.

Possivelmente, essa visão é apresentada junto com uma visão do sistema atual, com efeito comparativo, sendo ambas no mesmo nível de abstração e dentro de um mesmo documento inicial do projeto.

A visão pode incluir o protótipo de algumas telas do novo sistema, com a finalidade de mostrar a diferença do sistema novo para o velho ou ainda mostrar como será o comportamento de uma nova finalidade.

A visão do sistema pode incluir não só o funcionamento do sistema, mas também expectativas de comportamento e de efeitos do sistema no negócio. Deve ficar claro que a visão do sistema é uma declaração do usuário, não necessariamente um comprometimento do desenvolvedor.

A visão do sistema inclui também os requisitos já detectados, informalmente, pelo analista. Esses requisitos podem ser divididos em requisitos funcionais, requisitos de informação e requisitos não funcionais. Obviamente só estamos interessados em requisitos verdadeiros.

12.16. Descrevendo Requisitos

Normalmente as especificações de requisitos são escritas em linguagem natural (inglês ou português, por exemplo). O problema é que a forma como falamos e normalmente escrevemos é bastante ambígua. Isso exige que adotemos algumas técnicas básicas, principalmente um formato padronizado, um estilo de linguagem e uma organização que facilite a manipulação do conjunto de requisitos.

Algumas dicas para escrever requisitos são (K. Wiegers, 1999):

- use sentenças e parágrafos curtos;
- use a voz ativa;

- use verbos no futuro;
- use os termos de forma consistente e mantenha um glossário;
- para cada requisito, avalie se a partir de sua definição é possível determinar se ele está pronto ou não;
- garanta que todos os requisitos são verificáveis imaginando (e possivelmente documentando) uma forma de fazê-lo;
- verifique requisitos agregados (termos como “e” e “ou” são uma boa indicação) e divida-os, e
- mantenha um nível de detalhe único em todos os requisitos.

12.16.1. Exemplos de Requisitos

A seguir listamos alguns exemplos de requisitos bem descritos, seguindo o estilo de exemplos originais de Karl Wiegers:

- O sistema deverá permitir que um paciente marque uma consulta.
- O sistema deverá confirmar que a consulta foi aceita pelo paciente.
- O sistema deverá permitir que um condômino solicite a segunda via de sua conta condominial.
- O sistema deverá permitir um aviso ao condômino que não pagar sua no prazo correto.
- O sistema deverá permitir que um fornecedor cadastre um produto no catálogo.
- O sistema deverá informar ao sistema de estoques que um produto foi vendido.

Todas as sentenças acima usam, na verdade, uma expressão do tipo “deverá”, tradução do inglês “shall”, que é uma forma tradicional do americano definir uma ordem. Talvez as seguintes descrições sejam mais adequadas a nossa língua:

- O sistema permitirá que um paciente marque uma consulta.
- O sistema confirmará ao paciente que a consulta foi marcada.
- O sistema permitirá que um condômino solicite a segunda via de sua conta condominial.
- O sistema permitirá um aviso ao condômino que não pagar sua no prazo correto.
- O sistema permitirá que um fornecedor cadastre um produto no catálogo.
- O sistema informará ao sistema de estoques que um produto foi vendido.

Também devemos notar que as entradas do usuário são descritas como “deverá permitir”. Alguns autores, como o próprio Wiegers, admitem que uma especificação de requisito exija algo do usuário, como em:

- O paciente deverá especificar qual a especialidade médica que deseja.

Essa abordagem não é adequada, pois está fazendo uma exigência ao usuário e não ao sistema. Uma abordagem mais apropriada seria:

- O sistema exigirá que o usuário especifique a especialidade médica que deseja.

Desta forma deixamos claro que:

1. O requisito é do sistema
2. Cabe ao sistema exigir do usuário a resposta
3. Não cabe ao usuário saber o que fazer, mas sim ao sistema saber o que o usuário tem que fazer.
4. A especificação de requisitos do software não requer nada ao usuário.

Em conclusão, uma especificação de requisitos só deve exigir funcionalidade do sistema sendo definido. Entendemos que muitas vezes um sistema faz exigências ao ambiente. Essas exigências podem algo como “o usuário deve falar inglês” ou “o computador deve ser do compatível com chips XYZ/2001”. Todas as exigências que o sistema faz devem ser documentadas. Uma localização específica do documento para isso é a seção de suposições ou dependências, mas demandas que o sistema faz não podem ser vistas realmente como um requisito do sistema, apesar de possivelmente definir características do sistema.

12.17. Exercício

Exercício 12.1:

Escreva um conjunto de requisitos funcionais na forma de “O sistema deve...” para um aplicativo de celular que sirva como relógio e despertador, permitindo vários alarmes.

Exercício 12.2:

Procure na internet os seguintes documentos:

- *Software Requirements Specification for Cafeteria Ordering System prepared by Karl Wieggers v1.0 de 2002*, e
- *Cafeteria Ordering System Vision and Scope Document prepared by Karl Wieggers v1.0 de 2002*.

Preste atenção que o primeiro documento tem duas versões na rede, uma com casos de uso, outra com especificações em linguagem natural. Baixe os dois. Analise os três documentos e guarde-os para exercícios futuros.

Exercício 12.3:

Baseado no texto *Cafeteria Ordering System Vision and Scope Document prepared by Karl Wieggers v1.0 de 2002*, feito com linguagem Natural, crie um documento semelhante para o sistema descrito no Capítulo 8

Exercício 12.4:

Baseado no texto *Software Requirements Specification for Cafeteria Ordering System prepared by Karl Wieggers v1.0 de 2002*, feito com linguagem Natural, escreva um documento semelhante para o sistema descrito no Capítulo 8

Exercício 12.5:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita especifique no formato IEEE todos os requisitos que você encontrar.

Exercício 12.6:

Identifique todos os problemas na sua lista de requisitos para a Livraria Resolve. Defina perguntas no formato 5W2H para resolver esses problemas.

Casos de Uso

We thus do not model reality as it is, as object orientation is often said to do, but we model the reality as we want to see it and to highlight what is important in our application

(Ivar Jacobson)

Conteúdo

13.1.	Conceituação de Caso de Uso	175
13.2.	Conceitos Importantes nos Casos de Uso	176
13.3.	A Narrativa do Caso de Uso	182
13.4.	O Nível de Abstração	183
13.5.	O Escopo do Caso de Uso	184
13.6.	Casos de Uso Especiais	185
13.7.	Partes do Caso de Uso	185
13.8.	Exemplo de Um Caso de Uso	186
13.9.	Levantando Casos de Usos	187
13.10.	Diagramas de Caso de Uso	190
13.11.	O Que o Diagrama de Caso de Uso Não Informa	195

Por que casos de uso?

Junto com histórias do usuário, casos de uso são uma das principais formas de especificar requisitos, sendo ainda mais detalhadas que as primeiras. São usados em projetos ágeis ou prescritivos e um documento muito comum de se achar em fábricas de software.

Este capítulo apresenta Casos de Uso e Diagramas de Caso de Uso. É importante não confundir os dois conceitos. Um caso de uso é uma narrativa textual detalhada, acompanhada de informações que ajudam a entendê-la, normalmente apresentado na forma de um documento. Um diagrama de caso de uso é um gráfico que resume todos os casos de uso do sistema, não fornecendo nenhum detalhe, e que informa apenas uma visão global do escopo do sistema.

Casos de uso foram criados por Ivar Jacobson em 1986, aparecendo em um artigo da OOPSLA 87(Jacobson, 1987), mas só foram publicados em livro em 1992 (Jacobson, Christerson et al., 1992). Uma das melhores descrições de seu uso foi feita por Cockburn (2000), no livro *Writing Effective Use Cases*.

A parte principal de um caso de uso é uma sequência numerada de atividades que representam as ações que acontecem durante a interação de usuários com um sistema, como na descrição do cenário principal do caso de uso fictício “Avaliar Alunos” da Figura 13.6, que é um objetivo do ator professor e manda informações para o ator aluno.

1. Sistema Acadêmico apresenta tela de opções
2. Professor escolhe Entregar Notas
3. Sistema Acadêmico apresenta lista de cadeiras disponíveis
4. Professor escolhe cadeira
5. Sistema acadêmico apresenta lista de alunos com espaço para notas
6. Professor inclui notas para todos os alunos
7. Sistema Acadêmico registra notas
8. Sistema Acadêmico envia alteração de notas para todos Alunos por email
9. Sistema Acadêmico informa ao professor que as notas foram alteradas
10. Professor sai do Sistema Acadêmico

Figura 13.1.: Exemplo de cenário principal de caso de uso.

Casos de uso são uma das técnicas de maior sucesso na análise de sistemas, e apesar de serem identificados com a Análise Orientada a Objeto, não há nenhuma obrigação de serem usados apenas nesse contexto. A principal vantagem dos casos de uso sobre técnicas anteriores é serem descrições do comportamento concreto do software a ser desenvolvido, quando no passado se dava muita importância a descrições abstratas que eram confusas para o usuário final e deixam grande margem de interpretação, e consequentemente eram ambíguas para o desenvolvedor(Jacobson, 2004).

Casos de uso contam como o sistema funciona, ou funcionará, na visão de seus usuários, o que é sempre uma visão externa ao sistema. Eles devem ser facilmente lidos tanto pelos clientes, e outras partes interessadas, quanto pelos desenvolvedores. São, na prática, histórias contada

na forma de um passo a passo, de como o usuário usa o sistema e dos efeitos dessa utilização no estado do sistema.

Já um diagrama de caso de uso tem a aparência da Figura 13.2.

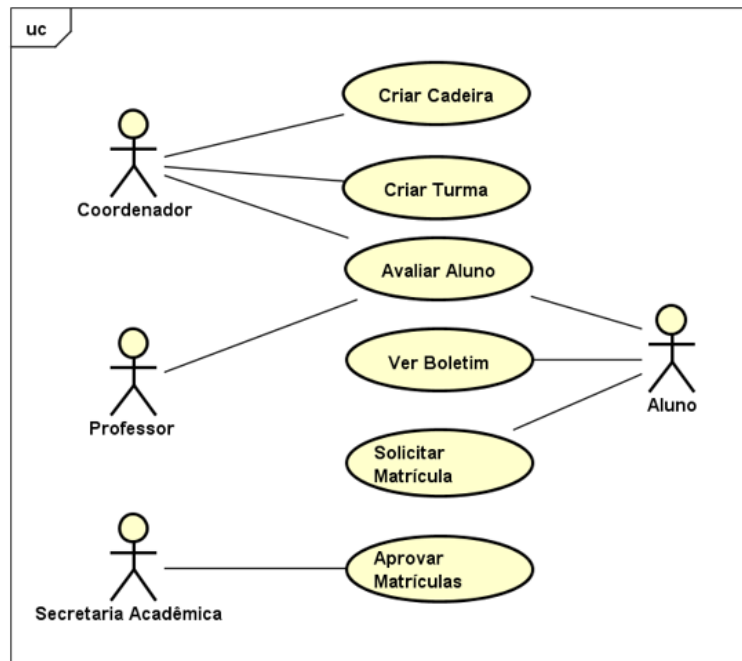


Figura 13.2.: Exemplo de um caso de uso.

Diagramas de caso de uso permitem ao seu leitor ter uma ideia do escopo do sistema. Eles apresentam os atores envolvidos e as funcionalidades esperadas do sistema de forma simples. Desde a origem da análise de sistemas é comum existir um diagrama cujo objetivo é mostrar o escopo do sistema, e o diagrama de caso de uso cumpre essa função na Análise Orientada a Objeto, sendo parte da especificação UMLOMG (2017). Basicamente eles substituíram os Diagramas de Contexto Gane e Sarson (1979) e Ruble (1997) nessa finalidade.

13.1. Conceituação de Caso de Uso

Um **caso de uso** é uma especificação, em forma de narrativa, de uma sequência de interações entre um sistema e os atores (agentes externos) que o usam de forma a alcançar um objetivo. Esse sistema pode ser um software ou uma organização complexa.

Cada caso de uso responde a uma solicitação de uma parte interessada, o **ator principal**, e descreve como o sistema vai se comportar enquanto interage com esse ator, de acordo com várias condições que podem ocorrer durante a interação (Cockburn, 2000).

Casos de uso podem ser simples ou complexos, devendo descrever, em um nível de detalhe desejado, algo que um usuário ou cliente quer que o sistema faça. Cada caso de uso descreve e define parte da funcionalidade de um sistema.

As definições de caso de uso, pelos seus criadores, evoluíram com o tempo. Jacobson, Spence e Bittner (2011a) citam duas definições, que são apresentadas a seguir.

- Um **caso de uso** mostra todas as formas de usar um sistema para alcançar um objetivo específico para um usuário específico. O conjunto de casos de uso fornece todas as formas úteis de usar o sistema e obter o valor que ele oferece.
- Um **caso de uso 2.0** é uma prática ágil e escalável que captura um conjunto de requisitos e guia o desenvolvimento incremental de um sistema para atingi-los.

Um caso de uso bem escrito deve descrever, de forma completa, um processo executado pelo sistema, contando uma história de como o ator principal tenta alcançar um objetivo específico ao usar o sistema. Na sua forma mais completa, um caso de uso apresenta vários cenários possíveis de sucesso ou falha na busca por esse objetivo(Cockburn, 2000) e ainda informações adicionais para o time.

O conjunto completo de casos de uso forma a especificação funcional de um sistema. Essa especificação é facilmente legível por usuários ou desenvolvedores, permitindo também sua validação e verificação. É importante notar que os diagramas de caso de uso têm um valor muito pequeno frente ao documento textual que é a descrição do caso de uso.

Um caso de uso deve:

- descrever uma tarefa de negócio que serve a um único objetivo de negócio;
- não ser orientado a uma linguagem de programação;
- ter o nível de detalhe apropriado;
- ser curto o suficiente para ser implementado por um desenvolvedor de software em uma versão do produto;
- ser descrito do ponto de vista externo, e
- ser consistente, tanto no nível de abstração quanto na escolha entre mostrar o sistema como uma caixa branca ou uma caixa preta.

Casos de uso também podem ser vistos como “contratos entre as partes interessadas e o sistema sobre o seu comportamento” Cockburn (2000).

Para isso, casos de uso devem descrever três tipos de ações(Cockburn, 2000):

- **interações entre dois atores**, sendo um deles normalmente o sistema;
- **validações**, que protegem uma ou mais partes interessadas, e
- **mudanças do estado interno do sistema**, que atendem uma ou mais partes interessadas.

13.2. Conceitos Importantes nos Casos de Uso

Os principais conceitos envolvidos em um caso de uso são:

- **atores**, e entre eles o **ator principal**, que participam do caso de uso;
- um **objetivo** do ator principal, que nomeia o caso de uso;
- os **cenários** que permitem atingir o objetivo ou levam a falhas, entre eles o **cenário principal** e vários possíveis **cenários alternativos**;
- as **condições** que geram os cenários alternativos;
- cada cenário é um **sequência de ações** que acontece durante a interação entre atores e o sistema, e
- cada ação é:
 - uma troca simples e unidirecional de informações entre dois atores;
 - uma validação, ou
 - uma mudança do estado interno do sistema.

13.2.1. Ator

Um ator é uma entidade externa ao sistema com comportamento próprio. Com isso queremos dizer que o sistema não pode controlar o comportamento do ator.

Os atores interagem com o sistema para alcançar seus objetivos. Em cada Caso de Uso, um ator é o ator principal, que visa alcançar com sucesso o seu objetivo principal por meio daquele caso de uso. Muitas vezes, o sistema invocará outros atores, que deverão cumprir suas responsabilidades para o ator principal alcançar seu objetivo, ou receberão informações.

Um ator não é a mesma coisa que um usuário. Um ator representa um papel dos usuários de um sistema. Tanto um ator pode representar um papel assumido por vários usuários, como o papel de correntista em um banco, para o qual existem milhares de usuários, quanto um usuário (pessoa real) pode ser representado por vários atores, como no caso de um funcionário do banco que é também correntista. Conceitualmente, em UML, usuários são instâncias de atores e cada ator define uma classe de usuários(OMG, 2017).

Os principais tipos de atores são:

- pessoas;
- organizações;
- equipamentos, e
- sistemas.

Um ator, nos diagramas de caso de uso, é representado por um “boneco de pauzinhos” sobre o nome que identifica o ator, como na Figuras 13.3 e Figura 13.4.



Figura 13.3.: Representação de um ator em um diagrama de caso de uso.



Figura 13.4.: Representação de dois atores identificados.

O sistema sendo descrito também é um ator, que conversa com o ator principal e com os outros atores.

É importante notar que existem partes interessadas que não são atores, pois nunca interagem com o sistema. Por exemplo, o patrocinador ou os próprios desenvolvedores, mas ainda assim podem ser afetadas pelas ações do sistema. Um exemplo possível seria um sistema de segurança pública que guarda informação sobre criminosos, mas esses criminosos nunca tem contato com o sistema.

13.2.2. Objetivos

Todo ator possui um ou mais objetivos ao usar o sistema. Alguns objetivos são ativos, isso quer dizer que o usuário invoca o sistema, se tornando o ator principal de um caso de uso, e outros podem ser ditos passivos, ou seja, o sistema vai informar o ator de algo ou invocar o ator para alguma ação.

Cada objetivo define e dá nome a um caso de uso. Esse objetivo deve deixar bem claro o que o ator deseja obter como funcionalidade do sistema, definindo de forma implícita o comportamento que ele espera do sistema.

Exemplos de objetivos são:

- em um sistema bancário, depositar dinheiro em conta corrente, sacar dinheiro de conta corrente, pagar boleto, transferir dinheiro para outra conta corrente, requisitar cheques;
- em um sistema de seleção de profissionais, enviar currículo, marcar entrevista, avaliar currículo, solicitar classificação dos candidatos, e
- em um sistema acadêmico, solicitar matrícula, avaliar alunos, informar presenças, emitir boletim.

Nomeando Casos de Uso

O objetivo é usado para nomear o caso de uso. Isso é feito no formato

<verbo no infinitivo><objeto>

como em “emitir boletim”. Para isso devem ser usados verbos que indiquem uma ação do ator, pois ele entra no sistema com a intenção de fazer algo. A Tabela 13.1 apresenta alguns verbos comumente usados para nomear casos de uso.

Tabela 13.1.: Verbos comumente usados para nomear casos de uso

Alterar	Descobrir	Permitir
Analisar	Encontrar	Preencher
Aprontar	Entregar	Preparar
Arranjar	Especificar	Projetar
Avaliar	Estabelecer	Providenciar
Buscar	Executar	Realizar
Classificar	Garantir	Recuperar
Completar	Identificar	Requisitar
Conseguir	Informar	Selecionar
Consultar	Monitorar	Solicitar
Definir	Mudar	Ver
	Notificar	

13.2.3. Cenários

Um caso de uso pode acontecer de acordo com vários cenários. Cada **cenário** descreve como uma instância específica do caso de uso pode acontecer, ou seja, as sequências específicas de ações que ocorrem na interação entre atores e o sistema.

13. Casos de Uso

Um desses cenários é o **cenário principal**, que narra como um ator alcança seu objetivo da forma mais fácil ou comum. O cenário principal é descrito de forma integral em todos os casos de uso e mostra como é esperado que a interação normal que atinge o objetivo aconteça. Por exemplo, para o caso de uso “sacar dinheiro”, solicitado pelo ator “correntista” para o sistema “caixa automático”, o cenário principal poderia ser o da Figura 13.5:

1. correntista toca na tela
2. caixa automático solicita cartão
3. correntista passa o cartão
4. caixa automático valida cartão
5. caixa automático solicita senha
6. correntista informa senha
7. caixa automático valida senha
8. caixa automático apresenta menu
9. correntista escolhe sacar da conta corrente
10. caixa automático solicita valor
11. correntista digita valor
12. caixa automático valida existência de saldo
13. caixa automático solicita cartão
14. correntista passa o cartão
15. caixa automática valida cartão
16. caixa automático fornece o dinheiro
17. caixa automático informa banco para debitar valor do saldo bancário do correntista
18. correntista retira o dinheiro
19. caixa automático fecha a sessão

Figura 13.5.: Exemplo de cenário principal do caso de uso **sacar dinheiro**.

O cenário acima é descrito na forma de uma **narrativa numerada**. Cada passo dessa narrativa é uma troca simples, e unidirecional, de informações, ou de uma mensagem, uma validação ou uma ação. Essa narrativa é o que deve acontecer se tudo der certo, ela não se preocupa, por exemplo, com o que fazer se não há saldo para retirar a quantia ou se a senha está errada. Também não se preocupa se o usuário quer fazer outra ação, como imprimir cheques.

Veja que no passo 17 o caso de uso envolve um outro ator, o banco, para o qual é enviada uma mensagem de alteração de saldo. Uma alternativa a esse passo seria “caixa automático debita valor do saque do saldo do correntista”. Esse é um tipo de passo que indica uma mudança no estado interno do sistema e também devemos considerar obrigatório, pois indica o contrato que o caso de uso cumpre.

Casos de uso também apresentam passos de validação, que tem como função proteger os interesses das partes interessadas. Os passos 4, 7, 12 e 15 fazem a validação e serão usados como locais onde condições definem alternativas.

Cenários, além do cenário principal, podem representar variantes normais do fluxo principal, casos raros, exceções e erros. Dessa maneira, podemos compreender um caso de uso como a descrição de uma coleção de cenários de sucesso ou falha que descrevem um determinado processo do sistema com a finalidade de atender um objetivo do usuário.

Na maior parte das vezes, os casos de uso também possuem vários **cenários alternativos**, alguns que também levam ao objetivo desejado, outros que levam a versões parciais desse objetivo e ainda cenários de falhas, onde o objetivo não é alcançado. Todos esses cenários também são

descritos nos casos de uso, porém normalmente de forma compactada, sendo mostrado apenas como eles diferem do caso de uso principal.

Os cenários diferem em função de condições específicas que podem acontecer na execução de uma instância do caso de uso. Por exemplo, se o caso de uso descreve uma retirada de dinheiro de uma conta bancária, o cenário principal considera que existe saldo suficiente na conta, enquanto cenários alternativos podem considerar que não existe saldo suficiente, que será necessário usar o cheque especial, ou que é tarde demais para retirar a quantia pedida.

No nosso exemplo, uma condição que leva a um cenário alternativo poderia ser “senha não confere” e outra poderia ser “não há saldo suficiente”. Nesse caso, cada condição levaria a um cenário alternativo.

Uma forma possível de descrever o cenário alternativo “senha não confere” seria:

1. caixa automático informa que senha não confere
2. voltar ao passo 5

Nesse caso ainda é possível terminar o caso de uso principal, o sistema volta para um passo anterior ao que aconteceu a alternativa e o ator tem mais uma chance.

Uma condição adicional seria “senha não confere pela terceira vez”, e o cenário relativo seria:

1. caixa automático informa que senha não confere
2. caixa automático informa ao banco para bloquear o cartão
3. caixa automático termina atendimento

Nesse caso não foi possível voltar ao cenário principal, o ator não conseguiu seu intento e o caso de uso terminou em falha.

Novamente, nesse último cenário, teríamos a opção de que o passo 2 fosse “caixa automático bloqueia cartão”, que indicaria uma mudança no estado do sistema.

As Figuras 13.6 e 13.7 ilustram o conceito de um caso de uso contendo um caminho principal e quatro condições que podem alterar a execução de uma instância do caso de uso. As figuras mostram, de forma abstrata, possíveis caminhos que formam cenários de execução desse caso de uso. Podemos ver, na representação da figura, que alguns caminhos alternativos podem permitir ainda alternativas adicionais (o caminho 2 é uma alternativa do caminho 1), e que em uma mesma execução de um caso de uso vários caminhos podem ser seguidos (os caminhos 1 e 3, por exemplo).

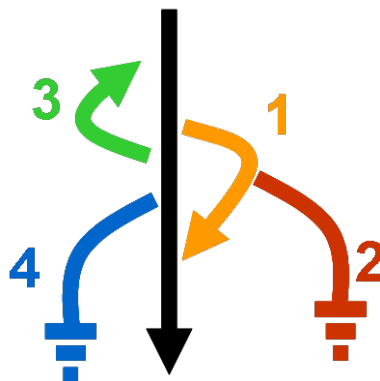


Figura 13.6.: Representação abstrata de um cenário principal e dos fluxos alternativos

Com essas figuras deve ficar claro um conceito: os cenários alternativos podem ser compostos de várias formas, mas com uma descrição compartimentada, não precisamos usar todas as instâncias

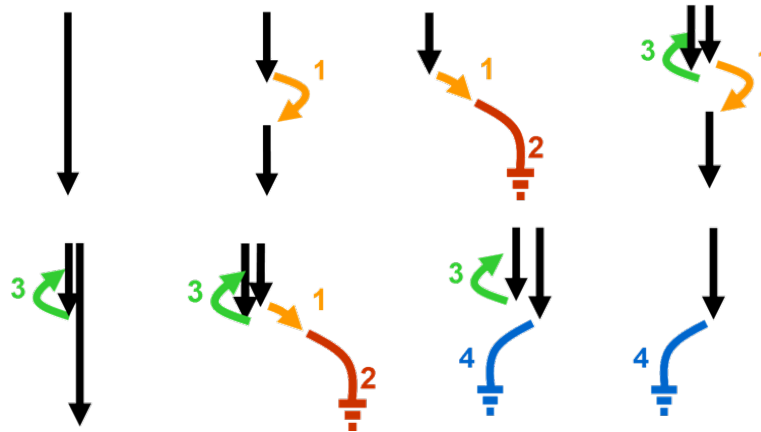


Figura 13.7.: Representação abstrata de cada alternativa investigada nesse caso.

possíveis dos cenários, por exemplo listar todos os cenários possíveis onde a alternativa 3 é chamada 1, 2, 3 ou infinitas vezes.

Também é importante notar que **não são usadas estruturas se-então-senão** ao escrever casos de uso. O que se usa são condições que podem acontecer em passos de um cenário e ordens como “vá para”. Cada cenário considera que todos seus passos de validação resultam em verdade.

Já o uso de laços, isto é, estruturas de repetição, pode ser feito de algumas formas, mas nunca na forma de um *for* ou *while* de uma linguagem de programação, mas sim como uma linguagem mais coloquial. (Cockburn, 2000) sugere:

- usar um passo onde algo pode ser feito várias vezes, como em “o comprador seleciona um ou mais produtos”, ou
- um passo não numerado que indica que alguns passos podem ser repetidos, como em “o comprador repete os passos 4-6 até ficar indicar que terminou”, que pode aparecer antes ou depois dos passos;

Por exemplo, um caso de uso onde o ator “professor” dá a nota para vários alunos poderia ser do da Figura 13.8.

- | |
|---|
| <ol style="list-style-type: none"> 1. professor seleciona DAR NOTAS
os passos 2-7 acontecem até o professor indicar que terminou 2. sistema mostra lista de alunos 3. professor escolhe aluno 4. sistema mostra registro do aluno 5. professor dá nota ao aluno 6. sistema altera a nota do aluno 7. sistema pergunta se o professor terminou 8. professor sai do sistema |
|---|

Figura 13.8.: Exemplo de cenário principal do caso de uso **atribuir notas**.

A Tabela 13.2 mostra uma notação alternativa para explicar os cenários possíveis a partir de alternativas.

Tabela 13.2.: Uma forma alternativa de representar os cenários possíveis a partir de um cenário principal e as alternativas associadas

Cenário 1	Cenário Principal			
Cenário 2	Cenário Principal	Alternativa 1		
Cenário 3	Cenário Principal	Alternativa 1	Alternativa 2	
Cenário 4	Cenário Principal	Alternativa 3		
Cenário 5	Cenário Principal	Alternativa 3	Alternativa 1	
Cenário 6	Cenário Principal	Alternativa 3	Alternativa 1	Alternativa 2
Cenário 7	Cenário Principal	Alternativa 4		
Cenário 8	Cenário Principal	Alternativa 3	Alternativa 4	

13.2.4. As Alternativas

As alternativas estão sempre associadas a uma condição ocorrendo no cenário principal, possivelmente em um momento de validação, ou, mais raramente, a um outro cenário importante.

Um cenário alternativo tem pelo menos quatro partes:

- a localização, mostrando o cenário, normalmente o principal, e o passo onde ocorre a condição;
- a condição, especificada na forma de uma expressão que pode ser avaliada como verdadeira ou falsa (lógica);
- a sequência de ações que corresponde a essa condição, e
- uma finalização que deixa claro o retorno ao fluxo principal ou a finalização de outra forma.

As finalizações possíveis para uma alternativa são:

- voltar ao início do caso de uso, comum em casos de falha onde é possível tentar outra vez;
- voltar a um passo já executado do caso de uso, trazendo o estado do sistema naquele passo;
- ir para um passo posterior ao passo onde ocorreu a condição, possivelmente o final;
- encerrar o caso de uso no cenário alternativo, com sucesso ou sucesso parcial;
- encerrar o caso de uso no cenário alternativo com falha, e
- abortar o caso de uso, em caso de falha grave.

13.2.5. Casos de Uso em UML

Em UML casos de uso descrevem o comportamento de um sistema, ou parte de sistema, como um componente ou classe, em consideração, denominado *subject*. Um caso de uso pode se aplicar a vários *subjects*, especificando um conjunto de comportamentos para os *subjects* a que é aplicado. Os atores são pessoas e outros sistemas que interagem com o *subject*. Um caso de uso gera um resultado que tem valor para os atores e outras partes interessadas no *subject* (OMG, 2017).

13.3. A Narrativa do Caso de Uso

A narrativa numerada é a forma de básica de descrição dos cenários do caso de uso. Essa forma de narrativa apresenta várias vantagens sobre outras formas já utilizadas de modelagem

de processos, como manter o contexto visível, eliminar dificuldades de compreensão para o usuário (causadas por linguagens técnicas com forte grau de abstração) e deixar claro qual o valor de cada função para os usuários.

Cada caso de uso possui uma narrativa principal, que mostra o que acontece no caso normal, e narrativas alternativas que tratam de atender a condições especiais que acontecem durante uma possível execução do caso de uso e fazem ele se afastar da narrativa principal.

Essa narrativa é uma especificação de uma sequência de ações que acontece interações entre o sistema e os agentes externos que usam esse sistema.

Segundo Cockburn (2000), cada passo da narrativa deve:

- usar uma gramática simples;
- mostrar quem está agindo;
- mostrar o processo andando para frente;
- mostrar a intenção do ator, não os movimentos, e
- conter um conjunto de ações razoável;

Outras formas de escrever o caso de uso são usadas no mercado, sendo que é comum que no início da análise seja usado um texto informal na forma de um parágrafo, muitas vezes tirado diretamente de uma entrevista.

13.4. O Nível de Abstração

Uma das formas mais interessantes de entender como desenvolver casos de uso é entender que eles podem ser descritos em diferentes níveis de abstração, desde um nível bastante abstrato até um nível detalhado em seus mínimos detalhes.

A idéia básica em torno desse conceito, proposto por Cockburn (2000), é que casos de uso de um nível mais abstrato explicam o porquê de um caso de uso de um nível mais baixo, enquanto o caso de uso de um nível mais baixo explica como o caso de uso do nível mais abstrato é realizado.

Podemos identificar cinco níveis de abstração para casos de uso (Cockburn, 2000):

- sumário de alto nível;
- sumário;
- objetivo do usuário;
- sub-função, e
- muito detalhado.

A abordagem adotada neste texto é trabalhar no nível de objetivo do usuário. Se necessário, para projetos grandes, é possível criar resumos no nível de sumário. Por exemplo, os casos de uso “sacar dinheiro”, “depositar dinheiro” e “transferir dinheiro” poderiam ser resumidos em um caso de uso de nível superior chamado “movimentar conta”.

Raramente são usados casos de uso mais detalhados do que no nível do objetivo do usuário, a não ser os casos de uso incluídos, que são normalmente do tipo sub-função.

Tabela 13.3.: Ícones que representam o nível de abstração dos casos de uso. Fonte:(Cockburn, 2000)

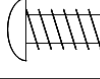
Nível	Ícone
Sumário de Alto Nível	
Sumário	
Objetivo do Usuário	
Sub-Função	
Nível Muito Detalhado	

13.5. O Escopo do Caso de Uso

Da mesma forma que definimos o nível de abstração de um caso de uso, podemos também definir o escopo do caso de uso em relação a organização ou sistema que ele descreve em níveis (Cockburn, 2000)

- organização, caixa preta;
- organização, caixa branca;
- sistema, caixa preta;
- sistema, caixa branca, e
- componente.

Tabela 13.4.: Ícones que representam o nível de escopo dos casos de uso. Fonte:(Cockburn, 2000)

Nível	Ícone
Organização, caixa preta	
Organização, caixa branca	
Sistema, caixa preta	
Organização, caixa branca	
Componente	

O nível que trabalhamos nesse texto é normalmente o nível de sistema de caixa preta. Nesse nível, tratamos de atores nomeados usando um sistema que é sempre chamado por seu nome.

No nível de sistema com caixa branca, passamos a nomear a parte do sistema que está sendo usada.

No nível organização e caixa preta, em vez de tratar do uso do sistema, se trata da interação do usuário com a organização. Assim, um correntista passa a conversar com o banco, um paciente com o hospital. No caso da caixa branca, o correntista passa a falar com o caixa ou com o gerente, o paciente com o médico ou a enfermeira. Também é possível usar a caixa branca em relação a setores da organização, como tesouraria e atendimento.

13.6. Casos de Uso Especiais

Alguns casos de uso não tem origem nos usuários ou cliente, mas em outras partes interessadas que precisam implantar e operar o sistema. Esses casos de uso são especiais e podem ser usados poucas vezes, mas podem ser bastante complexos. Alguns casos possíveis são:

- início do sistema;
- parada do sistema;
- manutenção da informação, inclusive com operações do tipo CRUD adicionais;
- adicionar nova funcionalidade com o sistema funcionando, em especial para sistemas que não devem parar;
- transferência do sistema para um novo ambiente, etc.

Create,
Read,
Update
Delete,
operações
que
sempre
podem se
realizadas
sobre os
dados
mesmo
sem ter
uma
semântica
do negóci

13.7. Partes do Caso de Uso

Um caso de uso completo é um documento com várias partes. Entre as partes possíveis que podemos usar estão:

- nome;
- identificação(número, etc.);
- escopo e nível de abstração(Cockburn, 2000);
- ator principal;
- outros atores;
- outras partes interessadas;
- pré-condições;
- gatilhos;
- pós-condições, em caso de sucesso;
- garantias, em caso de falha;
- cenário principal;
- cenários alternativos;
- requisitos especiais;
- variações tecnológicas ou de dados;
- justificativa;
- fonte ou origem;
- casos de uso relacionados;
- questões em aberto.

Já conhecemos a importância dos atores. A seguir discutiremos as outras partes do documento.

As **partes interessadas** a serem tratadas em um caso de uso são aquelas que não fazem parte do caso, mas podem ter interesse em seu resultado. Por exemplo, em um laboratório de exames clínicos, se um ator “técnico em análises clínicas” usa o caso de uso “registrar resultado

de exame”, o ator “paciente” é uma parte interessada, mesmo que só receba o resultado após o ator “médico” liberar o resultado.

As **pré-condições** de um caso de uso definem o que é sempre verdadeiro quando um caso de uso acontece. São condições que **não precisam ser testadas**, ou seja, são assumidas como verdadeiras no caso de uso. Um exemplo no caso de uso “registrar resultado de exame” seria “exame já está cadastrado”. Só devem ser comunicadas nessa seção questões dignas de nota, que constituam informação útil para o desenvolvimento e funcionamento do sistema.

Os **gatilhos** são eventos, como ações ou condições, que causam o início do caso de uso. Origem ou fonte registra quem ou o que, como um documento, demandou ou indicou a necessidade do caso de uso.

As **pós-condições** estabelecem o que deve ser verdadeiro após o caso de uso, no caso de sucesso, por exemplo garantir que caso um caixa automático tenha dado o dinheiro ao correntista que esse dinheiro tenha sido debitado da conta do mesmo. As **garantias** são o estado final caso haja um erro, por exemplo não alterando o saldo do correntista caso ele não tenha recebido o dinheiro.

Pré-condições, pós-condições e garantias são uma especificação do estado anterior e posterior do sistema e servem como parte do contrato do caso de uso.

Justificativa indica a motivação, o interesse para aquele caso de uso.

Requisitos especiais estão relacionados a demandas adicionais que são feitas a esse caso de uso específico e que não são gerais do sistema. Por exemplo, tecnologias específicas que devem ser usadas, velocidade, etc.

13.8. Exemplo de Um Caso de Uso

Caso de Uso: 1	Solicitar Boletim Oficial
<i>Escopo:</i>	Sistemas, caixa-branca
<i>Nível de abstração:</i>	Objetivo do usuário
<i>Ator principal:</i>	Aluno
<i>Ator secundário:</i>	Secretaria Acadêmica
<i>Outras partes interessadas:</i>	• Coordenador: Tem que assinar o documento
<i>Pré-condições:</i>	• O aluno está logado no sistema • O aluno tem notas em ao menos um período
<i>Pós-Condições:</i>	O pedido do aluno está incluído na lista de tarefas da Secretaria Acadêmica
<i>Gatilho:</i>	Aluno escolhe solicitar boletim oficial no menu
<i>Cenário Principal:</i>	

1. Sistema pede para o aluno confirmar o pedido
2. Aluno confirma o pedido
3. Sistema verifica se o aluno não tem nenhum pedido na fila
4. Sistema coloca o pedido do aluno na fila de tarefas da Secretaria Acadêmica de seu curso
5. Sistema avisa a Secretaria Acadêmica que há uma nova tarefa na fila
6. Sistema informa ao aluno que o pedido foi completado
7. Aluno aceita a mensagem

Alternativas:

3.a Aluno já tem um pedido na fila:

1. Sistema avisa Secretaria Acadêmica que um pedido foi feito
 2. Sistema avisa aluno que seu pedido já estava na fila e que um aviso foi dado a Secretaria Acadêmica
 3. Aluno aceita a mensagem
-

Requisitos Especiais:

O sistema deve responder ao aluno em até 5s

Variações de tecnologia:

Essa função deve funcionar na versão para telefone celular

Assuntos em Aberto:

Será necessário comprovar o pagamento de uma taxa?

13.9. Levantando Casos de Usos

O processo de levantar casos de uso é um processo investigativo e iterativo. Ele pode ser resumido em 6 passos, que devem ser feito iterativamente até alcançar a especificação desejada:

1. identificar os atores;
2. identificar os objetivos de cada ator, nomeando seus casos de uso;
3. escrever o cenário principal para cada objetivo;
4. determinar as condições que geram as alternativas nos cenários existentes;
5. escrever o cenário alternativo para cada condição, e
6. detalhar as variações de dados.

Para identificar os atores é necessário investigar não só os atores que são pessoas físicas, mas também que sistemas, subsistemas, organizações, grupo de pessoas vão interagir com o software. Muitas vezes um caso de uso que começa com um ator precisa se comunicar de alguma forma com outro ator, por exemplo, enviando mensagens. O resultado do primeiro passo é uma lista de atores, que poderá ser estendida nos próximos passos.

Sempre que um ator novo for encontrado, durante a elicitación de um caso de uso, ele deverá ser investigado para que se possa avaliar se ele também vai demandar casos de uso, isto é, se terá objetivos para usar o sistema.

Atores devem ser nomeados com um nome específico do negócio ou da área de aplicação. Assim um sistema acadêmico possui aluno, professor, coordenador, etc. Um consultório ou ambulatório possui enfermeiro, auxiliar de enfermagem, médico, paciente, atendente, acompanhante, etc. Uma loja de roupas possui cliente, estoquista, gerente, vendedor, etc. Usar nomes genéricos,

como usuário, demonstra má qualidade do modelo. O resultado do primeiro passo é uma lista de atores, que responde a pergunta “quem?”.

Para identificar o objetivo é importante entender o que cada ator precisa do sistema, quais suas necessidades e como elas devem ser demandadas ao sistema. O resultado do segundo passo é uma lista de casos de uso, que responde a pergunta “o que?” e um diagrama de casos de uso, que fornece uma visão geral do escopo do sistema. Com isso é possível ter uma lista delimitada e usável das funções do sistema.

Objetivos são escritos no formato verbo-objeto. Alguns exemplos são: solicitar boletim (para o ator aluno usando um sistema acadêmico) e lançar notas (para o ator professor no mesmo sistema).

No terceiro passo a função do analista de sistemas é determinar como o usuário alcançará seu objetivo usando o sistema. Esse passo a passo só deve ter as mensagens trocadas entre os agentes (atores e sistema). Não **pode ser usada nenhuma estrutura de controle**, como laços e decisões. O cenário deve capturar a intenção e responsabilidade de cada ator até alcançar o objetivo. Cada passo do cenário deve deixar bem clara a informação que é passada entre os atores. O resultado é uma descrição legível das funções do sistema na forma que seriam executadas se tudo ocorresse normalmente, sem nenhuma condição especial ser ativada.

Um cenário principal para um Sistema Acadêmico, que descreve o caso de uso “emitir boletim”, para atender o ator “aluno”, poderia ser descrito, nessa fase, como na Figura 13.9:

1. aluno informa DRE
2. sistema valida DRE
3. sistema acadêmico solicita senha
4. aluno informa senha
5. sistema acadêmico valida senha
6. sistema acadêmico apresenta menu
7. aluno escolhe emitir boletim
8. sistema acadêmico emite boletim
9. aluno sai do sistema

Figura 13.9.: Exemplo de cenário principal do caso de uso **emitir boletim**.

O quarto passo usa como referência o cenário principal, e outros cenários alternativos do caso de uso em uma interação mais avançada, determinando as alternativas a partir dos passos já descritos. Cada passo pode ter alternativas que levem ao fracasso, ao sucesso parcial ou ao sucesso total. Cada condição deve ser anotada, gerando uma lista de cenários alternativos que indica o cenário, o passo e a condição que pode ocorrer nesse passo.

No exemplo anterior, a lista de condições poderia ser:

- DRE não é validado
- senha não é validada
- se passam 2 minutos sem o aluno fazer uma ação

No quinta passo são resolvidas todas as alternativas. Isso é feito escrevendo o cenário, seguindo sempre as mesmas regras do cenário principal, e também indicando, ao final do cenário, se foi sucesso, parcial ou total, ou falha, e se é necessário voltar ao fluxo original.

A técnica do caso de uso descreve as alternativas apenas no que elas são diferentes do cenário principal, isto é, elas são normalmente menos passos do que o cenário principal e se referem a ele. O resultado dessa fase é uma versão completa dos casos de uso do sistema.

Finalmente, no último passo, são tratadas as questões que são de baixo nível para a análise, mas precisam ser resolvidas para a implementação. Por exemplo, como será emitido o boletim, que dados serão usados, etc.

Um bom caso de uso corresponde a um processo elementar da organização. Ele descreve a interação de um ator principal durante uma sessão única, onde um objetivo é atingido. Essa sessão **não** leva dias ou se divide em várias utilizações do sistema. Ele deve ser uma conversação, e não um passo único como “apagar um item”. Deve ser uma tarefa, concluída em uma sessão e que produz um resultado mensurável e partindo de uma situação onde as informações estão em um estado consistente e terminando em uma situação onde as informações também estão em um estado consistente, possivelmente diferente do estado inicial.

Casos de uso devem ser pensados como os motivos atômicos, isto é, mais simples, que o usuário tem para usar o sistema. Assim, em um sistema acadêmico, um ator “aluno” pode ter como objetivo de entrar no sistema “emitir o boletim” ou “fazer matrícula em cadeira”. É claro que um aluno específico pode entrar no sistema para fazer essas duas coisas, porém no processo de análise de sistemas elas são tratadas em separado, porque é possível entrar no sistema para fazer essas atividades, mas dentro delas não há nada menor para ser feito, logo elas são atômicas.

O mesmo exemplo pode ser dado com um correntista de banco. Ele pode usar o caixa automática para sacar dinheiro, ou para emitir cheques, logo esses são casos de uso. Ele também pode fazer as duas coisas em uma sessão, mas como essa atividade seria uma composição de outras duas que podem ser feitas isoladamente, isto é, podem ser um objetivo único de um correntista, então as duas atividades isoladas são casos de uso, mas a atividade composta não. Além disso, não faz sentido um usuário entrar no sistema só para fazer um passo menor do que isso, como “entrar senha”.

A lista a seguir mostra algumas perguntas que podem ajudar a eliciar os casos de uso.

- Quais são as tarefas de um ator no negócio?
- O ator precisa ser informado de que ocorrências que ocorrem no sistema?
- O ator precisa informar o sistema de mudanças no ambiente externo?

Cockburn (2000) propõe uma receita mais detalhada, com 12 passos:

1. encontre as fronteiras do sistema, usando um diagrama de contexto ou uma lista de entradas e saídas;
2. faça um *brainstorm* e list os atores primários, gerando a **lista de atores**;
3. faça um *brainstorm* e gere os objetivos principais dos atores perante o sistema, gerando uma lista de atores e objetivos;
4. escreva os casos de uso no nível de sumário de todos os casos encontrados;
5. reconsidere e revise os casos de uso estratégicos, adicione, retire, junte e separe objetivos;
6. escolha um caso de uso para expandir ou escreva narrativa para ficar acostumado com o material;
7. preencha as partes interessadas, interesses, pré-condições e garantias, e verifique esse trabalho;
8. escreva o cenário principal de sucesso, verifique frente os interesses e garantias;
9. faça um *brainstorm* e liste as principais condições de sucesso e falha alternativas;
10. escreva como os atores e os sistemas devem se comportar em cada alternativa;
11. extraia qualquer sub caso de uso que se mostre necessário, e
12. inicie do início e reajuste os casos de uso, adicione, retira, junte ou separe casos de uso, verifique para completude, legibilidade e condições de falha.

13.10. Diagramas de Caso de Uso

Um diagrama de caso de uso representa de forma muito abstrata todos os casos de uso de um sistema. Seus principais componentes são atores e casos de uso.

Um diagrama de caso de uso pode ser construído com apenas 3 figuras básicas: atores, casos de uso e associações, apresentadas na Figura 13.10,

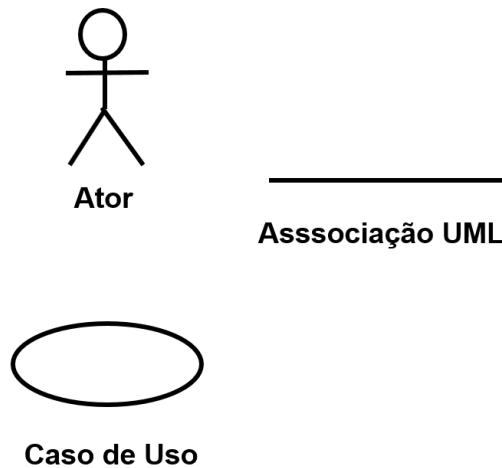


Figura 13.10.: Os elementos básicos de um diagrama de caso de uso são: atores, casos de uso e associações. Fonte: do autor

Outros símbolos, característicos dos casos de uso, também existem, representando relações entre atores e casos de uso. Esses símbolos são: herança, inclusão e extensão.

Finalmente, símbolos genéricos de UML como sistema/partição, comentário, etc., também podem ser usados.

Atores e casos de uso podem se relacionar de várias maneiras, como é mostrado na Tabela 13.6.

Tabela 13.6.: Associações que podem ocorrer entre atores e casos de uso.

	ator	caso de uso
ator	generalização	associação
caso de uso	associação	«include»
		«extend»
		generalização

13.10.1. Especificando o *Subject*

De acordo com o padrão UML(OMG, 2017), é possível indicar a que sistema caso de uso se refere, o que o padrão chama de *subject*, por meio de uma caixa, como feito na Figura 13.11.

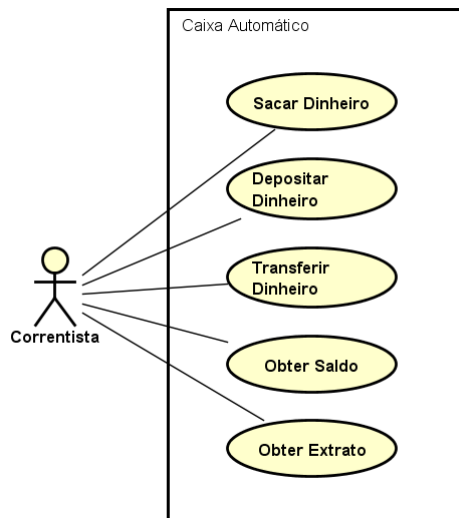


Figura 13.11.: Indicando o sistema ao qual os casos de uso se referem. Fonte:do autor.

13.10.2. Generalização entre Atores

Muitas vezes diferentes atores tem características comuns. Se estas características comuns puderem ser descritas como uma relação de especialização/generalização, podemos representar isso diretamente em um diagrama de caso de uso por meio da relação de generalização (usando uma seta com ponta triangular e linha cheia).

Múltiplos atores podem ter papéis comuns ao interagir com um caso de uso. A relação de generalização pode ser usada para simplificar relações entre muitos atores e um caso de uso. Ela também mostra que uma instância de um ator especializado por fazer tudo que outro tipo de ator faz.

Um exemplo típico é o da existência, em uma organização, de funcionários e gerentes. Normalmente, tudo que um funcionário pode fazer um gerente também pode fazer, mas a recíproca não é verdadeira. Assim, é possível criar vários casos de uso para o ator “funcionário” e fazer o ator “gerente” herdar de “funcionário”, adicionando-se então os casos de uso exclusivos de gerente. A Figura 13.12 mostra uma visão parcial de um sistema dedicado ao controle de funcionários, onde um ator gerente é uma especialização do ator funcionário. Dessa forma o gerente também pode fazer solicitar férias ou solicitar dispensa médica, mesmo que essa ligação não apareça diretamente no diagrama.

Também é possível usar a relação de generalização para criar atores abstratos, que representam um comportamento comum entre vários atores concretos. Um ator abstrato não existe na prática no mundo real, como o que é demonstrado no exemplo a seguir. No exemplo da Figura 13.13, 3 atores diferentes são considerados especialização de “*Profissional de Saúde*”, que é um ator abstrato, com o nome em itálico, que existe conceitualmente mesmo no mundo real, porém não existe nenhum exemplo de um profissional de saúde que não seja enfermeira, médico ou auxiliar de enfermagem.

13.10.3. Relações Entre Casos de Uso

Casos de uso podem se relacionar de 3 formas:

- pela inclusão de outro caso de uso;

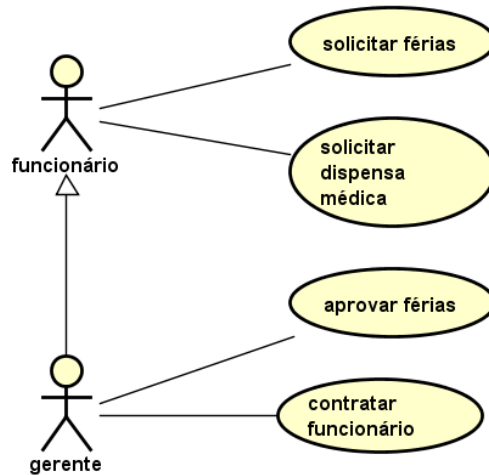


Figura 13.12.: Nesse diagrama de casos de uso, o gerente é uma especialização de funcionário, logo pode fazer tudo que o funcionário faz, mais alguns casos de uso que são exclusivos dele. Fonte: do autor

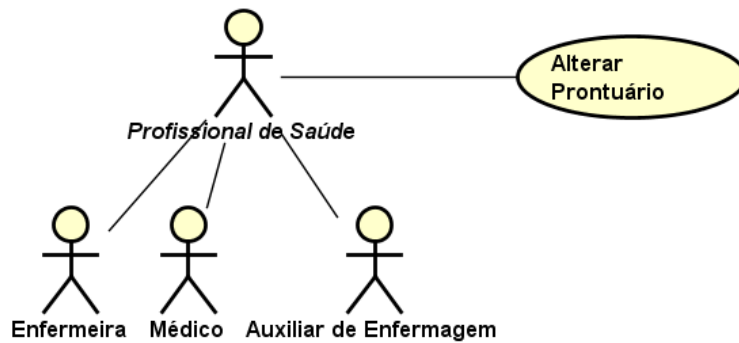


Figura 13.13.: Um *Profissional de Saúde* é a generalização de Enfermeira, Médico ou Auxiliar de Enfermagem, mas nenhum ator é um profissional de saúde se não for de um dos outros tipos. Fonte: do Autor.

- pela extensão de outro caso de uso, e
- pela generalização/especialização de outro caso de uso.

Em UML, esses relacionamentos são conhecidos como *include*, *extend*, e a generalização propriamente dita, sendo representados como na Figura 13.14.

Inclusão de Casos de Uso

O relacionamento de inclusão é o mais simples de se compreender, pois representa uma relação entre um caso de uso básico e um caso de uso incluído. Isso significa que caso de uso incluído é explicitamente inserido no caso de uso base, de forma semelhante a uma chamada de função. A Figura 13.15 apresenta um exemplo dessa associação.

O relacionamento de inclusão é usado para fatorar um comportamento comum entre dois ou mais casos de uso. Ele evita que tenhamos que descrever o mesmo comportamento duas vezes dentro dos respectivos casos de uso, aumentando a consistência e permitindo o reuso.

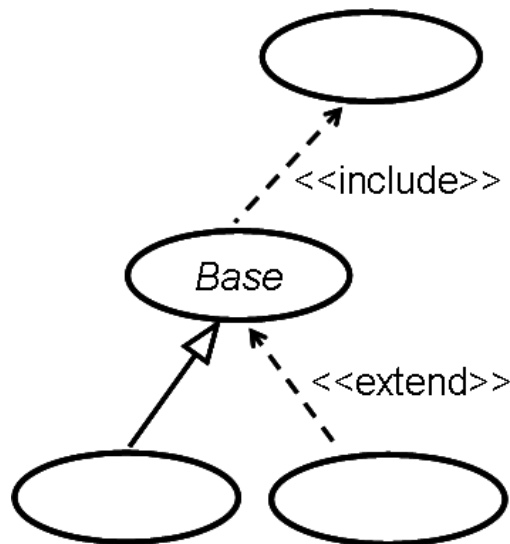


Figura 13.14.: Representações das associações entre casos de uso.

Também é possível usar o relacionamento de inclusão apenas para fatorar e encapsular comportamento de um caso de uso base, de forma a simplificar fluxo complexo de eventos ou remover da parte principal do caso de uso um comportamento que não é parte do objetivo primário. Nesse caso deve se considerar a complexidade do caso de uso descrito de forma completa em comparação com a complexidade de ter mais de um caso de uso sendo descrito para representar só um objetivo do usuário.

É importante entender que um caso de uso incluído é executado totalmente quando é chamado e, se não deve ser executado, a decisão é do caso de uso chamador. Alguns autores dizem que o caso de uso incluído é obrigatório, mas isso só é verdade dentro do fluxo onde ele ocorre, ou seja, um caso de uso incluído pode ocorrer em um cenário alternativo e, no caso, não ser aleatório¹.

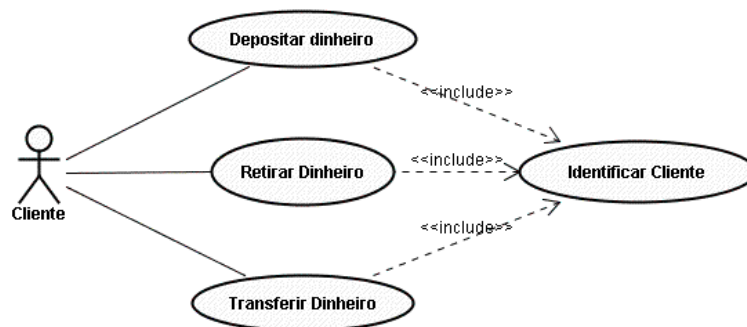


Figura 13.15.: Exemplo da associação de inclusão entre casos de uso

Extensão de Casos de Uso

A relação de extensão descreve que um caso de uso estende o comportamento de seu caso base, o que acontece apenas se uma condição de extensão for verdade. A relação de extensão

¹Mesmo assim, alguns lugares usam como regra que casos de uso incluídos estão sempre no cenário principal e casos de uso que estendem estão sempre em um cenário alternativo. Isso é apenas uma decisão local que pode tornar o trabalho mais fácil e consistente entre várias pessoas do grupo de análise de sistema, mas não tem base no padrão UML

é originária do uso de *patches* em sistemas que não podiam ter seus casos de uso originais alterados para aceitar novos comportamentos e pode ser compreendida como uma forma de *patch*, do mesmo jeito que a relação de inclusão pode ser compreendida como uma chamada de função. A Figura 13.16 apresenta um exemplo de relação de extensão em um caso de uso.

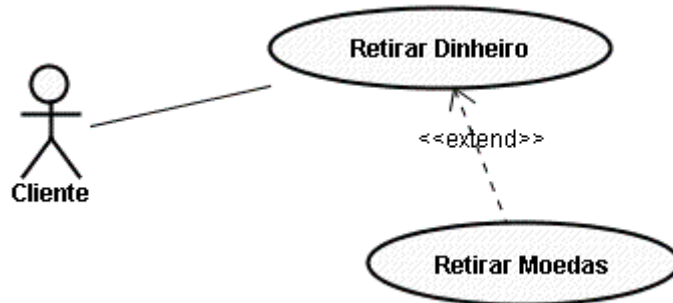


Figura 13.16.: Um exemplo da associação de extensão entre casos de uso.

Uma diferença principal entre inclusão e extensão é que a extensão sempre é chamada pelo caso de uso base, e só na extensão é que é decidido se ela deve fazer algo ou simplesmente retornar para o caso de uso base.

Extensões são tipicamente chamadas para fazer funções adicionais em cenários alternativos ou para descrever uma correção, um *patch*, no cenário principal. Por exemplo, uma possível extensão a um cenário do caso de uso “pagar com cartão de crédito”, em um terminal de ponto de venda², seria “pagar a prazo com cartão de crédito”.

Generalização de Casos de Uso

O relacionamento de generalização, como estamos habituados, descreve um comportamento geral compartilhado pelo caso de uso que herda (filho) com o seu parente. Ela descreve que o “filho” tem o mesmo comportamento geral do pai, porém com alguma diferenciação (especialização).

Esse relacionamento deve ser utilizado para mostrar comportamento, estrutura ou objetivos comuns entre diferentes casos de uso; para mostrar que os casos de uso “filhos” formam uma família de casos de uso com alguma similaridade; ou para assegurar que o comportamento comum se mantém consistente. A figura a seguir mostra um exemplo de herança.

Uma instância de caso de uso executando um caso especializado vai seguir o fluxo de eventos descritos pelo caso parente, inserindo comportamento adicional e modificando seu comportamento como definido no fluxo de eventos do caso especializado.

13.10.4. Usando Cardinalidades em Casos de Uso

O uso mais comum de casos de uso, tendo em vista sua utilidade, é sem cardinalidades nas associações, porém é correto usá-las segundo UML.

Com o uso de indicadores de cardinalidade, o que é parte de UML, algumas informações adicionais podem ser dadas. Quando essa cardinalidade é maior que um, a especificação UML

²A famosa máquina de cartão de crédito

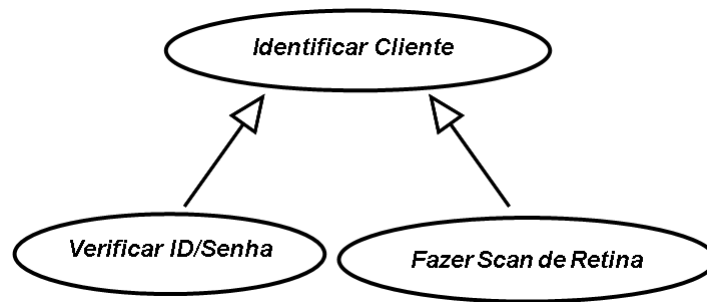


Figura 13.17.: Exemplo da associação de herança entre casos de uso

não define o que isso significa(OMG, 2017). Podem ser que os atores, múltiplos, tenham que agir em paralelo, ou em sequência, ou em outra forma.

No diagrama da Figura 13.18, existe um caso de uso apenas: aprovar empréstimo. Toda instância desse caso de uso precisa de um gerente para ser executado, porém em alguns casos especiais precisa também de um gerente regional. Gerentes e gerentes regionais, isto é, suas instâncias específicas, não precisam participar desse caso de uso. As cardinalidades marcadas indicam isso.

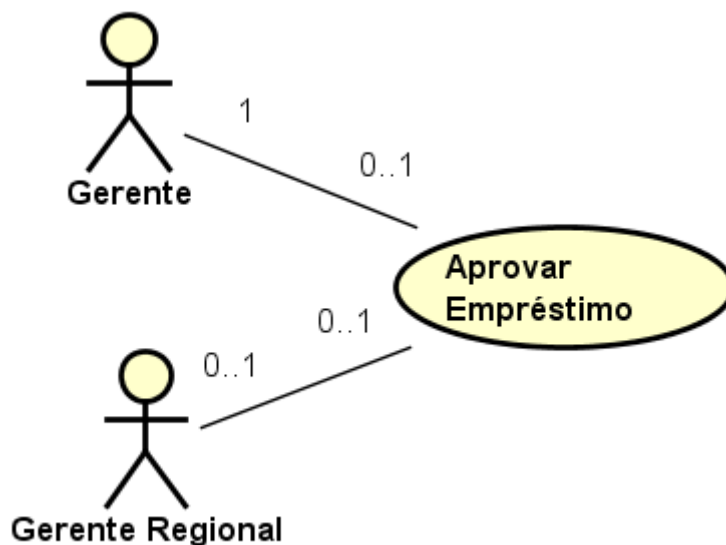


Figura 13.18.: Exemplo do uso de cardinalidades em casos de uso. Fonte: do autor

Também é possível que um ator execute várias instâncias de um caso de uso, ou que um caso de uso exija mais de uma instância de um ator.

13.11. O Que o Diagrama de Caso de Uso Não Informa

Um diagrama de caso de uso não dá algumas informações.

Os atores principais não são informados em um diagrama de caso de uso. Mais que isso, não há nenhuma informação que ajude a entender qual a participação de um ator em um caso de uso que esteja associado a vários atores. Para facilitar o entendimento, muitos diagramas, principalmente em livros, colocam o ator principal à esquerda do caso de uso e os outros atores à direita.

Assim vendo um ou mais atores associado a um caso de uso, não é possível saber:

- se ele é o ator principal;
- se ele envia ou recebe informações para o caso de uso;
- se ele participa de todas as instâncias do caso de uso, o que pode ser resolvido em alguns casos com a indicação de cardinalidade na associação, e
- se ele está no cenário principal ou só em algum cenário alternativo.

13.12. Exercícios

Exercício 13.1:

Escreva um conjunto de casos de uso para um aplicativo de celular que sirva como relógio e despertador, permitindo vários alarmes.

Exercício 13.2:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita identifique todos os atores e defina o nome de todos os casos de uso que pensa necessário. Para cada caso de uso imagine um cenário principal.

Histórias do Usuário

Since users don't know how to solve their problems, we need to stop asking ... and to involve them instead.

(Mike Cohn)

Conteúdo

14.1.	Conceituação de Histórias do Usuário	199
14.2.	Um <i>Template</i> Padrão para Histórias de Usuário	200
14.3.	INVEST	201
14.4.	Estados de Uma História do Usuário	204
14.5.	Exercícios	205

Por que histórias do usuário?

Junto com casos de uso, histórias do usuário são uma das duas principais formas de especificar os requisitos de um software sendo usadas atualmente, principalmente em projetos ágeis.

Histórias do usuário são uma prática ágil de registro das necessidades funcionais das partes interessadas, normalmente escritas em um cartão. O nome tem certa ambiguidade, sendo usado tanto para a prática como um todo quanto para o texto principal escrito no cartão. Em métodos ágeis como *eXtreme Programming (XP)* e *Scrum* elas são a principal forma de representação de um **requisito de software**,

O termo **história do usuário** foi cunhado por Kent Back (Beck, Cockburn et al., 2014), com a intenção de descrever uma prática com maior equilíbrio entre a área de negócio e a área de desenvolvimento, sendo uma maneira mais ágil de especificar requisitos. Um dos objetivos era que, para que os usuários tivessem acesso facilitado ao método, a linguagem deveria ser bem próxima a deles, evitando as formalidades e os jargões de TI dos métodos tradicionais.

Dessa forma, histórias do usuário são uma narrativa na perspectiva do usuário (Beck, Roden et al., 2014) escrita em linguagem natural. Segundo Adzic e Evans (2014), comparando histórias do usuário com requisitos levantados unicamente por desenvolvedores ou pessoas do negócio, “histórias do usuário implicam em um modelo totalmente diferente: requisitos por colaboração”.

Inicialmente histórias do usuário tinham uma definição muito informal, e até controversa (Beck, Cockburn et al., 2014; Beck, Roden et al., 2014). O entendimento e o acordo entre os praticantes dessa metodologia foram evoluindo com o tempo e a divulgação de seu uso, por meio de artigos e livros, e acabaram sendo de certa forma padronizadas no mercado em três conceitos principais que são tratados neste texto: o *template*, a sigla *INVEST* e a regra *Card, Conversation and Confirmation*.

Beck (1999) definiu histórias do usuário, dentro do contexto de *XP* como:

- uma coisa que o cliente quer que o sistema faça;
- que possa ser feita entre uma e cinco semanas, e
- que seja testável.

Nessa definição original, o prazo de uma a cinco semanas tem mais relação com o fato da história poder ser feita junto com outras nesse tempo. Alguns autores falam das histórias isoladamente como podendo ser feitas de meio dia até 2 semanas dias (Cohn, 2004). O prazo exato, porém, não é importante, mas sim o fato que uma história do usuário que é consistente com os prazos dos métodos ágeis, e com a prática de *time-box*.

Uma outra definição, dada por Jacobson, Lawson e Ng (2019) é “Alguma coisa que o sistema de software pode ser estendido para fazer, expressa em termos do valor que vai fornecer para o usuário do sistema.”

É importante compreender, porém, que **não há nenhuma limitação formal de como são escritas**, e o importante é que sejam manipuláveis e compreensíveis pelos usuários. Além disso, apesar da semelhança entre os nomes, **histórias do usuário não são casos de uso**, ou uma versão simplificada de casos de uso (Beck, Cockburn et al., 2014; Cohn, 2004).

Tipicamente, a parte mais visível das histórias do usuário são escritas em cartões como o apresentado na Figura 14.1.¹

Corrigir Nota

Alta prioridade

Como um professor eu desejo alterar a nota de um aluno para poder corrigir um erro de digitação ou um pedido de revisão de prova.

GX 31/1/2020 1 dia

Figura 14.1.: Exemplo de uma história do usuário em um cartão. Fonte: do autor.

¹São usados normalmente fichas pautadas de fichário

Um *time box* é um período de tempo previamente definido durante o qual uma equipe trabalha em direção a um objetivo que cabe dentro dele. Nessa técnica, intervalos de tempo são definidos antes, a funcionalidade a ser entregue deve ser compatível com ele.

Como em vários métodos ágeis, é comum que os cartões fiquem afixados a um quadro, como o quadro *Kanban*, ou ainda no *Sprint Backlog* ou no *Product Backlog* do *Scrum*.

Por que cartões?

Vários métodos de análise de sistemas, como *Volare*, CRC e Histórias do Usuário usam cartões, ou fichas de fichário. Essa prática é comum nos EUA, mas encontra até um pouco de resistência no Brasil. A questão é que nos EUA cartões são usados em várias técnicas de estudo, como o uso de *flashcards*, ou o fichamento (que também é usado no Brasil), então o profissional normalmente já tem certo hábito com eles.

14.1. Conceituação de Histórias do Usuário

Uma história do usuário “descreve uma funcionalidade que terá valor” para uma parte interessada de um sistema ou software (Cohn, 2004). Ela é composta de três aspectos, conhecidos como Cartão, Conversação e Confirmação (Cohn, 2004; Jeffries, 2001):

1. o **cartão** que contém uma descrição textual da história **usada para planejamento e como um lembrete**, ou a história do usuário propriamente dita, como a que está na Figura 14.1;
2. as **conversações** entre desenvolvedores e usuários sobre a história, e que levam a descoberta de seus detalhes, e
3. as **confirmações**, as quais garantem que a história está completa e funcionando como esperado.

Segundo K. E. Wiegers e Beatty (2013, p. 146), conversações servem para refinar as histórias do usuário e também para levar dessas histórias refinadas para testes de aceitação. Essas conversações podem levar ao desmembramento da história original em histórias menores e a detalhes adicionais.

Devemos deixar claro que **os cartões são apenas parte de usar histórias do usuário** como método, servindo como **ponto focal** do trabalho. A principal mudança, na verdade, é ter um ambiente participativo, onde usuários e desenvolvedores tem encontros face a face onde se estabelecem as **conversações**. Possivelmente, essas conversações podem levar a alguns documentos que serão usados pelos desenvolvedores, mas não há nenhum padrão pré-concebido a seguir na documentação.

Isso causa uma pequena confusão entre o método e o artefato histórias do usuário. O método é composto pelos três aspectos, o artefato, ou a descrição da história do usuário, é registrado em um cartão. Deve ficar claro, pelo contexto, qual dos significados é tratado em cada parte do texto.

O cartão propriamente dito pode ter várias informações. A principal é a descrição da história, possivelmente na forma de um dos *templates* apresentados a seguir neste texto, ou um definido na organização.

Outra prática comum é **dar um título a história**. Adzic e Evans (2014) recomenda nomear as histórias bem cedo e pensar se outras partes interessadas podem estar interessadas na história, deixando os detalhes para o final.

A Figura 14.2 mostra um cartão adaptado do modelo usado originalmente na Connextra (Patton e Economy, 2014a), com suas várias partes identificadas.

Card,
Conversa-
tion an
Confirma-
tion

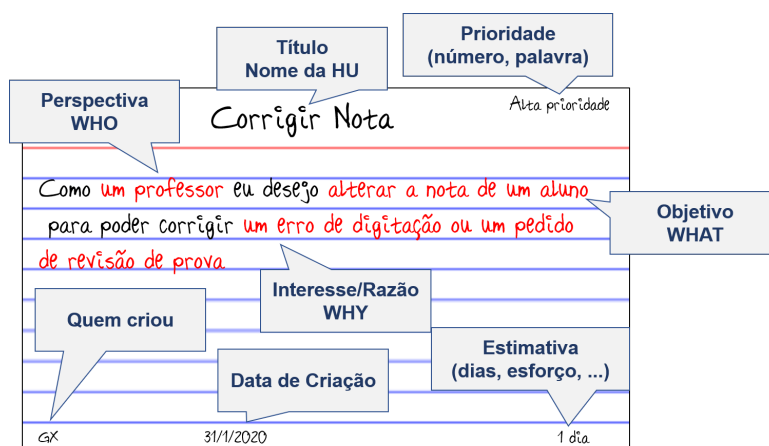


Figura 14.2.: Partes encontradas em um cartão de história do usuário, segundo um modelo original da Connextra (Patton e Economy, 2014a).

14.2. Um *Template* Padrão para Histórias de Usuário

North (2013) sugeriu um formato para histórias de usuário em um grupo do Yahoo, conhecido como o *template* da Connextra (Adzic e Evans, 2014), que acabou se tornando uma referência na área:

Como um < papel de usuário > eu desejo < objetivo > de forma a < razão >.

onde o *objetivo* descreve uma funcionalidade do sistema e a *razão* descreve porque aquela funcionalidade traz valor ao usuário. É **importante que a razão não seja apenas uma nova maneira de dizer o objetivo**, pois nesse caso o valor não está claro.

Exemplo de histórias do usuário que seguem esse padrão são:

- como um sócio do clube eu desejo marcar a quadra de futebol de forma a jogar futebol com amigos, e
- como um gerente eu desejo emitir o relatório de vendas por cliente de forma a descobrir quais são os clientes que mais gastam.

Adzic e Evans (2014) ainda recomenda que as histórias do usuário tenham duas propriedades: descrever uma mudança de comportamento, para as partes interessadas, e descrever uma mudança no sistema, para a equipe de desenvolvimento.

14.2.1. Outros formatos possíveis

O formato padrão responde 3 perguntas sobre o caso de uso: quem, o que e por que. É possível modificar esse formato para responder mais perguntas do 5W2H. Dois formatos possíveis são:

Como um < papel de usuário >, < prazo >, eu desejo < objetivo > de forma a < razão >.

o que responde ainda quando, como em “Como gerente, até o dia 30 de maio, eu quero o relatório de vendas por cliente ordenado por valor, porque quero conhecer meus melhores clientes.” ou ainda

Como um < papel de usuário >, < prazo >, eu desejo < objetivo > de forma a < razão >.

User
Story
Template

o que responde ainda “por quanto?”, como no exemplo “Como gerente, até o dia 30 de maio, eu quero o relatório de vendas por cliente ordenado por valor, porque quero conhecer meus melhores clientes, gastando no máximo 1 semana de desenvolvimento.”

Cohn (2004) propõe também histórias escritas em formatos mais livres, como em “Um usuário pode postar seu currículo no *website*”. Já Adzic e Evans (2014) lembra que o importante é que as histórias do usuário estimulem uma boa discussão sobre o requisito, e sugere experimentar com diferentes formatos, como perguntas, imagens.

Exemplos de histórias do usuário escritas de forma mais livre são fornecidos por Cohn (2004) para um sistema de busca de empregos:

- um candidato pode incluir seu currículo;
- um candidato pode editar seu currículo;
- um candidato pode apagar seu currículo;
- um candidato pode marcar o currículo como inativo, e
- um resumo pode incluir educação, empregos anteriores, publicações, salários anteriores e um objetivo.

Chamamos a atenção para a diferença entre essas histórias do usuário e como requisitos do sistema segundo a norma IEEE. Os requisitos demandam funcionalidades do sistema, no formato “o sistema deve”, enquanto as histórias do usuários dizem o que o usuário pode fazer, no formato “um usuário pode”.

14.3. INVEST

Ao desenvolver as histórias é necessário levar em consideração que uma boa história deve seguir alguns princípios que são definidos pelo acrônimo INVEST (Cohn, 2004; Wake, 2003):

- **I**ndependente;
- **N**egociável;
- com **V**alor;
- **E**stimável;
- Pequena (*Small*), e
- **T**estável.

14.3.1. Independente

Cada história do usuário deve ser independente, sempre que possível, de outras. O benefício da independência é evitar problemas de estimativa, priorização e planejamento (Cohn, 2004). Elas devem poder ser implementadas em qualquer ordem (Wake, 2003). Isso não é sempre possível, já que algumas funcionalidades, como a produção de relatórios, precisam ser feitas uma de cada vez, e a primeira vez vai afetar, normalmente facilitando, as outras.

Também no caso de estimativas de história com alta dependência fica difícil acertar, pois a implementação de uma vai afetar a implementação da outra, mas pode não ser claro qual o impacto final. Isso pode implicar em juntar algumas histórias de modo a deixar as que as co-dependências (Cohn, 2004) estejam todas na mesma história.

Por exemplo, suponha que em um caixa automático para pagamento de estacionamento existam várias histórias, com os seguintes nomes:

- o motorista deseja **pagar com cartão de crédito** para poder sair do estacionamento com seu carro;
- o motorista deseja **pagar com cartão de débito** para poder sair do estacionamento com seu carro;
- o motorista deseja **pagar com notas** para poder sair do estacionamento com seu carro, e
- o motorista deseja **pagar com aplicativo** para poder sair do estacionamento com seu carro;

Claramente, a implementação de uma dessas histórias pode afetar as outras, tornando-as mais fáceis. A solução de juntar as histórias criaria uma só no lugar delas, usando como ação “pagar estacionamento”. Por outro lado, essa história seria grande, talvez até mesmo um épico, o que pode levar a necessitar de um outro tratamento, mais tarde no projeto.

Outra opção seria manter as histórias em separado, mas criar duas estimativas, uma para a história sendo feita isoladamente, outra para a história sendo feita depois de uma outra estar feita (Cohn, 2004).

Wake (2012) descreve 3 formas de dependência:

1. sobreposição;
2. ordem, e
3. contenção.

Dependência de Sobreposição

A **sobreposição** acontece quando existem um conjunto de funcionalidades e as histórias do usuário cobrem algumas dessas funcionalidades, com interseção entre elas. Um sinal de que isto está acontecendo são histórias com a conjunção “e”, como em “Cliente compra e vende itens” e “Cliente compra e devolve itens”. Isso deveria ser dividido em histórias diferentes como nomes como (Wake, 2012):

- cliente compra itens;
- cliente vende itens, e
- cliente devolve itens

Dependência de Ordem

Nesse caso há uma história que precisa ser implementada antes da outra. Os motivos podem ser muitos, inclusive ser da natureza do problema, já que um item só pode ser devolvido se for comprado. Esse tipo de dependência não pode ser eliminado, mas normalmente a prioridade do negócio vai estar na mesma ordem. ordem (Wake, 2012).

Dependência de Contenção

Nesse caso existe uma organização hierárquica das histórias, onde uma contém outras. Todos os métodos tem que tratar isso de alguma forma, já que a abstração de refinamento sucessivo é muito prática e utilizada praticamente por todos para entender melhor como um sistema se organiza.

Histórias de usuário de nível muito alto de abstração recebem nomes como temas ou épicos (Wake, 2012) e existem formas específicas de tratá-las.

A característica principal da dependência de contenção é que a decomposição hierárquica não é uma estratégia de sequenciamento (Wake, 2012). Então é importante atacar as partes principais, seguindo normalmente uma estratégia de desenvolver o **menor produto viável** e construir versões melhores, agregando o maior valor possível, em sequência.

14.3.2. Negociável

Uma história do usuário é na verdade um tópico de conversação entre usuário e desenvolvedores, logo elas devem ser escritas de forma a serem negociáveis. A negociação promove o entendimento e compromisso entre as partes (Jacobson, Lawson e Ng, 2019).

14.3.3. Com Valor

Cada história precisa ter um valor para o cliente, ou outra parte interessada que não seja da equipe de desenvolvimento (Wake, 2003), como o comprador. Histórias são sempre escritas com a perspectiva do usuário.

Esse conceito está relacionado com a prática geral dos métodos ágeis de desenvolver os sistemas por fatias verticais, isso é, sempre entregando funcionalidade ao cliente, em vez de camadas onde funcionalidades internas, de base, são feitas antes, e mais tarde as camadas externas, que atendem os clientes, são feitas.

A questão do interesse de outras partes interessadas é importante, porque usuários não estão interessados com certos requisitos do sistema. Cohn (2004) cita, por exemplo, a necessidade de seguir alguma norma internacional, que pode ser do interesse do comprador mas não dos usuários. Um história válida, então, seria “A equipe de desenvolvimento irá fornecer documentação que atende as normas de auditoria da ISO 9001”(Cohn, 2004).

14.3.4. Estimável

Para poder ser útil no processo de planejamento, é importante que as histórias do usuário sejam estimáveis Wake (2003) em relação ao esforço necessário para implementá-las.

Estimativas são sempre feitas pelos desenvolvedores. É preciso reafirmar a prática que as pessoas do negócio decidem o que deve ser feito, mas são os desenvolvedores que decidem como e o prazo.

Estimar histórias e dar uma avaliação do esforço necessário para desenvolvê-las, em alguma medida combinada no projeto. Dois métodos usados no mercado são usar pesos relativos, como a sequência (1,2,3,5,8,13,20,40,100) usada no *Planning Poker* ou tamanhos de camisa, como (XP,P,M,G,XG).

Os principais motivos para histórias não poderem ser estimadas são (Cohn, 2005):

- os desenvolvedores não entenderam o domínio, isto é, não entenderam como a história do usuário deve acontecer no seu contexto, e devem discutir mais a história com os usuários;
- os desenvolvedores não sabem como implementar, por exemplo, por falta de domínio da tecnologia, e
- a história é muito grande e é muito difícil estimar o esforço necessários, sendo um épico.

O menor produto viável (MVP) é a versão mais simples de um produto que pode ser lançada com uma quantidade mínima de esforço de desenvolvimento

O *Planning Poker* é uma técnica de estimativa baseada na técnica do Delphi que usa reuniões face

É importante notar que normalmente as histórias do usuário são usadas junto com *time-boxes*, o que implica em elas terem um tamanho máximo, correspondente ao que pode ser feito dentro desse intervalo de tempo.

14.3.5. Pequena

O tamanho da história do usuário é um fator importante na sua utilidade tanto no projeto quanto no planejamento do mesmo. Histórias muito pequenas são fáceis de implementar, mas podem trazer pouco valor, enquanto histórias muito grandes trazem muito valor, mas também causam dificuldades para estimá-las ou completá-las dentro de um *time-box*. Encontrar um tamanho ideal para uma história é parte do trabalho de colaboração entre desenvolvedores e clientes. Esse tamanho deve ser pequeno de forma que ela possa ser implementada em um ciclo de desenvolvimento (Jacobson, Lawson e Ng, 2019)

Histórias grandes demais podem ser de dois tipos (Cohn, 2004):

- **histórias compostas**, que são épicos com várias histórias menores dentro dele, ou
- **histórias complexas**, que são inerentemente grandes e difícil de ser particionadas em histórias menores.

Um exemplo de história composta é a história “Um correntista pode movimentar sua conta” em um banco, que pode ser facilmente convertida em um conjunto de histórias como:

- um correntista pode depositar valor em conta;
- um correntista pode sacar dinheiro da conta;
- um correntista pode transferir valor de sua conta para outra, e
- um correntista pode pagar boleto.

Histórias grandes podem, e normalmente devem, ser quebradas. Cada história menor devem manter uma interação significativa com o usuário, mesmo que pequena ou especializada. Uma dica para escolher um pedaço de uma história do usuário durante o processo de divisão é olhar os testes e verificar seus objetivos (Jacobson, Lawson e Ng, 2019).

14.3.6. Testável

Esse critério é um dos mais importantes. Normalmente é usado com técnicas como *Test Driven Development*, onde o teste é implementado antes. Também ajuda a que desenvolvedor e usuário concordem com o que vai ser feito (Jacobson, Lawson e Ng, 2019).

14.4. Estados de Uma História do Usuário

De acordo com o SEMAT, uma história do usuário pode passar por 4 estágios (Jacobson, Lawson e Ng, 2019):

1. identificada, onde tem seu valor claramente expresso e está no *backlog* do produto;
2. pronta para desenvolvimento, onde os detalhes foram discutidos e está claro o que é necessário para cumprir seus requisitos;
3. em progresso, onde o time está desenvolvendo a história, e
4. verificada, onde um representante dos usuários verificou a história.

Já um cartão de história pode passar por 3 estados (Jacobson, Lawson e Ng, 2019):

1. valor expresso, o que pode ser feito em alguns dos formatos aceitos;
2. critério de aceitação expresso, e
3. conversação capturada, onde foram feitas as discussões que detalham os requisitos da história e elas foram possivelmente registradas no cartão ou em um sistema a parte.

Finalmente, um caso de teste passa também por 3 estados (Jacobson, Lawson e Ng, 2019):

1. critério de aceitação capturado;
2. roteiro de teste pronto, e
3. roteiro de teste automatizado.

14.5. Exercícios

Exercício 14.1:

Escreva um conjunto de histórias do usuário para um aplicativo de celular que sirva como relógio e despertador, permitindo vários alarmes.

Exercício 14.2:

Vá para o site <http://jogodeanalisedesistemas.xexeo.net/> e visite a Livraria Resolve. A partir da sua visita especifique no todas as histórias do usuário que encontrou.

Parte III.

Gerenciando de Software

O Que é um Projeto

Let our advance worrying become
advance thinking and planning.

(Winston Churchill)

Conteúdo

15.1.	Recursos e Entregas	208
15.2.	Partes Interessadas	208
15.3.	Sucesso, Fracasso e Riscos	208
15.4.	Pontos Críticos ou Pontos Chave de Sucesso	210
15.5.	Conclusão	210

Este capítulo faz uma introdução ao que é um projeto.

Segundo o PMI (2017), um **projeto** é “um esforço temporário empreendido para criar um produto, serviço ou resultado exclusivo. Os projetos e as operações diferem, principalmente, no fato que os projetos são temporários e exclusivos, enquanto as operações são contínuas e repetitivas.”

Exemplos de projeto, de complexidades diversas, são:

- construir uma casa;
- elaborar um plano de marketing;
- consertar um equipamento;
- fazer uma inspeção de segurança;
- preparar um curso de treinamento, e
- desenvolver um software.

Os projetos tem como características(Kerzner, 2017, pg.2):

- ter um **objetivo específico**, cujo escopo pode ser elaborado tanto no início quanto, mais modernamente, ao longo do seu ciclo de vida(PMI, 2017, pg. 13);

- possuir um **prazo**, no qual deve ser completado dentro de certas especificações;
- ter **início e fim** definidos;
- ter uma **limitação de recursos**, e verba, quando aplicável,
- utilizar recursos, humanos ou não, e
- ser multifuncionais.

15.1. Recursos e Entregas

Projetos entregam alguma coisa aos seus clientes. Existem entregas físicas, como mesas, estradas, protótipos e equipamentos, e entregas abstratas, como software, relatórios e estudos, ou mesmo o resultado de um treinamento, como o aprendizado de um tema como “programação em C” por algumas pessoas. Um projeto sempre tem pelo menos uma entrega, que é o seu objetivo final, mas normalmente existem também entregas intermediárias, principalmente quando há um contrato, e a parte contratada precisa fornecer essas **entregas intermediárias** para comprovar que está fazendo sua parte e receber um pagamento, previamente acordado, da parte contratante.

Para poder fazer suas entregas, um projeto precisa utilizar vários tipos de recursos. Os tipos mais comuns são (Kerzner, 2017, pgs. 11, 12):

- dinheiro;
- pessoas;
- equipamentos e ferramentas;
- locais;
- infraestrutura;
- materiais consumíveis, e
- informação e tecnologia.

Outro item que poderia ser chamado de recurso é o tempo. Um projeto ocorre em um espaço de tempo, partindo de uma data de início e possuindo uma data de fim estimado. Atrasos no fim do projeto afetam negativamente o entendimento do seu sucesso, porém acabar um projeto antes do previsto nem sempre é considerado um sucesso adicional. Pela necessidade de gastar dinheiro continuamente durante um projeto, para garantir sua execução, um atraso normalmente aumenta o custo total do projeto.

15.2. Partes Interessadas

Todo projeto possui **partes interessadas**, que são indivíduos ou organizações que afetam ou são afetadas pelo projeto (PMI, 2017), seja favorável ou desfavoravelmente (PMI, 2017). Esses efeitos podem surgir ao longo, como o barulho causado em uma obra na rua, ou só ao fim do projeto, como a poluição que pode ser causada quando uma usina térmica de energia começa a funcionar. Vantagens e desvantagens geradas por um projeto podem causar conflitos de interesse entre as partes interessadas. Partes interessadas são tratadas detalhadamente no Capítulo 3.

15.3. Sucesso, Fracasso e Riscos

O sucesso de um projeto acontece quando ele se completa (Kerzner, 2017, pg. 6):

- dentro do prazo;
- dentro do orçamento;
- com o desempenho apropriado ou no nível especificado;
- sendo aceito pelo clientes e usuários;
- satisfazendo as partes interessadas (PMI, 2017, pg. 552)
- **com mudanças mínimas de escopo acordadas entre as partes interessadas;**
- sem causar distúrbio ao fluxo de trabalho da organização, e
- quando desejado, sem mudar a cultura da organização¹.

As mudanças de escopo são normais em projetos Kerzner (2017, pg. 6), provocadas pelo acontecimento dos riscos ou por mudanças não previstas no ambiente ou nas demandas das partes interessadas. Quanto mais longo o projeto, maior o risco de mudanças importantes. Mudanças, porém, podem gerar riscos adicionais, como afetar a moral da equipe. Em projetos que seguem a linha prescritiva de gerência, ou são mantidos sobre contratos fortes, as mudanças são mantidas em um mínimo necessário e exigem a concordância não só do cliente como da equipe de desenvolvimento Kerzner (2017).

15.3.1. A Incerteza dos Projetos e os Riscos

Todo projeto está associado a um nível de incerteza de seus resultados e entregas (Satpathy et al., 2016), não sendo possível garantir com absoluta certeza que um projeto terminará dentro de todos os parâmetros combinados. As técnicas de gerência de projeto tem como objetivo diminuir essas incertezas, principalmente com o uso de técnicas de análise de risco e do uso de estimativas e margens de erro.

Uma questão importante de projetos são seus riscos, ou seja, os desafios que podem ocorrer ao longo do mesmo e que temos de evitar ou mitigar para completar o projeto com sucesso. Riscos acontecem com uma probabilidade, gerando um custo adicional, se acontecerem, e impactando o projeto de alguma forma, até mesmo com seu cancelamento. Um risco típico de projetos de software é perder pessoal

Para garantir o sucesso do projeto é preciso gerenciá-lo ativamente. Existem várias formas de gerenciar projetos, e elas incluem atividades para iniciar, planejar, executar, monitorar e controlar e fechar projetos. O PMBOK (PMI, 2017) é uma fonte de referência importante que consolida várias técnicas tradicionais de gerência de projetos, como documentos necessários, diagramas de rede, uso da estrutura analítica de projeto e outras. Ele possui uma parte adicional para métodos ágeis e outra para projetos de software.

15.3.2. Benefícios da Gerência de Projeto

Os benefícios potenciais de gerenciar um projeto são Kerzner (2017, pg. 3):

- identificação clara das responsabilidades funcionais para garantir que todas as atividades estão com responsáveis, mesmo com a troca de profissionais;
- minimizar a necessidade de relatos contínuos;
- identificar os limites de tempo para todos os agendamentos;
- identificar a metodologia para análise de custo-benefício;
- medir o que foi alcançado em relação ao plano;
- identificação antecipada de problemas para permitir ações corretivas;

¹alguns projetos tem o objetivo de mudar a cultura

- melhor capacidade de estimação e planejamento, e
- conhecimento dos objetivos que podem não ser alcançado ou vão ser excedidos.

15.4. Pontos Críticos ou Pontos Chave de Sucesso

Os pontos críticos (ou chave) de sucesso para um projeto são aquelas questões que, não estando diretamente relacionada ao desenvolvimento propriamente dito, são essenciais para o bom andamento do projeto.

Exemplos: compromisso de certos funcionários, fornecimento de certa informação, chegada de alguma máquina ou software, compromisso de entrega de dados ou equipamentos pelo cliente, etc.

Os pontos críticos devem ser levantados detalhadamente, pois os compromissos do desenvolvedor só poderão ser cumpridos caso sejam resolvidos satisfatoriamente.

15.5. Conclusão

O desenvolvimento de um software é um projeto, e como tal deve ser gerenciado para aumentar as chances que tenha sucesso. Além disso, desenvolver software é um tipo especial de projeto que tem se mostrado, desde o início, muito desafiador, inclusive criando uma nova área de trabalho, a Engenharia de Software.

A análise de sistemas é uma das tarefas existentes em um projeto de software, e, dentro de todas as outras, tem como finalidade definir os requisitos verdadeiros desse projeto.

Gestão de Projetos de Software

Plans are worthless. Planning is everything.

(Dwight D. Eisenhower)

Conteúdo

16.1.	Técnicas Tradicionais de Gestão de Projeto	212
16.2.	Uso das Técnicas Tradicionais	219
16.3.	Técnicas Ágeis de Gestão de Projeto	219

Por que Gestão de Projetos de Software?

To be done

Projetos podem ser divididos em quatro grandes momentos: iniciação, planejamento, execução e fechamento. Eles podem acontecer em sequência e em alguma forma cíclica. Esses momentos também podem ser únicos ou se repetir em sequência, de acordo com fases, que podem também ocorrer em paralelo.

No desenvolvimento de software essas fases e forma de sequenciamento ou paralelismo são definidas de acordo com um Processo de Desenvolvimento de Software.

O PMI (2017) define um processo de gerenciamento de projeto com cinco grupo de atividades: Iniciação, Planejamento, Execução, Encerramento, e Monitoramento e Controle. Para cada grupo define um conjunto de processos. Alguns desses processos são tratados ao longo deste livro. Por exemplo, no grupo de Iniciação se encontra o processo de “Identificar as Partes Interessadas”, tratadas no Capítulo 3. Já no grupo de Planejamento se encontram processos como “Coletar os Requisitos” e “Definir o escopo”, que também são tratadas em outros capítulos.

Nesse capítulo são discutidas algumas ferramentas que servem para o planejamento e depois continuam a ser usadas para a monitoramento e controle. A ideia aqui é tratar das questões ligadas a entrega de resultados do projeto no tempo esperado.

16.1. Técnicas Tradicionais de Gestão de Projeto

Entre as técnicas mais tradicionais para o planejamento existem três que são destinadas, simultaneamente ao planejamento e ao acompanhamento do projeto: PERT/CPM, Controlgrama de Gantt e WBS/EAP.

Essas técnicas apresentam como maior vantagem a facilidade de uso, porém elas são desafiadas pela singularidade de cada projeto de software e pela linearidade com que tratam o desenvolvimento de projetos. Na verdade, elas são muito adequadas para a construção de navios e prédios, mas apresentam problemas quando usadas no desenvolvimento de software.

É possível perguntar por que elas são ainda estudadas e usadas? A verdade é que elas fornecem uma boa visão do que poderia acontecer se “tudo desse certo” e os riscos que são corridos caso algo dê errado.

Hoje em dia quase toda técnica ou ferramenta ágil apresenta uma versão do Controlgrama de Gantt ou do WBS/EAP. Já as redes PERT/CPM são menos usadas, mas ainda são úteis para planejamento de projetos que não contam só com software, ou para planejamento a prazo mais longo. Além disso todas essas técnicas acabam, muitas vezes, sendo exigências contratuais.

16.1.1. Estrutura Analítica de Projeto

Uma **Estrutura Analítica de Projeto (EAP)**, ou *Work Breakdown Structure (WBS)* é uma subdivisão das entregas ou do trabalho do projeto em componentes menores e mais facilmente gerenciáveis (PMI, 2017). O resultado é um visão estruturada do que será entregue.

Ele é estruturada na forma de um árvore exaustiva, hierárquica, do mais geral para o mais específico, e orientada por (PMI, 2017):

- entregas;
- fases de ciclo de vida, ou
- sub-projetos

O objetivo de uma EAP é identificar os elementos terminais da hierarquia, ou seja, os produtos, serviços e resultados a serem entregues pelo projeto (PMI, 2017).

Na sua representação, o nível superior contém apenas um nó que representa todo o projeto.

No nível mais baixo estão os **pacotes de trabalho**, que podem ser usados para agrupar atividades onde o trabalho é planejado, estimado, monitorado e controlado. Nesse caso, trabalho se refere aos produtos ou entregáveis que são o resultado da atividade, não a atividade (PMI, 2017).

Os pacotes de trabalho, como itens terminais, não são subdivididos. Eles devem poder ser estimáveis em termo de recursos, orçamento e duração necessários, ligados por dependência e agendados. Caso um candidato a pacote de trabalho não possa ser estimado, deve ser subdividido até que isso seja possível.

Os níveis intermediários podem ser organizados de várias formas diferentes:

- fases do projeto
- entregas principais
- subcomponentes

A EAP deve incluir 100% do trabalho definido pelo escopo do projeto e capturar todas as entregas — internas, externas ou intermediárias — de forma ao trabalho estar completo, incluídas as tarefas de gerenciamento de projeto. Isso vale para todos os níveis da hierarquia, isto é, a cada nível, o projeto deve estar descrito de forma completa em relação aquele nível. Obviamente, nenhum trabalho fora do escopo do projeto pode estar na EAP.

A EAP é uma técnica tradicional no planejamento do projeto e é necessário por indicar que entregas são desejadas, apesar de não ainda discutir o momento que são desejadas, mas possivelmente apenas sua ordem. Hoje em dia há formas similares a EAP usadas em projetos ágeis, como *Story Maps* (Patton e Economy, 2014b).

A lista numerada da Figura 16.1 representa uma possível EAP para um projeto de software que segue um processo simples em cascata. A mesma EAP está representada de forma gráfica na Figura 16.2.

1. Software de Agenda Telefônica
 - 1.1. Especificação
 - 1.1.1. Especificação da tela de inclusão
 - 1.1.2. Especificação da tela de edição
 - 1.1.3. Especificação da tela de exclusão
 - 1.1.4. Especificação da tela de listagem
 - 1.1.5. Especificação da tela de busca
 - 1.2. Projeto
 - 1.2.1. Projeto da tela de inclusão
 - 1.2.2. Projeto da tela de edição
 - 1.2.3. Projeto da tela de exclusão
 - 1.2.4. Projeto da tela de listagem
 - 1.2.5. Projeto da tela de busca
 - 1.3. Programação
 - 1.3.1. Programação da tela de inclusão
 - 1.3.2. Programação da tela de edição
 - 1.3.3. Programação da tela de exclusão
 - 1.3.4. Programação da tela de listagem
 - 1.3.5. Programação da tela de busca
 - 1.4. Testes
 - 1.4.1. Testes da tela de inclusão
 - 1.4.2. Testes da tela de edição
 - 1.4.3. Testes da tela de exclusão
 - 1.4.4. Testes da tela de listagem
 - 1.4.5. Testes da tela de busca

Figura 16.1.: EAP em formato texto, com o mesmo significado da Figura 16.2. Fonte: do autor

16.1.2. Controlgrama de Gantt

Os **Controlgramas de Gantt**, desenvolvidos por Henry L. Gantt no início do século XX, são representações gráficas utilizadas para o planejamento e gestão de projetos. O diagrama ilustra visualmente a linha do tempo de um projeto, representando cada tarefa como uma barra horizontal, cujo comprimento corresponde à duração da tarefa. Isso permite que gestores de

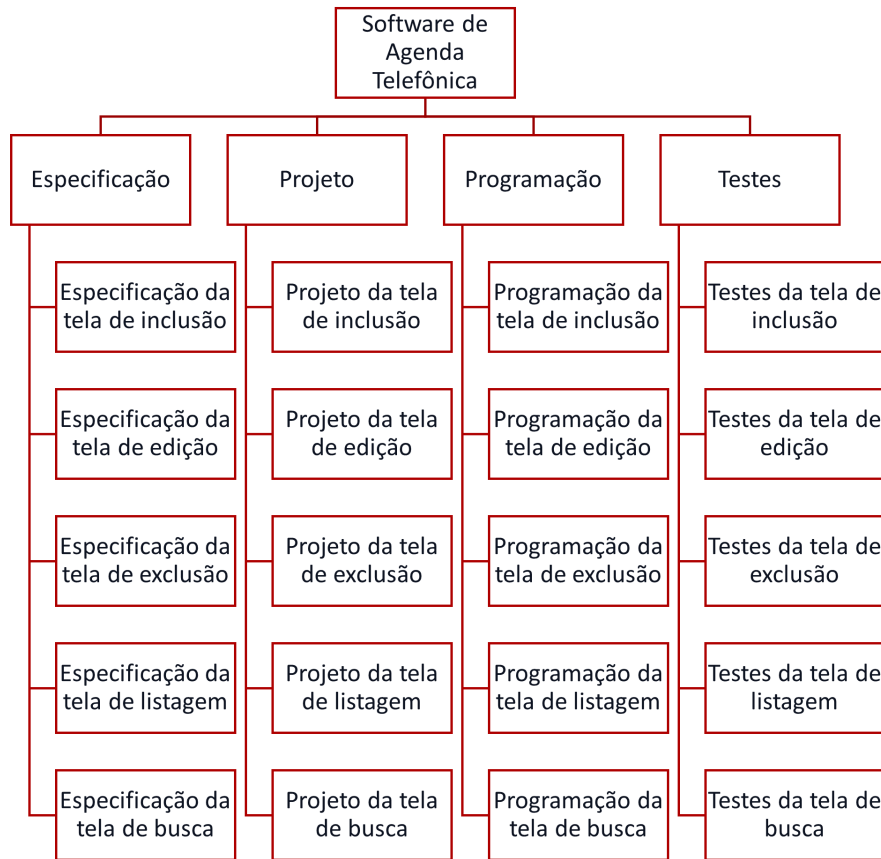


Figura 16.2.: Diagrama com modelo EAP equivalente ao da Figura 16.1. Fonte: do autor

projeto e membros da equipe visualizem e monitorem facilmente o progresso das tarefas ao longo do tempo. Um exemplo dessa visualização é dado na Figura 16.3.

O Controlgrama de Gantt, como visto na Figura 16.3 também mostrar as relações entre as tarefas, isso é dependências, nas quais uma tarefa não pode começar até que outra seja concluída. Elas são geralmente indicadas com setas conectando as barras relevantes. Essa relação, porém, é mais clara nos Diagramas de Rede.

Embora os Controlgramas de Gantt sejam úteis para retratar a linha do tempo eles têm limitações. Por exemplo, podem não representar facilmente a complexidade de projetos grandes ou os recursos alocados para cada tarefa. No entanto, softwares modernos de gestão de projetos frequentemente combinam a visualização do Diagrama de Gantt com ferramentas e funcionalidades adicionais para tratar destas preocupações.

16.1.3. Redes PERT/CPM

O *Program Evaluation and Review Technique* (**PERT**) e o *Critical Path Method* (**CPM**) são técnicas que têm como objetivo auxiliar no planejamento e controle de projetos. Essas técnicas são frequentemente empregadas na gestão de projetos para determinar a duração total do projeto e identificar o caminho crítico, que é a sequência de tarefas que não pode sofrer atrasos sem afetar a duração total do projeto.

O PERT foi originalmente desenvolvido durante os anos 1950 pela Marinha dos EUA, com contribuições da Lockheed Martin e da Booz Allen Hamilton, para o programa de mísseis balísticos Polaris (Malcolm et al., 1959). Paralelamente, o CPM foi desenvolvido por Morgan

16. Gestão de Projetos de Software

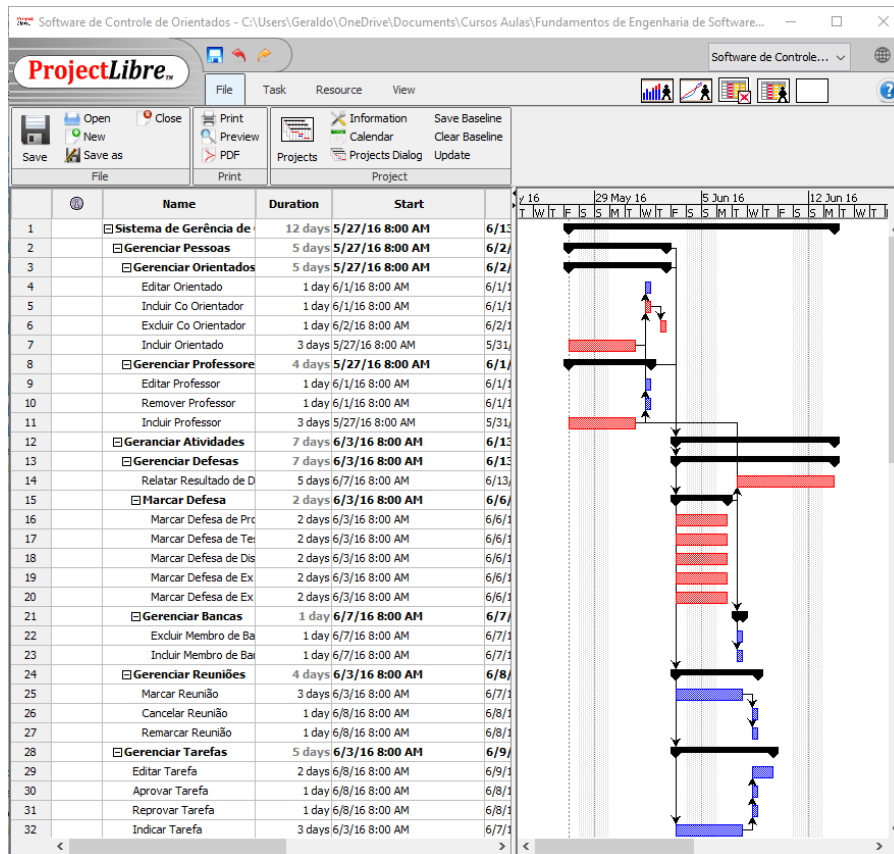


Figura 16.3.: Controlgrama de Gantt feito com o software Project Libre. Fonte: do autor

R. Walker da DuPont e James E. Kelley Jr. da Remington Rand, para abordar desafios de planejamento e programação de projetos (Kelley Jr e Walker, 1961).

Ambas as técnicas, PERT e CPM, utilizam a representação gráfica de um projeto em forma de rede, onde os nós representam eventos e as ligações representam as atividades, como na Figura 16.4, que representa as atividades da Tabela 16.1.

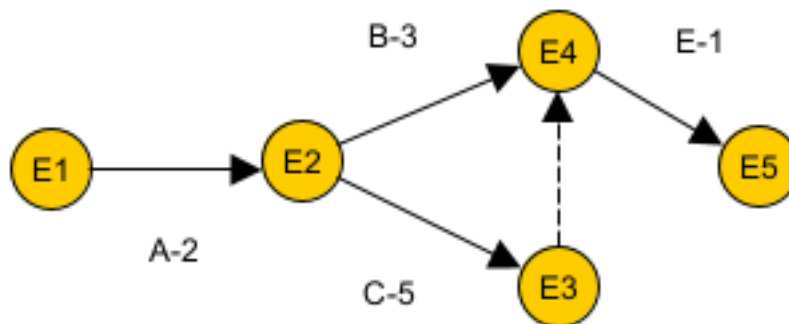


Figura 16.4.: Modelo PERT original para a Tabela 16.1

Hoje em dia, porém, é comum representar de forma inversa, sendo o nó representando as atividades e as ligações as dependências entre elas (Kerzner, 2017), criando redes de precedência. Muitas vezes, nesse caso, são chamados apenas de **Diagramas de Rede**, como o mostrado na Figura 16.5, feita com o software MS Project. O projeto da Tabela 16.1 está apresentado na Figura 16.6, feita no software Project Libre, que é Open Source, mas menos configurável.

Essas softwares normalmente também permitem a criação de diferentes tipos de dependência: a normal sendo fim para início, mas também início com início, fim com fim, início com fim e até mesmo a partir de percentuais do trabalho completo Kerzner, 2017.

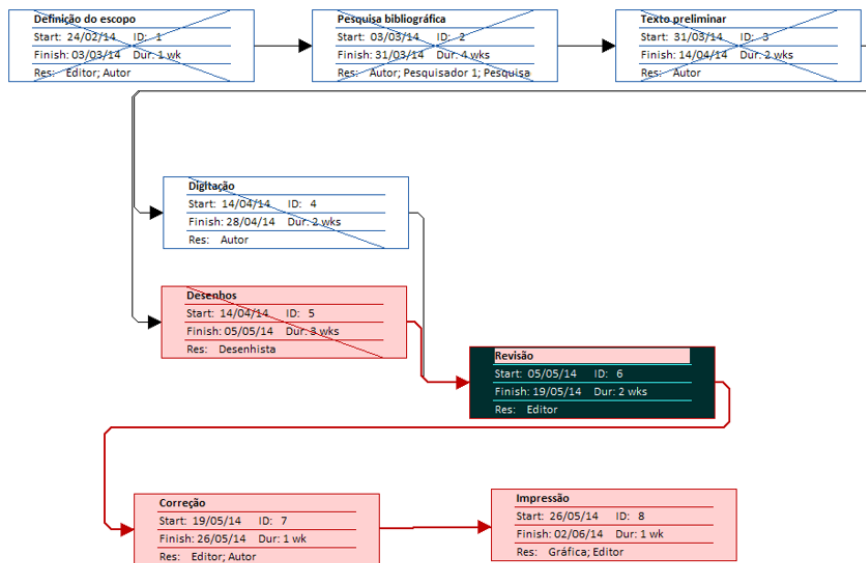


Figura 16.5.: Um diagrama de rede construído com o software MSProject, software altamente configurável. Vemos atividades completas, em andamento. Também é possível notar as atividades no caminho crítico. Fonte: do autor

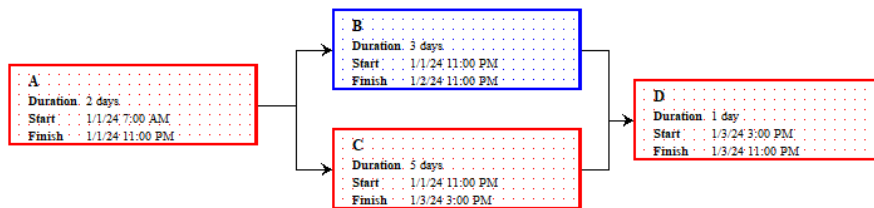


Figura 16.6.: Diagrama de rede para a Tabela 16.1, feito no software Project Libre. Fonte: o autor

A distinção principal entre PERT e CPM reside no tratamento das estimativas de tempo. Enquanto o PERT usa três estimativas de tempo (otimista, mais provável e pessimista) para determinar uma estimativa esperada e um desvio padrão para as atividades, o CPM usa apenas uma estimativa determinística.

Cálculo do Tempos a partir das Dependências

O principal uso dos diagramas de rede é calcular o tempo total do projeto, as folgas e o caminho crítico. O caminho crítico é o conjunto de atividades que se sofrerem atrasos levarão ao atraso do projeto.

A folga pode ser chamada de folga livre ou folga total. A folga livre é o tempo que uma atividade pode ser atrasada sem afetar o início da atividade imediatamente seguinte na sequência. A folga total é o tempo total que uma atividade pode ser atrasada sem atrasar a conclusão do projeto como um todo. Isso é, uma tarefa pode ultrapassar sua folga livre, mas mesmo assim não atrasar o projeto como todo. A explicação comum é que seu atraso vai ser compensando pela folga livre de uma outra atividade subsequente.

Tabela 16.1.: Tabela de tarefas de um projeto

Tarefa	Duração	Antecedentes
A	2	
B	3	A
C	5	A
D	1	B,C

Tabela 16.2.: Tabela de tarefas de um projeto, com as datas de início e término mais cedo

Tarefa	Duração	Antecedentes	Início mais cedo	Término mais cedo
A	2		1	2
B	3	A	3	5
C	5	A	3	7
D	1	B,C	8	8

Considerando um projeto com o tempo orçado em dias, dado um dia de início mais cedo de um projeto, é possível calcular a data de término mais cedo de um projeto simplesmente navegando pela rede e iniciando cada tarefa no dia seguinte ao que todas as tarefas de que dependem terminam. Por exemplo, considere as quatro tarefas e suas durações e dependências descritas na Tabela 16.1. Se a tarefa A começar no dia 1 de janeiro, então ela se encerrará no dia 2 de janeiro, perfazendo dois dias de trabalho. As tarefas B e C, que dependem de A acabar para começar, começarão no dia 3 de janeiro, e a tarefa B irá até o dia 5. A tarefa C, porém, é mais longa e irá até o dia 7. Assim, a tarefa D, que depende de B e C, só poderá começar dia 8. A tarefa B tem uma folga livre e total de dois dias. Se o projeto tiver que acabar no dia 8, as outras tarefas tem folga livre e total igual a zero. As tarefas A,B e D perfazem o caminho crítico.

Após o preenchimento das datas mais cedo de início e término das tarefas, é possível fazer a conta de volta, calculando as datas mais tarde e as folgas.

A partir da Tabela 16.2 podemos calcular, de trás para frente, as datas de término e início mais tarde das tarefas. Como decidimos que o projeto deve acabar na data de término mais cedo, a tarefa D fica com as datas do tipo mais tarde iguais as do tipo mais cedo. A diferença acontece na tarefa . A data de início mais tarde dela é o dia 7, já que a tarefa D, que depende dela, tem como data de início mais tarde o dia 8. Assim existe uma diferença entre a data de término mais tarde e a data de término mais cedo, logo ela tem uma folga livre de 2 dias. Já a tarefa B não tem folga, e também a tarefa A. O resultado pode ser visto na Tabela 16.3

O desenho de uma rede PERT/CPM permite fazer essas contas rapidamente. Para projetos com muitas atividades é recomendado usar um software dedicado.

16.1.4. Algoritmo para cálculo das datas em uma rede PERT/CPM

Para calcular as datas de início e término de atividades em uma rede de atividades, podemos seguir os seguintes passos:

Tabela 16.3.: Tabela de tarefas de um projeto, com as datas de início e término mais cedo e mais tarde.

Tarefa	Duração	Antecedentes	Início mais cedo	Término mais cedo	Início mais tarde	Término mais tarde
A	2		1	2	1	2
B	3	A	3	5	5	7
C	5	A	3	7	3	7
D	1	B,C	8	8	8	8

Datas de Início Mais Cedo (IMC) e Término Mais Cedo (TMC)

1. Considere a primeira atividade com $IMC = 1$.
2. Para todas as atividades com o IMC, atualize o TMC da atividade como:

$$TMC = IMC + \text{duração da atividade} - 1$$

3. Para todas as atividades subsequentes diretamente dependentes desta atividade que tiverem todas suas atividades dependentes com o TMC calculado, atualize o IMC delas como:

$$IMC (\text{atividade subsequente}) = \max(TMC (\text{atividades anteriores})) + 1$$

onde o máximo é tomado sobre todas as atividades das quais a atividade subsequente depende.

4. Repita os passos 2 e 3 para todas as atividades até o fim da rede.

Datas de Término Mais Tarde (TMT) e Início Mais Tarde (IMT)

1. Inicialize a atividade final com TMT igual ao seu TMC (determinado no passo anterior). Ou seja, o projeto terminará no TMC da última atividade.
2. Calcule o IMT da atividade final como:

$$IMT = TMT - \text{duração da atividade} + 1$$

3. Para todas as atividades anteriores diretamente relacionadas à essa atividade, atualize o TMT delas como:

$$TMT (\text{atividade anterior}) = \min(IMT (\text{atividades subsequentes})) - 1$$

onde o mínimo é tomado sobre todas as atividades que dependem diretamente da atividade anterior.

4. Atualize o IMT da atividade anterior como:

$$IMT = TMT - \text{duração da atividade} + 1$$

5. Repita os passos 3 e 4 retrocedendo para todas as atividades até o início da rede.

As folgas de cada atividade podem ser calculadas como:

$$\text{Folga} = IMT - IMC$$

Uma folga de zero indica que a atividade está no caminho crítico.

16.2. Uso das Técnicas Tradicionais

Hoje em dia, devido aos softwares disponíveis, como MSProject ou Project Libre, essas técnicas são todas usadas de forma conjunta, sendo possível passar de uma visão para outra com facilidade.

16.3. Técnicas Ágeis de Gestão de Projeto

16.3.1. Kanban

Parte IV.

Implementando o Software

Nessa parte deve ser tratada a gestão de configuração.

Testing shows the presence, not the
absence of bugs

(*Edsger W. Dijkstra*)

Conteúdo

17.1.	Terminologia	222
17.2.	Introdução aos Conceitos de Teste	223

Testes servem para mostrar empiricamente que um fragmento específico de software cumpre um conjunto dado de requisitos em um ambiente dado em um momento específico do tempo. Eles permitem verificar o comportamento do sistema, sendo uma técnica dinâmica que busca revelar falhas no software, isto é, encontrar defeitos.

Verificar significa que algo está dentro de suas especificações. Essa palavra é normalmente utilizada junto com a palavra validar, que significa que algo faz o que deveria fazer, como o usuário deseja. Diferenciar esses dois conceitos é algo importante na Engenharia de Software, pois nos mostra que mesmo fazendo o que prometemos, podemos não entregar o desejado.

Segundo Pressman e Maxim (2019), testar é “Testar é o processo de exercitar um programa com a intenção específica de achar erros antes de entregá-lo ao usuário final”. Já a ISO/IEC/IEEE (2017) define teste como “Atividade em que um sistema ou componente é executado sob condições especiais, os resultados são observados ou registrados, e uma avaliação é feita sobre algum aspecto do sistema ou componente”.

17.1. Terminologia

Para discutir testes, é necessário conhecer uma terminologia adequada:

- **bug**: são coisas que o software não devia fazer, ou coisas que o software não faz, mas devia fazer (Telles e Hsieh, 2001), sinônimo de falta ou defeito (ISO/IEC/IEEE, 2017).
- **crash**: a falha (*failure*) completa e repentina de um sistema computacional ou componente (ISO/IEC/IEEE, 2017).
- **defeito (defect)**: uma imperfeição ou deficiência em um produto do trabalho que não atende seus requisitos ou especificações, e precisa ser ou reparado ou trocado (ISO/IEC/IEEE, 2017).
- **erro (error)**: uma ação humana que produz um resultado incorreto, ou a diferença entre um valor ou condição computada, observada ou medida e o valor ou condição verdadeiro, especificado ou teoricamente correto, ou ainda um estado errôneo do sistema (ISO/IEC/IEEE, 2017).
- **erro (mistake)**: ação humana que produz um resultado incorreto (ISO/IEC/IEEE, 2017).
- **falha (failure)**: o fim da habilidade de um sistema de desempenhar uma função necessária ou sua inabilidade de desempenhar dentro de limites previamente especificados; um desvio externamente visível da especificação do sistema (ISO/IEC/IEEE, 2017).
- **falta ou defeito (fault)**: manifestação de um erro em software, defeito em um sistema ou uma representação de um sistema que, se executado/ativado, poderia potencialmente resultar em um erro (ISO/IEC/IEEE, 2017).

Resumindo essas definições, um erro é um problema interno e causa um defeito, que é percebido externamente. Um erro (*mistake*) é causado por um humano que gera um defeito (*fault*), que pode se transformar em um erro encontrado ou um defeito que vai gerar uma falha no ambiente do usuário.

Um processo de qualidade deve descobrir como os erros estão sendo inseridos. A qualidade do processo diminui a inserção de erros. O processo de qualidade do produto detecta os erros antes de se transformarem em defeitos, causando falhas para o usuário. A partir dessa ideia é possível definir o ciclo de vida do defeito, que se inicia com a inserção, passa pela detecção e termina quando é removido.

17.2. Introdução aos Conceitos de Teste

Não é possível provar que um programa está absolutamente correto. Mesmo tentativas de construir programas corretos por meio da lógica, ou de outro processo formal, falharam. O objetivo do teste é encontrar problemas, para corrigi-los.

É praticamente impossível testar um programa completamente. Para isso seria necessário testar todos os caminhos possíveis do código, todas as entradas válidas e todas as entradas inválidas. É fácil provar que isso é impossível: imagine um programa que permite a entrada de uma quantidade indeterminada de dados em um arquivo ou base de dados. Para testá-lo completamente, seria necessário entrar todas as entradas possíveis, tanto corretas quanto incorretas, em todas as ordenações possíveis. Isso certamente levaria um tempo enorme.

Dessa análise, podemos concluir que leva mais tempo para testar do que gostaríamos, e temos que fazer opções, ou planos de teste, formados de casos de teste. Partimos do princípio que nunca encontraremos o último erro, e tomamos uma postura probabilística.

Uma prática importante é a do **testes regressivos**, que significa executar todos os testes do software a cada mudança, para ter indícios que a última mudança não gerou um erro que já foi resolvido antes. Na prática acontece um ciclo da seguinte forma, quando um defeito é encontrado na operação:

1. Acontece o defeito;
2. O cliente ou usuário faz uma reclamação, que pode ser mal documentada;
3. É preciso criar um teste que replique o defeito;
4. É feita a suposição que o teste ativa o erro que provoca o defeito descrito pelo usuário (pode ser outro erro que provoca o mesmo defeito);
5. Criamos novos testes para delimitar o defeito;
6. Buscamos o erro (*mistake*);
7. Corrigimos o erro;
8. Corrigimos outros erros associados;
9. Criamos novos testes para verificar que tudo funciona, e
10. Executamos os testes.

Testes possuem uma cobertura, isso é, eles cobrem, em certa extensão, os requisitos de um sistema ou produto de software (ISO, 2017).

Parte V.

Medindo o Software

Medidas e Estimativas do Tamanho do Software

Measurement is the first step that leads to control and eventually to improvement. If you can't measure something, you can't understand it. If you can't understand it, you can't control it. If you can't control it, you can't improve it.

(H. James Harrington)

Conteúdo

18.1.	Esforço	227
18.2.	Algumas Medidas Conceitualmente Simples	228
18.3.	Visão Geral dos Métodos de Estimativa	231
18.4.	Métodos de Estimativa Baseado em Opinião	232

Ao iniciar um projeto de software, ou qualquer outro projeto, é necessário ter uma estimativa da quantidade de tempo e recursos que serão usadas para desenvolvê-lo. Essa estimativa é feita a partir de outra: a estimativa de tamanho do software. Para essa estimativa ser feita é preciso, antes, entender como esse tamanho pode ser medido. Isso foi visto parcialmente no Capítulo ??, mas existe muito material sobre o assunto tanto na área de Gestão de Projetos (PMI, 2017) quanto na área de Engenharia de Software (IEEE Computer Society, 2014a; Pressman e Maxim, 2014).

Apesar de muito estudada, a estimativa de projetos de software ainda é um problema em aberto. Isso acontece não só pela dificuldade inerente de prever alguma coisa, mas também por alguns motivos adicionais. O primeiro motivo importante é a falta de maturidade do processo de muitas organizações, que resulta na falta de dados que ajudem a fazer previsões com baixa margem de erro. Em muitos lugares ainda são praticadas estimativas *ad-hoc*, baseadas na

experiência dos desenvolvedores ou gerentes, e ainda sujeitas a pressões de clientes e da alta gerência. Em outros, mesmo com processos mais definidos, não há dados suficientes sobre o desempenho da equipe em projetos anteriores e similares para fazer boas previsões. Finalmente, muitos projetos de software começam com uma definição incerta do que vai ser feito, tendo um foco inicial no problema a ser resolvido, o que dificulta também a previsão. Os métodos ágeis atacam alguns desses problemas evitando fazer grandes estimativas do trabalho total do projeto no seu início, porém isso impossibilita certas formas de contrato.

Quanto mais tradicional for o desenvolvimento, seja por cultura da empresa, seja por imposição do cliente, mais será necessário fazer e reduzir os erros dessa estimativa, inclusive por motivos contratuais. Além disso, ela influencia todas as expectativas das partes interessadas, incluindo, principalmente: o orçamento, a funcionalidade entregue e o prazo do projeto. Já nos projetos ágeis, o foco é na entrega de valor em prazos pré-delimitados e o projeto é visto de outra forma.

Logo, o objetivo da estimativa de um projeto de software exige, em um certo momento no tempo, no início ou ao longo do projeto, prever os recursos necessários, cujo custo é basicamente dominado pelos recursos humanos¹, e o tempo necessário para terminar o projeto, dando ferramentas ao gestor para planejar as próximas fases. Isso é feito com um grau de incerteza, que é maior no início, por ser uma previsão baseada em informações limitadas, mas é importante lutar para que essa incerteza seja pequena e conhecida. Ao longo do projeto as estimativas continuam e são constantemente comparadas e revisadas. Como há um aumento da quantidade de informações, normalmente se tornam mais precisas. No limite, ao final do projeto, a previsão dos recursos e tempo necessário para o projeto é perfeita, apesar de ser inútil.

Este livro é basicamente dedicado a responder a pergunta **o que** devemos desenvolver, de algumas formas, mas não seria completo se não apresentasse também alguma maneira de avaliar o tamanho do software que está sendo proposto, principalmente após o processo de análise. Este capítulo e os próximos tentam mostrar como essas outras perguntas podem ser respondidas. Para isso trataremos de algumas medidas de tamanho possíveis para o software, respondendo a pergunta 2, e discutiremos como essas medidas podem ser previstas, respondendo a pergunta 3. Finalmente mostraremos, com alguma simplificação, como o método COCOMO II (Barry W. Boehm et al., 2000) pode ser usado para calcular o esforço, a equipe média e o tempo de desenvolvimento.

18.1. Esforço

Tendo em vista o forte domínio do valor dos recursos humanos em projeto de software, a principal forma de entender a demanda de recursos e tempo para um projeto de software é a quantidade de trabalho que será necessária para construí-lo. Em Engenharia de Software, o trabalho necessário para desenvolver um produto é conhecido como **esforço**. O esforço é medido em pessoas-hora ou pessoas-mês, isto é, a quantidade total de horas, ou meses, trabalhados por uma quantidade de pessoas para fazer o produto.

Assim, um software pequeno pode precisar de 4 pessoas-mês, dando a ideia que 2 pessoas podem fazê-lo em 2 meses, ou 1 pessoa pode fazê-lo em quatro meses. Porém, é difícil imaginar que 8 pessoas possam fazê-lo em 15 dias. Acontece que sistemas de informação são um pouco como bebês: não podemos ter a gestação de um bebê com nove mães em um mês, também não podemos colocar milhares de pessoas para fazer um software em um dia. Ao contrário, se colocarmos pessoas demais é possível que a necessidade de gerenciar atrapalhe a produção (Brooks,

¹Ou proporcional a eles

1995). Na verdade, Barry W. Boehm et al. (2000) encontraram uma relação entre o esforço necessário e o tempo necessário para fazer um sistema, e consequentemente o tamanho médio da equipe, incluída no método COCOMO II.

A medição do esforço de um projeto em andamento ou terminado é ao mesmo tempo uma prática fácil e difícil. Em um projeto onde toda a equipe está dedicada, o esforço pode ser medido simplesmente considerando o tempo e a quantidade de pessoas envolvidas no projeto. Já em casos onde há vários colaboradores parciais, ou se deseja uma contagem mais exata, é necessário que todo o trabalho seja registrado e contado detalhadamente.

A pergunta principal do processo de estimativa de software, então, é o esforço necessário para desenvolver uma certa funcionalidade, dentro de um contexto. A expectativa de um gerente de projeto é que esse esforço seja estimado diretamente a partir de uma especificação, de preferência o mais cedo possível. O que se espera, e há uma confirmação pelo método COCOMO (Barry W. Boehm et al., 2000), é que o esforço necessário seja função, pelo menos em maior parte, do tamanho do software.

Disso podemos concluir que uma das mais importantes informações que podemos ter sobre um produto de software é seu **tamanho**. O problema é que tamanho, em software, é um conceito com muitos significados. Por exemplo, você pode simplesmente medir o tamanho de um programa na forma compilada, e isso tem impacto no processo de distribuição do produto, como tamanho de mídia ou ocupação da rede. Porém, esse tamanho tem pouco significado em relação a questões muito mais importantes, como é o esforço necessário para construí-lo. Grande parte do tamanho do programa a ser distribuído, por exemplo, pode vir de bibliotecas anexadas a ele. Qual o tamanho de um programa “Hello World!” para um sistema operacional de janelas e para um sistema de linha de comando²? Assim, entre as medidas propostas, existem algumas mais úteis do que outras, em cada contexto.

Dessa forma, temos uma sequência de questões a serem respondidas:

1. O que o software fará?
2. Como medir o tamanho de um software?
3. Como prever o tamanho do software que faz o que foi pedido?
4. Qual o **esforço** necessário para desenvolver um software desse tamanho?
5. Qual a equipe média e o tempo de desenvolvimento necessário para realizar esse **esforço**?

18.2. Algumas Medidas Conceitualmente Simples

Algumas medidas de software são razoavelmente simples de entender, o que permite que elas sejam usadas e compreendidas por pessoas com pouco ou nenhum treinamento, como por exemplo:

- linhas de código fonte;
- pontos de história, e
- tamanhos de camisa.

Essas medidas, apresentadas a seguir, podem ser usadas em métodos de estimativas baseados em especialistas, que serão o tema da próxima seção.

²Em um pequeno experimento, a diferença foi de cinco vezes

18.2.1. A Medida Mais Simples: Linhas de Código

A forma mais conhecida de medir o tamanho de software é a contagem de (milhares) linhas de código fonte, conhecida como LOCs, KLOCs, SLOCs ou KSLOCs, do inglês (*Kilo*) (*Source*) *Lines of Code*. A ideia de contar as linhas de código do produto de software e ter uma medida direta e facilmente verificada.

Só que a questão não é tão fácil.

Primeiro, linguagens diferentes vão apresentar uma quantidade de linhas de código diferentes para a mesma funcionalidade, como mostram as Listagens 18.1 e 18.2. Como comparar o tamanho de dois programas em linguagens diferentes então?

Programa 18.1: Programa Hello World para Win32(Rösler, 1994)

```
; Hello world em Assembler para arquitetura Win32

TITLE Hello world in win32. Tasm

VERSION T310
Model use32 Flat,StdCall

start_code segment byte public 'code' use32
begin:
    Call MessageBox, 0, offset sHallo, offset caption, 0
    Call ExitProcess, 0
start_code Ends

start_data segment byte public 'data' use32

sHallo db 'Hello world',0
caption db "Hi",0

start_data
```

Programa 18.2: Programa Hello World em Pytho 3(Rösler, 1994)

```
# Hello world em Python 3
print("Hello World")
```

Segundo, o que contar como uma linha de código? Por que uma sentença com o mesmo significado pode ser escrita em um número diferente de linhas. Além disso, contam-se os comentários ou não? Contam-se os espaços em brancos destinados a facilitar a leitura? Esse problema, ao menos, foi resolvido com uma definição do que deve, e do que não deve, ser contado, por meio de padrões razoavelmente complicados. porém esse trabalho é difícil, também dependente de linguagem, e pode gerar dúvidas em função do estilo de programação.

Linhas de código tem como vantagens ser intuitivas, mesmo considerando os complicados padrões que dizem o que pode e o que não pode ser contado, e permitem a automação da contagem. A automatização acaba sendo importante na análise estatística de projetos, que levou ao método COCOMO II, por exemplo, e pode gerar adaptações locais aos valores das constantes desse método e de suas variantes(Barry W. Boehm et al., 2000).

Seus problemas, porém, são muitos:

- não têm relação com o esforço de análise e projeto, sendo ligadas apenas ao esforço de programação;
- não tem relação real com a funcionalidade entregue;
- variam de acordo com linguagem, estilo de programação, experiência do programador, uso de bibliotecas, etc.;
- não atendem as tecnologias mais modernas, com uso de GUI e geração de código;
- quando usadas como parâmetro de remuneração, incentivam soluções mais longas, e
- são difíceis de usar como medida inicial de estimativa.

Provavelmente a maneira mais difundida de contar linhas de código é a fornecida pelo Software Engineering Institute em um manual de 242 páginas, que inclui uma *check-list* de cinco páginas para exemplificar como isso pode ser feito. Para linhas de código fonte lógicas (LSC) as instruções básicas sugerem contar instruções executáveis ou não e declarações, e não contar comentários e linhas de código. Além disso, devem ser contadas linhas programadas, geradas, convertidas, copiadas e modificadas, mas não linhas removidas (Park, 1992).

Tendo em vista o quadro de problemas da medida de linha de código, é possível pensar que uma medida realmente interessante sobre software não fala do artefato criado, mas sim do resultado fornecido pelo artefato. Assim, outras medidas podem fornecer uma ideia melhor do software.

Isso significa entender que o verdadeiro valor do software não está nas medidas diretas que podemos fazer, mas sim no que ele fornece ao usuário, e que esse tamanho que é interessante.

Essa foi a motivação da criação da medida **Pontos de Função**, que é tratada no Capítulo 19.

18.2.2. Pontos de História

“**Pontos de história**” é uma medida ágil usada como estimativa do esforço necessário para desenvolver uma história do usuário. Pontos de História são bastante informais e, devido a forma como são definidos e usados, não podem ser comparados entre projetos.

Segundo Cohn (2005) “pontos de histórias são uma unidade de medida para expressar o tamanho geral de uma história do usuário, funcionalidade ou outra peça de trabalho”. Além disso, Cohn (2005) avisa que “O valor bruto que damos não é importante. O que importa são os valores relativos.”

Pontos de história podem ser medidos em uma escala qualquer, porém o mercado praticamente estabeleceu o uso de um conjunto de valores que aproxima uma série de Fibonacci: 0, 1/2, 1, 2, 3, 5, 8, 13, 20, 40 e 100. Esses números devem ser vistos como uma referência de comparação.

Essa sequência, em vez de usar qualquer número, tem uma função: evitar discussões infrutíferas e aceitar uma margem de incerteza em estimativas. Dessa forma se evita que em uma discussão para estimar o tamanho de uma história se perca tempo discutindo se é 12 ou 13, por exemplo. Os números dessa série, então, tem a propriedade de mostrarem não só tamanhos claramente distintos, mas também com uma importância nessa distinção: cada número é, aproximadamente, a soma dos dois anteriores, assim a comparação entre tamanho histórias pode ficar mais clara.

Para fazer a estimativa, em cada início de projeto, a história do usuário mais simples, para aquele projeto específico, recebe o valor 2 como estimativa do seu esforço. A partir daí, os outros valores são usados no processo de estimativa escolhido, tentando mostrar a relação proporcional de esforço necessário em uma tarefa em relação a tarefa de peso 2. Claro que nenhuma história

no início do projeto vai ser pontuada com um valor menor que 2, mas é comum aparecerem histórias menores com o tempo.

A forma normalmente escolhida para fazer a estimativa é o *Planning Poker* (Cohn, 2005, p 56), principalmente em projetos ágeis.

A estimativa de pontos de história é calibrada ao longo do projeto, pois a equipe vai aprendendo a estimar melhor ao realizar as histórias.

Alguns autores, como ISBS (2010), propõem que a equipe analise a relação entre os pontos de histórias atribuídos a uma história de usuário e as horas efetivamente gastas para implementá-las. Isso pode trazer uma visão mais clara aos membros da equipe, porém não é necessariamente vantajoso frente a experiência real da equipe no projeto e depende, para ser uma ação prática, do método de gestão levantar essas informações com facilidade, como acontece com o uso de algumas ferramentas de gestão de projetos ágeis.

Velocidade da Equipe

A velocidade de uma equipe ágil é o número de pontos de história que ela faz, em média, em cada ciclo (ou *sprint*). No início do projeto, como a equipe não sabe ainda estimar muito bem os tamanhos das histórias, essa velocidade tende a variar. Ao longo do projeto ela costuma se tornar estável e até mesmo diminuir com o aumento da produtividade da equipe. Vigiar a velocidade é uma tarefa dos gestores ágeis, porém ela é apenas um sinalizador, não é razoável utilizá-la como um número absoluto de produtividade, porque as equipes vão adaptando suas estimativas ao longo do projeto.

18.2.3. Tamanhos de Camisa

Uma medida curiosa relatada por Dimitrov (2020), Rubin (2013) e vários outros autores é medir o tamanho de software fazendo uma analogia com tamanhos de camisa. Nesse caso, usado em métodos ágeis, cada história do usuário recebe um valor entre os tamanhos que encontramos para camisetas. Uma seleção possível de valores seria: XP, P, M, G, XG, XXG.

*T-Shirt
Sizing*

Para isso funcionar, é possível manter uma tabela que associe cada tamanho com uma margem de variação, possivelmente em horas de trabalho, para estimativas de artefatos pequenos, mas talvez de custo se são estimados softwares completos, por exemplo.

Essa escala estimula a comparação e a abstração, porém dificulta a determinação de valores como a velocidade da equipe e o tamanho global de um projeto ou de uma fase de projeto.

Rubin (2013) considera que medidas de tamanho de camisa associadas a faixas de valores de custo são boas para o planejamento de portfólio.

18.3. Visão Geral dos Métodos de Estimativa

Como visto, os motivos principais para se medir software têm relação com as tarefas de gerenciar projetos de software. Precisamos saber o tamanho para entender quanto trabalho fizemos ou vamos fazer, qual a dificuldade de fazê-lo, ou qual será o custo. Entender o tamanho de um produto permite responder perguntas como:

- Qual o custo de desenvolver o sistema?

- Qual o esforço para desenvolver o sistema?
- Quantas pessoas serão necessárias para fazer esse software?
- Em quanto tempo o sistema ficará pronto?
- Que recursos são necessários para o sistema executar?
- Qual o valor da nossa carteira de software?

Métodos de estimativa de projeto de software podem ser divididos em três grandes grupos (ISBS, 2010):

- métodos macro, que buscam estimativas de alto nível;
 - uso de equações;
 - comparação, com projetos similares, e
 - analogia, que busca um projeto completo muito similar;
- métodos micro, que buscam estimativas a partir dos detalhes, baseados na estrutura analítica do projeto ou em outra forma de quebrar sucessivamente o projeto em tarefas menores que podem ser estimadas com mais facilidade;
- métodos usando IA, que não serão tratadas neste texto.

Estimar software sem o uso de IA sempre implica em ter, em algum momento, a opinião de pessoas. Algumas práticas se baseiam fortemente nas opiniões de especialistas, talvez um único especialista. Outros, principalmente os aplicados em processos ágeis, se utilizam da equipe e tendem a cultivar o aprendizado e praticar revisões nas estimativas.

A questão aqui é realmente ligada ao tipo de contrato de desenvolvimento que está sendo feito. Um contrato tradicional não tem como escapar de fazer uma estimativa mais formal, considerar riscos e cobrar valores que vão cobrir o custo e gerar lucro. Um contrato por tempo de serviço, onde a funcionalidade é determinada de tempos em tempos pelo cliente, facilita o processo de estimar, principalmente se este intervalo de tempo for curto.

É importante deixar claro que o problema de estimar software não é complicado para pequenas tarefas, mas extremamente difícil para grandes sistemas. Mesmo dividindo um software com uma metodologia como a Estrutura Analítica de Projeto, há sempre o problema de integrar as partes e a complexidade do sistema não é o somatório das complexidades das partes. Pelo contrário, quanto maior o sistema, mais complexo é construí-lo. Essa afirmação é corroborada pelos resultados da pesquisa que levou ao método COCOMO II (Barry W. Boehm et al., 2000).

18.4. Métodos de Estimativa Baseado em Opinião

Alguns métodos simples para estimar baseado em opiniões são:

- Método de Estimativa do PERT;
- Método de Delphi, e
- *Planning Poker*.

18.4.1. O Método de Estimativa do PERT

Uma maneira simples de fazer uma estimativa através da opinião de especialistas é a usada pelo método PERT (ISBS, 2010; PMI, 2017), que estima um valor a partir da fórmula:

$$T_e = \frac{T_o + 4 \times T_\mu + T_p}{6} \quad (18.1)$$

onde T_p é a estimativa mais pessimista, T_o a estimativa mais otimista e T_μ a estimativa média. Essas três estimativas são idealmente calculadas a partir de várias opiniões dadas por pessoas diferentes, mas podem ser até mesmo fornecidas por um especialista único.

18.4.2. O Método de Delphi

Outra forma conhecida de fazer estimativas é o método de Delphi (Dalkey, 1967; Helmer-Hirschberg, 1967; ISBS, 2010), cuja prática original é um ciclo iterativo onde:

1. especialistas recebem uma ou mais perguntas para fazer sua estimativa;
2. as estimativas são coletadas e relatadas a todos os especialistas de forma anônima;
3. os especialistas podem revisar e dar justificativas para suas novas estimativas, e
4. o ciclo é repetido até as opiniões convergirem.

Esse método, proposto em 1966, recebe o nome do Oráculo de Delphi, ou Delfos, e vem sendo reconstruído de várias formas (ISBS, 2010), como em reuniões onde não há anonimato e em aplicativos na rede, porém a que tem mais feito sucesso nos últimos anos é o *Planning Poker*.

18.4.3. O Planning Poker

No *Planning Poker* (Cohn, 2005, p 56) cada membro da equipe deve fazer uma estimativa do esforço necessário para desenvolver um história do usuário. Geralmente esse esforço é medido em valores arbitrários, como os **pontos de história**, descritos na seção 18.2.2.

Para o Planning Poker é necessário um baralho especial, ou um aplicativo que simule esse baralho, e que possua a sequência de pontos de história usada pela sua equipe e algumas cartas adicionais, como uma carta com uma interrogação, que indica que o estimador está sem ideia de como estimar, e uma xícara de café, que indica que o estimador está cansado e precisa parar um pouco.

As rodadas, inspiradas no Método de Delphi, são iniciadas com os membros da equipe ouvindo e questionando um *product owner* ou um analista sobre como deve ser o comportamento da história do usuário sendo estimada (Figura 18.1a). A cada rodada, cada membro do time escolhe uma carta em segredo com um número desejado (Figura 18.1b). As cartas são todas mostradas simultaneamente (Figura 18.1c) e, então, se as estimativas forem diferentes, o mais otimista e o mais pessimista devem justificar seu voto (Figura 18.1d) e o processo é refeito até que uma condição de parada seja aceita (Figura 18.1e e 18.1f).

A questão da condição de parada é um pouco controversa. Normalmente o número de rodadas é limitado a 3, com a expectativa que **um número comum a todos seja alcançado**. Caso isso não aconteça, existem várias regras de como agir: tentar mais algumas vezes, adiar a avaliação para outra ocasião, propor a quebra da história em histórias menores, usar o valor mais frequente (Cohn, 2005), usar a previsão mais pessimista (Maximini, 2018), usar a previsão mais otimista (ISBS, 2010), ou até mesmo gerar uma solução *ad-hoc* de consenso naquele instante. Essa decisão também depende da variação das estimativas. Um caso onde todas são iguais menos uma é diferente de um caso onde as estimativas estão espalhadas entre 3 números.

O Oráculo de Delfos foi um templo na Grécia Antiga onde sacerdotes faziam previsões do futuro normalmente ambíguas.

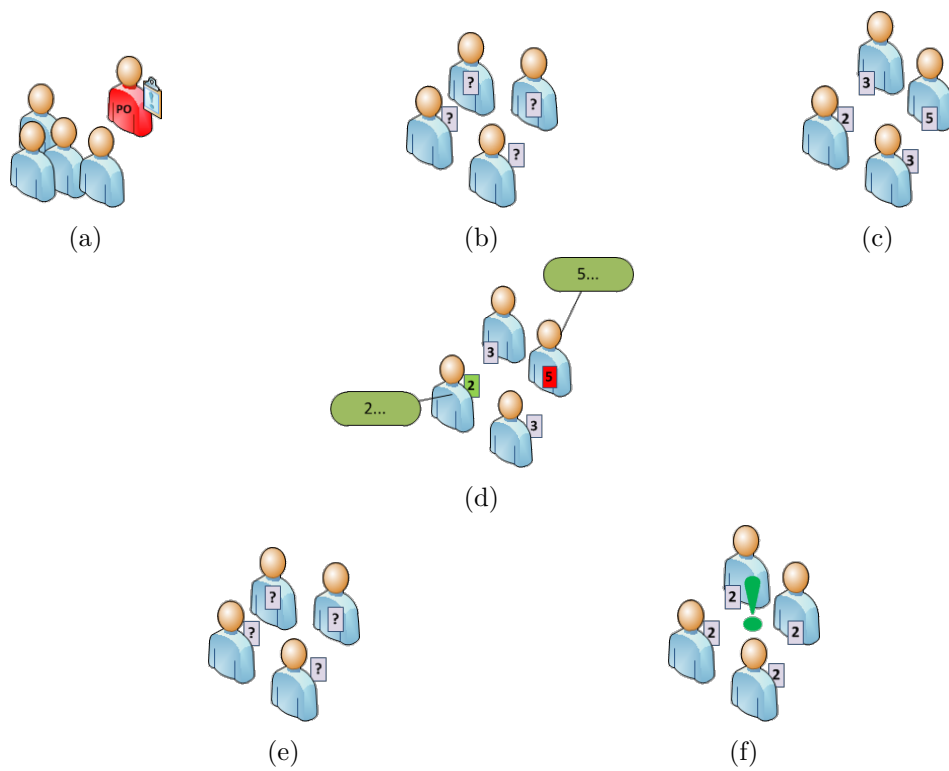


Figura 18.1.: Ilustração da sequência do Planning Poker com duas rodadas

Análise de Pontos de Função

When you can measure what you are speaking about and express it in numbers, you know that on which you are discoursing

(Lord Kelvin)

Conteúdo

19.1.	Visão Geral de Pontos de Função	236
19.2.	Procedimento de Contagem	236
19.3.	Funções Transacionais	237
19.4.	Funções de Dados	237
19.5.	Identificando Funções de Negócio	238
19.6.	Identificando Entradas	239
19.7.	Identificando Arquivos Internos	240
19.8.	As Perguntas	243
19.9.	Cálculo dos Pfs Finais	243
19.10.	Conclusão	244

Por que Pontos de Função?

Ao fazer uma modelagem de sistema, é importante ter alguma ideia do esforço necessário para desenvolvê-lo. A Análise de Pontos de Função é uma prática aceita em todo mundo que permite avaliar o tamanho previsto de um software.

Até 1979 não tínhamos uma boa medida do tamanho do software em função da sua funcionalidade como vista pelo usuário. Apenas nesse ano, (Albrecht, 1979) apresentou uma medida conhecida como Pontos de Função, cujo objetivo era servir como avaliador e preditor do tamanho

de um sistema. Atualmente Pontos de Função são uma medida internacionalmente aceita e uma das formas possíveis de se fazer um contrato com o governo no Brasil (Brasil, 2018)¹.

19.1. Visão Geral de Pontos de Função

Um Ponto de Função (PF) é uma medida abstrata e relativa que conta “o número de funções de negócio entregues ao usuário”. Um relatório simples, por exemplo, pode medir “4 Pontos de Função”. Da mesma forma que um metro ou um litro, Pontos de Função só fazem sentido quando comparados com um padrão. Assim, um sistema com 1.000 PF entrega o dobro de funcionalidade de que um sistema com 500 PF. Com o tempo, aprendemos a ter uma ideia absoluta do tamanho de um sistema a partir da contagem de seus PFs (IFPUG, 2012).

Pontos de Função avaliam o tamanho do software a partir de seis características principais (IFPUG, 2012):

- entradas;
- saídas;
- consultas;
- arquivos lógicos internos;
- interfaces lógicas externas, e
- ajustes de complexidade.

A maneira padronizada de contar pontos de função é fornecida pelo *International Function Points User Group (IFPUG)*(IFPUG, 2012), que fornece aos seus associados um manual contendo detalhes do que deve e do que não deve ser contado. Esse capítulo é apenas uma introdução ao método, servindo para fazer cálculos aproximados do número de pontos de função.

19.2. Procedimento de Contagem

O procedimento genérico de contagem de Pontos de Função é representado na Figura 19.1.

Ele se inicia com a determinação do **propósito da contagem**, isto é, a explicitação do motivo da contagem estar sendo realizada. Normalmente esse propósito estará relacionado a fornecer uma resposta a um problema de negócio existente, como a contratação de um serviço.

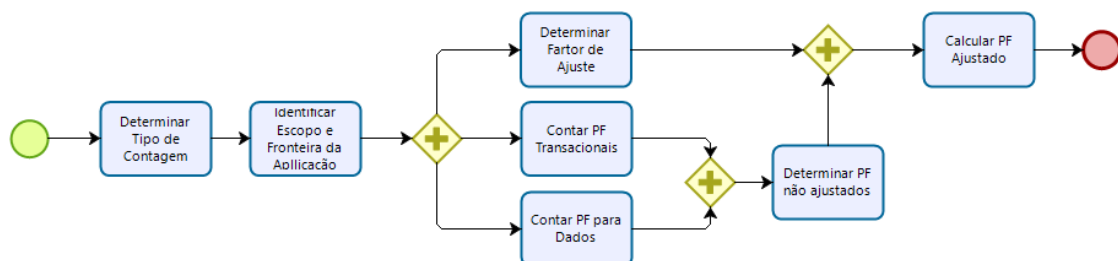


Figura 19.1.: O processo geral de contagem de Pontos de Função. Fonte: do autor

¹A especificação brasileira de Pontos de Função para o governo é mais complexa na sua forma de usar, mas parte dos mesmos princípios.

A partir do propósito podemos determinar o tipo de contagem. São considerados três tipos de contagem:

1. **projeto de desenvolvimento**, onde medimos as funcionalidades entregues ao usuário em uma versão onde o software é desenvolvido desde o início. Inclui as funcionalidades necessárias para a conversão de dados, mesmo que usadas uma única vez;
2. **projeto de melhoria**, onde medimos as modificações que alteram, adicionam ou removem funcionalidades em uma aplicação existente. Inclui as funcionalidades necessárias para a conversão de dados, mesmo que usadas uma única vez, e
3. **aplicação**, onde medimos uma aplicação já instalada. Também é chamada de *baseline* e é normalmente realizada no fim de um projeto de desenvolvimento e mantida atualizada nos projetos de melhoria.

O escopo da contagem define um conjunto ou um subconjunto do sistema, e permite dizer se uma funcionalidade deve ou não ser contada.

A fronteira da aplicação delimita o software medido, definindo sua interface com o mundo exterior. Ela servirá não só para considerarmos se uma função deve ou não ser contada, mas também para considerar se um arquivo lógico deve ser contado como interno ou externo a aplicação.

Definido o tipo de contagem, a fronteira e o escopo da contagem, que influenciarão a forma final de contagem, identificamos as funções de negócio como percebidas pelo usuário, dividindo-as em 5 grupos, agrupados em 2 tipos: funções transacionais e funções de dados.

19.3. Funções Transacionais

Um processo elementar é a menor atividade significativa para o usuário de forma indivisível. Isso significa que o usuário o vê como um processo único, completo e independente de outros, que se inicia com o sistema em um estado consistente e termina com o sistema em um estado consistente.

Um processo elementar será contado como uma de três possíveis formas de função transacional:

- **saídas externas** (SE ou EO), são informações de negócio que o usuário final pode receber, representando relatórios, telas e mensagens de erro como um todo e não em suas partes individuais;
- **consultas externas** (CE ou EQ), que são saídas simples e imediatas, sem alteração na base, usualmente caracterizáveis por chaves simples de consulta, e
- **entradas externas** (EE ou EI), que são processos elementares que processam informações de negócio recebidas pelo sistema de fora da fronteira da aplicação e cuja finalidade principal é manter um Arquivo Lógico Interno .

19.4. Funções de Dados

As funções de dados estão divididas em dois tipos:

- **arquivos lógicos internos** (ALI ou ILF), que contém os dados permanentes, relevantes para o usuário e mantidos e utilizados pelo sistema. O sistema cria, altera e apaga esses dados, e

- **arquivos de interface externos** (AIE ou EIF), que contém dados permanentes e relevantes para os usuários, guardados em algum lugar por outra aplicação, mas referenciados pela aplicação em questão, sendo que outro sistema mantém esses dados.

O segundo passo é determinar a complexidade de cada função de negócio. A complexidade fornece um peso para cada função de negócio encontrada. O somatório do número de funções multiplicadas pelo seu peso fornece o número básico de pontos de função. Esse número é um indicador preliminar do tamanho do sistema.

Finalmente, no terceiro passo, o número básico de pontos de funções é corrigido em função de fatores que aumentam ou diminuem a complexidade do sistema.

19.5. Identificando Funções de Negócio

Para identificar as funções de negócio devemos partir de algum documento que aponte as funções **aprovadas** e pelo usuário e **úteis** para o negócio. Não devem ser contadas funções necessárias por causa da tecnologia aplicada. Basicamente, só é cobrado o que o usuário pode ver e está disposto a pagar. Também é importante que as funções de negócio sejam cobradas **como o usuário as percebem**. Isto significa que não interessa se estamos usando um ou vinte arquivos para guardar uma informação, mas sim de quantas formas o usuário pode acessar essa informação.

Além disso, devemos identificar as funções seguindo certa ordem. A ordem é importante porque encontrar um tipo de função de negócio ajuda a encontrar as funções de outro tipo. Assim, em um sistema novo devemos usar a ordem: saídas, consultas, entradas, arquivos e interfaces. Por outro lado, em um sistema já existente devemos usar a ordem arquivos, interfaces, saídas, consultas e entradas.

Para serem contados as funções devem:

- beneficiar claramente o usuário;
- ser especificamente aprovadas pelo usuário, e
- influenciar em algum grau mensurável o projeto, desenvolvimento, implementação e suporte à aplicação.

No manual da IFPUG podemos encontrar vários exemplos que nos permitem dirimir as dúvidas de como realizar a contagem. A técnica é simples, porém difícil de dominar.

19.5.1. Identificando Saídas

Para contar as saídas é necessário contar todas as informações que o sistema gera para o ambiente de forma procedimental. Tipicamente, saídas são relatórios impressos, telas, janelas e arquivos gerados para outras aplicações.

Devem ser contadas como saídas distintas cada formato utilizado. Basicamente, se for necessário fazer outro procedimento para produzir a informação, contamos como uma saída distinta. Também contamos cada tipo de lógica utilizada para fazer gerar a informação. Assim, se um relatório de vendas contém as vendas por vendedor e por loja, contaremos como duas saídas, pois são necessários procedimentos lógicos distintos para calcular cada um desses valores. Linhas de sumário e total, porém, não são contadas como novas saídas.

Uma saída externa pode ter uma “parte de entrada”, para, por exemplo, selecionar os registros necessários em um relatório, usando alguns campos como filtro. Essa “parte entrada” não é contada a parte, já está considerada nessa contagem.

19.5.2. Identificando Consultas

Consultas são, na prática, saídas simplificadas. Normalmente utilizadas para achar informações para modificá-las ou apagá-las. São sempre no monitor, não existe uma consulta em relatório de papel. Uma consulta não pode calcular nenhum valor. Em caso de cálculo de qualquer valor, temos uma saída.

19.6. Identificando Entradas

Entradas permitem adicionar, modificar e apagar informações. Se uma tela permite estas 3 funções, são contadas 3 entradas. Normalmente as funções de modificar e apagar ainda exigem consultas correspondentes para achar a informação que será alterada.

Um comando específico para o sistema executar algo é uma entrada.

Mensagens de erro que fazem parte de um processo não são contadas isoladamente, mas sim como um DET. Por exemplo, se você esquecer de colocar um campo obrigatório e receber uma mensagem de erro, não deve contar uma saída a mais, e sim essa mensagem como um DET da entrada ou consulta.

Mensagens de notificação, por outro lado, são processos elementares e devem ser contados como uma saída a parte. Por exemplo, ao tentar comprar um produto que não existe e receber uma mensagem com essa notificação foi feito um processamento, que é contado como uma saída externa adicional.

19.6.1. Entendendo as Lógicas de Processamento

A lógicas de processamento, ou formas de processamento lógico, são as características que uma transação tem de acordo com os requisitos solicitados especificamente pelo usuário.

A partir dessas lógicas de processamento podemos determinar, como indicado na Tabela 34, qual o tipo de função transacional que deve ser contado.

Tabela 19.1.: Tabela de apoio a determinação do tipo de função transacional. P=possível, N=Não permitida, O=obrigatório, O*= obrigatório que pelo menos uma das condições aconteça.

Forma de Processamento Lógico	Tipo de Função Transacional		
	EE	SE	CE
Realiza validação dos dados	P	P	P
Realiza cálculos ou fórmulas matemáticas	P	O*	N
Converte valores equivalentes	P	P	P
Filtra dados e seleciona usando critérios específicos	P	P	P
para comparar múltiplos conjuntos de dados			
Analisa condições para determinar qual é aplicável	P	P	P
Altera ou inclui ao menos um ILF	O*	O*	N
Referencia ao menos um ILF ou EIF	P	P	O
Recupera dados ou informações de controle	P	P	O
Cria dados derivados	P	O*	N
Altera o comportamento do sistema	O*	O*	N
Prepara e apresenta informações fora das fronteiras do sistema	P	O	O
É capaz de aceitar dados ou informação de controle que entra pela fronteira da aplicação	O	P	P
Reordena ou reorganiza um conjunto de dados	P	P	P

19.7. Identificando Arquivos Internos

Arquivos representam um grupamento lógico requerido pelo usuário. Podem incluir uma ou mais tabelas ou entidades.

Esse é uma das partes mais difíceis da contagem de pontos de função, pois devemos separar o que o usuário pensa do modelo que criamos. Nosso modelo muitas vezes usa vários grupos de dados, ou tabelas, ou entidades, para modelar algo que o usuário vê como um conceito único. Mesmo na modelagem conceitual, a tendência do analista é incluir entidades que o usuário não “vê” naturalmente.

Um Tipo de Elemento de Registro (RET, Record Element Type) é um subgrupo de elementos de dados dentro de um arquivo ou interface. Na prática, em um modelo de dados, um arquivo do usuário (ILF) é composto de um ou mais objetos do modelo.

Outra característica difícil de contar é que cada forma de acesso a um arquivo lógico conta novamente. Assim, por exemplo, se o usuário exige acessar um automóvel tanto por sua placa quanto por seu número do chassi, temos 2 arquivos lógicos para contar.

Exemplos: Uma nota fiscal é um arquivo lógico, com dois RETs: dados da nota, item de nota.

19.7.1. Identificando Arquivos Externos

Arquivos Externos representam um grupamento lógico requerido pelo usuário. Podem incluir uma ou mais tabelas ou entidades.

Arquivos Externos são mantidos por outras aplicações. Arquivos importados contam também como Entrada Externa, arquivos exportados contam também como Consulta Externa ou Saída Externa.

19.7.2. Identificando Itens de Dados (DETs)

Itens de dados ou elementos de dados (DETs) campos únicos, reconhecidos pelos usuários, desconsiderando-se recursão e repetição. DETs também podem invocar ações.

Exemplos de DETs em entradas externas são:

- campos de entrada de dados;
- mensagens de erro;
- valores calculados que são guardados;
- botões;
- mensagens de confirmação, e
- campos repetidos contam apenas como um DET.

Exemplos de DETs em saídas externas são:

- campos em relatório;
- valores calculados
- mensagens de erro;
- cabeçalhos de coluna que são gerados, e dinamicamente em um relatório

Exemplos de DETs em consultas externas (além dos anteriores que se enquadrem) são:

- campos usados em filtros de procura, e
- o clique do mouse.

Em GUIs, botões onde só se pode fazer uma seleção entre muitas (normalmente radio buttons) devem ser contados como um DET apenas. Já *check boxes* são normalmente contadas uma a uma. Botões de comando devem ser contados como um elemento de dados levando em conta o fato de executarem uma função.

Tabela 19.2.: Cálculo da complexidade das saídas externas.

Arquivos Referenciados	Itens de dados referenciados		
	1-5	6-19	20+
0-1	Simples(4)	Simples(4)	Média(5)
2-3	Simples(4)	Média(5)	Complexa(7)
4+	Média(5)	Complexa(7)	Complexa(7)

Tabela 19.3.: Cálculo da complexidade das entradas externas.

Arquivos Referenciados	Itens de dados referenciados		
	1-5	6-19	20+
0-1	Simples(3)	Simples(3)	Média(4)
2	Simples(3)	Média(4)	Complexa(6)
3+	Média(4)	Complexa(6)	Complexa(6)

Tabela 19.4.: Cálculo da complexidade das consultas externas.

Arquivos Referenciados	Itens de dados referenciados		
	1-5	6-19	20+
0-1	Simples(3)	Simples(3)	Média(4)
2-3	Simples(3)	Média(4)	Complexa(6)
4+	Média(4)	Complexa(6)	Complexa(6)

Tabela 19.5.: Cálculo da complexidade dos arquivos lógicos internos.

RETs	Itens de dados referenciados		
	1-19	20-50	51+
1	Simples(7)	Simples(7)	Média(10)
2-5	Simples(7)	Média(10)	Complexa(15)
6+	Média(10)	Complexa(15)	Complexa(15)

Tabela 19.6.: Cálculo da complexidade das interfaces lógicas externas.

RETs	Itens de dados referenciados		
	1-19	20-50	51+
1	Simples(5)	Simples(5)	Média(7)
2-5	Simples(5)	Média(7)	Complexa(10)
6+	Média(7)	Complexa(10)	Complexa(10)

19.8. As Perguntas

São 14 as perguntas que devem ser feitas e ajudaram a determinar a quantidade de PF relativa a um sistema. Cada uma deve ser respondida com um número, de 0 a 5, indicando a importância da característica que se pergunta sobre o sistema, da seguinte forma:

- 0 - Não tem influência
- 1 - Influência incidental
- 2 - Influência moderada
- 3 - Influência média
- 4 - Influência significativa
- 5 - Influência essencial em todo o sistema

Para cada pergunta, o padrão IFPUG determina tipos de respostas padronizadas que nos permitem dar a resposta (entre 0 e 5) mais facilmente, como é exemplificado no item 1 (Comunicação de Dados). Foge ao objetivo desse texto fornecer um detalhamento completo do padrão de contagem, que pode ser obtido em (IFPUG, 2010).

- Quantas facilidades de comunicação existem para facilitar a transferência ou troca de informação com a aplicação ou sistema?
- Como será tratada a distribuição de dados e processamento?
- O usuário exige tempos de resposta ou throughput, ou seja, o desempenho é crítico?
- Quão fortemente é utilizada a plataforma (hardware) onde a aplicação vai ser executada?
- Qual a frequência das transações (diárias, semanais, altas o suficiente para exigir um estudo de desempenho)?
- Que percentagem das informações é inserida on-line? o Se mais de 30
- A aplicação é projetada para eficiência para o usuário final?
- Quantas ILFs são alteradas por transações on-line?
- A aplicação tem processamento lógico ou matemático extensivo?
- A aplicação é desenvolvida para atender um ou muitos tipos de usuários diferentes?
- Qual a dificuldade de conversão e instalação?
- Qual a eficiência e grau de automação de inicialização, backup e recuperação?
- A aplicação foi especialmente projetada, desenvolvida e suportada para funcionar em locais diferentes em diferentes organizações?
- A aplicação foi especialmente projetada, desenvolvida e suportada para facilitar mudanças?

19.9. Cálculo dos Pfs Finais

Os PF são calculados em etapas: contam-se os números de entradas, saídas, consultas, arquivos e interfaces do sistema; Para cada entrada se determina um grau de complexidade, de acordo com as tabelas complexidade da função, e somando os resultados (Tabela 40) se obtém a contagem básica de pontos de função; responde-se a uma série de perguntas, as quais fornecem, cada uma, um valor de 0 a 5 ($p_i, 0 \leq i \leq 5$), calcula-se o número de pontos de função com a equação:

$$PF = \text{total-de-2} \times (0,65 + 0,01 \times \sum(p_i)). \quad (19.1)$$

Devemos notar que se o cálculo de PF for usado para fazer previsões seguindo o método COCOMO, apenas a contagem básica é necessária (só devem ser feitos os passos 1 e 2).

		Complexidade					
Função	Contagem Total	Simple	×	Média	×	Complexa	Total
Saídas Externas			4		5		7 =
Consultas Externas			3		4		6 =
Entradas Externas			3		4		6 =
Arquivos Lógicos Internos			7		10		15 =
Arquivos de Interface Externos			5		7		10 =
						Total	

Figura 19.2.: Planilha de registro para ajudar a contar os Pontos de Função

19.10. Conclusão

Este capítulo é apenas uma introdução aos Pontos de Função. Este método é bastante padronizado e tem muitas dicas de como contar, mas para nossa necessidade, ter uma ideia do tamanho do sistema, a forma de contagem aqui apresentada é razoável. Não foram feitas simplificações no método básico, mas nos padrões são tratados detalhes que podem levar a diferenças na contagem entre um profissional e a realizada por quem usou esse capítulo. Isso não é muito grave, já que o autor conhece casos onde empresas diferentes contam Pontos de Função de um mesmo sistema de forma diferente, com disparidades de 100%.

O melhor também é que seja feita apenas a contagem básica, por alguns motivos. O primeiro é que ela é mais aceita no mundo todo, sendo que alguns países só normatizaram ela. O segundo é que, para dar as respostas as perguntas existem definições bem mais precisas no padrão da IFPUG, e só demos uma leitura geral. O terceiro é que algumas dessas perguntas são muito antigas em relação a tecnologia.

Uma Visão Reduzida do Modelo COCOMO II

Success in all types of organization depends increasingly on the development of customized software solutions, yet more than half of software projects now in the works will exceed both their schedules and their budgets by more than 50%.

(Barry Boehm)

Conteúdo

20.1.	Dois Modelos em Um	246
20.2.	Esforço em Função do Tamanho	246
20.3.	Existe uma Relação Entre Esforço e Tempo	247

Por que COCOMO II?

Conhecer as relações entre o tamanho do projeto e o esforço necessário para desenvolvê-lo, e ainda o tempo necessário para esse esforço, é um passo importante na finalização da análise de sistemas e ainda para permitir escolhas de escopo.

COCOMO II (Constructive Cost Model II) é um método de previsão de custos de software (Barry W. Boehm et al., 2000), focado no esforço em pessoas-mês necessários para terminar um projeto. É um método baseado em estatísticas recolhidas em centenas de projetos de software. Por ser um método totalmente aberto, uma empresa pode calibrar o método para refletir o seu ambiente de desenvolvimento.

Atualmente é possível conseguir gratuitamente um software COCOMO, o que facilita o cálculo de custos de projeto de sistemas de informação¹.

Neste texto serão usadas as fórmulas e constantes do *COCOMO II Model Definition Manual version 1.4*(B. Boehm, Abts e Brad Clark, 1997).

20.1. Dois Modelos em Um

O método COCOMO II possui dois modelos, um chamado *Early Design Model*, adequado ao momento inicial de estimativa do projeto, e outro chamado *Post Architecture Model*, adequado ao momento onde a arquitetura do ciclo de vida de software está desenvolvida.

O *Early Design Model* usa como métrica de entrada a contagem de pontos de função não ajustada, e produz como métrica de saída o esforço necessário para completar o projeto.

20.2. Esforço em Função do Tamanho

O modelo básico do COCOMO II calcula o esforço em Pessoas-Mês em função do tamanho do software. As fórmulas, porém, se baseiam no tamanho do software em SLOCs, logo deve ser usada uma tabela de conversão de Pontos de Função para linhas de código, como a mostrada na Tabela 20.1.

$$PM_{\text{NOMINAL}} = A \times (\text{Size})^B \quad (20.1)$$

Onde A captura o esforço multiplicativo do aumento de tamanho e o expoente B captura as economias, e deseconomias, de escala que são encontradas em projetos de tamanhos diferentes(B. Boehm, Abts e Brad Clark, 1997). A Figura 20.1 exemplifica o efeito dessas constantes.

Linguagem	SLOC/PF
Assembly (com macros)	213
C	128
C++	128
Pascal	91
Planilha Eletrônica	6

Tabela 20.1.: Conversão de Pontos de Função não ajustados para linhas de código. Fonte: (B. Boehm, Abts e Brad Clark, 1997)

¹Uma lista de softwares na rede pode ser encontrada em <https://sites.google.com/cos.ufrj.br/livroanalisedesistemas>

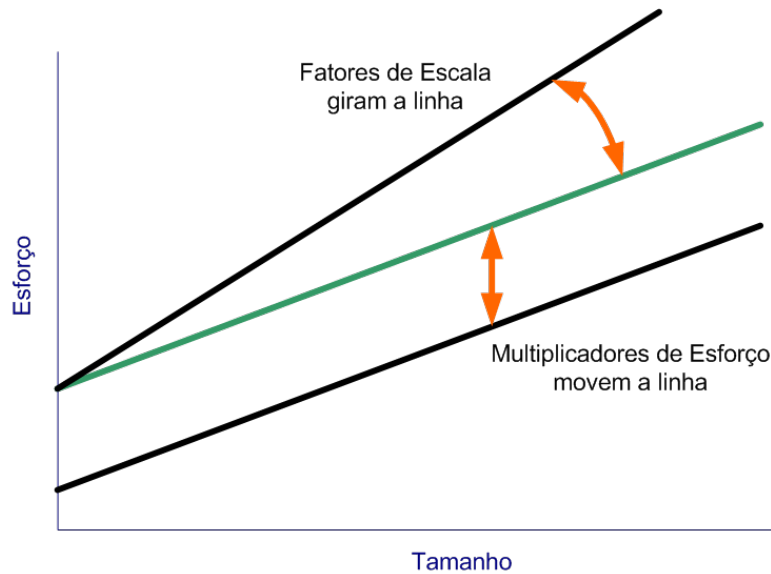


Figura 20.1.: Efeitos das constantes A e B na relação entre tamanho e esforço

20.3. Existe uma Relação Entre Esforço e Tempo

Um contribuição importante do método COCOMO II é determinar que existe uma relação entre o esforço necessário para desenvolver software e o tempo necessário para completar esse esforço. Antes mesmo de conhecer esse método, é importante deixar claro essa relação, pois muitos gerentes acreditam que podem espremer o tempo necessário para o desenvolvimento de um produto de software, o que leva as bastante conhecidas “Marchas da Morte” e, é claro, projetos atrasados, porque foram planejados de forma errada (Yourdon, 1994; Yourdon e Becker, 1997).

A suposição de que basta colocar mais gente, ou trabalhar mais horas, é suficiente para acelerar o tempo de desenvolvimento de um projeto, pode ser facilmente contestada com exemplos absurdos. Se esperamos que um software pode ser feito por 1 pessoa em 8 meses, é razoável imaginar, apesar de estar tecnicamente errado, que 2 pessoas poderiam fazê-lo em 4 meses. Porém, muito dificilmente 16 pessoas poderiam fazê-lo em 15 dias. Esse tipo de conta foi chamado de “Mito do Homem-Mês” (Brooks, 1995).

A relação entre o tempo de desenvolvimento e o esforço necessário, que apresentamos em sua forma completa a seguir, é parte importante do modelo COCOMO

$$T_{dev} = [C \times PM_{NS}]^{D+0,2 \times (E-B)} \times \frac{SCED\%}{100} \quad (20.2)$$

Nessa fórmula PM_{NS} é a quantidade nominal de pessoas-mês, $SCED\%$ é a compressão necessária no tempo de desenvolvimento, B, C e D são constantes calibráveis e E é um coeficiente calculado a partir de fatores de escala.

Os valores das constantes estão na tabela

O principal significado dessa fórmula pode ser associado ao fato que um projeto de desenvolvimento não pode ser comprimido, no tempo, além de certos limites. Coloquialmente, dizemos que software, como um bebê em gestação, pode até ficar bem feito se apressarmos o processo, porém não adianta pegar nove mães para fazer um bebê em um mês.

Tabela 20.2.: Valores das constantes no modelo COCOMO II padrão

A	2.94
B	0.91
C	3.67
D	0.28

20.3.1. Cálculo dos Fatores de Escala

O valor do coeficiente E é calculado pela equação:

$$E = B + 0.01 \times \sum_{j=1}^N SF_j \quad (20.3)$$

Na fase inicial do projeto, os fatores de escala são 5 ($N = 5$), e representam a precedência do sistema (PREC), a flexibilidade do projeto (FLEX), o risco do projeto e da arquitetura (RESL), a coesão da equipe (TEAM) e a maturidade do processo (PMAT). Cada um desses fatores de escala pode variar entre os conceitos:

- *Very Low* (muito baixa);
- *Low* (baixa);
- *Nominal* (nominal ou média);
- *High* (alta);
- *Very High* (muito alta), e
- *Extra High* (extramamente alta).

Originalmente, esses valores deveriam variar entre 5 (VL) e 0 (EH) , porém foram corrigidos estatisticamente. Em situação nominal para todos os casos, o valor de E seria a 1,06, porém o software COCOMO usa outros valores, calibrados, o valor de E acaba sendo 1,10.

$$PM = A \times \prod_{j=1}^{17} c_i \times \left(\left(1 + \frac{BRAK}{100} \right) \times PM_{NS} \right)^E \quad (20.4)$$

$$T_{DEV} = 3,67 \times (PM_{NS})^{0,32} \quad (20.5)$$

$$PM = 2,94 \times MLDC^{1,1} \quad (20.6)$$

Bibliografia

- Adiel Teixeira de Almeida Danielle Morais, Hannu Nurmi (2019). *Systems, Procedures And Voting Rules In Context: A Primer For Voting Rule Selection*. Advances In Group Decision And Negotiation Vol. 9. Springer (ver p. 55).
- Adzic, Gojko e David Evans (2014). *Fifty Quick Ideas to Improve your User Stories*. Neuri Consulting (ver pp. 198–201).
- Albrecht, A.J (1979). “Measuring Application Development Productivity”. Em: *Proc. IBM Applic. Dev. Symposium*. Monterey, CA, pp. 89–92 (ver p. 235).
- Almquist, Eric, Jamie Cleghorn e Lori Sherer (2018). “The B2B Elements of Value”. *Harvard Business Review*, pp. 72–81. URL: <https://hbr.org/2018/03/the-b2b-elements-of-value> (acesso em 09/02/2020) (ver pp. 49–51, 65).
- Almquist, Eric, John Senior e Nicolas Bloch (1 de set. de 2016). “The Elements of Value”. *Harvard Business Review*, pp. 46–53. URL: <https://hbr.org/2016/09/the-elements-of-value> (acesso em 09/02/2020) (ver pp. 48, 65).
- Amy, Douglas J. (2000). *Behind the Ballot Box: A Citizen’s Guide to Voting Systems*. Greenwood Publishing Group (ver p. 55).
- Andersen, Erling S, Kristoffer V Grude e Tor Haug (2009). *Goal Directed Project Management: Effective techniques and strategies*. 4ª ed. London: Kogan Page (ver p. 40).
- Anderson, Eric (2014). *Value-based Pricing: A Guide for Software Companies*. Tech Press (ver p. 64).
- Anklesaria, Jimmy (2008). *Supply Chain Cost Management. The AIM & DRIVE Process for Achieving Extraordinary Results*. New York: Amacom - American Management Association (ver pp. 59, 60).
- Beck, Kent (29 de set. de 1999). *eXtreme Programming eXplained: Embrace Change*. 1ª ed. USA: Addison-Wesley Longman Publishing Co., Inc. (ver p. 198).
- Beck, Kent, Mike Beedle et al. (2001). *Manifesto for Agile Software Development*. Online, Agile Alliance. Documento assinado por 17 praticantes de software em reunião em Snowbird, Utah. URL: <https://agilemanifesto.org/> (ver pp. 89, 94).
- Beck, Kent, Alistair Cockburn et al. (10 de nov. de 2014). *User Story And Use Case Comparison*. URL: <http://wiki.c2.com/?UserStoryAndUseCaseComparison> (acesso em 13/01/2020) (ver pp. 197, 198).
- Beck, Kent, Martha Roden et al. (8 de jan. de 2014). *User Story*. URL: <http://wiki.c2.com/?UserStory> (acesso em 13/01/2020) (ver p. 198).

- Belchior, Arnaldo Dias (1997). “Um Modelo Fuzzy para Avaliação da Qualidade de Software”. D.Sc. Tese de dout. Rio de Janeiro, Brasil: Programa de Engenharia de Sistemas e Computação, COPPE/UFRJ. 1-185 (ver pp. ix, 78, 80, 82).
- Bellovin, Steve (11 de ago. de 1997). *Software error may have contributed to Guam crash*. Ed. por Peter G. Neumann. The RISKS Digest Forum on Risks to the Public in Computers, Related Systems ACM Committee on Computers e Public Policy. URL: <http://catless.ncl.ac.uk/Risks/19/29#subj1> (acesso em 20/12/2019) (ver p. 12).
- Benington, Herbert D. (29 de jun. de 1956). “Production of Large Computer Programs”. Em: *Symposium on Advanced Programming Methods for Digital Computers*. Washington, D.C.: Office of Naval Research, Dept. of the Navy, pp. 15–27 (ver p. 87).
- BeSeen (17 de ago. de 2015). *The value triangle – managing customers’ expectations*. BeSeen. URL: <https://www.beseen-marketing.co.uk/marketing-blog/the-value-triangle> (acesso em 09/02/2020) (ver p. 63).
- Biffi, Stefan et al., ed. (2006). *Value-Based Software Engineering*. Berlin, Heidelberg: Springer (ver p. 62).
- Blanchard, Kenneth e Spencer Johnson (1983). *The One Minute Manager*. 10th anniversary. Berkley Trade (ver p. 138).
- Boehm, B. W. (mai. de 1988). “A spiral model of software development and enhancement”. *Computer* 21.5, pp. 61–72. ISSN: 1558-0814. DOI: 10.1109/2.59 (ver pp. 34, 89).
- Boehm, Barry, Chris Abts e and Sunita Devnani-Chulani Brad Clark (1997). *COCOMO II Model Definition Manual version 1.4* (ver pp. xiii, 106, 246).
- Boehm, Barry e Hasan Kitapci (2006). “The WinWin Approach: Using a Requirements Negotiation Tool for Rationale Capture and Use”. Em: *Rationale Management in Software Engineering*. Ed. por Raymond Dutoit Allen H. and McCall, Ivan Mistrik e Barbara Paech. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 173–190. ISBN: 978-3-540-30998-7. DOI: 10.1007/978-3-540-30998-7_8. URL: https://doi.org/10.1007/978-3-540-30998-7_8 (ver p. 34).
- Boehm, Barry W. et al. (2000). *Software Cost Estimation with Cocomo II with Cdrom*. 1st. USA: Prentice Hall PTR. ISBN: 0130266922 (ver pp. 63, 227–229, 232, 245).
- Brasil (2018). *Roteiro de Métricas de Software do SISP: versão 2.3*. Roteiro. Versão 2.3. Ministério do Planejamento, Desenvolvimento e Gestão. Secretaria de Tecnologia da Informação e Comunicação - Setic. (ver pp. 5, 236).
- Brooks, Frederick P. (1978). *The Mythical Man-Month. Essays on Software Engineering*. 1^a ed. USA: Addison-Wesley Longman Publishing Co., Inc. ISBN: 0201006502 (ver p. 63).
- (1995). *The Mythical Man-Month (Anniversary Ed.) Essays on Software Engineering*. 2^a ed. USA: Addison-Wesley Longman Publishing Co., Inc. ISBN: 0201835959 (ver pp. 227, 247).
- Cairncross, A. (1951). *Introduction to Economics*. Butterworth (ver pp. 44, 45).
- Carr, Nicholas G. (22 de jan. de 2005). “Does Not Compute”. *The New York Times*. ISSN: 0362-4331. URL: <http://www.nytimes.com/2005/01/22/opinion/does-not-compute.html> (acesso em 04/03/2017) (ver p. 12).
- Christel, M. e K. Kand (set. de 1992). *Issues in Requirements Elicitation*. Technical Report CMU/SEI-92-TR-012. Software Engineering Institute / CMU, p. 80. URL: https://resources.sei.cmu.edu/asset_files/TechnicalReport/1992_005_001_16478.pdf (acesso em 24/02/2020) (ver pp. x, xiii, 157, 159, 160).
- CMMI Product Team (nov. de 2010). *CMMI for Development, Version 1.3 Improving processes for developing better products and services*. CMU/SEI-2010-TR-033. URL: http://cmmiinstitute.com/system/files/models/CMMI_for_Development_v1.3.pdf (ver pp. 83, 98).

- Cockburn, Alistair (jan. de 2000). *Writing Effective Use Cases*. Addison-Wesley (ver pp. xiii, 110, 174–176, 181, 183–185, 189).
- Cohn, Mike (2004). *User Stories Applied: For Agile Software Development*. USA: Addison Wesley Longman Publishing Co., Inc. ISBN: 0321205685 (ver pp. 56, 110, 153, 198, 199, 201–204).
- (2005). *Agile Estimating and Planning*. Prentice Hall (ver pp. 55, 106, 107, 114, 203, 230, 231, 233).
- Conklin, Jeff (18 de nov. de 2005). *Dialogue Mapping: Building Shared Understanding of Wicked Problems*. 1 edition. Chichester, England ; Hoboken, NJ: Wiley. 266 pp. ISBN: 978-0-470-01768-5 (ver p. 14).
- Constantine, Larry L. (out. de 1993). “Work organization”. *Communications of the ACM* 36.10, pp. 35–43. DOI: 10.1145/163430.163435. URL: <https://doi.org/10.1145/163430.163435> (ver p. 106).
- Craigen, Dan, Susan Gerhart e Ted Ralston (1993). “An International Survey of Industrial Applications of Formal Methods”. Em: *Z User Workshop, London 1992*. Ed. por J. P. Bowen e J. E. Nicholls. London: Springer London, pp. 1–5 (ver p. 152).
- Czarnacka-Chrobot, Beata (jul. de 2012). “What Is the Cost of One IFPUG Method Function Point? – Case Study”. Em: *The 11th International Conference on Software Engineering Research and Practice (SERP’12), The 2012 World Congress in Computer Science, Computer Engineering & Applied Computing (WORLDCOMP’12)*. The 11th International Conference on Software Engineering Research and Practice (SERP’12), The 2012 World Congress in Computer Science, Computer Engineering & Applied Computing (WORLDCOMP’12), Las Vegas, Nevada, USA: CSREA Press (ver p. 5).
- Dahl, Robert A. (jul. de 1957). “The Concept of Power”. *Behavioral Science* 2.3, pp. 201–215 (ver p. 36).
- Dal Zot, Wili e Manuela Longoni de Castro (2015). *Matemática Financeira: fundamentos e aplicações*. Porto Alegre: Bookman, p. 151 (ver p. 57).
- Dalkey, Norman Crolee (out. de 1967). *Delphi*. Technical Report P-3704. Santa Monica, California: RAND Corporation. URL: <https://www.rand.org/pubs/papers/P3704.html> (acesso em 16/01/2020) (ver pp. 55, 114, 233).
- Davis, Alan M. (set. de 1992). “Operational Prototyping: A New Development Approach”. *IEEE Software* 9.5. Artigo seminal que propõe o modelo de prototipagem operacional, pp. 70–78. DOI: 10.1109/52.156899. URL: <https://doi.org/10.1109/52.156899> (ver p. 91).
- Defeo, Joseph e J.M. Juran (9 de jun. de 2010). *Juran’s Quality Handbook: The Complete Guide to Performance Excellence 6/e*. 6 edition. New York: McGraw-Hill Education. 1136 pp. ISBN: 978-0-07-162973-7 (ver p. 74).
- Denning, Stephen (2018). “The Age of Agile”. *Strategy & Leadership* 46.2, pp. 3–10. DOI: 10.1108/SL-01-2018-0003 (ver p. 95).
- digital.ai (2022). *15th State of Agile Report. Agile adoption accelertes across the enterprise*. White Paper (ver pp. 104, 113).
- Dijkstra, Edsger W. (out. de 1972). “The Humble Programmer”. *Commun. ACM* 15.10, pp. 859–866. ISSN: 0001-0782. DOI: 10.1145/355604.361591. URL: <http://doi.acm.org/10.1145/355604.361591> (ver p. 13).
- Dimitrov, Dimitri (2020). *Software Project Estimation. Intelligent Forecasting, Project Control, and Cliente Relationship Management*. Apress (ver p. 231).
- Drucker, Peter F. (1974). *agement: Tasks, Responsibilities, Practices*. New York: Truman Talley Books (ver p. 47).
- Erdogmus, Hakan, John Favaro e Michael Halling (2006). “Valuation of Software Initiatives Under Uncertainty: Concepts, Issues, and Techniques”. Em: *Value-Based Software Engineering*. Ed. por Stefan Biffl et al. Berlin, Heidelberg: Springer (ver p. 62).

- Extra (18 de jun. de 2019). *Se quer levar mais de 10 quilos, pague, sem problema nenhum', diz Bolsonaro sobre fim do despacho gratuito*. URL: <https://extra.globo.com/noticias/economia/se-quer-levar-mais-de-10-quilos-pague-sem-problema-nenhum-diz-bolsonaro-sobre-fim-do-despacho-gratuito-23747656.html> (acesso em 30/01/2020) (ver p. 69).
- Facetation (12 de mai. de 2015). *What is the history of the RACI chart?* URL: <http://facetation.blogspot.com/2015/05/what-is-history-of-raci-chart.html> (acesso em 06/10/2020) (ver p. 40).
- FATTO (2020). *Quanto pagar por um ponto de função?* URL: <https://docplayer.com.br/13499465-Quanto-pagar-por-um-ponto-de-funcao.html> (acesso em 02/02/2020) (ver p. 5).
- Fedotov, Alex (22 de fev. de 2019). *Septem Circumstantiae, five W's and H or 'six serving-men'*. URL: <https://alxfed.github.io/blog/posts/2019/02/22/Septem-Circumstantiae.html> (acesso em 08/01/2020) (ver p. 68).
- Finocchio Jr., José (2013). *Project Model Canvas: gerenciamento de projetos sem burocracia*. Rio de Janeiro: Campus, Elsevier (ver pp. ix, xii, 102, 103, 105, 106, 109, 112, 114).
- Flores, Fernando (2012). *Conversation for Action and Collected Essays: Instilling a Culture of Commitment in Working Relationships*. Ed. por Maria Flores Letelier. North Charleston, SC: CreateSpace Independent Publishing Platform (ver p. 69).
- Folha Online (29 de mai. de 2006). *Volkswagen anuncia recall de 123 mil veÃculos Gol, Fox e Kombi*. Folha Online - Dinheiro. URL: <http://www1.folha.uol.com.br/folha/dinheiro/ult91u108087.shtml> (acesso em 03/03/2017) (ver p. 12).
- Franceschini, Fiorenzo (2016). *Advanced Quality Function Deployment*. CRC Press, p. 208. ISBN: 9781420025439 (ver pp. 34, 51).
- Freeman, R Edward (1984). *Strategic Management: A Stakeholder Approach*. Boston: Pitman Publishing (ver p. 14).
- French, John R. P. e Bertram H. Raven (1959). "The Bases of Social Power". Em: *Studies in Social Power*. Ed. por Dorwin Cartwright. Ann Arbor, MI: Institute for Social Research, pp. 150–167 (ver pp. 36–38).
- Gane, Chris P. e Trish Sarson (1979). *Structured Systems Analysis: Tools and Techniques*. 1ª ed. Prentice Hall Professional Technical Reference. ISBN: 0138545472 (ver pp. 62, 109, 110, 152, 175).
- Garvin, David A. (1984). "What Does Product Quality Really Mean?" *MIT Sloan Management Review*. URL: <http://sloanreview.mit.edu/article/what-does-product-quality-really-mean/> (acesso em 06/03/2017) (ver p. 77).
- Gibbs, Wayt W. (set. de 1994). "Software's Chronic Crisis". *Scientific American*, pp. 86–100 (ver pp. 12, 14).
- Gillenson, Mark L. e Robert Goldberg (1984). *Strategic Planning, Systems Analysis, and Database Design: The Continuous Flow Approach*. John Wiley & Sons (ver p. 147).
- Gladys S. W. Lam, Ronald G. Ross and (out. de 2015). *Building Business Solutions: Business Analysis with Business Rules*. 2ª ed. sponsored by IIBA: International Institute of Business Analysis. Business Rule Solutions LLC., p. 304 (ver p. 29).
- Gray, Dave, Sunni Brown e James Macanufo (2010). *Gamestorming*. Sebastopol, CA: O'Reilly (ver pp. 55, 108, 111, 115).
- Green, Paul E. e V. Srinivasan (2000). *Conjoint Analysis in Marketing: New Developments with Implications for Research and Practice*. American Marketing Association (ver p. 64).
- Greenlaw, Steven A., David Shapiro e Timothy Taylor (2017). *Principles of Microeconomics fpr AP @Courses*. 2ª ed. OpenStax, Rice University (ver p. 46).

- Gross, John M. e Kenneth R. McInnis (2003). *Kanban Made Simple: Demystifying and Applying Toyota's Legendary Manufacturing Process*. New York: AMACOM (ver p. 105).
- Hastie, Shane e Stéphane Wojewoda (4 de out. de 2015). *Standish Group 2015 Chaos Report - Q&A with Jennifer Lynch*. URL: <https://www.infoq.com/articles/standish-chaos-2015/> (acesso em 15/12/2019) (ver p. 19).
- Hayes, Adam (25 de jun. de 2019). *Internal Rate of Return – IRR*. Investopedia. URL: <https://www.investopedia.com/terms/i/irr.asp> (acesso em 08/02/2020) (ver pp. 58, 62).
- Head, Robert V. (out. de 2002). “Getting Sabre off the Ground”. *IEEE Annals of the History of Computing* 24 (4), pp. 32–39. DOI: 10.1109/MAHC.2002.1114868 (ver p. 9).
- Helmer-Hirschberg, Olaf (mar. de 1967). *Analysis of the Future: The Delphi Method*. Technical Report P-3558. Santa Monica, California: RAND Corporation. URL: <https://www.rand.org/pubs/papers/P3558.html> (acesso em 16/01/2020) (ver pp. 55, 233).
- Higgins, J.M. (1994). *101 Creative Problem Solving Techniques: The Handbook of New Ideas for Business*. New Management Publishing Company. ISBN: 9781883629007. URL: https://books.google.com.br/books?id=Q1%5C_9LcYV03kC (ver p. 71).
- Highsmith, Jim (2002). *Agile Software Development Ecosystems*. Addison-Wesley. ISBN: 978-0201760439 (ver pp. 94, 95).
- Hoda, Rashina, James Noble e Stuart Marshall (2017). “Becoming agile: A grounded theory of agile transitions in practice”. *Empirical Software Engineering* 22.6, pp. 2543–2578. DOI: 10.1007/s10664-016-9463-9 (ver p. 95).
- Holanda Ferreira, Aurélio Buarque de (1986). *Novo Dicionário da Língua Portuguesa*. 2ª ed. Nova Fronteira (ver p. 44).
- IEEE (25 de jun. de 1998). *Recommended Practice for Software Requirements Specifications*. Standard ISO/IEC/IEEE 830:1998. IEEE (ver pp. 156, 168, 169).
- (dez. de 2010). *ISO/IEC/IEEE International Standard - Systems and software engineering – Vocabulary*. Standard 24765:2010(E). IEEE, pp. 1–418. DOI: 10.1109/IEEESTD.2010.5733835 (ver p. 151).
 - (2017a). *IEEE Recommended Practice on Software Reliability*. Standard IEEE Std 1633™-2016. New York: IEEE (ver p. 83).
 - (2017b). *ISO/IEC/IEEE International Standard - Systems and software engineering–Vocabulary*. Standard ISO/IEC/IEEE 24765:2017(E). ISO/IEC/IEEE, pp. 1–541. DOI: 10.1109/IEEESTD.2017.8016712 (ver pp. 8, 109).
 - (nov. de 2018). *ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering*. Standard 29148:2018. IEEE, pp. 1–104. DOI: 10.1109/IEEESTD.2018.8559686 (ver pp. 162, 163, 167–169).
- IEEE Computer Society (17 de jan. de 2014a). *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0*. Ed. por Pierre Bourque e Richard E. Fairley. 3 edition. IEEE Computer Society Press. 346 pp. ISBN: 978-0-7695-5166-1 (ver pp. viii, ix, 17, 20, 74–76, 83, 104, 226).
- (2014b). *Software Engineering Competency Model Version 1.0 SWECOM*. New York: IEEE Computer Society (ver pp. 23–25).
- IFPUG (2010). *Function Point Counting Practices Manual Release 4.3.1*. Standard. International Function Point Users Group (ver pp. 5, 56, 114, 243).
- (2012). *The IFPUG Guide to IT and Software Measurement*. Auerbach Publications (ver pp. 106, 236).
- IIBA (2011). *Um guia para o Corpo de Conhecimento de Análise de Negócios (Guia BABOK) Version 2.0*. Ontário, Canadá: International Institute of Business Analysis (ver pp. 52, 65, 110, 165).

- ISBS, International Software Benchmarking Standards Group - (2010). *Practical Software Project Estimation: A Toolkit for Estimating Software Development Effort & Duration*. Ed. por Peter Hill. McGraw Hill (ver pp. 231–233).
- ISO (15 de jun. de 2001). *Software Engineering – Product Quality – Part 1: Quality Model*. Standard ISO/IEC 9126-1:2001. Geneva, Switzerland: International Standards Organization (ver p. 75).
- (nov. de 2004). *Information technology — Process assessment — Part 1: Concepts and vocabulary*. Standard ISO/IEC 15504-1:2004. Geneva, Switzerland: ISO International Standards Organization (ver p. 83).
 - (set. de 2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Standard ISO/IEC 25010:2011(en). Geneva, Switzerland: ISO International Standards Organization (ver pp. ix, 75, 79–81).
 - (1 de mar. de 2015a). *Information technology – Process assessment – Concepts and terminology*. Standard ISO/IEC 33001. Geneva, Switzerland: ISO International Standards Organization (ver p. 83).
 - (15 de set. de 2015b). *Quality management systems — Fundamentals and vocabulary*. Standard ISO 9000:2015. Geneva, Switzerland: ISO International Standards Organization, p. 51 (ver pp. 75, 85).
 - (2017). *Systems And Software Engineering. Software Life Cycle Processes (British Standard)*. Standard BS ISO/IEC/IEEE 12207:2017. International Standards Organization (ver pp. 83, 84, 86, 87, 224).
- ISO/IEC/IEEE (2017). “ISO/IEC/IEEE International Standard - Systems and software engineering– Vocabulary”. *ISO/IEC/IEEE 24765:2017(E)*, pp. 1–541. DOI: 10 . 1109 / IEEESTD . 2017 . 8016712 (ver pp. 222, 223).
- Jacobson, Ivar (dez. de 1987). “Object-Oriented Development in an Industrial Environment”. *SIGPLAN Not.* 22.12, pp. 183–191. ISSN: 0362-1340. DOI: 10 . 1145 / 38807 . 38824. URL: <https://doi.org/10.1145/38807.38824> (ver p. 174).
- (1 de ago. de 2004). “Use cases – Yesterday, today, and tomorrow”. *Software & Systems Modeling* 3.3, pp. 210–220. ISSN: 1619-1374. DOI: 10 . 1007 / s10270 - 004 - 0060 - 3. URL: <https://doi.org/10.1007/s10270-004-0060-3> (ver p. 174).
- Jacobson, Ivar, Grady Booch e James Rumbaugh (1999). *The Unified Software Development Process*. 1ª ed. Boston, MA: Addison-Wesley Professional. ISBN: 9780201571691 (ver pp. 89, 94).
- Jacobson, Ivar, Magnus Christerson et al. (1992). *Object-oriented software engineering: a use case driven approach*. ACM Press Series. Reading: Addison-Wesley. ISBN: 9780201544350 (ver pp. 152, 153, 174).
- Jacobson, Ivar, Harold ”Bud” Lawson e Pan-Wei Ng (19 de jul. de 2019). *The Essentials of Modern Software Engineering: Free the Practices from the Method Prisons!* ACM Books. 400 pp. ISBN: 978-1-947487-24-6 (ver pp. 21, 38, 39, 105, 111, 137, 198, 203–205).
- Jacobson, Ivar, Paul E. McMahon e Roland Racko (2015). *24 Questions SEMAT and Essence: The Why’s, What’s and How’s to See the Difference*. Rel. técn. Ivar Jacobson International (ver p. 22).
- Jacobson, Ivar, Pan-Wei Ng et al. (26 de jan. de 2013). *The Essence of Software Engineering: Applying the SEMAT Kernel*. 1 edition. Upper Saddle River, NJ: Addison-Wesley Professional. 352 pp. ISBN: 978-0-321-88595-1 (ver pp. viii, 21–23, 27, 33).
- Jacobson, Ivar, Ian Spence e Kurt Bittner (dez. de 2011a). *Use-Case 2.0. The Guide to Succeeding with Use Cases*. White Paper. Ivar Jacobson International S.A. (ver pp. 152, 175).

- Jacobson, Ivar, Ian Spence e Kurt Bittner (dez. de 2011b). *Use-Case 2.0: The Guide to Succeeding with Use Cases* (ver pp. 110, 113).
- Jeffries, Ron (30 de ago. de 2001). *Essential XP: Card, Conversation, Confirmation*. URL: <https://ronjeffries.com/xprog/articles/expcardconversationconfirmation/> (acesso em 13/01/2020) (ver p. 199).
- Johnson, Phil (8 de ago. de 2012). *Curiosity about lines of code*. ITworld. URL: <http://www.itworld.com/article/2725085/big-data/curiosity-about-lines-of-code.html> (acesso em 04/03/2017) (ver p. 15).
- Jones, C. e O. Bonsignour (2012). *The Economics of Software Quality*. Addison-Wesley. ISBN: 9780132582209 (ver pp. xii, 84).
- Jones, Caper (abr. de 2005). “Software Cost Estimating Methods for Large Projects”. *Crosstalk The Journal of Defense Software Engineering* (April 2005). Disponível via WayBack Machine. URL: <http://www.stsc.hill.af.mil/crosstalk/2005/04/0504Jones.html> (ver p. 166).
- Kelley Jr, J. E. e M. R. Walker (1961). “Critical-path planning and scheduling”. Em: *Proceedings of the Eastern Joint Computer Conference*, pp. 160–173 (ver p. 215).
- Kendall, Kenneth E. e Julie E. Kendall (2013). *Systems Analysis and Design*. 9^a ed. Pearson (ver p. 149).
- Kenton, Will (25 de jun. de 2019). *Net Present Value (NPV)*. Investopedia. URL: <https://www.investopedia.com/terms/n/npv.asp> (acesso em 08/02/2020) (ver p. 61).
- Kerzner, Harold (2017). *Project Management. A system approach to planning, scheduling and controlling*. 12^a ed. Hoboken, New Jersey: John Wiley & Sons (ver pp. 63, 104, 108, 113, 207–209, 215, 216).
- Kim, W.C. e R. Mauborgne (2005). *A Estratégia Do Oceano Azul*. Elsevier. ISBN: 9788535215243 (ver p. 51).
- King, J. E. e Michael McLure (2014). *History of the Concept of Value*. Discussion Paper 14.06. The University of Western Australia, Department of Economics. URL: <https://ideas.repec.org/p/uwa/wpaper/14-06.html> (ver p. 45).
- Kotler, Philip, Gary Armstrong et al. (2017). *Principles of Marketing*. 7th European Edition. Pearsom (ver p. 46).
- Kotler, Philip e Kevin Lane Keller (2012). *Marketing Management*. 14^a ed. Boston: Prentice Hall (ver pp. 46, 47).
- (2013). *Administração de Marketing*. 14^a ed. São Paulo: Pearson (ver p. 47).
- Krasner, Herb (1 de jan. de 2021). *The Cost of Poor Software Quality in the US: A 2020 Report*. type. CISQ Consortium for Information & Software Quality. 46 pp. URL: <https://www.it-cisq.org/pdf/CPSQ-2020-report.pdf> (acesso em 26/06/2021) (ver p. 12).
- Kruchten, Philippe (1998). *The Rational Unified Process: An Introduction*. 1^a ed. Boston: Addison-Wesley (ver pp. 89, 93, 94).
- Krugman, Paul e Robin Wells (2013). *Microeconomics*. 3^a ed. New York: Worth Publishers (ver pp. 45, 46).
- Kumar, G. Prananna (2020). *Quality Function Deployment*. 12 Manage The Executive Fast Track Knowledge Center. URL: https://www.12manage.com/methods_akao_quality_function_deployment.html (acesso em 11/02/2020) (ver p. 34).
- Ladkin, Peter (7 de mai. de 1996). *The Cali and Puerto Plata B757 Crashes*. Ed. por Peter G. Neumann. Volume 18 Issue 10. The RISKS Digest Forum on Risks to the Public in Computers, Related Systems ACM Committee on Computers e Public Policy. URL: <http://catless.ncl.ac.uk/Risks/18/10#subj1%7D> (acesso em 20/12/2019) (ver p. 12).
- Leffingwell, Dean e Don Widrig (1999). *Managing Software Requirements: A Unified Approach*. USA: Addison-Wesley Longman Publishing Co., Inc. ISBN: 0201615932 (ver p. 52).

- Levenson, Nancy G. e Clark S. Turner (jul. de 1993). “An Investigation of the Therac-25 Accidents”. *Computer* 26.7, pp. 18–41 (ver p. 11).
- Levere, Jane L. (16 de mar. de 2001). “TECHNOLOGY; Electronic Data Systems to Buy Sabre Airline Computer Unit”. *The New York Times*, Section C, Page 8. URL: <https://www.nytimes.com/2001/03/16/business/technology-electronic-data-systems-to-buy-sabre-airline-computer-unit.html> (acesso em 02/02/2020) (ver p. 9).
- Lions, J. L. (19 de jul. de 1996). *ARIANE 5 Flight 501 Failure Report by the Inquiry Board*. Report. Paris: European Space Agency (ver p. 12).
- Magee, Joe C. e Adam D. Galinsky (2008). “Social Hierarchy: The Self-Reinforcing Nature of Power and Status”. *The Academy of Management Annals* 2.1, pp. 351–398. DOI: 10.1080/19416520802211628 (ver p. 37).
- Malcolm, D. G. et al. (1959). “Application of a technique for research and development program evaluation”. *Operations research* 7.5, pp. 646–669 (ver p. 214).
- Maximini, Dominik (2018). *The Scrum Culture. Introducing Agile Methos in Organizations*. 2ª ed. Management for Professionals. Springer (ver p. 233).
- McCall, J.A., P. K. Richards e G.F. Walters (nov. de 1977). *Factors in Software Quality: Concepts and Definitions of Software Quality*. Rel. técn. RADC-TR-77-369. General Electric. URL: <http://oai.dtic.mil/oai/oai?verb=getRecord&metadataPrefix=html&identifier=ADA049055%7D> (ver pp. ix, 77, 78).
- McConnell, Jane (2020). “Digital goods and consumer perception”. *Digital Economy Quarterly* 6, pp. 15–28 (ver p. 64).
- McMenamin, Stephen M. e John F. Palmer (1984). *Essential Systems Analysis*. USA: Yourdon Press. ISBN: 0917072308 (ver pp. 109, 137, 152, 161).
- Meskanen, T e H Nurmi (2006). “Distance from Consensus: A Theme and Variations”. Em: *Mathematics and Democracy. Studies in Choice and Welfare*. Ed. por Pukelsheim F. Simeone B. Berlin, Heidelberg: Springer. DOI: 10.1007/3-540-35605-3_9 (ver p. 55).
- Mitchell, Ronald K., Bradley R. Agle e Donna J. Wood (1997). “Toward a Theory of Stakeholder Identification and Salience: Defining the Principle of Who and What Really Counts”. *The Academy of Management Review* 22.4, pp. 853–886. ISSN: 0363-7425. DOI: 10.2307/259247. URL: <http://www.jstor.org/stable/259247%7D> (acesso em 15/01/2017) (ver pp. viii, 36, 38, 39, 119).
- Montoni, Mariano Angel, Ana Regina Rocha e Kival Chaves Weber (2009). “MPS.BR: a successful program for software process improvement in Brazil”. *Software Process: Improvement and Practice* 14.5, pp. 289–300. DOI: <https://doi.org/10.1002/spip.428>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spip.428>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spip.428> (ver p. 98).
- Moorman, Jan (9 de out. de 2012). *Leveraging the Kano Model for Optimal Results*. Article No :882. UX Magazine. URL: <https://uxmag.com/articles/leveraging-the-kano-model-for-optimal-results> (acesso em 08/02/2020) (ver pp. xii, 53, 54, 65).
- Muñoz, Carlos e Daniel Torres (2017). “Price perception in digital products: A behavioral study”. *Journal of Digital Economy* 9, pp. 123–140 (ver p. 64).
- Nakagawa, Marcelo (2012). *Ferramenta: 5W2H – Plano De Ação Estratégia e Gestão Para Empreendedores*. URL: <https://www.sebrae.com.br/Sebrae/Portal%5C%20Sebrae/Anexos/5W2H.pdf> (acesso em 20/08/2021) (ver pp. 67, 107).
- Naur, Peter e Brian Randell, ed. (jan. de 1969). *Software Engineering. Report on a conference sponsored by the NATO SCIENCE COMMITTEE Garmisch, Germany, 7th to 11th October 1968* (ver pp. 13, 104).
- Niquete, Paul (2006). *Softword: Provenance for the Word 'Software'*. URL: <http://niquette.com/books/softword/tocsoft.html> (acesso em 02/02/2017) (ver p. 8).

- Nocera, Julia (2019). “Customer-centric pricing strategies in software engineering”. *Software Economics Review* 15, pp. 34–56 (ver p. 64).
- North, Dan (16 de nov. de 2013). *User Story Template*. URL: <http://wiki.c2.com/?UserStoryTemplate> (acesso em 13/01/2020) (ver p. 200).
- OMG (2013). *Business Process Model and Notation (BPMN) Version 2.0.2*. Standard. Object Management Group. URL: <https://www.omg.org/spec/BPMN/2.0.2/PDF> (ver pp. 126, 127, 129–132, 134).
- (5 de dez. de 2017). *OMG® Unified Modeling Language® (OMG UML®). version 2.5.1*. specification formal/2017-12-05. Object Management Group (ver pp. 153, 175, 177, 182, 190, 195).
 - (out. de 2018). *Essence – Kernel and Language for Software Engineering Methods: Version 1.2*. specification. Object Management Group. URL: [%5Curl%7Bhttps://www.omg.org/spec/Essence/%7D](https://www.omg.org/spec/Essence/) (ver pp. 108, 137).
- Osterwalder, A., Y. Pigneur e T. Clark (2010). *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Strategyzer series. Hoboken, NJ: Wiley (ver pp. ix, 103, 104).
- Park, Robert E. (set. de 1992). *Software Size Measurement: A Framework for Counting Source Statements*. Technical Report CMU/SEI-92-TR-020 ESC-TR-92-020. Software Engineering Institute, Carnegie Mellon University. URL: https://resources.sei.cmu.edu/asset_files/TechnicalReport/1992_005_001_16082.pdf (acesso em 16/01/2020) (ver p. 230).
- Patton, Jeff e Peter Economy (2014a). *User Story Mapping: Discover the Whole Story, Build the Right Product*. 1st. O’Reilly Media, Inc. ISBN: 1491904909 (ver pp. x, 199, 200).
- (2014b). *User Story Mapping: Discover the Whole Story, Build the Right Product*. 1st. O’Reilly Media, Inc. (ver p. 213).
- PMI (2017). *Um Guia do Conhecimento em Gerenciamento de Projetos (Guia PMBOK®)*. Sexta Edição. Newtown Square, PA: Project Management Institute. ISBN: 9788502223745 (ver pp. viii, 28, 30, 32, 35, 38, 39, 71, 104, 111, 113, 114, 207–209, 211, 212, 226, 232).
- Poppendieck, Mary e Tom Poppendieck (2011). *Lean Software Development: An Agile Toolkit*. Addison-Wesley Professional. ISBN: 978-0321150783 (ver p. 95).
- Prajapati, Jaydeep (jul. de 2024). *The Biggest Software Failures of 2024*. Publicado em *Java Revisited* via Medium. URL: <https://medium.com/javarevisited/the-biggest-software-failures-of-2024-8e9413350f4c> (acesso em 02/08/2025) (ver p. 13).
- Pressman, Roger S. (2020). *Software Engineering: A Practitioner’s Approach*. McGraw-Hill (ver pp. 64, 88).
- Pressman, Roger S. e Bruce Maxim (23 de jan. de 2014). *Software Engineering: A Practitioner’s Approach*. 8 edition. New York, NY: McGraw-Hill Education. 976 pp. ISBN: 978-0-07-802212-8 (ver pp. ix, 8, 16, 18, 74, 76, 226).
- (2019). *Software Engineering: A Practitioner’s Approach*. 9ª ed. New York, NY: McGraw-Hill. 976 pp. ISBN: 978-1-260-54800-6 (ver pp. 104–107, 112, 222).
- Radek (2025). *Chaos Report – why this study about IT project management is so unique*. URL: <https://thestory.is/en/journal/chaos-report/> (acesso em 05/08/2025) (ver p. 16).
- Raven, Bertram H. (1993). “The Bases of Power: Origins and Recent Developments”. *Journal of Social Issues* 49.4, pp. 227–251. DOI: 10.1111/j.1540-4560.1993.tb01191.x (ver p. 36).
- (2008). “The Bases of Power and the Power/Interaction Model of Interpersonal Influence”. *Analyses of Social Issues and Public Policy* 8.1, pp. 1–22 (ver pp. 36–38).
- Ricardo, David (1996). *Princípios de Economia Política e Tributação*. Os Economistas. texto original de 1821. São Paulo: Editora Nova Cultural Ltda. (ver p. 45).
- Robertson, James e Suzzane Robertson (1998). *Complete Systems Analysis*. New York: Dorser House (ver p. 52).

- Robertson, James e Suzzane Robertson (2006). *Mastering the Requirements Process*. 2^a ed. Addison-Wesley Professional. ISBN: 0321419499 (ver pp. x, 53, 65, 152, 156, 157, 164).
- Rösler, Wolfram (1994). *The Hello World Collection*. URL: <http://helloworldcollection.de/> (acesso em 15/01/2020) (ver p. 229).
- Ross, Amanda (2018). “Dynamic pricing strategies for subscription-based software”. *Journal of Software Economics* 12, pp. 22–45 (ver p. 64).
- Ross, Timothy J. (2017). *Fuzzy logic with engineering applications*. Southern Gate, Chichester, West Sussex, United Kingdom: John Wiley & Sons (ver p. 55).
- Royce, Winston W. (1970). “Managing the Development of Large Software Systems”. Em: *Proceedings of IEEE WESCON*. Vol. 26. Publicado pela TRW e reimpresso pela IEEE, pp. 1–9 (ver p. 90).
- Rubin, Kenneth S. (2013). *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Pearson Education (ver pp. 39, 112, 231).
- Ruble, David A. (1997). *Practical Analysis & Design for Client/Server and GUI Systems*. Upper Saddle River: Yourdon Press (ver pp. 62, 107, 109, 137, 175).
- Satpathy, Tridibesh et al. (2016). *A Guide to the SCRUM BODY OF KNOWLEDGE (SBOK™ Guide)*. *A Comprehensive Guide to Deliver Projects using Scrum*. 2016^a ed. SCRUMStudy, VMEdU, Inc (ver pp. 52, 58, 63, 105, 116, 209).
- Savage, Neil (jun. de 2020). “An Animating Spirit”. *Communications of the ACM* 63.6, pp. 10–12. DOI: 10.1145/3392520 (ver p. 11).
- Scheel, Henrik von et al. (2015). “Phase 1: Process Concept Evolution”. Em: *The Complete Business Process Handbook*. Elsevier, pp. 1–9. DOI: 10.1016/b978-0-12-799959-3.00001-x (ver p. 122).
- SEMAT Inc. (2019). *ESSENCE User Guide*. URL: <http://semat.org/essence-user-guide%7D> (acesso em 20/12/2019) (ver p. 22).
- Serrat, Olivier (2017). “The Five Whys Technique”. Em: *Knowledge Solutions: Tools, Methods, and Approaches to Drive Organizational Performance*. Singapore: Springer Singapore, pp. 307–310. ISBN: 978-981-10-0983-9. DOI: 10.1007/978-981-10-0983-9_32. URL: https://doi.org/10.1007/978-981-10-0983-9_32 (acesso em 10/02/2020) (ver p. 70).
- Sloan, Michael C. (2010). “Aristotle’s Nicomachean Ethics as the Original Locus for the Septem Circumstantiae”. *Classical Philology* 105.3, pp. 236–251. ISSN: 0009837X, 1546072X. URL: <http://www.jstor.org/stable/10.1086/656196> (ver p. 68).
- Smith, Adam (2003). *The Wealth of Nations*. texto original de 1854. Bantam Classics. ISBN: 0553585975 (ver p. 45).
- Sommerville, Ian (3 de abr. de 2015). *Software Engineering*. 10 edition. Boston: Pearson. 816 pp. ISBN: 978-0-13-394303-0 (ver pp. ix, x, 110, 111, 152, 155, 156, 158).
- Sommerville, Ian e Pete Sawyer (1997). *Requirements Engineering: A Good Practice Guide*. 1st. USA: John Wiley & Sons, Inc. ISBN: 0471974447 (ver p. 151).
- Sowa, J.F. e J.A. Zachman (1992). “Extending and Formalizing the Framework for Information Systems Architecture”. *IBM Systems Journal* 31.3, p. 590 (ver p. 69).
- Sr., Lance B. Coleman (2015). *The Customer-Driven Organization: Employing the Kano Model*. 1^a ed. Productivity Press (ver p. 65).
- Stapenhurst, Tim (2009). *The Benchmarking Book*. Butterworth. ISBN: 0750677775 (ver p. 139).
- Stapleton, Jennifer, ed. (2003). *DSDM: Business Focused Development*. 2^a ed. London: DSDM Consortium, Addison Wesley (ver pp. 53, 108, 110, 115).
- Strathern, Paul (2003). *Uma Breve História da Economia*. Zahar (ver p. 45).
- Tang, Mei e Hao Li (2020). “Market behavior in emerging economies: A software perspective”. *Emerging Markets Journal* 8, pp. 89–104 (ver p. 64).

- Telles, M. e Y. Hsieh (2001). *The Science of Debugging*. Sebastapol, CA: O'Reilly Media (ver p. 223).
- The Standish Group (1994). *The Chaos Report 1994*. Report. The Standish Group (ver p. 17).
- (2015). *Chaos Report 2015*. Report. Standish Group (ver pp. xii, 16, 17, 19, 105).
- Thomsett, Rob (2002). *Radical Project Management*. Prentice Hall PTR. ISBN: 013458886X (ver p. 113).
- Tracy, Brian (2015). *Creativity & Problem Solving*. New York: AMACOM (ver pp. 108, 111, 115).
- Travis, Gregory (s.d.). “How the Boeing 737 Max Disaster Looks to a Software Developer: Design shortcuts meant to make a new plane seem like an old, familiar one are to blame” (). URL: <https://spectrum.ieee.org/how-the-boeing-737-max-disaster-looks-to-a-software-developer> (ver p. 11).
- Tricentis (2019). *Software FAILS Watch*. White Paper. Versão 5th Edition. Tricentis (ver p. 12).
- Tukey, John W. (1958). “The Teaching of Concrete Mathematics”. *The American Mathematical Monthly* 65.1, pp. 1–9. ISSN: 0002-9890. DOI: 10.2307/2310294. URL: www.jstor.org/stable/2310294 (acesso em 20/12/2019) (ver p. 8).
- Urban, Joseph E. (1992). *Software Prototyping and Requirements Engineering*. Rel. técn. Documento técnico que descreve a classificação de protótipos com base em práticas da indústria. ITT Systems & Sciences Corporation. URL: <https://csiac.dtic.mil/wp-content/uploads/2021/06/SW-Prototyping-and-Requirements-Engineering.pdf> (ver p. 90).
- Vaughan-Nichols, Steven J. (7 de nov. de 2017). *MINIX: Intel's hidden in-chip operating system*. URL: <https://www.zdnet.com/article/minix-intels-hidden-in-chip-operating-system/> (acesso em 02/02/2020) (ver p. 9).
- Venners, B. (8 de dez. de 2003). *Design by Contract: A Conversation with Bertrand Meyer*. URL: www.artima.com/intv/contracts.html (ver p. 84).
- Volare (2020). *Prioritisation Analysis*. URL: <https://www.volere.org/prioritisation-analysis/> (acesso em 24/02/2020) (ver p. 166).
- Wake, Bill (17 de ago. de 2003). *INVEST in Good Stories, and SMART Tasks*. URL: <https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/> (acesso em 14/01/2020) (ver pp. 201, 203).
- (8 de fev. de 2012). *Independent Stories in the INVEST Model*. URL: <https://xp123.com/articles/independent-stories-in-the-invest-model/> (acesso em 14/01/2020) (ver pp. 202, 203).
- Wieggers, Karl (mai. de 1999). “Writing Quality Requirements”. *Software Development* 7.5, pp. 44–48. ISSN: 1070-8588 (ver p. 170).
- Wieggers, Karl E. e Joy Beatty (2013). *Software Requirements*. 3ª ed. Redmond, WA, USA: Microsoft Press (ver pp. xiii, 36, 152, 153, 157, 158, 161, 199).
- Yourdon, Edward (1989). *Modern Structured Analysis*. Englewood Cliffs, N.J.: Yourdon Press/Prentice-Hall. ISBN: 0135986249 (ver p. 152).
- (1994). *Decline and Fall of the American Programmer*. 1st. USA: Prentice Hall PTR. ISBN: 013191958X (ver p. 247).
- Yourdon, Edward e Paul D. Becker (1997). *Death March: The Complete Software Developer's Guide to Surviving Mission Impossible Projects*. USA: Prentice Hall PTR. ISBN: 0137483104 (ver p. 247).
- Zachman, J.A. (1987). “A Framework for Information Systems Architecture”. *IBM Systems Journal* 26.3, p. 276 (ver p. 69).

Índice Remissivo

- RFP*, **152**
- Request for Proposal*, **152**
- Work Breakdown Structure*, **212**
- future value*, **58**
- present value*, **58**
- 100-pontos, **52**
- 5W2H, **67**
- 5w2h, **67**
- alpha, **21**
- arquivos de interface externos, **238**
- arquivos lógicos internos, **237**
- ator, **176**
- ator principal, **175, 176**
- bem sucedidos, **16**
- benefício, **62**
- BPMN, **125**
 - atividade, **127**
 - Gateway, **132**
 - gateway
 - baseado em eventos, **133**
 - complexo, **134**
 - exclusivo, **132**
 - inclusivo, **133**
 - paralelo, **133**
 - paralelos baseado em eventos, **133**
 - participante, **126**
 - processo
 - público, **126**
 - processos
 - privados, **126**
 - subprocesso, **127**
 - tarefa, **127**
- bug, **223**
- Capability Maturity Model, **83**
- Capability Maturity Model Integration, **83**
- caso de uso, **175, 176**
 - diagrama, **190**
- caso de uso 2.0, **176**
- casos de uso, **173**
- causa raiz, **138**
- cenário, **176, 178**
 - alternativo, **176**
 - principal, **176**
- cenário alternativo, **179**
- cenário principal, **179**
- ciclo de vida, **86**
- CMM, **83**
- CMMI, **83**
- colaboração, **126**
- com problemas, **16**
- condição, **176**
- consultas externas, **237**
- Controlgramas de Gantt, **213**
- coreografia, **126**
- CPM, **214**
- crash, **223**
- custo, **4**
- custo de oportunidade, **46**
- defeito (defect), **223**
- demanda, **47**
 - estados, **47**
- Desdobramento da Função Qualidade, **34**
- desejo, **46**
- diagrama de caso de uso, **190**

- Diagrama de Causa Raiz, **143**
- Diagrama de Espinha de Peixe, **143**
- Diagramas de Rede, **215**
- dinheiro de brinquedo, **52**

- EAP, **212**
- elicitação, **157**
- Engenharia de Software, 7, **17**
- entradas externas, **237**
- entrega intermediária, 208
- erro (error), **223**
- erro (mistake), **223**
- esforço, **227**
- especificação de requisitos, **151**
- estimativa de tamanho software, 226
- Estrutura Analítica de Projeto, **212**
- estruturas de dados, **8**

- falha (failure), **223**
- falta ou defeito (fault), **223**
- ferramentas, 18
- fluxo condicional, **134**
- fluxo de exceção, **134**
- fluxo de mensagem, **134**
- fluxo de sequência normal, **134**
- fluxo default, **134**
- fluxo não controlado, **134**
- fracassos, **16**
- Framework Cynefin, **3**
- Framework de Zachman, 69
- funcionalidade, **16**
- função utilidade, **45**

- garantias, **186**

- hardware, **8**
- história do usuário, 197, **197**
- histórias complexas, **204**
- histórias compostas, **204**

- IMC, **218**
- IMT, **218**
- incrementos, **91**
- informação, **8**
- instrução, **8**
- interesse, 33, **34**
- Início Mais Cedo, **218**
- Início Mais Tarde, **218**
- IRACIS, **62**
- ISO 25000, **75**
- ISO 25010, **79**

- ISO 9126, **75**
- iteração, **91**

- juro, **58**

- Kano, 53
- kernel, 21

- latência de decisão, **20**
- legitimidade, **38**
- lista de atores, **189**

- markup, **45**
- Matriz de Engajamento, **36**
- menor produto viável, **203**
- Modelagem de negócio, 121
- modelagem de processos, **122**
- Modelagem de Processos de Negócio, **122**
- modelos de ciclo de vida, **86**
- modo de trabalho, 22
- moeda virtual, **165**
- montante, **58**
- MoSCoW, **52, 165**
- MTBF, **155**
- Mundo Real, 2
- método, **21**
- métodos, 18

- narrativa, 179
 - numerada, 179
- necessidade, 46

- objetivo, 33, **34, 176**
- oportunidade, 22, 136, **137**
 - tecnológica, 148
- oportunidades
 - de negócio, 148
- orçamento, **16**

- pacote de trabalho, 212
- pagamento, **58**
- parte interessada, 185
- partes interessadas, 18, **22, 27, 208**
- PERT, **214**
- PERT/CPM, 214
- planning poker, 231, 233
- poder, 36, **38**
- pontos de função, 235
- Pontos de história, **230**
- pontos de história, **233**
- prazo, **16, 58**
- prestação, **58**

ÍNDICE REMISSIVO

- preço, 4
- principal, **58**
- princípio da diminuição da utilidade
 - marginal, **45**
- Princípio de Pareto, **140**
- problema, 136
- problemas de negócio, **138**
- problemas funcionais, **138**
- problemas operacionais, **138**
- processo, **122**
- processo de negócio, **122**
- processos, 18
- projeto, **207**
- prática, 21
- práticas, 18
- pré-condição, 186
- pós-condição, 186
- QFD, 34
- qualidade, 18, **75**
- qualidade de software, 74
- Quality Function Deployment, 34
- QWAN, **77**
- redes de precedência, 215
- remuneração, **58**
- requisito, 22, **151**
 - arbitrário, 161
 - do negócio, 161
 - do sistema, 161
 - do software, 161
 - do usuário, 161
 - elicitação, 157
 - especificação, 151
 - falso, 161
 - tecnológico, 161
- requisito de software, **197**
- requisitos de desempenho, 156
- requisitos de interface, 156
- requisitos essenciais, **152**
- requisitos funcionais, 154, **154**
- requisitos não funcionais, **154**, 155
- restrições, **154**
- saldo, **58**
- saliência, **38**
- satisfação, **47**
- saídas externas, **237**
- sistema de software, **22**
- sobreposição, **202**
- software, 8, **8**
- stakeholders, **27**
- tamanho do software, 226, 228
- taxa, **58**
- taxa de juros, **58**
- Testes, 222
- testes regressivos, **224**
- time, **22**
- TMC, **218**
- TMT, **218**
- trabalho, **22**
- Término Mais Cedo, **218**
- Término Mais Tarde, **218**
- ubíquo, **11**
- urgência, **39**
- user story, 197
- usuário, **29**
- usuário externo, **30**
- usuário final, **29**
- usuário interno, **31**
- usuário onisciente, **30**
- utilidade, 45
- valor, **9**, 18, 43, 44, **44**, 47, 62, **62**
- valor atual, **58**
- valor de resgate, **58**
- valor descontado, **58**
- valor futuro, **58**
- valor presente, **58**
- Volere, **166**
- WBS, **212**
- wicked problems, **14**